

Tomasz Drabas, Denny Lee

Foreword by: **Holden Karau**

Principal Software Engineer at IBM Spark Technology Center

Learning PySpark

Build data-intensive applications locally and deploy
at scale using the combined powers of Python and
Spark 2.0



Packt>

Learning PySpark

Build data-intensive applications locally and deploy at scale using the combined powers of Python and Spark 2.0

Tomasz Drabas

Denny Lee



BIRMINGHAM - MUMBAI

Learning PySpark

Copyright © 2017 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the authors, nor Packt Publishing, and its dealers and distributors will be held liable for any damages caused or alleged to be caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

First published: February 2017

Production reference: 1220217

Published by Packt Publishing Ltd.
Livery Place
35 Livery Street
Birmingham B3 2PB, UK.

ISBN 978-1-78646-370-8

www.packtpub.com

Credits

Authors

Tomasz Drabas

Denny Lee

Reviewer

Holden Karau

Commissioning Editor

Amey Varangaonkar

Acquisition Editor

Prachi Bisht

Content Development Editor

Amrita Noronha

Technical Editor

Akash Patel

Copy Editor

Safis Editing

Project Coordinator

Shweta H Birwatkar

Proofreader

Safis Editing

Indexer

Aishwarya Gangawane

Graphics

Disha Haria

Production Coordinator

Aparna Bhagat

Cover Work

Aparna Bhagat

Foreword

Thank you for choosing this book to start your PySpark adventures, I hope you are as excited as I am. When Denny Lee first told me about this new book I was delighted-one of the most important things that makes Apache Spark such a wonderful platform, is supporting both the Java/Scala/JVM worlds and Python (and more recently R) worlds. Many of the previous books for Spark have been focused on either all of the core languages, or primarily focused on JVM languages, so it's great to see PySpark get its chance to shine with a dedicated book from such experienced Spark educators. By supporting both of these different worlds, we are able to more effectively work together as Data Scientists and Data Engineers, while stealing the best ideas from each other's communities.

It has been a privilege to have the opportunity to review early versions of this book, which has only increased my excitement for the project. I've had the privilege of being at some of the same conferences and meetups and watching the authors introduce new concepts in the world of Spark to a variety of audiences (from first timers to old hands), and they've done a great job distilling their experience for this book. The experience of the authors shines through with everything from their explanations to the topics covered. Beyond simply introducing PySpark they have also taken the time to look at *up and coming* packages from the community, such as GraphFrames and TensorFrames.

I think the community is one of those often-overlooked components when deciding what tools to use, and Python has a great community and I'm looking forward to you joining the Python Spark community. So, enjoy your adventure; I know you are in good hands with Denny Lee and Tomek Drabas. I truly believe that by having a diverse community of Spark users we will be able to make better tools useful for everyone, so I hope to see you around at one of the conferences, meetups, or mailing lists soon :)

Holden Karau

P.S.

I owe Denny a beer; if you want to buy him a Bud Light lime (or lime-a-rita) for me I'd be much obliged (although he might not be quite as amused as I am).

About the Authors

Tomasz Drabas is a Data Scientist working for Microsoft and currently residing in the Seattle area. He has over 13 years of experience in data analytics and data science in numerous fields: advanced technology, airlines, telecommunications, finance, and consulting he gained while working on three continents: Europe, Australia, and North America. While in Australia, Tomasz has been working on his PhD in Operations Research with a focus on choice modeling and revenue management applications in the airline industry.

At Microsoft, Tomasz works with big data on a daily basis, solving machine learning problems such as anomaly detection, churn prediction, and pattern recognition using Spark.

Tomasz has also authored the *Practical Data Analysis Cookbook* published by Packt Publishing in 2016.

I would like to thank my family: Rachel, Skye, and Albert – you are the love of my life and I cherish every day I spend with you! Thank you for always standing by me and for encouraging me to push my career goals further and further. Also, to my family and my in-laws for putting up with me (in general).

There are many more people that have influenced me over the years that I would have to write another book to thank them all. You know who you are and I want to thank you from the bottom of my heart!

However, I would not have gotten through my PhD if it was not for Czesia Wieruszewska; Czesiu - dziękuję za Twoją pomoc bez której nie rozpocząłbym mojej podróży po Antypodach. Along with Krzys Krzysztozek, you guys have always believed in me! Thank you!

Denny Lee is a Principal Program Manager at Microsoft for the Azure DocumentDB team—Microsoft's blazing fast, planet-scale managed document store service. He is a hands-on distributed systems and data science engineer with more than 18 years of experience developing Internet-scale infrastructure, data platforms, and predictive analytics systems for both on-premise and cloud environments.

He has extensive experience of building greenfield teams as well as turnaround/change catalyst. Prior to joining the Azure DocumentDB team, Denny worked as a Technology Evangelist at Databricks; he has been working with Apache Spark since 0.5. He was also the Senior Director of Data Sciences Engineering at Concur, and was on the incubation team that built Microsoft's Hadoop on Windows and Azure service (currently known as HDInsight). Denny also has a Masters in Biomedical Informatics from Oregon Health and Sciences University and has architected and implemented powerful data solutions for enterprise healthcare customers for the last 15 years.

I would like to thank my wonderful spouse, Hua-Ping, and my awesome daughters, Isabella and Samantha. You are the ones who keep me grounded and help me reach for the stars!

About the Reviewer

Holden Karau is transgender Canadian, and an active open source contributor. When not in San Francisco working as a software development engineer at IBM's Spark Technology Center, Holden talks internationally on Spark and holds office hours at coffee shops at home and abroad. Holden is a co-author of numerous books on Spark including High Performance Spark (which she believes is the gift of the season for those with expense accounts) & Learning Spark. Holden is a Spark committer, specializing in PySpark and Machine Learning. Prior to IBM she worked on a variety of distributed, search, and classification problems at Alpine, Databricks, Google, Foursquare, and Amazon. She graduated from the University of Waterloo with a Bachelor of Mathematics in Computer Science. Outside of software she enjoys playing with fire, welding, scooters, poutine, and dancing.

www.PacktPub.com

eBooks, discount offers, and more

Did you know that Packt offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at www.PacktPub.com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at customercare@packtpub.com for more details.

At www.PacktPub.com, you can also read a collection of free technical articles, sign up for a range of free newsletters and receive exclusive discounts and offers on Packt books and eBooks.



<https://www.packtpub.com/mapt>

Get the most in-demand software skills with Mapt. Mapt gives you full access to all Packt books and video courses, as well as industry-leading tools to help you plan your personal development and advance your career.

Why subscribe?

- Fully searchable across every book published by Packt
- Copy and paste, print, and bookmark content
- On demand and accessible via a web browser

Customer Feedback

Thanks for purchasing this Packt book. At Packt, quality is at the heart of our editorial process. To help us improve, please leave us an honest review on this book's Amazon page at <https://www.amazon.com/dp/1786463709>.

If you'd like to join our team of regular reviewers, you can email us at customerreviews@packtpub.com. We award our regular reviewers with free eBooks and videos in exchange for their valuable feedback. Help us be relentless in improving our products!

Table of Contents

Preface	vii
Chapter 1: Understanding Spark	1
What is Apache Spark?	2
Spark Jobs and APIs	3
Execution process	3
Resilient Distributed Dataset	4
DataFrames	6
Datasets	6
Catalyst Optimizer	6
Project Tungsten	7
Spark 2.0 architecture	8
Unifying Datasets and DataFrames	9
Introducing SparkSession	10
Tungsten phase 2	11
Structured streaming	13
Continuous applications	14
Summary	15
Chapter 2: Resilient Distributed Datasets	17
Internal workings of an RDD	17
Creating RDDs	18
Schema	20
Reading from files	20
Lambda expressions	21
Global versus local scope	23
Transformations	24
The .map(...) transformation	24
The .filter(...) transformation	25
The .flatMap(...) transformation	26

The .distinct(...) transformation	26
The .sample(...) transformation	27
The .leftOuterJoin(...) transformation	27
The .repartition(...) transformation	28
Actions	29
The .take(...) method	29
The .collect(...) method	29
The .reduce(...) method	29
The .count(...) method	31
The .saveAsTextFile(...) method	31
The .foreach(...) method	32
Summary	32
Chapter 3: DataFrames	33
Python to RDD communications	34
Catalyst Optimizer refresh	35
Speeding up PySpark with DataFrames	36
Creating DataFrames	38
Generating our own JSON data	38
Creating a DataFrame	39
Creating a temporary table	39
Simple DataFrame queries	42
DataFrame API query	42
SQL query	42
Interoperating with RDDs	43
Inferring the schema using reflection	43
Programmatically specifying the schema	44
Querying with the DataFrame API	46
Number of rows	46
Running filter statements	46
Querying with SQL	47
Number of rows	47
Running filter statements using the where Clauses	48
DataFrame scenario – on-time flight performance	49
Preparing the source datasets	50
Joining flight performance and airports	50
Visualizing our flight-performance data	52
Spark Dataset API	53
Summary	54

Chapter 4: Prepare Data for Modeling	55
Checking for duplicates, missing observations, and outliers	56
Duplicates	56
Missing observations	60
Outliers	64
Getting familiar with your data	66
Descriptive statistics	67
Correlations	70
Visualization	71
Histograms	72
Interactions between features	76
Summary	77
Chapter 5: Introducing MLlib	79
Overview of the package	80
Loading and transforming the data	80
Getting to know your data	85
Descriptive statistics	85
Correlations	87
Statistical testing	89
Creating the final dataset	90
Creating an RDD of LabeledPoints	90
Splitting into training and testing	91
Predicting infant survival	91
Logistic regression in MLlib	91
Selecting only the most predictable features	93
Random forest in MLlib	94
Summary	96
Chapter 6: Introducing the ML Package	97
Overview of the package	97
Transformer	98
Estimators	101
Classification	101
Regression	103
Clustering	103
Pipeline	104
Predicting the chances of infant survival with ML	105
Loading the data	105
Creating transformers	106
Creating an estimator	107
Creating a pipeline	107

Fitting the model	108
Evaluating the performance of the model	109
Saving the model	110
Parameter hyper-tuning	111
Grid search	111
Train-validation splitting	115
Other features of PySpark ML in action	116
Feature extraction	116
NLP - related feature extractors	116
Discretizing continuous variables	119
Standardizing continuous variables	120
Classification	122
Clustering	123
Finding clusters in the births dataset	124
Topic mining	124
Regression	127
Summary	129
Chapter 7: GraphFrames	131
Introducing GraphFrames	134
Installing GraphFrames	134
Creating a library	135
Preparing your flights dataset	138
Building the graph	140
Executing simple queries	141
Determining the number of airports and trips	142
Determining the longest delay in this dataset	142
Determining the number of delayed versus on-time/early flights	142
What flights departing Seattle are most likely to have significant delays?	143
What states tend to have significant delays departing from Seattle?	144
Understanding vertex degrees	145
Determining the top transfer airports	146
Understanding motifs	147
Determining airport ranking using PageRank	149
Determining the most popular non-stop flights	151
Using Breadth-First Search	152
Visualizing flights using D3	154
Summary	155

Chapter 8: TensorFrames	157
What is Deep Learning?	157
The need for neural networks and Deep Learning	161
What is feature engineering?	163
Bridging the data and algorithm	164
What is TensorFlow?	166
Installing Pip	168
Installing TensorFlow	169
Matrix multiplication using constants	170
Matrix multiplication using placeholders	171
Running the model	172
Running another model	172
Discussion	173
Introducing TensorFrames	174
TensorFrames – quick start	175
Configuration and setup	176
Launching a Spark cluster	176
Creating a TensorFrames library	176
Installing TensorFlow on your cluster	176
Using TensorFlow to add a constant to an existing column	177
Executing the Tensor graph	178
Blockwise reducing operations example	179
Building a DataFrame of vectors	180
Analysing the DataFrame	180
Computing elementwise sum and min of all vectors	181
Summary	182
Chapter 9: Polyglot Persistence with Blaze	183
Installing Blaze	184
Polyglot persistence	185
Abstracting data	186
Working with NumPy arrays	186
Working with pandas' DataFrame	188
Working with files	189
Working with databases	192
Interacting with relational databases	192
Interacting with the MongoDB database	194
Data operations	194
Accessing columns	194
Symbolic transformations	195
Operations on columns	197

Table of Contents

Reducing data	198
Joins	200
Summary	202
Chapter 10: Structured Streaming	203
What is Spark Streaming?	203
Why do we need Spark Streaming?	206
What is the Spark Streaming application data flow?	207
Simple streaming application using DStreams	208
A quick primer on global aggregations	213
Introducing Structured Streaming	218
Summary	222
Chapter 11: Packaging Spark Applications	223
The spark-submit command	223
Command line parameters	224
Deploying the app programmatically	227
Configuring your SparkSession	227
Creating SparkSession	228
Modularizing code	229
Structure of the module	229
Calculating the distance between two points	231
Converting distance units	231
Building an egg	232
User defined functions in Spark	232
Submitting a job	233
Monitoring execution	236
Databricks Jobs	237
Summary	241
Index	243

Preface

It is estimated that in 2013 the whole world produced around 4.4 zettabytes of data; that is, 4.4 *billion* terabytes! By 2020, we (as the human race) are expected to produce ten times that. With data getting larger literally by the second, and given the growing appetite for making sense out of it, in 2004 Google employees Jeffrey Dean and Sanjay Ghemawat published the seminal paper *MapReduce: Simplified Data Processing on Large Clusters*. Since then, technologies leveraging the concept started growing very quickly with Apache Hadoop initially being the most popular. It ultimately created a Hadoop ecosystem that included abstraction layers such as Pig, Hive, and Mahout – all leveraging this simple concept of map and reduce.

However, even though capable of chewing through petabytes of data daily, MapReduce is a fairly restricted programming framework. Also, most of the tasks require reading and writing to disk. Seeing these drawbacks, in 2009 Matei Zaharia started working on Spark as part of his PhD. Spark was first released in 2012. Even though Spark is based on the same MapReduce concept, its advanced ways of dealing with data and organizing tasks make it 100x faster than Hadoop (for in-memory computations).

In this book, we will guide you through the latest incarnation of Apache Spark using Python. We will show you how to read structured and unstructured data, how to use some fundamental data types available in PySpark, build machine learning models, operate on graphs, read streaming data, and deploy your models in the cloud. Each chapter will tackle different problem, and by the end of the book we hope you will be knowledgeable enough to solve other problems we did not have space to cover here.

What this book covers

Chapter 1, Understanding Spark, provides an introduction into the Spark world with an overview of the technology and the jobs organization concepts.

Chapter 2, Resilient Distributed Datasets, covers RDDs, the fundamental, schema-less data structure available in PySpark.

Chapter 3, DataFrames, provides a detailed overview of a data structure that bridges the gap between Scala and Python in terms of efficiency.

Chapter 4, Prepare Data for Modeling, guides the reader through the process of cleaning up and transforming data in the Spark environment.

Chapter 5, Introducing MLlib, introduces the machine learning library that works on RDDs and reviews the most useful machine learning models.

Chapter 6, Introducing the ML Package, covers the current mainstream machine learning library and provides an overview of all the models currently available.

Chapter 7, GraphFrames, will guide you through the new structure that makes solving problems with graphs easy.

Chapter 8, TensorFrames, introduces the bridge between Spark and the Deep Learning world of TensorFlow.

Chapter 9, Polyglot Persistence with Blaze, describes how Blaze can be paired with Spark for even easier abstraction of data from various sources.

Chapter 10, Structured Streaming, provides an overview of streaming tools available in PySpark.

Chapter 11, Packaging Spark Applications, will guide you through the steps of modularizing your code and submitting it for execution to Spark through command-line interface.

For more information, we have provided two bonus chapters as follows:

Installing Spark: <https://www.packtpub.com/sites/default/files/downloads/InstallingSpark.pdf>

Free Spark Cloud Offering: <https://www.packtpub.com/sites/default/files/downloads/FreeSparkCloudOffering.pdf>

What you need for this book

For this book you need a personal computer (can be either Windows machine, Mac, or Linux). To run Apache Spark, you will need Java 7+ and an installed and configured Python 2.6+ or 3.4+ environment; we use the Anaconda distribution of Python in version 3.5, which can be downloaded from <https://www.continuum.io/downloads>.

The Python modules we randomly use throughout the book come preinstalled with Anaconda. We also use GraphFrames and TensorFrames that can be loaded dynamically while starting a Spark instance: to load these you just need an Internet connection. It is fine if some of those modules are not currently installed on your machine – we will guide you through the installation process.

Who this book is for

This book is for everyone who wants to learn the fastest-growing technology in big data: Apache Spark. We hope that even the more advanced practitioners from the field of data science can find some of the examples refreshing and the more advanced topics interesting.

Conventions

In this book, you will find a number of styles of text that distinguish between different kinds of information. Here are some examples of these styles, and an explanation of their meaning.

Code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles are shown as follows:

A block of code is set as follows:

```
data = sc.parallelize([
    ('Amber', 22), ('Alfred', 23), ('Skye', 4), ('Albert', 12),
    ('Amber', 9)])
```

When we wish to draw your attention to a particular part of a code block, the relevant lines or items are set in bold:

```
rdd1 = sc.parallelize([('a', 1), ('b', 4), ('c', 10)])
rdd2 = sc.parallelize([('a', 4), ('a', 1), ('b', '6'), ('d', 15)])
rdd3 = rdd1.leftOuterJoin(rdd2)
```


Any command-line input or output is written as follows:

```
java -version
```

New terms and **important words** are shown in bold. Words that you see on the screen, in menus or dialog boxes for example, appear in the text like this: "Clicking the **Next** button moves you to the next screen."

[ Warnings or important notes appear in a box like this.]

[ Tips and tricks appear like this.]

Reader feedback

Feedback from our readers is always welcome. Let us know what you think about this book – what you liked or may have disliked. Reader feedback is important for us to develop titles that you really get the most out of.

To send us general feedback, simply send an e-mail to feedback@packtpub.com, and mention the book title via the subject of your message.

If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, see our author guide on www.packtpub.com/authors.

Customer support

Now that you are the proud owner of a Packt book, we have a number of things to help you to get the most from your purchase.

Downloading the example code

You can download the example code files for all Packt books you have purchased from your account at <http://www.packtpub.com>. If you purchased this book elsewhere, you can visit <http://www.packtpub.com/support> and register to have the files e-mailed directly to you.

All the code is also available on GitHub: <https://github.com/drabastomek/learningPySpark>.

You can download the code files by following these steps:

1. Log in or register to our website using your e-mail address and password.
2. Hover the mouse pointer on the **SUPPORT** tab at the top.
3. Click on **Code Downloads & Errata**.
4. Enter the name of the book in the **Search** box.
5. Select the book for which you're looking to download the code files.
6. Choose from the drop-down menu where you purchased this book from.
7. Click on **Code Download**.

Once the file is downloaded, please make sure that you unzip or extract the folder using the latest version of:

- WinRAR / 7-Zip for Windows
- Zipeg / iZip / UnRarX for Mac
- 7-Zip / PeaZip for Linux

The code bundle for the book is also hosted on GitHub at <https://github.com/PacktPublishing/Learning-PySpark>. We also have other code bundles from our rich catalog of books and videos available at <https://github.com/PacktPublishing/>. Check them out!

Downloading the color images of this book

We also provide you with a PDF file that has color images of the screenshots/diagrams used in this book. The color images will help you better understand the changes in the output. You can download this file from https://www.packtpub.com/sites/default/files/downloads/LearningPySpark_ColorImages.pdf.

Errata

Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you find a mistake in one of our books—maybe a mistake in the text or the code—we would be grateful if you would report this to us. By doing so, you can save other readers from frustration and help us improve subsequent versions of this book. If you find any errata, please report them by visiting <http://www.packtpub.com/submit-errata>, selecting your book, clicking on the **errata submission form** link, and entering the details of your errata. Once your errata are verified, your submission will be accepted and the errata will be uploaded on our website, or added to any list of existing errata, under the Errata section of that title. Any existing errata can be viewed by selecting your title from <http://www.packtpub.com/support>.

Piracy

Piracy of copyright material on the Internet is an ongoing problem across all media. At Packt, we take the protection of our copyright and licenses very seriously. If you come across any illegal copies of our works, in any form, on the Internet, please provide us with the location address or website name immediately so that we can pursue a remedy.

Please contact us at copyright@packtpub.com with a link to the suspected pirated material.

We appreciate your help in protecting our authors, and our ability to bring you valuable content.

Questions

You can contact us at questions@packtpub.com if you are having a problem with any aspect of the book, and we will do our best to address it.

1

Understanding Spark

Apache Spark is a powerful open source processing engine originally developed by Matei Zaharia as a part of his PhD thesis while at UC Berkeley. The first version of Spark was released in 2012. Since then, in 2013, Zaharia co-founded and has become the CTO at Databricks; he also holds a professor position at Stanford, coming from MIT. At the same time, the Spark codebase was donated to the Apache Software Foundation and has become its flagship project.

Apache Spark is fast, easy to use framework, that allows you to solve a wide variety of complex data problems whether semi-structured, structured, streaming, and/or machine learning / data sciences. It also has become one of the largest open source communities in big data with more than 1,000 contributors from 250+ organizations and with 300,000+ Spark Meetup community members in more than 570+ locations worldwide.

In this chapter, we will provide a primer to understanding Apache Spark. We will explain the concepts behind Spark Jobs and APIs, introduce the Spark 2.0 architecture, and explore the features of Spark 2.0.

The topics covered are:

- What is Apache Spark?
- Spark Jobs and APIs
- Review of Resilient Distributed Datasets (RDDs), DataFrames, and Datasets
- Review of Catalyst Optimizer and Project Tungsten
- Review of the Spark 2.0 architecture

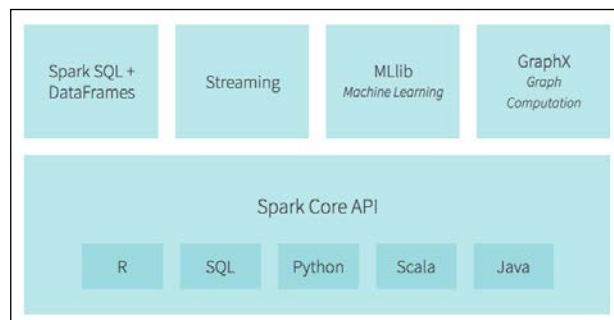
What is Apache Spark?

Apache Spark is an open-source powerful distributed querying and processing engine. It provides flexibility and extensibility of MapReduce but at significantly higher speeds: Up to 100 times faster than Apache Hadoop when data is stored in memory and up to 10 times when accessing disk.

Apache Spark allows the user to read, transform, and aggregate data, as well as train and deploy sophisticated statistical models with ease. The Spark APIs are accessible in Java, Scala, Python, R and SQL. Apache Spark can be used to build applications or package them up as libraries to be deployed on a cluster or perform *quick* analytics interactively through notebooks (like, for instance, Jupyter, Spark-Notebook, Databricks notebooks, and Apache Zeppelin).

Apache Spark exposes a host of libraries familiar to data analysts, data scientists or researchers who have worked with Python's `pandas` or R's `data.frames` or `data.tables`. It is important to note that while Spark DataFrames will be *familiar* to `pandas` or `data.frames` / `data.tables` users, there are some differences so please temper your expectations. Users with more of a SQL background can use the language to shape their data as well. Also, delivered with Apache Spark are several already implemented and tuned algorithms, statistical models, and frameworks: MLlib and ML for machine learning, GraphX and GraphFrames for graph processing, and Spark Streaming (DStreams and Structured). Spark allows the user to combine these libraries seamlessly in the same application.

Apache Spark can easily run locally on a laptop, yet can also easily be deployed in standalone mode, over YARN, or Apache Mesos - either on your local cluster or in the cloud. It can read and write from a diverse data sources including (but not limited to) HDFS, Apache Cassandra, Apache HBase, and S3:



Source: Apache Spark is the smartphone of Big Data <http://bit.ly/1QsgaNj>



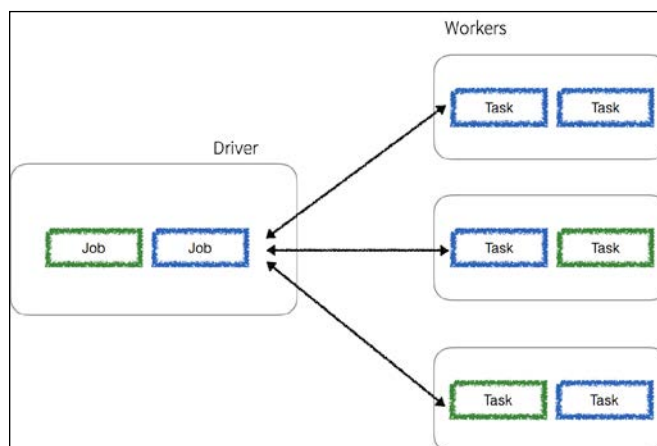
For more information, please refer to: Apache Spark is the Smartphone of Big Data at <http://bit.ly/1QsgaNj>

Spark Jobs and APIs

In this section, we will provide a short overview of the Apache Spark Jobs and APIs. This provides the necessary foundation for the subsequent section on Spark 2.0 architecture.

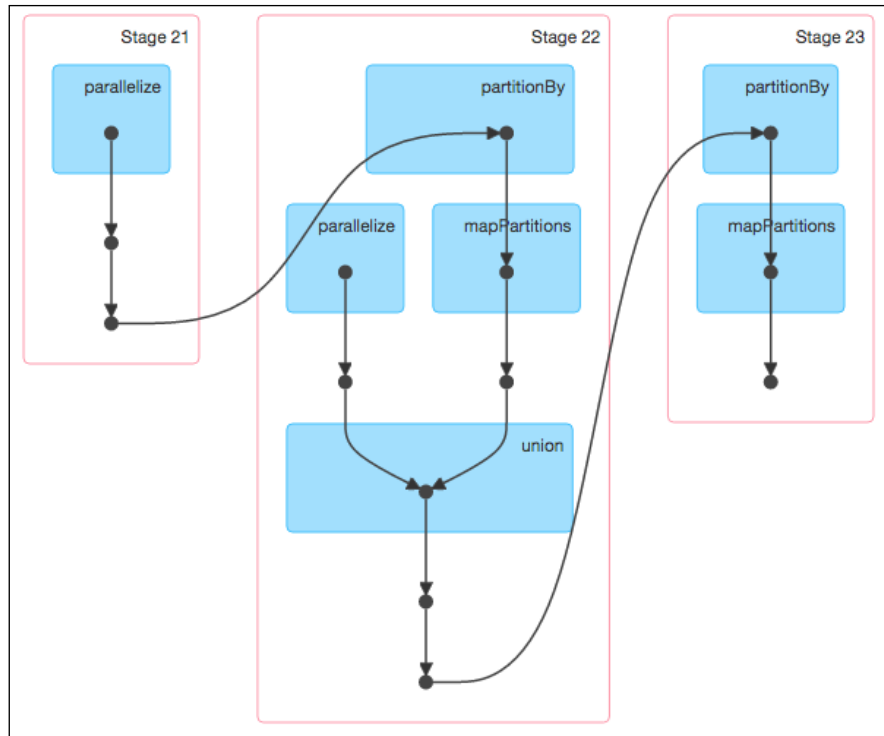
Execution process

Any Spark application spins off a single driver process (that can contain multiple jobs) on the *master* node that then directs executor processes (that contain multiple tasks) distributed to a number of *worker* nodes as noted in the following diagram:



The driver process determines the number and the composition of the task processes directed to the executor nodes based on the graph generated for the given job. Note, that any worker node can execute tasks from a number of different jobs.

A Spark job is associated with a chain of object dependencies organized in a direct acyclic graph (DAG) such as the following example generated from the Spark UI. Given this, Spark can optimize the scheduling (for example, determine the number of tasks and workers required) and execution of these tasks:



[ For more information on the DAG scheduler, please refer to <http://bit.ly/29WTiK8>.]

Resilient Distributed Dataset

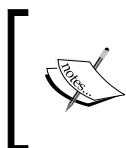
Apache Spark is built around a distributed collection of immutable Java Virtual Machine (JVM) objects called Resilient Distributed Datasets (RDDs for short). As we are working with Python, it is important to note that the Python data is stored within these JVM objects. More of this will be discussed in the subsequent chapters on RDDs and DataFrames. These objects allow any job to perform calculations very quickly. RDDs are calculated against, cached, and stored in-memory; a scheme that results in orders of magnitude faster computations compared to other traditional distributed frameworks like Apache Hadoop.

At the same time, RDDs expose some coarse-grained transformations (such as `map(...)`, `reduce(...)`, and `filter(...)` which we will cover in greater detail in Chapter 2, *Resilient Distributed Datasets*), keeping the flexibility and extensibility of the Hadoop platform to perform a wide variety of calculations. RDDs apply and log transformations to the data in parallel, resulting in both increased speed and fault-tolerance. By registering the transformations, RDDs provide data lineage - a form of an ancestry tree for each intermediate step in the form of a graph. This, in effect, guards the RDDs against data loss - if a partition of an RDD is lost it still has enough information to recreate that partition instead of simply depending on replication.



If you want to learn more about data lineage check this link
<http://ibm.co/2ao9B1t>.

RDDs have two sets of parallel operations: *transformations* (which return pointers to new RDDs) and *actions* (which return values to the driver after running a computation); we will cover these in greater detail in later chapters.



For the latest list of transformations and actions, please refer to the Spark Programming Guide at <http://spark.apache.org/docs/latest/programming-guide.html#rdd-operations>.

RDD transformation operations are *lazy* in a sense that they do not compute their results immediately. The transformations are only computed when an action is executed and the results need to be returned to the driver. This delayed execution results in more fine-tuned queries: Queries that are optimized for performance. This optimization starts with Apache Spark's DAGScheduler - the stage oriented scheduler that transforms using *stages* as seen in the preceding screenshot. By having separate RDD *transformations* and *actions*, the DAGScheduler can perform optimizations in the query including being able to avoid *shuffling* the data (the most resource intensive task).

For more information on the DAGScheduler and optimizations (specifically around narrow or wide dependencies), a great reference is the *Narrow vs. Wide Transformations* section in *High Performance Spark* in Chapter 5, *Effective Transformations* (<https://smile.amazon.com/High-Performance-Spark-Practices-Optimizing/dp/1491943203>).

DataFrames

DataFrames, like RDDs, are immutable collections of data distributed among the nodes in a cluster. However, unlike RDDs, in DataFrames data is organized into named columns.



If you are familiar with Python's `pandas` or R `data.frames`, this is a similar concept.

DataFrames were designed to make large data sets processing even easier. They allow developers to formalize the structure of the data, allowing higher-level abstraction; in that sense DataFrames resemble tables from the relational database world. DataFrames provide a domain specific language API to manipulate the distributed data and make Spark accessible to a wider audience, beyond specialized data engineers.

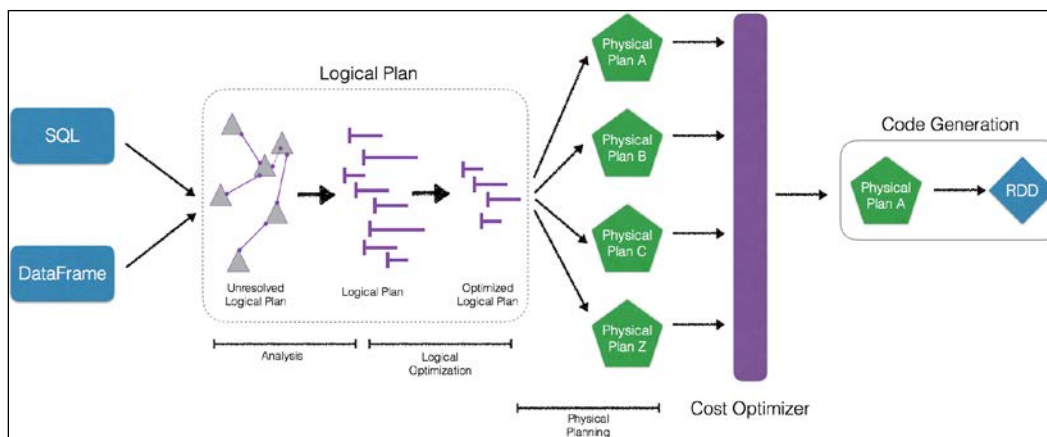
One of the major benefits of DataFrames is that the Spark engine initially builds a logical execution plan and executes generated code based on a physical plan determined by a cost optimizer. Unlike RDDs that can be significantly slower on Python compared with Java or Scala, the introduction of DataFrames has brought performance parity across all the languages.

Datasets

Introduced in Spark 1.6, the goal of Spark Datasets is to provide an API that allows users to easily express transformations on domain objects, while also providing the performance and benefits of the robust Spark SQL execution engine. Unfortunately, at the time of writing this book Datasets are only available in Scala or Java. When they are available in PySpark we will cover them in future editions.

Catalyst Optimizer

Spark SQL is one of the most technically involved components of Apache Spark as it powers both SQL queries and the DataFrame API. At the core of Spark SQL is the Catalyst Optimizer. The optimizer is based on functional programming constructs and was designed with two purposes in mind: To ease the addition of new optimization techniques and features to Spark SQL and to allow external developers to extend the optimizer (for example, adding data source specific rules, support for new data types, and so on):



For more information, check out *Deep Dive into Spark SQL's Catalyst Optimizer* (<http://bit.ly/271I7Dk>) and *Apache Spark DataFrames: Simple and Fast Analysis of Structured Data* (<http://bit.ly/29QbcOV>)

Project Tungsten

Tungsten is the codename for an umbrella project of Apache Spark's execution engine. The project focuses on improving the Spark algorithms so they use memory and CPU more efficiently, pushing the performance of modern hardware closer to its limits.

The efforts of this project focus, among others, on:

- Managing memory explicitly so the overhead of JVM's object model and garbage collection are eliminated
- Designing algorithms and data structures that exploit the memory hierarchy
- Generating code in runtime so the applications can exploit modern compilers and optimize for CPUs
- Eliminating virtual function dispatches so that multiple CPU calls are reduced
- Utilizing low-level programming (for example, loading immediate data to CPU registers) to speed up the memory access and optimizing Spark's engine to efficiently compile and execute simple loops



For more information, please refer to

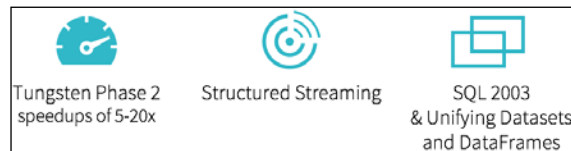
Project Tungsten: Bringing Apache Spark Closer to Bare Metal (<https://databricks.com/blog/2015/04/28/project-tungsten-bringing-spark-closer-to-bare-metal.html>)

Deep Dive into Project Tungsten: Bringing Spark Closer to Bare Metal [SSE 2015 Video and Slides] (<https://spark-summit.org/2015/events/deep-dive-into-project-tungsten-bringing-spark-closer-to-bare-metal/>) and

Apache Spark as a Compiler: Joining a Billion Rows per Second on a Laptop (<https://databricks.com/blog/2016/05/23/apache-spark-as-a-compiler-joining-a-billion-rows-per-second-on-a-laptop.html>)

Spark 2.0 architecture

The introduction of Apache Spark 2.0 is the recent major release of the Apache Spark project based on the key learnings from the last two years of development of the platform:



Source: Apache Spark 2.0: Faster, Easier, and Smarter <http://bit.ly/2ap7qd5>

The three overriding themes of the Apache Spark 2.0 release surround performance enhancements (via Tungsten Phase 2), the introduction of structured streaming, and unifying Datasets and DataFrames. We will describe the Datasets as they are part of Spark 2.0 even though they are currently only available in Scala and Java.



Refer to the following presentations by key Spark committers for more information about Apache Spark 2.0:

Reynold Xin's Apache Spark 2.0: Faster, Easier, and Smarter webinar
<http://bit.ly/2ap7qd5>

Michael Armbrust's Structuring Spark: DataFrames, Datasets, and Streaming
<http://bit.ly/2ap7qd5>

Tathagata Das' A Deep Dive into Spark Streaming <http://bit.ly/2aHt1w0>

Joseph Bradley's Apache Spark MLlib 2.0 Preview: Data Science and Production
<http://bit.ly/2aHrOVN>

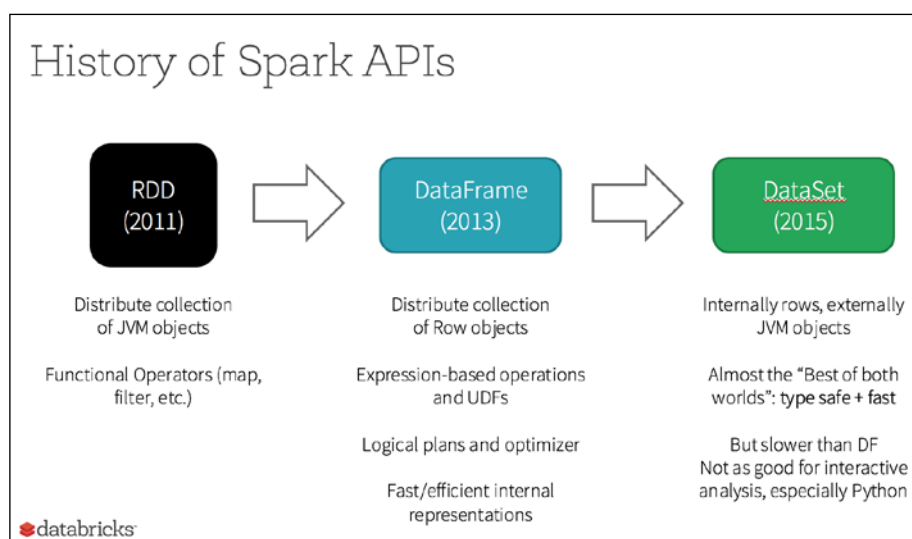
Unifying Datasets and DataFrames

In the previous section, we stated out that Datasets (at the time of writing this book) are only available in Scala or Java. However, we are providing the following context to better understand the direction of Spark 2.0.

Datasets were introduced in 2015 as part of the Apache Spark 1.6 release. The goal for datasets was to provide a type-safe, programming interface. This allowed developers to work with semi-structured data (like JSON or key-value pairs) with compile time type safety (that is, production applications can be checked for errors before they run). Part of the reason why Python does not implement a Dataset API is because Python is not a type-safe language.

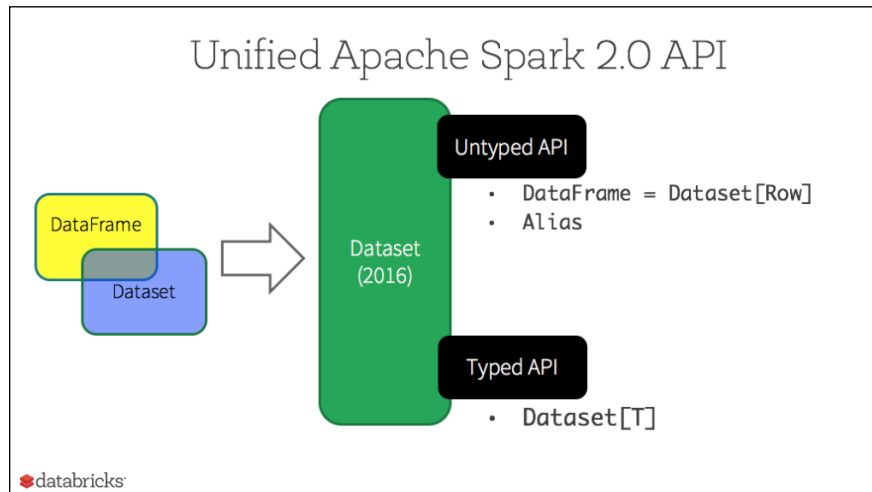
Just as important, the Datasets API contain high-level domain specific language operations such as `sum()`, `avg()`, `join()`, and `group()`. This latter trait means that you have the flexibility of traditional Spark RDDs but the code is also easier to express, read, and write. Similar to DataFrames, Datasets can take advantage of Spark's catalyst optimizer by exposing expressions and data fields to a query planner and making use of Tungsten's fast in-memory encoding.

The history of the Spark APIs is denoted in the following diagram noting the progression from RDD to DataFrame to Dataset:



Source: From Webinar Apache Spark 1.5: What is the difference between a DataFrame and a RDD?
<http://bit.ly/29JPJSA>

The unification of the DataFrame and Dataset APIs has the potential of creating breaking changes to backwards compatibility. This was one of the main reasons Apache Spark 2.0 was a major release (as opposed to a 1.x minor release which would have minimized any breaking changes). As you can see from the following diagram, DataFrame and Dataset both belong to the new Dataset API introduced as part of Apache Spark 2.0:



Source: A Tale of Three Apache Spark APIs: RDDs, DataFrames, and Datasets <http://bit.ly/2accSNA>

As noted previously, the Dataset API provides a type-safe, object-oriented programming interface. Datasets can take advantage of the Catalyst optimizer by exposing expressions and data fields to the query planner and Project Tungsten's Fast In-memory encoding. But with DataFrame and Dataset now unified as part of Apache Spark 2.0, DataFrame is now an alias for the Dataset Untyped API. More specifically:

```
DataFrame = Dataset[Row]
```

Introducing SparkSession

In the past, you would potentially work with `SparkConf`, `SparkContext`, `SQLContext`, and `HiveContext` to execute your various Spark queries for configuration, Spark context, SQL context, and Hive context respectively. The `SparkSession` is essentially the combination of these contexts including `StreamingContext`.

For example, instead of writing:

```
df = sqlContext.read \
    .format('json').load('py/test/sql/people.json')
```

now you can write:

```
df = spark.read.format('json').load('py/test/sql/people.json')
```

or:

```
df = spark.read.json('py/test/sql/people.json')
```

The `SparkSession` is now the entry point for reading data, working with metadata, configuring the session, and managing the cluster resources.

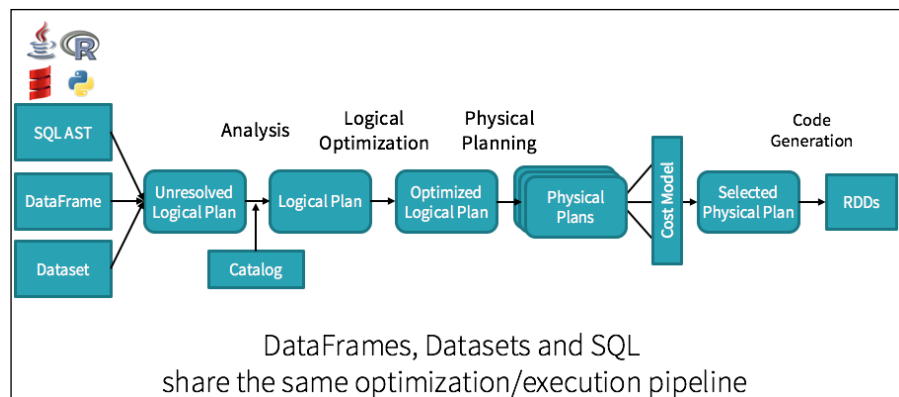
Tungsten phase 2

The fundamental observation of the computer hardware landscape when the project started was that, while there were improvements in *price per performance* in RAM memory, disk, and (to an extent) network interfaces, the *price per performance* advancements for CPUs were not the same. Though hardware manufacturers could put more cores in each socket (i.e. improve performance through parallelization), there were no significant improvements in the actual core speed.

Project Tungsten was introduced in 2015 to make significant changes to the Spark engine with the focus on improving performance. The first phase of these improvements focused on the following facets:

- **Memory Management and Binary Processing:** Leveraging application semantics to manage memory explicitly and eliminate the overhead of the JVM object model and garbage collection
- **Cache-aware computation:** Algorithms and data structures to exploit memory hierarchy
- **Code generation:** Using code generation to exploit modern compilers and CPUs

The following diagram is the updated Catalyst engine to denote the inclusion of Datasets. As you see at the right of the diagram (right of the Cost Model), **Code Generation** is used against the selected physical plans to generate the underlying RDDs:



Source: Structuring Spark: DataFrames, Datasets, and Streaming <http://bit.ly/2cJ508x>

As part of Tungsten Phase 2, there is the push into *whole-stage* code generation. That is, the Spark engine will now generate the byte code at compile time for the entire Spark stage instead of just for specific jobs or tasks. The primary facets surrounding these improvements include:

- **No virtual function dispatches:** This reduces multiple CPU calls that can have a profound impact on performance when dispatching billions of times
- **Intermediate data in memory vs CPU registers:** Tungsten Phase 2 places intermediate data into CPU registers. This is an order of magnitude reduction in the number of cycles to obtain data from the CPU registers instead of from memory
- **Loop unrolling and SIMD:** Optimize Apache Spark's execution engine to take advantage of modern compilers and CPUs' ability to efficiently compile and execute simple `for` loops (as opposed to complex function call graphs)

For a more in-depth review of Project Tungsten, please refer to:

- *Apache Spark Key Terms, Explained* <https://databricks.com/blog/2016/06/22/apache-spark-key-terms-explained.html>
- *Apache Spark as a Compiler: Joining a Billion Rows per Second on a Laptop* <https://databricks.com/blog/2016/05/23/apache-spark-as-a-compiler-joining-a-billion-rows-per-second-on-a-laptop.html>

- *Project Tungsten: Bringing Apache Spark Closer to Bare Metal* <https://databricks.com/blog/2015/04/28/project-tungsten-bringing-spark-closer-to-bare-metal.html>

Structured Streaming

As quoted by Reynold Xin during Spark Summit East 2016:

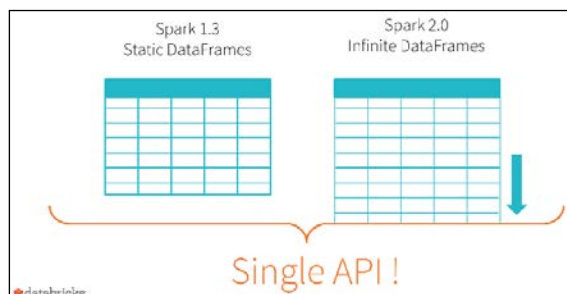
"The simplest way to perform streaming analytics is not having to reason about streaming."

This is the underlying foundation for building Structured Streaming. While streaming is powerful, one of the key issues is that streaming can be difficult to build and maintain. While companies such as Uber, Netflix, and Pinterest have Spark Streaming applications running in production, they also have dedicated teams to ensure the systems are highly available.



For a high-level overview of Spark Streaming, please review Spark Streaming: What Is It and Who's Using It? <http://bit.ly/1Qb10f6>

As implied previously, there are many things that can go wrong when operating Spark Streaming (and any streaming system for that matter) including (but not limited to) late events, partial outputs to the final data source, state recovery on failure, and/or distributed reads/writes:

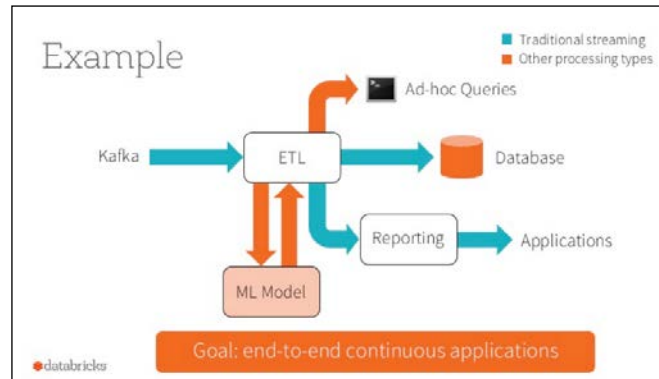


Source: A Deep Dive into Structured Streaming <http://bit.ly/2aHt1w0>

Therefore, to simplify Spark Streaming, there is now a single API that addresses both batch and streaming within the Apache Spark 2.0 release. More succinctly, the high-level streaming API is now built on top of the Apache Spark SQL Engine. It runs the same queries as you would with Datasets/DataFrames providing you with all the performance and optimization benefits as well as benefits such as event time, windowing, sessions, sources, and sinks.

Continuous applications

Altogether, Apache Spark 2.0 not only unified DataFrames and Datasets but also unified streaming, interactive, and batch queries. This opens a whole new set of use cases including the ability to aggregate data into a stream and then serving it using traditional JDBC/ODBC, to change queries at run time, and/or to build and apply ML models in for many scenario in a variety of latency use cases:



Source: Apache Spark Key Terms, Explained <https://databricks.com/blog/2016/06/22/apache-spark-key-terms-explained.html>.

Together, you can now build end-to-end **continuous applications**, in which you can issue the same queries to batch processing as to real-time data, perform ETL, generate reports, update or track specific data in the stream.



For more information on continuous applications, please refer to Matei Zaharia's blog post *Continuous Applications: Evolving Streaming in Apache Spark 2.0 - A foundation for end-to-end real-time applications* <http://bit.ly/2aJaSOr>.

Summary

In this chapter, we reviewed what is Apache Spark and provided a primer on Spark Jobs and APIs. We also provided a primer on Resilient Distributed Datasets (RDDs), DataFrames, and Datasets; we will dive further into RDDs and DataFrames in subsequent chapters. We also discussed how DataFrames can provide faster query performance in Apache Spark due to the Spark SQL Engine's Catalyst Optimizer and Project Tungsten. Finally, we also provided a high-level overview of the Spark 2.0 architecture including the Tungsten Phase 2, Structured Streaming, and Unifying DataFrames and Datasets.

In the next chapter, we will cover one of the fundamental data structures in Spark: The Resilient Distributed Datasets, or RDDs. We will show you how to create and modify these schema-less data structures using transformers and actions so your journey with PySpark can begin.

Before we do that, however, please, check the link <http://www.tomdrabas.com/site/book> for the Bonus Chapter 1 where we outline instructions on how to install Spark locally on your machine (unless you already have it installed). Here's a direct link to the manual: <https://www.packtpub.com/sites/default/files/downloads/InstallingSpark.pdf>.

2

Resilient Distributed Datasets

Resilient Distributed Datasets (RDDs) are a distributed collection of immutable JVM objects that allow you to perform calculations very quickly, and they are the *backbone* of Apache Spark.

As the name suggests, the dataset is distributed; it is split into chunks based on some key and distributed to executor nodes. Doing so allows for running calculations against such datasets very quickly. Also, as already mentioned in *Chapter 1, Understanding Spark*, RDDs keep track (log) of all the transformations applied to each chunk to speed up the computations and provide a fallback if things go wrong and that portion of the data is lost; in such cases, RDDs can recompute the data. This data lineage is another line of defense against data loss, a complement to data replication.

The following topics are covered in this chapter:

- Internal workings of an RDD
- Creating RDDs
- Global versus local scopes
- Transformations
- Actions

Internal workings of an RDD

RDDs operate in parallel. This is the strongest advantage of working in Spark: Each transformation is executed in parallel for enormous increase in speed.

The transformations to the dataset are lazy. This means that any transformation is only executed when an action on a dataset is called. This helps Spark to optimize the execution. For instance, consider the following very common steps that an analyst would normally do to get familiar with a dataset:

1. Count the occurrence of distinct values in a certain column.
2. Select those that start with an A.
3. Print the results to the screen.

As simple as the previously mentioned steps sound, if only items that start with the letter A are of interest, there is no point in counting distinct values for all the other items. Thus, instead of following the execution as outlined in the preceding points, Spark could only count the items that start with A, and then print the results to the screen.

Let's break this example down in code. First, we order Spark to map the values of A using the `.map(lambda v: (v, 1))` method, and then select those records that start with an 'A' (using the `.filter(lambda val: val.startswith('A'))` method). If we call the `.reduceByKey(operator.add)` method it will reduce the dataset and *add* (in this example, count) the number of occurrences of each key. All of these steps **transform** the dataset.

Second, we call the `.collect()` method to execute the steps. This step is an **action** on our dataset - it finally counts the distinct elements of the dataset. In effect, the action might reverse the order of transformations and filter the data first before mapping, resulting in a smaller dataset being passed to the reducer.



Do not worry if you do not understand the previous commands yet - we will explain them in detail later in this chapter.

Creating RDDs

There are two ways to create an RDD in PySpark: you can either

`.parallelize(...)` a collection (list or an array of some elements):

```
data = sc.parallelize(
    [('Amber', 22), ('Alfred', 23), ('Skye', 4), ('Albert', 12),
     ('Amber', 9)])
```

Or you can reference a file (or files) located either locally or somewhere externally:

```
data_from_file = sc.\
    textFile(
        '/Users/drabast/Documents/PySpark_Data/VS14MORT.txt.gz',
        4)
```



We downloaded the Mortality dataset `VS14MORT.txt` file from (accessed on July 31, 2016) ftp://ftp.cdc.gov/pub/Health_Statistics/NCHS/Datasets/DVS/mortality/mort2014us.zip; the record schema is explained in this document http://www.cdc.gov/nchs/data/dvs/Record_Layout_2014.pdf. We selected this dataset on purpose: The encoding of the records will help us to explain how to use UDFs to transform your data later in this chapter. For your convenience, we also host the file here: <http://tomdrabas.com/data/VS14MORT.txt.gz>

The last parameter in `sc.textFile(..., n)` specifies the number of partitions the dataset is divided into.



A rule of thumb would be to break your dataset into two-four partitions for each in your cluster.

Spark can read from a multitude of filesystems: Local ones such as NTFS, FAT, or Mac OS Extended (HFS+), or distributed filesystems such as HDFS, S3, Cassandra, among many others.



Be wary where your datasets are read from or saved to: The path cannot contain special characters `[]`. Note, that this also applies to paths stored on Amazon S3 or Microsoft Azure Data Storage.

Multiple data formats are supported: Text, parquet, JSON, Hive tables, and data from relational databases can be read using a JDBC driver. Note that Spark can automatically work with compressed datasets (like the Gzipped one in our preceding example).

Depending on how the data is read, the object holding it will be represented slightly differently. The data read from a file is represented as `MapPartitionsRDD` instead of `ParallelCollectionRDD` when we `.parallelize(...)` a collection.

Schema

RDDs are *schema-less* data structures (unlike DataFrames, which we will discuss in the next chapter). Thus, parallelizing a dataset, such as in the following code snippet, is perfectly fine with Spark when using RDDs:

```
data_heterogenous = sc.parallelize([
    ('Ferrari', 'fast'),
    {'Porsche': 100000},
    ['Spain', 'visited', 4504]
]).collect()
```

So, we can mix almost anything: a tuple, a dict, or a list and Spark will not complain.

Once you `.collect()` the dataset (that is, run an action to bring it back to the driver) you can access the data in the object as you would normally do in Python:

```
data_heterogenous[1]['Porsche']
```

It will produce the following:

```
100000
```

The `.collect()` method returns all the elements of the RDD to the driver where it is serialized as a list.



We will talk more about the caveats of using `.collect()` later in this chapter.

Reading from files

When you read from a text file, each row from the file forms an element of an RDD.

The `data_from_file.take(1)` command will produce the following (somewhat unreadable) output:

```
Out[7]: ['          1
2101 M1087 432311 4M4          2014U7CN
I64 238 070 24 0111I64
01 I64
01 11          100 601']
```

To make it more readable, let's create a list of elements so each line is represented as a list of values.

Lambda expressions

In this example, we will extract the useful information from the cryptic looking record of `data_from_file`.



Please refer to our GitHub repository for this book for the details of this method. Here, due to space constraints, we will only present an abbreviated version of the full method, especially where we create the Regex pattern. The code can be found here: https://github.com/drabastomek/learningPySpark/tree/master/Chapter03/LearningPySpark_Chapter03.ipynb.

First, let's define the method with the help of the following code, which will parse the unreadable row into something that we can use:

```
def extractInformation(row):
    import re
    import numpy as np
    selected_indices = [
        2,4,5,6,7,9,10,11,12,13,14,15,16,17,18,
        ...
        77,78,79,81,82,83,84,85,87,89
    ]
    record_split = re\
        .compile(
            r'([\s]{19}) ([0-9]{1}) ([\s]{40})
            ...
            ([\s]{33}) ([0-9\s]{3}) ([0-9\s]{1}) ([0-9\s]{1}) '
        )
    try:
        rs = np.array(record_split.split(row))[selected_indices]
    except:
        rs = np.array(['-99'] * len(selected_indices))
    return rs
```




A word of caution here is necessary. Defining pure Python methods can slow down your application as Spark needs to continuously switch back and forth between the Python interpreter and JVM. Whenever you can, you should use built-in Spark functions.

Next, we import the necessary modules: The `re` module as we will use regular expressions to parse the record, and `NumPy` for ease of selecting multiple elements at once.

Finally, we create a `Regex` object to extract the information as specified and parse the row through it.



We will not be delving into details here describing Regular Expressions. A good compendium on the topic can be found here <https://www.packtpub.com/application-development/mastering-python-regular-expressions>.

Once the record is parsed, we try to convert the list into a `NumPy` array and return it; if this fails we return a list of default values `-99` so we know this record did not parse properly.



We could implicitly filter out the malformed records by using `.flatMap(...)` and return an empty list `[]` instead of `-99` values. Check this for details: <http://stackoverflow.com/questions/34090624/remove-elements-from-spark-rdd>

Now, we will use the `extractInformation(...)` method to split and convert our dataset. Note that we pass only the method signature to `.map(...)`: the method will *hand over* one element of the RDD to the `extractInformation(...)` method at a time in each partition:

```
data_from_file_conv = data_from_file.map(extractInformation)
```

```
Out[4]: [array(['1', ' ', '2', '1', '01', 'M', '1', '087', ' ', '43', '23', '1  
1',  
        ' ', '4', 'M', '4', '2014', 'U', '7', 'C', 'N', ' ', ' ', 'I64  
,  
        '238', '070', ' ', '24', '01', '11I64', ' ', ' ',  
,  
,  
        ' ', ' ', ' ', ' ', ' ', ' ', ' ',  
,  
        ' ', ' ', ' ', ' ', ' ', ' ', ' ',  
,  
        'I64', ' ', ' ', ' ', ' ', ' ', ' ', ' ', '01',  
        ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', '01',  
,  
        ' ', '1', '1', '100', '6'],  
dtype='<U40')]
```

One of the things that you, as a prospective PySpark user, need to get used to is the inherent parallelism of Spark. Even if you are proficient in Python, executing scripts in PySpark requires shifting your thinking a bit.

However, taking the same code and deploying it to a cluster might cause a lot of head-scratching if you are not careful. This requires understanding how Spark executes a job on the cluster.

The driver then instructs the workers to execute their tasks and return the results to the driver when done. Before that happens, however, the driver prepares each task's closure: A set of variables and methods present on the driver for the worker to execute its task on the RDD.

This set of variables and methods is inherently *static* within the executors' context, that is, each executor gets a *copy* of the variables and methods from the driver. If, when running the task, the executor alters these variables or overwrites the methods, it does so **without** affecting either other executors' copies or the variables and methods of the driver. This might lead to some unexpected behavior and runtime bugs that can sometimes be really hard to track down.



Check out this discussion in PySpark's documentation for a more hands-on example: <http://spark.apache.org/docs/latest/programming-guide.html#local-vs-cluster-modes>.

Transformations

Transformations shape your dataset. These include mapping, filtering, joining, and transcoding the values in your dataset. In this section, we will showcase some of the transformations available on RDDs.



Due to space constraints we include only the most often used transformations and actions here. For a full set of methods available we suggest you check PySpark's documentation on RDDs <http://spark.apache.org/docs/latest/api/python/pyspark.html#pyspark.RDD>.

Since RDDs are schema-less, in this section we assume you know the schema of the produced dataset. If you cannot remember the positions of information in the parsed dataset we suggest you refer to the definition of the `extractInformation(...)` method on GitHub, code for Chapter 03.

The `.map(...)` transformation

It can be argued that you will use the `.map(...)` transformation most often. The method is applied to each element of the RDD: In the case of the `data_from_file_conv` dataset, you can think of this as a transformation of each row.

In this example, we will create a new dataset that will convert year of death into a numeric value:

```
data_2014 = data_from_file_conv.map(lambda row: int(row[16]))
```

Running `data_2014.take(10)` will yield the following result:

```
Out[11]: [2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014, -99]
```



If you are not familiar with lambda expressions, please refer to this resource: https://pythonconquerstheuniverse.wordpress.com/2011/08/29/lambda_tutorial/.

You can of course bring more columns over, but you would have to package them into a tuple, dict, or a list. Let's also include the 17th element of the row along so that we can confirm our `.map(...)` works as intended:

```
data_2014_2 = data_from_file_conv.map(
    lambda row: (row[16], int(row[16])):
data_2014_2.take(5)
```

The preceding code will produce the following result:

```
Out[12]: [('2014', 2014),
          ('2014', 2014),
          ('2014', 2014),
          ('2014', 2014),
          ('2014', 2014),
          ('2014', 2014),
          ('2014', 2014),
          ('2014', 2014),
          ('2014', 2014),
          ('-99', -99)]
```

The `.filter(...)` transformation

Another most often used transformation is the `.filter(...)` method, which allows you to select elements from your dataset that fit specified criteria. As an example, from the `data_from_file_conv` dataset, let's count how many people died in an accident in 2014:

```
data_filtered = data_from_file_conv.filter(
    lambda row: row[16] == '2014' and row[21] == '0')
data_filtered.count()
```



Note that the preceding command might take a while depending on how fast your computer is. For us, it took a little over two minutes to return a result.

The `.flatMap(...)` transformation

The `.flatMap(...)` method works similarly to `.map(...)`, but it returns a flattened result instead of a list. If we execute the following code:

```
data_2014_flat = data_from_file_conv.flatMap(lambda row: (row[16],
int(row[16]) + 1))
data_2014_flat.take(10)
```

It will yield the following output:

```
Out[14]: ['2014', 2015, '2014', 2015, '2014', 2015, '2014', 2015,
          '2014', 2015]
```

You can compare this result with the results of the command that generated `data_2014_2` previously. Note, also, as mentioned earlier, that the `.flatMap(...)` method can be used to filter out some malformed records when you need to parse your input. Under the hood, the `.flatMap(...)` method treats each row as a list and then simply *adds* all the records together; by passing an empty list the malformed records is dropped.

The `.distinct(...)` transformation

This method returns a list of distinct values in a specified column. It is extremely useful if you want to get to know your dataset or validate it. Let's check if the gender column contains only males and females; that would verify that we parsed the dataset properly. Let's run the following code:

```
distinct_gender = data_from_file_conv.map(
    lambda row: row[5]).distinct()
distinct_gender.collect()
```

This code will produce the following output:

```
Out[22]: ['-99', 'M', 'F']
```

First, we extract only the column that contains the gender. Next, we use the `.distinct()` method to select only the distinct values in the list. Lastly, we use the `.collect()` method to return the print of the values on the screen.



Note that this is an expensive method and should be used sparingly and only when necessary as it shuffles the data around.

The `.sample(...)` transformation

The `.sample(...)` method returns a randomized sample from the dataset. The first parameter specifies whether the sampling should be with a replacement, the second parameter defines the fraction of the data to return, and the third is seed to the pseudo-random numbers generator:

```
fraction = 0.1
data_sample = data_from_file_conv.sample(False, fraction, 666)
```

In this example, we selected a randomized sample of 10% from the original dataset. To confirm this, let's print the sizes of the datasets:

```
print('Original dataset: {0}, sample: {1}'\
      .format(data_from_file_conv.count(), data_sample.count()))
```

The preceding command produces the following output:

```
Original dataset: 2631171, sample: 263247
```

We use the `.count()` action that counts all the records in the corresponding RDDs.

The `.leftOuterJoin(...)` transformation

`.leftOuterJoin(...)`, just like in the SQL world, joins two RDDs based on the values found in both datasets, and returns records from the left RDD with records from the right one appended in places where the two RDDs match:

```
rdd1 = sc.parallelize([('a', 1), ('b', 4), ('c', 10)])
rdd2 = sc.parallelize([('a', 4), ('a', 1), ('b', '6'), ('d', 15)])
rdd3 = rdd1.leftOuterJoin(rdd2)
```

Running `.collect(...)` on the `rdd3` will produce the following:

```
Out[52]: [('c', (10, None)), ('b', (4, '6')), ('a', (1, 4)), ('a', (1, 1))]
```

 This is another expensive method and should be used sparingly and only when necessary as it shuffles the data around causing a performance hit.

What you can see here are all the elements from RDD `rdd1` and their corresponding values from RDD `rdd2`. As you can see, the value `'a'` shows up two times in `rdd3` and `'a'` appears twice in the RDD `rdd2`. The value `b` from the `rdd1` shows up only once and is joined with the value `'6'` from the `rdd2`. There are two things *missing*: Value `'c'` from `rdd1` does not have a corresponding key in the `rdd2` so the value in the returned tuple shows as `None`, and, since we were performing a left outer join, the value `'d'` from the `rdd2` disappeared as expected.

If we used the `.join(...)` method instead we would have got only the values for `'a'` and `'b'` as these two values intersect between these two RDDs. Run the following code:

```
rdd4 = rdd1.join(rdd2)
rdd4.collect()
```

It will result in the following output:

```
Out[48]: [('b', (4, '6')), ('a', (1, 4)), ('a', (1, 1))]
```

Another useful method is `.intersection(...)`, which returns the records that are equal in both RDDs. Execute the following code:

```
rdd5 = rdd1.intersection(rdd2)
rdd5.collect()
```

The output is as follows:

```
Out[88]: [('a', 1)]
```

The `.repartition(...)` transformation

Repartitioning the dataset changes the number of partitions that the dataset is divided into. This functionality should be used sparingly and only when really necessary as it shuffles the data around, which in effect results in a significant hit in terms of performance:

```
rdd1 = rdd1.repartition(4)
len(rdd1.glom().collect())
```

The preceding code prints out 4 as the new number of partitions.

The `.glom()` method, in contrast to `.collect()`, produces a list where each element is another list of all elements of the dataset present in a specified partition; the main list returned has as many elements as the number of partitions.

Actions

Actions, in contrast to transformations, execute the scheduled task on the dataset; once you have finished transforming your data you can execute your transformations. This might contain no transformations (for example, `.take(n)` will just return `n` records from an RDD even if you did not do any transformations to it) or execute the whole chain of transformations.

The `.take(...)` method

This is most arguably the most useful (and used, such as the `.map(...)` method). The method is preferred to `.collect(...)` as it only returns the `n` top rows from a single data partition in contrast to `.collect(...)`, which returns the whole RDD. This is especially important when you deal with large datasets:

```
data_first = data_from_file_conv.take(1)
```

If you want somewhat randomized records you can use `.takeSample(...)` instead, which takes three arguments: First whether the sampling should be with replacement, the second specifies the number of records to return, and the third is a seed to the pseudo-random numbers generator:

```
data_take_sampled = data_from_file_conv.takeSample(False, 1, 667)
```

The `.collect(...)` method

This method returns all the elements of the RDD to the driver. As we have just provided a caution about it, we will not repeat ourselves here.

The `.reduce(...)` method

The `.reduce(...)` method reduces the elements of an RDD using a specified method.

You can use it to sum the elements of your RDD:

```
rdd1.map(lambda row: row[1]).reduce(lambda x, y: x + y)
```

This will produce the sum of 15.

We first create a list of all the values of the `rdd1` using the `.map(...)` transformation, and then use the `.reduce(...)` method to process the results. The `reduce(...)` method, on each partition, runs the summation method (here expressed as a `lambda`) and returns the sum to the driver node where the final aggregation takes place.



A word of caution is necessary here. The functions passed as a reducer need to be **associative**, that is, when the order of elements is changed the result does not, and **commutative**, that is, changing the order of operands does not change the result either.

The example of the associativity rule is $(5 + 2) + 3 = 5 + (2 + 3)$, and of the commutative is $5 + 2 + 3 = 3 + 2 + 5$. Thus, you need to be careful about what functions you pass to the reducer.

If you ignore the preceding rule, you might run into trouble (assuming your code runs at all). For example, let's assume we have the following RDD (with one partition only!):

```
data_reduce = sc.parallelize([1, 2, .5, .1, 5, .2], 1)
```

If we were to reduce the data in a manner that we would like to divide the current result by the subsequent one, we would expect a value of 10:

```
works = data_reduce.reduce(lambda x, y: x / y)
```

However, if you were to partition the data into three partitions, the result will be wrong:

```
data_reduce = sc.parallelize([1, 2, .5, .1, 5, .2], 3)
```

```
data_reduce.reduce(lambda x, y: x / y)
```

It will produce 0.004.

The `.reduceByKey(...)` method works in a similar way to the `.reduce(...)` method, but it performs a reduction on a key-by-key basis:

```
data_key = sc.parallelize(
    [('a', 4), ('b', 3), ('c', 2), ('a', 8), ('d', 2), ('b', 1),
     ('d', 3)], 4)
data_key.reduceByKey(lambda x, y: x + y).collect()
```

The preceding code produces the following:

```
Out[122]: [('b', 4), ('c', 2), ('a', 12), ('d', 5)]
```

The .count(...) method

The `.count(...)` method counts the number of elements in the RDD. Use the following code:

```
data_reduce.count()
```

This code will produce 6, the exact number of elements in the `data_reduce` RDD.

The `.count(...)` method produces the same result as the following method, but it does not require moving the whole dataset to the driver:

```
len(data_reduce.collect()) # WRONG -- DON'T DO THIS!
```

If your dataset is in a key-value form, you can use the `.countByKey()` method to get the counts of distinct keys. Run the following code:

```
data_key.countByKey().items()
```

This code will produce the following output:

```
Out[132]: dict_items([('a', 2), ('b', 2), ('d', 2), ('c', 1)])
```

The .saveAsTextFile(...) method

As the name suggests, the `.saveAsTextFile(...)` the RDD and saves it to text files: Each partition to a separate file:

```
data_key.saveAsTextFile(
    '/Users/drabast/Documents/PySpark_Data/data_key.txt')
```

To read it back, you need to parse it back as all the rows are treated as strings:

```
def parseInput(row):
    import re
    pattern = re.compile(r'\\(\\'([a-z]))\\', ([0-9]))')
    row_split = pattern.split(row)
    return (row_split[1], int(row_split[2]))

data_key_reread = sc \
    .textFile(
        '/Users/drabast/Documents/PySpark_Data/data_key.txt') \
    .map(parseInput)
data_key_reread.collect()
```

The list of keys read matches what we had initially:

```
Out[159]: [('a', 4), ('b', 3), ('c', 2), ('a', 8), ('d', 2), ('b', 1), ('d', 3)]
```

The .foreach(...) method

This is a method that applies the same function to each element of the RDD in an iterative way; in contrast to `.map(...)`, the `.foreach(...)` method applies a defined function to each record in a one-by-one fashion. It is useful when you want to save the data to a database that is not natively supported by PySpark.

Here, we'll use it to print (to CLI - not the Jupyter Notebook) all the records that are stored in `data_key` RDD:

```
def f(x):
    print(x)

data_key.foreach(f)
```

If you now navigate to CLI you should see all the records printed out. Note, that every time the order will most likely be different.

Summary

RDDs are the backbone of Spark; these schema-less data structures are the most fundamental data structures that we will deal with within Spark.

In this chapter, we presented ways to create RDDs from text files, by means of the `.parallelize(...)` method as well as by reading data from text files. Also, some ways of processing unstructured data were shown.

Transformations in Spark are lazy - they are only applied when an action is called. In this chapter, we discussed and presented the most commonly used transformations and actions; the PySpark documentation contains many more <http://spark.apache.org/docs/latest/api/python/pyspark.html#pyspark.RDD>.

One major distinction between Scala and Python RDDs is speed: Python RDDs can be much slower than their Scala counterparts.

In the next chapter we will walk you through a data structure that made PySpark applications perform *on par* with those written in Scala - the DataFrames.

3

DataFrames

A `DataFrame` is an immutable distributed collection of data that is organized into named columns analogous to a table in a relational database. Introduced as an experimental feature within Apache Spark 1.0 as `SchemaRDD`, they were renamed to `DataFrames` as part of the Apache Spark 1.3 release. For readers who are familiar with Python `Pandas DataFrame` or R `DataFrame`, a Spark `DataFrame` is a similar concept in that it allows users to easily work with structured data (for example, data tables); there are some differences as well so please temper your expectations.

By imposing a structure onto a distributed collection of data, this allows Spark users to query structured data in Spark SQL or using expression methods (instead of lambdas). In this chapter, we will include code samples using both methods. By structuring your data, this allows the Apache Spark engine – specifically, the Catalyst Optimizer – to significantly improve the performance of Spark queries. In earlier APIs of Spark (that is, `RDDs`), executing queries in Python could be significantly slower due to communication overhead between the Java JVM and Py4J.



If you are familiar with working with `DataFrames` in previous versions of Spark (that is Spark 1.x), you will notice that in Spark 2.0 we are using `SparkSession` instead of `SQLContext`. The various Spark contexts: `HiveContext`, `SQLContext`, `StreamingContext`, and `SparkContext` have merged together in `SparkSession`. This way you will be working with this session only as an entry point for reading data, working with metadata, configuration, and cluster resource management.

For more information, please refer to *How to use SparkSession in Apache Spark 2.0*(<http://bit.ly/2br0Fr1>).

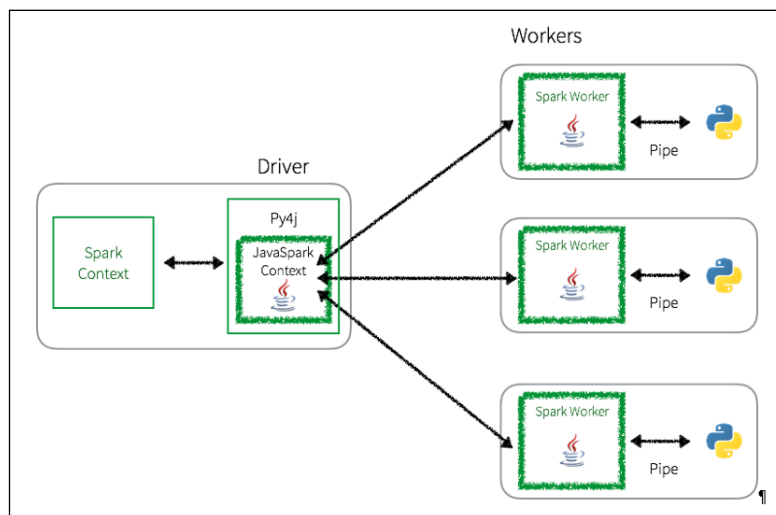
In this chapter, you will learn about the following:

- Python to RDD communications
- A quick refresh of Spark's Catalyst Optimizer
- Speeding up PySpark with DataFrames
- Creating DataFrames
- Simple DataFrame queries
- Interoperating with RDDs
- Querying with the DataFrame API
- Querying with Spark SQL
- Using DataFrames for an on-time flight performance

Python to RDD communications

Whenever a PySpark program is executed using RDDs, there is a potentially large overhead to execute the job. As noted in the following diagram, in the PySpark driver, the Spark Context uses Py4j to launch a JVM using the JavaSparkContext. Any RDD transformations are initially mapped to PythonRDD objects in Java.

Once these tasks are pushed out to the Spark Worker(s), PythonRDD objects launch Python subprocesses using pipes to send *both code and data* to be processed within Python:



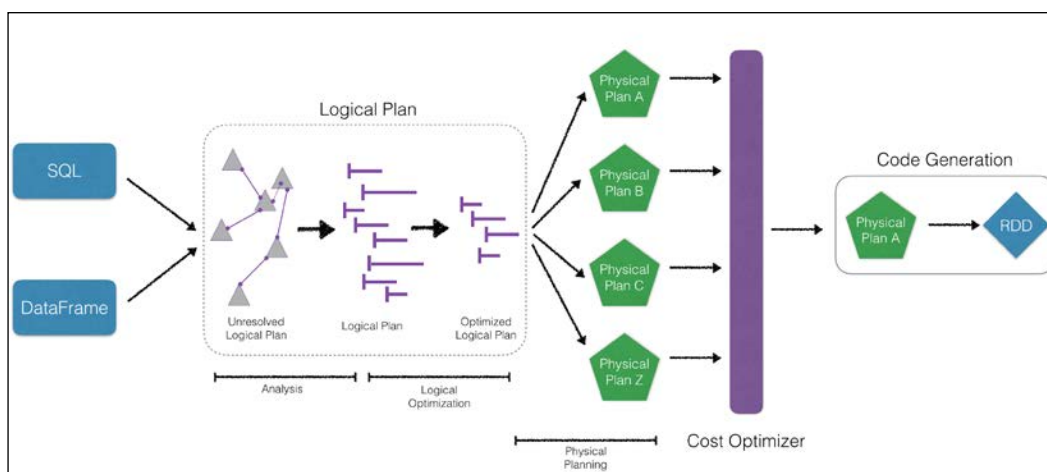
While this approach allows PySpark to distribute the processing of the data to multiple Python subprocesses on multiple workers, as you can see, there is a lot of context switching and communications overhead between Python and the JVM.



An excellent resource on PySpark performance is Holden Karau's *Improving PySpark Performance: Spark performance beyond the JVM*: <http://bit.ly/2bx89bn>.

Catalyst Optimizer refresh

As noted in *Chapter 1, Understanding Spark*, one of the primary reasons the Spark SQL engine is so fast is because of the **Catalyst Optimizer**. For readers with a database background, this diagram looks similar to the logical/physical planner and cost model/cost-based optimization of a **relational database management system (RDBMS)**:



The significance of this is that, as opposed to immediately processing the query, the Spark engine's Catalyst Optimizer compiles and optimizes a logical plan and has a cost optimizer that determines the most efficient physical plan generated.



As noted in earlier chapters, while the Spark SQL Engine has both rules-based and cost-based optimizations that include (but are not limited to) predicate push down and column pruning. Targeted for the Apache Spark 2.2 release, the jira item [SPARK-16026] *Cost-based Optimizer Framework* at <https://issues.apache.org/jira/browse/SPARK-16026> is an umbrella ticket to implement a cost-based optimizer framework beyond broadcast join selection. For more information, please refer to the *Design Specification of Spark Cost-Based Optimization* at <http://bit.ly/21i1t4T>.

As part of **Project Tungsten**, there are further improvements to performance by generating byte code (code generation or *codegen*) instead of interpreting each row of data. Find more details on Tungsten in the *Project Tungsten* section in *Chapter 1, Understanding Spark*.

As previously noted, the optimizer is based on functional programming constructs and was designed with two purposes in mind: to ease the adding of new optimization techniques and features to Spark SQL, and to allow external developers to extend the optimizer (for example, adding data-source-specific rules, support for new data types, and so on).



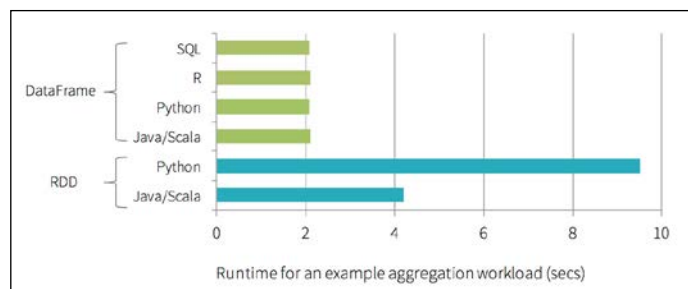
For more information, please refer to Michael Armbrust's excellent presentation, *Structuring Spark: SQL DataFrames, Datasets, and Streaming* at <http://bit.ly/2cJ508x>.

For further understanding of the *Catalyst Optimizer*, please refer to *Deep Dive into Spark SQL's Catalyst Optimizer* at <http://bit.ly/2bDVB1T>.

Also, for more information on *Project Tungsten*, please refer to *Project Tungsten: Bringing Apache Spark Closer to Bare Metal* at <http://bit.ly/2bQI1KY>, and *Apache Spark as a Compiler: Joining a Billion Rows per Second on a Laptop* at <http://bit.ly/2bDWtnc>.

Speeding up PySpark with DataFrames

The significance of DataFrames and the *Catalyst Optimizer* (and *Project Tungsten*) is the increase in performance of PySpark queries when compared to non-optimized RDD queries. As shown in the following figure, prior to the introduction of DataFrames, Python query speeds were often twice as slow as the same Scala queries using RDD. Typically, this slowdown in query performance was due to the communications overhead between Python and the JVM:



Source: *Introducing DataFrames in Apache-spark for Large Scale Data Science* at <http://bit.ly/2b1DBI1>

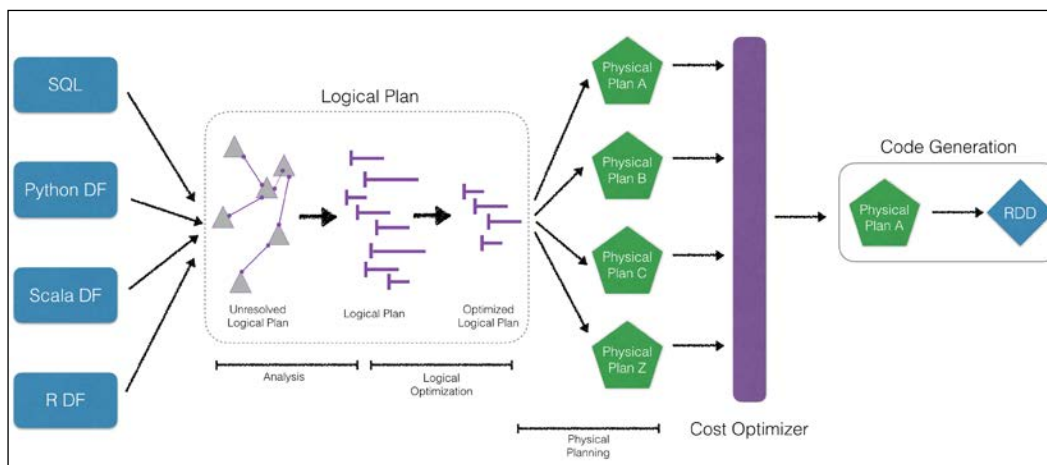
With DataFrames, not only was there a significant improvement in Python performance, there is now performance parity between Python, Scala, SQL, and R.



It is important to note that while, with DataFrames, PySpark is often significantly faster, there are some exceptions. The most prominent one is the use of Python UDFs, which results in round-trip communication between Python and the JVM. Note, this would be the worst-case scenario which would be similar if the compute was done on RDDs.

Python can take advantage of the performance optimizations in Spark even while the codebase for the Catalyst Optimizer is written in Scala. Basically, it is a Python wrapper of approximately 2,000 lines of code that allows PySpark DataFrame queries to be significantly faster.

Altogether, Python DataFrames (as well as SQL, Scala DataFrames, and R DataFrames) are all able to make use of the Catalyst Optimizer (as per the following updated diagram):





For more information, please refer to the blog post *Introducing DataFrames in Apache Spark for Large Scale Data Science* at <http://bit.ly/2b1DBI1>, as well as Reynold Xin's Spark Summit 2015 presentation, *From DataFrames to Tungsten: A Peek into Spark's Future* at <http://bit.ly/2bQN92T>.

Creating DataFrames

Typically, you will create DataFrames by importing data using `SparkSession` (or calling `spark` in the PySpark shell).



In Spark 1.x versions, you typically had to use `sqlContext`.

In future chapters, we will discuss how to import data into your local file system, **Hadoop Distributed File System (HDFS)**, or other cloud storage systems (for example, S3 or WASB). For this chapter, we will focus on generating your own DataFrame data directly within Spark or utilizing the data sources already available within Databricks Community Edition.



For instructions on how to sign up for the Community Edition of Databricks, see the bonus chapter, *Free Spark Cloud Offering*.

First, instead of accessing the file system, we will create a DataFrame by generating the data. In this case, we'll first create the `stringJSONRDD` RDD and then convert it into a DataFrame. This code snippet creates an RDD comprised of swimmers (their ID, name, age, and eye color) in JSON format.

Generating our own JSON data

Below, we will generate initially generate the `stringJSONRDD` RDD:

```
stringJSONRDD = sc.parallelize("""
    { "id": "123",
      "name": "Katie",
      "age": 19,
      "eyeColor": "brown"
    }""",
  """{
    "id": "234",
```

```

"name": "Michael",
"age": 22,
"eyeColor": "green"
}""",
"""{
"id": "345",
"name": "Simone",
"age": 23,
"eyeColor": "blue"
}""")
)

```

Now that we have created the RDD, we will convert this into a DataFrame by using the `SparkSession.read.json` method (that is, `spark.read.json(...)`). We will also create a temporary table by using the `.createOrReplaceTempView` method.



In Spark 1.x, this method was `.registerTempTable`, which is being deprecated as part of Spark 2.x.

Creating a DataFrame

Here is the code to create a DataFrame:

```
swimmersJSON = spark.read.json(stringJSONRDD)
```

Creating a temporary table

Here is the code for creating a temporary table:

```
swimmersJSON.createOrReplaceTempView("swimmersJSON")
```

As noted in the previous chapters, many RDD operations are transformations, which are not executed until an action operation is executed. For example, in the preceding code snippet, the `sc.parallelize` is a transformation that is executed when converting from an RDD to a DataFrame by using `spark.read.json`. Notice that, in the screenshot of this code snippet notebook (near the bottom left), the Spark job is not executed until the second cell containing the `spark.read.json` operation.



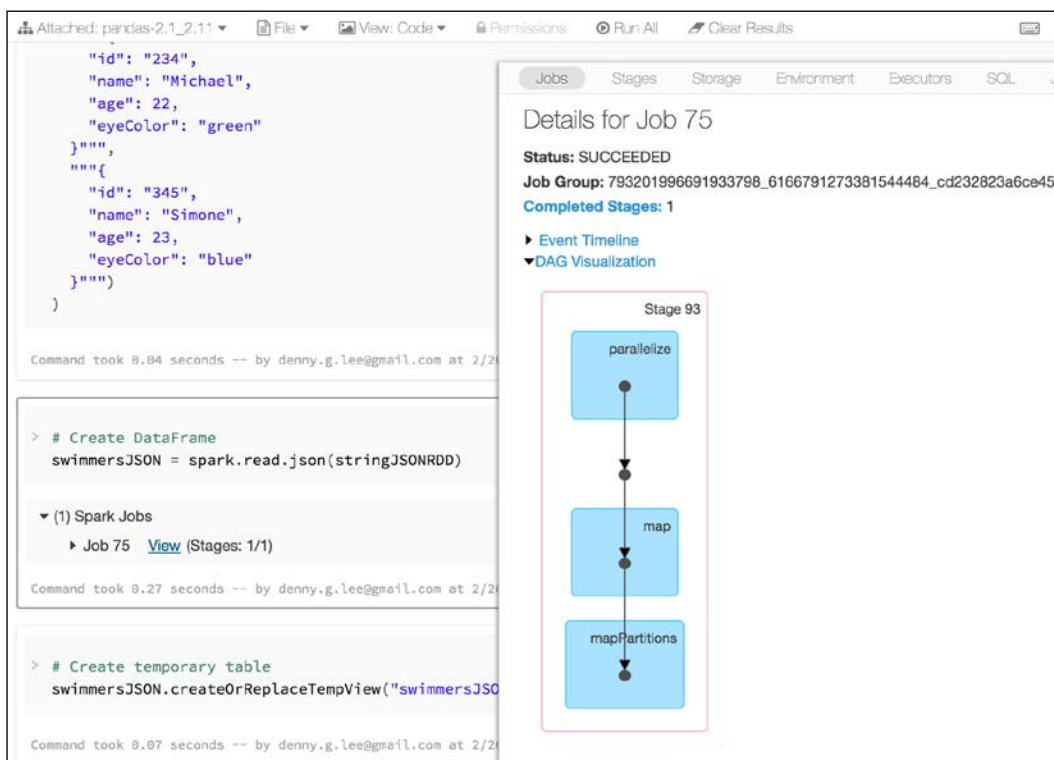
These are screenshots from Databricks Community Edition, but all the code samples and Spark UI screenshots can be executed/viewed in any flavor of Apache Spark 2.x.

To further emphasize the point, in the right pane of the following figure, we present the DAG graph of execution.



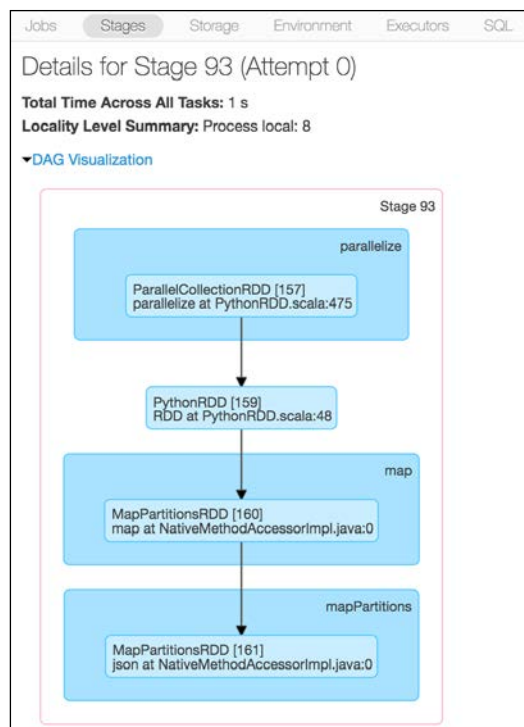
A great resource to better understand the Spark UI DAG visualization is the blog post *Understanding Your Apache Spark Application Through Visualization* at <http://bit.ly/2cSemkv>.

In the following screenshot, you can see the Spark job's `parallelize` operation is from the first cell generating the RDD `stringJSONRDD`, while the `map` and `mapPartitions` operations are the operations required to create the DataFrame:



Spark UI of the DAG visualization of the `spark.read.json(stringJSONRDD)` job.

In the following screenshot, you can see the *stages* for the `parallelize` operation are from the first cell generating the RDD `stringJSONRDD`, while the `map` and `mapPartitions` operations are the operations required to create the DataFrame:



Spark UI of the DAG visualization of the stages within the `spark.read.json(stringJSONRDD)` job.

It is important to note that `parallelize`, `map`, and `mapPartitions` are all RDD *transformations*. Wrapped within the DataFrame operation, `spark.read.json` (in this case), are not only the RDD transformations, but also the *action* which converts the RDD into a DataFrame. This is an important call out, because even though you are executing DataFrame *operations*, to debug your operations you will need to remember that you will be making sense of *RDD operations* within the Spark UI.

Note that creating the temporary table is a DataFrame transformation and not executed until a DataFrame action is executed (for example, in the SQL query to be executed in the following section).



DataFrame transformations and actions are similar to RDD transformations and actions in that there is a set of operations that are lazy (transformations). But, in comparison to RDDs, DataFrames operations are not as lazy, primarily due to the Catalyst Optimizer. For more information, please refer to Holden Karau and Rachel Warren's book *High Performance Spark*, <http://highperformancespark.com/>.

Simple DataFrame queries

Now that you have created the `swimmersJSON` DataFrame, we will be able to run the DataFrame API, as well as SQL queries against it. Let's start with a simple query showing all the rows within the DataFrame.

DataFrame API query

To do this using the DataFrame API, you can use the `show(<n>)` method, which prints the first `n` rows to the console:

 Running the `.show()` method will default to present the first 10 rows.

```
# DataFrame API
swimmersJSON.show()
```

This gives the following output:

```
► (2) Spark Jobs
+---+-----+---+-----+
|age|eyeColor| id|  name|
+---+-----+---+-----+
| 19|  brown|123|  Katie|
| 22|  green|234|Michael|
| 23|  blue|345| Simone|
+---+-----+---+-----+
Command took 0.22s
```

SQL query

If you prefer writing SQL statements, you can write the following query:

```
spark.sql("select * from swimmersJSON").collect()
```

This will give the following output:

```
► (1) Spark Jobs
Out[6]:
[Row(age=19, eyeColor=u'brown', id=u'123', name=u'Katie'),
 Row(age=22, eyeColor=u'green', id=u'234', name=u'Michael'),
 Row(age=23, eyeColor=u'blue', id=u'345', name=u'Simone')]
Command took 0.17s
```

We are using the `.collect()` method, which returns all the records as a list of **Row** objects. Note that you can use either the `collect()` or `show()` method for both DataFrames and SQL queries. Just make sure that if you use `.collect()`, this is for a small DataFrame, since it will return all of the rows in the DataFrame and move them back from the executors to the driver. You can instead use `take(<n>)` or `show(<n>)`, which allow you to limit the number of rows returned by specifying `<n>`:



Note that, if you are using Databricks, you can use the `%sql` command and run your SQL statement directly within a notebook cell, as noted.

```
> %sql
-- Query Data
select * from swimmersJSON
```

▶ (3) Spark Jobs

age	eyeColor	id	name
19	brown	123	Katie
22	green	234	Michael
23	blue	345	Simone

Command took 0.23 seconds -- by denny.g.lee@gmail.com at 2/20/2017, 10:18:30 AM on pandas-2.1.2.11

Interoperating with RDDs

There are two different methods for converting existing RDDs to DataFrames (or Datasets[T]): inferring the schema using reflection, or programmatically specifying the schema. The former allows you to write more concise code (when your Spark application already knows the schema), while the latter allows you to construct DataFrames when the columns and their data types are only revealed at run time. Note, **reflection** is in reference to *schema reflection* as opposed to Python *reflection*.

Inferring the schema using reflection

In the process of building the DataFrame and running the queries, we skipped over the fact that the schema for this DataFrame was automatically defined. Initially, row objects are constructed by passing a list of key/value pairs as `**kwargs` to the row class. Then, Spark SQL converts this RDD of row objects into a DataFrame, where the keys are the columns and the data types are inferred by sampling the data.

 The `**kwargs` construct allows you to pass a variable number of parameters to a method at runtime.

Going back to the code, after initially creating the `swimmersJSON` DataFrame, without specifying the schema, you will notice the schema definition by using the `printSchema()` method:

```
# Print the schema
swimmersJSON.printSchema()
```

This gives the following output:

```
root
 |-- age: long (nullable = true)
 |-- eyeColor: string (nullable = true)
 |-- id: string (nullable = true)
 |-- name: string (nullable = true)

Command took 0.07s
```

But what if we want to specify the schema because, in this example, we know that the `id` is actually a `long` instead of a `string`?

Programmatically specifying the schema

In this case, let's programmatically specify the schema by bringing in Spark SQL data types (`pyspark.sql.types`) and generate some `.csv` data for this example:

```
# Import types
from pyspark.sql.types import *

# Generate comma delimited data
stringCSVRDD = sc.parallelize([
    (123, 'Katie', 19, 'brown'),
    (234, 'Michael', 22, 'green'),
    (345, 'Simone', 23, 'blue')
])
```

First, we will encode the schema as a string, per the `[schema]` variable below. Then we will define the schema using `StructType` and `StructField`:

```
# Specify schema
schema = StructType([
    StructField("id", LongType(), True),
    StructField("name", StringType(), True),
    StructField("age", LongType(), True),
    StructField("eyeColor", StringType(), True)
])
```

Note, the `StructField` class is broken down in terms of:

- `name`: The name of this field
- `dataType`: The data type of this field
- `nullable`: Indicates whether values of this field can be null

Finally, we will apply the schema (`schema`) we created to the `stringCSRDD` RDD (that is, the generated `.csv` data) and create a temporary view so we can query it using SQL:

```
# Apply the schema to the RDD and Create DataFrame
swimmers = spark.createDataFrame(stringCSRDD, schema)

# Creates a temporary view using the DataFrame
swimmers.createOrReplaceTempView("swimmers")
```

With this example, we have finer-grain control over the schema and can specify that `id` is a long (as opposed to a string in the previous section):

```
swimmers.printSchema()
```

This gives the following output:

```
root
|-- id: long (nullable = true)
|-- name: string (nullable = true)
|-- age: long (nullable = true)
|-- eyeColor: string (nullable = true)

Command took 0.04s
```



In many cases, the schema can be inferred (as per the previous section) and you do not need to specify the schema, as in this preceding example.

Querying with the DataFrame API

As noted in the previous section, you can start off by using `collect()`, `show()`, or `take()` to view the data within your DataFrame (with the last two including the option to limit the number of returned rows).

Number of rows

To get the number of rows within your DataFrame, you can use the `count()` method:

```
swimmers.count()
```

This gives the following output:

```
Out[13]: 3
```

Running filter statements

To run a filter statement, you can use the `filter` clause; in the following code snippet, we are using the `select` clause to specify the columns to be returned as well:

```
# Get the id, age where age = 22
swimmers.select("id", "age").filter("age = 22").show()

# Another way to write the above query is below
swimmers.select(swimmers.id, swimmers.age).filter(swimmers.age == 22).
show()
```

The output of this query is to choose only the `id` and `age` columns, where `age = 22`:



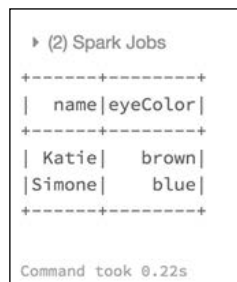
```
► (2) Spark Jobs
+----+----+
| id | age |
+----+----+
| 234 | 22  |
+----+----+

Command took 0.22s
```

If we only want to get back the name of the swimmers who have an eye color that begins with the letter `b`, we can use a SQL-like syntax, like, as shown in the following code:

```
# Get the name, eyeColor where eyeColor like 'b%'
swimmers.select("name", "eyeColor").filter("eyeColor like 'b%'").show()
```

The output is as follows:



```
► (2) Spark Jobs
+-----+-----+
| name | eyeColor |
+-----+-----+
| Katie | brown |
| Simone | blue |
+-----+-----+
Command took 0.22s
```

Querying with SQL

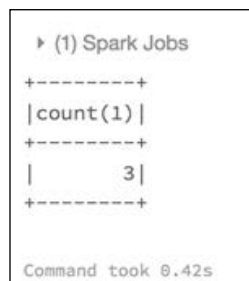
Let's run the same queries, except this time, we will do so using SQL queries against the same DataFrame. Recall that this DataFrame is accessible because we executed the `.createOrReplaceTempView` method for `swimmers`.

Number of rows

The following is the code snippet to get the number of rows within your DataFrame using SQL:

```
spark.sql("select count(1) from swimmers").show()
```

The output is as follows:



```
► (1) Spark Jobs
+-----+
| count(1) |
+-----+
| 3 |
+-----+
Command took 0.42s
```

Running filter statements using the where Clauses

To run a filter statement using SQL, you can use the `where` clause, as noted in the following code snippet:

```
# Get the id, age where age = 22 in SQL
spark.sql("select id, age from swimmers where age = 22").show()
```

The output of this query is to choose only the `id` and `age` columns where `age = 22`:

```
▶ (2) Spark Jobs
+----+----+
| id | age |
+----+----+
| 234 | 22 |
+----+----+
Command took 0.27s
```

As with the DataFrame API querying, if we want to get back the name of the swimmers who have an eye color that begins with the letter `b` only, we can use the `like` syntax as well:

```
spark.sql(
  "select name, eyeColor from swimmers where eyeColor like 'b%'"
).show()
```

The output is as follows:

```
▶ (2) Spark Jobs
+-----+-----+
| name | eyeColor |
+-----+-----+
| Katie | brown |
| Simone | blue |
+-----+-----+
Command took 0.27s
```



For more information, please refer to the *Spark SQL, DataFrames, and Datasets Guide* at <http://bit.ly/2cd1wyx>.



An important note when working with Spark SQL and DataFrames is that while it is easy to work with CSV, JSON, and a variety of data formats, the most common storage format for Spark SQL analytics queries is the *Parquet* file format. It is a columnar format that is supported by many other data processing systems and Spark SQL supports both reading and writing Parquet files that automatically preserves the schema of the original data. For more information, please refer to the latest *Spark SQL Programming Guide > Parquet Files* at: <http://spark.apache.org/docs/latest/sql-programming-guide.html#parquet-files>. Also, there are many performance optimizations that pertain to Parquet, including (but not limited to) *Automatic Partition Discovery and Schema Migration for Parquet* at <https://databricks.com/blog/2015/03/24/spark-sql-graduates-from-alpha-in-spark-1-3.html> and *How Apache Spark performs a fast count using the parquet metadata* at <https://github.com/dennyglee/databricks/blob/master/misc/parquet-count-metadata-explanation.md>.

DataFrame scenario – on-time flight performance

To showcase the types of queries you can do with DataFrames, let's look at the use case of on-time flight performance. We will analyze the *Airline On-Time Performance and Causes of Flight Delays: On-Time Data* (<http://bit.ly/2ccJPPM>), and join this with the airports dataset, obtained from the *Open Flights Airport, airline, and route data* (<http://bit.ly/2ccK5hw>), to better understand the variables associated with flight delays.



For this section, we will be using Databricks Community Edition (a free offering of the Databricks product), which you can get at <https://databricks.com/try-databricks>. We will be using visualizations and pre-loaded datasets within Databricks to make it easier for you to focus on writing the code and analyzing the results.

If you would prefer to run this on your own environment, you can find the datasets available in our GitHub repository for this book at <https://github.com/drabastomek/learningPySpark>.

Preparing the source datasets

We will first process the source airports and flight performance datasets by specifying their file path location and importing them using SparkSession:

```
# Set File Paths
flightPerfFilePath =
"/databricks-datasets/flights/departuredelays.csv"
airportsFilePath =
"/databricks-datasets/flights/airport-codes-na.txt"

# Obtain Airports dataset
airports = spark.read.csv(airportsFilePath, header='true',
inferSchema='true', sep='\t')
airports.createOrReplaceTempView("airports")

# Obtain Departure Delays dataset
flightPerf = spark.read.csv(flightPerfFilePath, header='true')
flightPerf.createOrReplaceTempView("FlightPerformance")

# Cache the Departure Delays dataset
flightPerf.cache()
```

Note that we're importing the data using the CSV reader (`com.databricks.spark.csv`), which works for any specified delimiter (note that the airports data is tab-delimited, while the flight performance data is comma-delimited). Finally, we cache the flight dataset so subsequent queries will be faster.

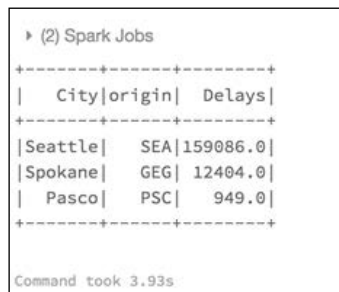
Joining flight performance and airports

One of the more common tasks with DataFrames/SQL is to join two different datasets; it is often one of the more demanding operations (from a performance perspective). With DataFrames, a lot of the performance optimizations for these joins are included by default:

```
# Query Sum of Flight Delays by City and Origin Code
# (for Washington State)
spark.sql("""
select a.City,
f.origin,
sum(f.delay) as Delays
from FlightPerformance f
join airports a
on a.IATA = f.origin
where a.State = 'WA'
""")
```

```
group by a.City, f.origin
order by sum(f.delay) desc"""
).show()
```

In our scenario, we are querying the total delays by city and origin code for the state of Washington. This will require joining the flight performance data with the airports data by **International Air Transport Association (IATA)** code. The output of the query is as follows:



▶ (2) Spark Jobs

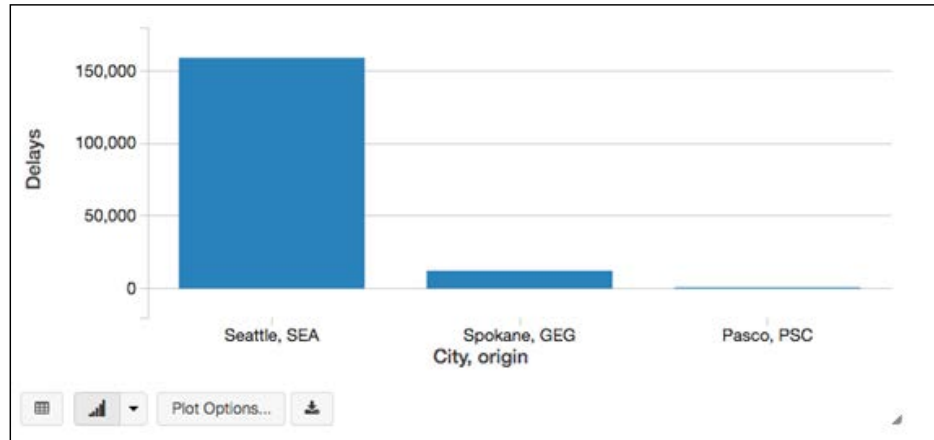
City	origin	Delays
Seattle	SEA	159086.0
Spokane	GEG	12404.0
Pasco	PSC	949.0

Command took 3.93s

Using notebooks (such as Databricks, iPython, Jupyter, and Apache Zeppelin), you can more easily execute and visualize your queries. In the following examples, we will be using the Databricks notebook. Within our Python notebook, we can use the `%sql` function to execute SQL statements within that notebook cell:

```
%sql
-- Query Sum of Flight Delays by City and Origin Code (for Washington
State)
select a.City, f.origin, sum(f.delay) as Delays
  from FlightPerformance f
    join airports a
      on a.IATA = f.origin
 where a.State = 'WA'
 group by a.City, f.origin
 order by sum(f.delay) desc
```

This is the same as the previous query, but due to formatting, easier to read. In our Databricks notebook example, we can quickly visualize this data into a bar chart:

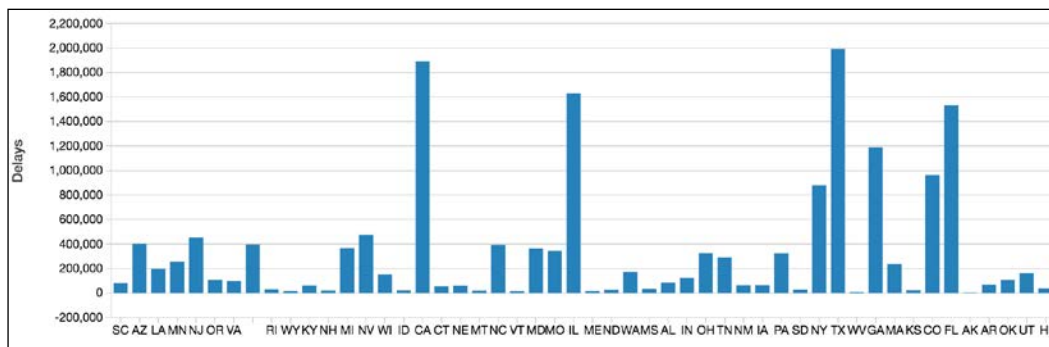


Visualizing our flight-performance data

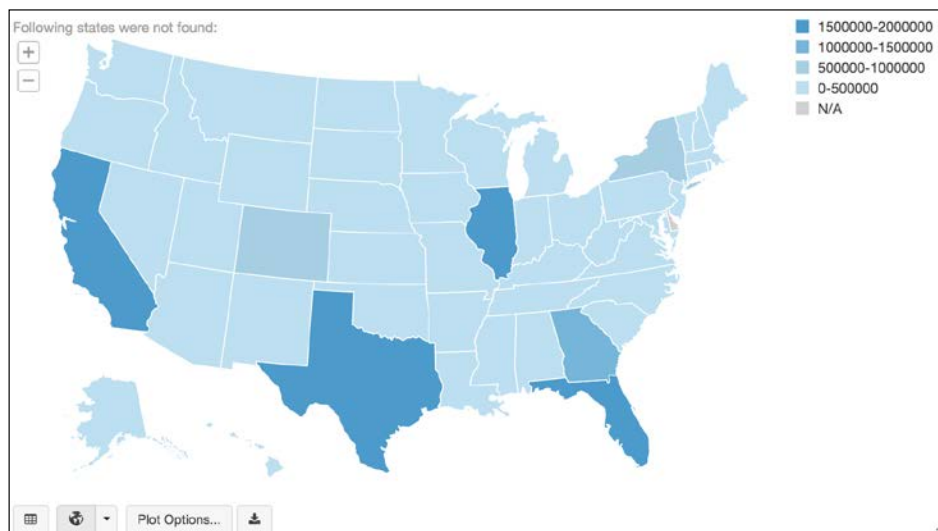
Let's continue visualizing our data, but broken down by all states in the continental US:

```
%sql
-- Query Sum of Flight Delays by State (for the US)
select a.State, sum(f.delay) as Delays
  from FlightPerformance f
    join airports a
      on a.IATA = f.origin
 where a.Country = 'USA'
 group by a.State
```

The output bar chart is as follows:



But, it would be cooler to view this data as a map; click on the bar chart icon at the bottom-left of the chart, and you can choose from many different native navigations, including a map:

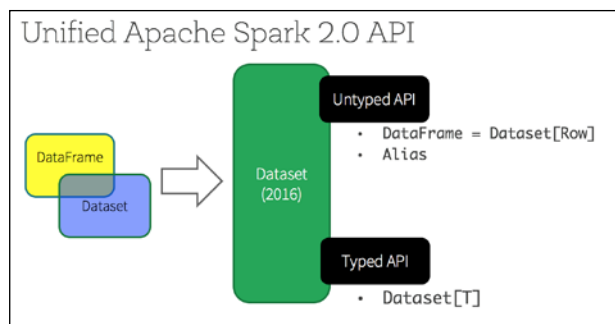


One of the key benefits of DataFrames is that the information is structured similar to a table. Therefore, whether you are using notebooks or your favorite BI tool, you will be able to quickly visualize your data.


 You can find the full list of `pyspark.sql.DataFrame` methods at <http://bit.ly/2bkUGnT>.
 You can find the full list of `pyspark.sql.functions` at <http://bit.ly/2bTAzLT>.

Spark Dataset API

After this discussion about Spark DataFrames, let's have a quick recap of the Spark Dataset API. Introduced in Apache Spark 1.6, the goal of Spark Datasets was to provide an API that allows users to easily express transformations on domain objects, while also providing the performance and benefits of the robust Spark SQL execution engine. As part of the Spark 2.0 release (and as noted in the diagram below), the DataFrame APIs is merged into the Dataset API thus unifying data processing capabilities across all libraries. Because of this unification, developers now have fewer concepts to learn or remember, and work with a single high-level and *type-safe* API – called Dataset:



Conceptually, the Spark DataFrame is an *alias* for a collection of generic objects `Dataset[Row]`, where a `Row` is a generic *untyped* JVM object. `Dataset`, by contrast, is a collection of *strongly-typed* JVM objects, dictated by a case class you define, in Scala or Java. This last point is particularly important as this means that the `Dataset` API is *not supported* by PySpark due to the lack of benefit from the type enhancements. Note, for the parts of the `Dataset` API that are not available in PySpark, they can be accessed by converting to an `RDD` or by using `UDFs`. For more information, please refer to the jira [SPARK-13233]: Python Dataset at <http://bit.ly/2dbfoFT>.

Summary

With Spark DataFrames, Python developers can make use of a simpler abstraction layer that is also potentially significantly faster. One of the main reasons Python is initially slower within Spark is due to the communication layer between Python sub-processes and the JVM. For Python DataFrame users, we have a Python wrapper around Scala DataFrames that avoids the Python sub-process/JVM communication overhead. Spark DataFrames has many performance enhancements through the Catalyst Optimizer and Project Tungsten which we have reviewed in this chapter. In this chapter, we also reviewed how to work with Spark DataFrames and worked on an on-time flight performance scenario using DataFrames.

In this chapter, we created and worked with DataFrames by generating the data or making use of existing datasets.

In the next chapter, we will discuss how to transform and understand your own data.

4

Prepare Data for Modeling

All data is dirty, irrespective of what the source of the data might lead you to believe: it might be your colleague, a telemetry system that monitors your environment, a dataset you download from the web, or some other source. Until you have tested and proven to yourself that your data is in a clean state (we will get to what clean state means in a second), you should neither trust it nor use it for modeling.

Your data can be stained with duplicates, missing observations and outliers, non-existent addresses, wrong phone numbers and area codes, inaccurate geographical coordinates, wrong dates, incorrect labels, mixtures of upper and lower cases, trailing spaces, and many other more subtle problems. It is your job to clean it, irrespective of whether you are a data scientist or data engineer, so you can build a statistical or machine learning model.

Your dataset is considered technically clean if none of the aforementioned problems can be found. However, to clean the dataset for modeling purposes, you also need to check the distributions of your features and confirm they fit the predefined criteria.

As a data scientist, you can expect to spend 80-90% of your time *massaging* your data and getting familiar with all the features. This chapter will guide you through that process, leveraging Spark capabilities.

In this chapter, you will learn how to do the following:

- Recognize and handle duplicates, missing observations, and outliers
- Calculate descriptive statistics and correlations
- Visualize your data with matplotlib and Bokeh

Checking for duplicates, missing observations, and outliers

Until you have fully tested the data and proven it worthy of your time, you should neither trust it nor use it. In this section, we will show you how to deal with duplicates, missing observations, and outliers.

Duplicates

Duplicates are observations that appear as distinct rows in your dataset, but which, upon closer inspection, look the same. That is, if you looked at them side by side, all the features in these two (or more) rows would have exactly the same values.

On the other hand, if your data has some form of an ID to distinguish between records (or associate them with certain users, for example), then what might initially appear as a duplicate may not be; sometimes systems fail and produce erroneous IDs. In such a situation, you need to either check whether the same ID is a real duplicate, or you need to come up with a new ID system.

Consider the following example:

```
df = spark.createDataFrame([
    (1, 144.5, 5.9, 33, 'M'),
    (2, 167.2, 5.4, 45, 'M'),
    (3, 124.1, 5.2, 23, 'F'),
    (4, 144.5, 5.9, 33, 'M'),
    (5, 133.2, 5.7, 54, 'F'),
    (3, 124.1, 5.2, 23, 'F'),
    (5, 129.2, 5.3, 42, 'M'),
], ['id', 'weight', 'height', 'age', 'gender'])
```

As you can see, we have several issues here:

- We have two rows with IDs equal to 3 and they are exactly the same
- Rows with IDs 1 and 4 are the same — the only thing that's different is their IDs, so we can safely assume that they are the same person
- We have two rows with IDs equal to 5, but that seems to be a recording issue, as they do not seem to be the same person

This is a very easy dataset with only seven rows. What do you do when you have millions of observations? The first thing I normally do is to check if I have any duplicates: I compare the counts of the full dataset with the one that I get after running a `.distinct()` method:

```
print('Count of rows: {}'.format(df.count()))
print('Count of distinct rows: {}'.format(df.distinct().count()))
```

Here's what you get back for our DataFrame:

```
Count of rows: 7
Count of distinct rows: 6
```

If these two numbers differ, then you know you have, what I like to call, pure duplicates: rows that are exact copies of each other. We can drop these rows by using the `.dropDuplicates(...)` method:

```
df = df.dropDuplicates()
```

Our dataset will then look as follows (once you run `df.show()`):

id	weight	height	age	gender
4	144.5	5.9	33	M
1	144.5	5.9	33	M
5	129.2	5.3	42	M
5	133.2	5.7	54	F
2	167.2	5.4	45	M
3	124.1	5.2	23	F

We dropped one of the rows with ID 3. Now let's check whether there are any duplicates in the data irrespective of ID. We can quickly repeat what we have done earlier, but using only columns other than the ID column:

```
print('Count of ids: {}'.format(df.count()))
print('Count of distinct ids: {}'.format(
    df.select([
        c for c in df.columns if c != 'id'
    ]).distinct().count()))
)
```

We should see one more row that is a duplicate:

```
Count of ids: 6
Count of distinct ids: 5
```

We can still use the `.dropDuplicates(...)`, but will add the `subset` parameter that specifies only the columns other than the `id` column:

```
df = df.dropDuplicates(subset=[
    c for c in df.columns if c != 'id'
])
```

The `subset` parameter instructs the `.dropDuplicates(...)` method to look for duplicated rows using only the columns specified via the `subset` parameter; in the preceding example, we will drop the duplicated records with the same weight, height, age, and gender but not `id`. Running the `df.show()`, we get the following cleaner dataset as we dropped the row with `id = 1` since it was identical to the record with `id = 4`:

id	weight	height	age	gender
5	133.2	5.7	54	F
4	144.5	5.9	33	M
2	167.2	5.4	45	M
3	124.1	5.2	23	F
5	129.2	5.3	42	M

Now that we know there are no full rows duplicated, or any identical rows differing only by ID, let's check if there are any duplicated IDs. To calculate the total and distinct number of IDs in one step, we can use the `.agg(...)` method:

```
import pyspark.sql.functions as fn

df.agg(
    fn.count('id').alias('count'),
    fn.countDistinct('id').alias('distinct')
).show()
```

Here's the output of the preceding code:

count	distinct
5	4

In the previous example, we first import all the functions from the `pyspark.sql` module.



This gives us access to a vast array of various functions, too many to list here. However, we strongly encourage you to study the PySpark's documentation at <http://spark.apache.org/docs/2.0.0/api/python/pyspark.sql.html#module-pyspark.sql.functions>.

Next, we use the `.count(...)` and `.countDistinct(...)` to, respectively, calculate the number of rows and the number of distinct `ids` in our `DataFrame`. The `.alias(...)` method allows us to specify a friendly name to the returned column.

As you can see, we have five rows in total, but only four distinct IDs. Since we have already dropped all the duplicates, we can safely assume that this might just be a fluke in our ID data, so we will give each row a unique ID:

```
df.withColumn('new_id', fn.monotonically_increasing_id()).show()
```

The preceding code snippet produced the following output:

id	weight	height	age	gender	new_id
5	133.2	5.7	54	F	25769803776
4	144.5	5.9	33	M	171798691840
2	167.2	5.4	45	M	592705486848
3	124.1	5.2	23	F	1236950581248
5	129.2	5.3	42	M	1365799600128

The `.monotonicallymonotonically_increasing_id()` method gives each record a unique and increasing ID. According to the documentation, as long as your data is put into less than roughly 1 billion partitions with less than 8 billions records in each, the ID is guaranteed to be unique.



A word of caution: in earlier versions of Spark the `.monotonicallymonotonically_increasing_id()` method would not necessarily return the same IDs across multiple evaluations of the same `DataFrame`. This, however, has been fixed in Spark 2.0.

Missing observations

You will frequently encounter datasets with *blanks* in them. The missing values can happen for a variety of reasons: systems failure, people error, data schema changes, just to name a few.

The simplest way to deal with missing values, if your data can afford it, is to drop the whole observation when any missing value is found. You have to be careful not to drop too many: depending on the distribution of the missing values across your dataset it might severely affect the usability of your dataset. If, after dropping the rows, I end up with a very small dataset, or find that the reduction in data size is more than 50%, I start checking my data to see what features have the most holes in them and perhaps exclude those altogether; if a feature has most of its values missing (unless a missing value bears a meaning), from a modeling point of view, it is fairly useless.

The other way to deal with the observations with missing values is to impute some value in place of those *Nones*. Given the type of your data, you have several options to choose from:

- If your data is a discrete Boolean, you can turn it into a categorical variable by adding a third category — *Missing*
- If your data is already categorical, you can simply extend the number of levels and add the *Missing* category as well
- If you're dealing with ordinal or numerical data, you can impute either mean, median, or some other predefined value (for example, first or third quartile, depending on the distribution shape of your data)

Consider a similar example to the one we presented previously:

```
df_miss = spark.createDataFrame([
    (1, 143.5, 5.6, 28, 'M', 100000),
    (2, 167.2, 5.4, 45, 'M', None),
    (3, None, 5.2, None, None, None),
    (4, 144.5, 5.9, 33, 'M', None),
    (5, 133.2, 5.7, 54, 'F', None),
    (6, 124.1, 5.2, None, 'F', None),
    (7, 129.2, 5.3, 42, 'M', 76000),
], ['id', 'weight', 'height', 'age', 'gender', 'income'])
```

In our example, we deal with a number of missing values categories.

Analyzing *rows*, we see the following:

- The row with ID 3 has only one useful piece of information—the height
- The row with ID 6 has only one missing value—the age

Analyzing *columns*, we can see the following:

- The income column, since it is a very personal thing to disclose, has most of its values missing
- The weight and gender columns have only one missing value each
- The age column has two missing values

To find the number of missing observations per row, we can use the following snippet:

```
df_miss.rdd.map(
    lambda row: (row['id'], sum([c == None for c in row]))
).collect()
```

It produces the following output:

```
Out[9]: [(1, 0), (2, 1), (3, 4), (4, 1), (5, 1), (6, 2), (7, 0)]
```

It tells us that, for example, the row with ID 3 has four missing observations, as we observed earlier.

Let's see what values are missing so that when we count missing observations in columns, we can decide whether to drop the observation altogether or impute some of the observations:

```
df_miss.where('id == 3').show()
```

Here's what we get:

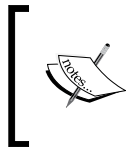
```
+---+-----+-----+-----+-----+-----+
| id|weight|height| age|gender|income|
+---+-----+-----+-----+-----+-----+
|  3|  null|   5.2|null|  null|  null|
+---+-----+-----+-----+-----+-----+
```

Let's now check what percentage of missing observations are there in each column:

```
df_miss.agg([
    (1 - (fn.count(c) / fn.count('*'))).alias(c + '_missing')
    for c in df_miss.columns
]).show()
```


This generates the following output:

id_missing	weight_missing	height_missing	age_missing	gender_missing	income_missing
0.0	0.1428571428571429	0.0	0.2857142857142857	0.1428571428571429	0.7142857142857143



The `*` argument to the `.count(...)` method (in place of a column name) instructs the method to count all rows. On the other hand, the `*` preceding the list declaration instructs the `.agg(...)` method to treat the list as a set of separate parameters passed to the function.

So, we have 14% of missing observations in the `weight` and `gender` columns, twice as much in the `height` column, and almost 72% of missing observations in the `income` column. Now we know what to do.

First, we will drop the `'income'` feature, as most of its values are missing.

```
df_miss_no_income = df_miss.select([
    c for c in df_miss.columns if c != 'income'
])
```

We now see that we do not need to drop the row with ID 3 as the coverage in the `'weight'` and `'age'` columns has enough observations (in our simplified example) to calculate the mean and impute it in the place of the missing values.

However, if you decide to drop the observations instead, you can use the `.dropna(...)` method, as shown here. Here, we will also use the `thresh` parameter, which allows us to specify a threshold on the number of missing observations per row that would qualify the row to be dropped. This is useful if you have a dataset with tens or hundreds of features and you only want to drop those rows that exceed a certain threshold of missing values:

```
df_miss_no_income.dropna(thresh=3).show()
```

The preceding code produces the following output:

id	weight	height	age	gender
1	143.5	5.6	28	M
2	167.2	5.4	45	M
4	144.5	5.9	33	M
5	133.2	5.7	54	F
6	124.1	5.2	null	F
7	129.2	5.3	42	M

On the other hand, if you wanted to impute the observations, you can use the `.fillna(...)` method. This method accepts a single integer (long is also accepted), float, or string; all missing values in the whole dataset will then be filled in with that value. You can also pass a dictionary of a form `{ '<colName>': <value_to_impute> }`. This has the same limitation, in that, as the `<value_to_impute>`, you can only pass an integer, float, or string.

If you want to impute a mean, median, or other calculated value, you need to first calculate the value, create a dictionary with such values, and then pass it to the `.fillna(...)` method.

Here's how we do it:

```
means = df_miss_no_income.agg(
    *[fn.mean(c).alias(c)
      for c in df_miss_no_income.columns if c != 'gender']
).toPandas().to_dict('records')[0]

means['gender'] = 'missing'

df_miss_no_income.fillna(means).show()
```

The preceding code will produce the following output:

id	weight	height	age	gender
1	143.5	5.6	28	M
2	167.2	5.4	45	M
3	140.2833333333	5.2	40	missing
4	144.5	5.9	33	M
5	133.2	5.7	54	F
6	124.1	5.2	40	F
7	129.2	5.3	42	M

We omit the gender column as one cannot calculate a mean of a categorical variable, obviously.

We use a double conversion here. Taking the output of the `.agg(...)` method (a PySpark DataFrame), we first convert it into a pandas' DataFrame and then once more to a dictionary.



Note that calling the `.toPandas()` can be problematic, as the method works essentially in the same way as `.collect()` in RDDs. It collects all the information from the workers and brings it over to the driver. It is unlikely to be a problem with the preceding dataset, unless you have thousands upon thousands of features.

The `records` parameter to the `.to_dict(...)` method of `pandas` instructs it to create the following dictionary:

```
{'age': 40.399999999999999,
 'height': 5.4714285714285706,
 'id': 4.0,
 'weight': 140.28333333333333}
```

Since we cannot calculate the average (or any other numeric metric of a categorical variable), we added the `missing` category to the dictionary for the `gender` feature. Note that, even though the mean of the `age` column is 40.40, when imputed, the type of the `df_miss_no_income.age` column was preserved—it is still an integer.

Outliers

Outliers are those observations that deviate significantly from the distribution of the rest of your sample. The definitions of *significance* vary, but in the most general form, you can accept that there are no outliers if all the values are roughly within the $Q1-1.5IQR$ and $Q3+1.5IQR$ range, where `IQR` is the interquartile range; the `IQR` is defined as a difference between the upper- and lower-quartiles, that is, the 75th percentile (the `Q3`) and 25th percentile (the `Q1`), respectively.

Let's, again, consider a simple example:

```
df_outliers = spark.createDataFrame([
    (1, 143.5, 5.3, 28),
    (2, 154.2, 5.5, 45),
    (3, 342.3, 5.1, 99),
    (4, 144.5, 5.5, 33),
    (5, 133.2, 5.4, 54),
    (6, 124.1, 5.1, 21),
    (7, 129.2, 5.3, 42),
], ['id', 'weight', 'height', 'age'])
```

Now we can use the definition we outlined previously to flag the outliers.

First, we calculate the lower and upper cut off points for each feature. We will use the `.approxQuantile(...)` method. The first parameter specified is the name of the column, the second parameter can be either a number between 0 or 1 (where 0.5 means to calculate median) or a list (as in our case), and the third parameter specifies the acceptable level of an error for each metric (if set to 0, it will calculate an exact value for the metric, but it can be really expensive to do so):

```
cols = ['weight', 'height', 'age']
bounds = {}

for col in cols:
    quantiles = df_outliers.approxQuantile(
        col, [0.25, 0.75], 0.05
    )

    IQR = quantiles[1] - quantiles[0]

    bounds[col] = [
        quantiles[0] - 1.5 * IQR,
        quantiles[1] + 1.5 * IQR
    ]
```

The bounds dictionary holds the lower and upper bounds for each feature:

```
Out[17]: {'age': [9.0, 51.0],
          'height': [4.8999999999999995, 5.6],
          'weight': [115.0, 146.84999999999997]}
```

Let's now use it to flag our outliers:

```
outliers = df_outliers.select(*['id'] + [
    (
        (df_outliers[c] < bounds[c][0]) |
        (df_outliers[c] > bounds[c][1])
    ).alias(c + '_o') for c in cols
])
outliers.show()
```

The preceding code produces the following output:

id	weight_o	height_o	age_o
1	false	false	false
2	true	false	false
3	true	false	true
4	false	false	false
5	false	false	true
6	false	false	false
7	false	false	false

We have two outliers in the weight feature and two in the age feature. By now you should know how to extract these, but here is a snippet that lists the values significantly differing from the rest of the distribution:

```
df_outliers = df_outliers.join(outliers, on='id')
df_outliers.filter('weight_o').select('id', 'weight').show()
df_outliers.filter('age_o').select('id', 'age').show()
```

The preceding code will give you the following output:

id	weight
3	342.3
2	154.2

id	age
5	54
3	99

Equipped with the methods described in this section, you can quickly clean up even the biggest of datasets.

Getting familiar with your data

Although we would strongly discourage such behavior, you can build a model without knowing your data; it will most likely take you longer, and the quality of the resulting model might be less than optimal, but it is doable.



In this section, we will use the dataset we downloaded from <http://packages.revolutionanalytics.com/datasets/ccFraud.csv>. We did not alter the dataset itself, but it was GZipped and uploaded to <http://tomdrabas.com/data/LearningPySpark/ccFraud.csv.gz>. Please download the file first and save it in the same folder that contains your notebook for this chapter.

The head of the dataset looks as follows:

```
"custID","gender","state","cardholder","balance","numTrans","numIntlTrans","creditLine","fraudRisk"
1,1,35,1,3000,4,14,2,0
2,2,2,1,0,9,0,18,0
3,2,2,1,0,27,9,16,0
4,1,15,1,0,12,0,5,0
5,1,46,1,0,11,16,7,0
```

Thus, any serious data scientist or data modeler will become acquainted with the dataset before starting any modeling. As a first thing, we normally start with some descriptive statistics to get a feeling for what we are dealing with.

Descriptive statistics

Descriptive statistics, in the simplest sense, will tell you the basic information about your dataset: how many non-missing observations there are in your dataset, the mean and the standard deviation for the column, as well as the min and max values.

However, first things first—let's load our data and convert it to a Spark DataFrame:

```
import pyspark.sql.types as typ
```

First, we load the only module we will need. The `pyspark.sql.types` exposes all the data types we can use, such as `IntegerType()` or `FloatType()`.



For a full list of available types check <http://spark.apache.org/docs/latest/api/python/pyspark.sql.html#module-pyspark.sql.types>.

Next, we read the data in and remove the header line using the `.filter(...)` method. This is followed by splitting the row on each comma (since this is a `.csv` file) and converting each element to an integer:

```
fraud = sc.textFile('ccFraud.csv.gz')
header = fraud.first()

fraud = fraud \
    .filter(lambda row: row != header) \
    .map(lambda row: [int(elem) for elem in row.split(',')])
```

Next, we create the schema for our DataFrame:

```
fields = [
    *[
        typ.StructField(h[1:-1], typ.IntegerType(), True)
        for h in header.split(',')
    ]
]
schema = typ.StructType(fields)
```

Finally, we create our DataFrame:

```
fraud_df = spark.createDataFrame(fraud, schema)
```

Having created our `fraud_df` DataFrame, we can calculate the basic descriptive statistics for our dataset. However, you need to remember that even though all of our features appear as numeric in nature, some of them are categorical (for example, gender or state).

Here's the schema of our DataFrame:

```
fraud_df.printSchema()
```

The representation is shown here:

```
root
|-- custID: integer (nullable = true)
|-- gender: integer (nullable = true)
|-- state: integer (nullable = true)
|-- cardholder: integer (nullable = true)
|-- balance: integer (nullable = true)
|-- numTrans: integer (nullable = true)
|-- numIntlTrans: integer (nullable = true)
|-- creditLine: integer (nullable = true)
|-- fraudRisk: integer (nullable = true)
```

Also, no information would be gained from calculating the mean and standard deviation of the `custId` column, so we will not be doing that.

For a better understanding of categorical columns, we will count the frequencies of their values using the `.groupby(...)` method. In this example, we will count the frequencies of the `gender` column:

```
fraud_df.groupby('gender').count().show()
```

The preceding code will produce the following output:

gender	count
1	6178231
2	3821769

As you can see, we are dealing with a fairly imbalanced dataset. What you would expect to see is an equal distribution for both genders.



It goes beyond the scope of this chapter, but if we were building a statistical model, you would need to take care of these kinds of biases. You can read more at http://www.va.gov/VETDATA/docs/SurveysAndStudies/SAMPLE_WEIGHT.pdf.

For the truly numerical features, we can use the `.describe()` method:

```
numerical = ['balance', 'numTrans', 'numIntlTrans']
desc = fraud_df.describe(numerical)
desc.show()
```

The `.show()` method will produce the following output:

summary	balance	numTrans	numIntlTrans
count	10000000	10000000	10000000
mean	4109.9199193	28.9351871	4.0471899
stddev	3996.847309737077	26.553781024522852	8.602970115863767
min	0	0	0
max	41485	100	60

Even from these relatively few numbers we can tell quite a bit:

- All of the features are positively skewed. The maximum values are a number of times larger than the average.
- The coefficient of variation (the ratio of mean to standard deviation) is very high (close or greater than 1), suggesting a wide spread of observations.

Here's how you check the skewness (we will do it for the 'balance' feature only):

```
fraud_df.agg({'balance': 'skewness'}).show()
```

The preceding code produces the following output:

```
+-----+
| skewness(balance) |
+-----+
| 1.1818315552995033 |
+-----+
```

A list of aggregation functions (the names are fairly self-explanatory) includes:

`avg()`, `count()`, `countDistinct()`, `first()`, `kurtosis()`, `max()`, `mean()`, `min()`, `skewness()`, `stddev()`, `stddev_pop()`, `stddev_samp()`, `sum()`, `sumDistinct()`, `var_pop()`, `var_samp()` and `variance()`.

Correlations

Another highly useful measure of mutual relationships between features is correlation. Your model would normally include only those features that are highly correlated with your target. However, it is almost equally important to check the correlation between the features; including features that are highly correlated among them (that is, are *collinear*) may lead to unpredictable behavior of your model, or might unnecessarily complicate it.



I talk more about multicollinearity in my other book, *Practical Data Analysis Cookbook*, Packt Publishing (<https://www.packtpub.com/big-data-and-business-intelligence/practical-data-analysis-cookbook>), in *Chapter 5, Introducing MLlib*, under the section titled *Identifying and tackling multicollinearity*.

Calculating correlations in PySpark is very easy once your data is in a DataFrame form. The only difficulties are that the `.corr(...)` method supports the Pearson correlation coefficient at the moment, and it can only calculate pairwise correlations, such as the following:

```
fraud_df.corr('balance', 'numTrans')
```

In order to create a correlations matrix, you can use the following script:

```
n_numerical = len(numerical)

corr = []

for i in range(0, n_numerical):
    temp = [None] * i

    for j in range(i, n_numerical):
        temp.append(fraud_df.corr(numerical[i], numerical[j]))
    corr.append(temp)
```

The preceding code will create the following output:

```
Out[30]: [[1.0, 0.00044523140172659576, 0.00027139913398184604],
          [None, 1.0, -0.0002805712819816179],
          [None, None, 1.0]]
```

As you can see, the correlations between the numerical features in the credit card fraud dataset are pretty much non-existent. Thus, all these features can be used in our models, should they turn out to be statistically sound in explaining our target.

Having checked the correlations, we can now move on to visually inspecting our data.

Visualization

There are multiple visualization packages, but in this section we will be using `matplotlib` and `Bokeh` exclusively to give you the best tools for your needs.

Both of the packages come preinstalled with Anaconda. First, let's load the modules and set them up:

```
%matplotlib inline
import matplotlib.pyplot as plt
plt.style.use('ggplot')
```

```
import bokeh.charts as chrt
from bokeh.io import output_notebook

output_notebook()
```

The `%matplotlib inline` and the `output_notebook()` commands will make every chart generated with `matplotlib` or `Bokeh`, respectively, appear within the notebook and not as a separate window.

Histograms

Histograms are by far the easiest way to visually gauge the distribution of your features. There are three ways you can generate histograms in PySpark (or a Jupyter notebook):

- Aggregate the data in workers and return an aggregated list of bins and counts in each bin of the histogram to the driver
- Return all the data points to the driver and allow the plotting libraries' methods to do the job for you
- Sample your data and then return them to the driver for plotting.

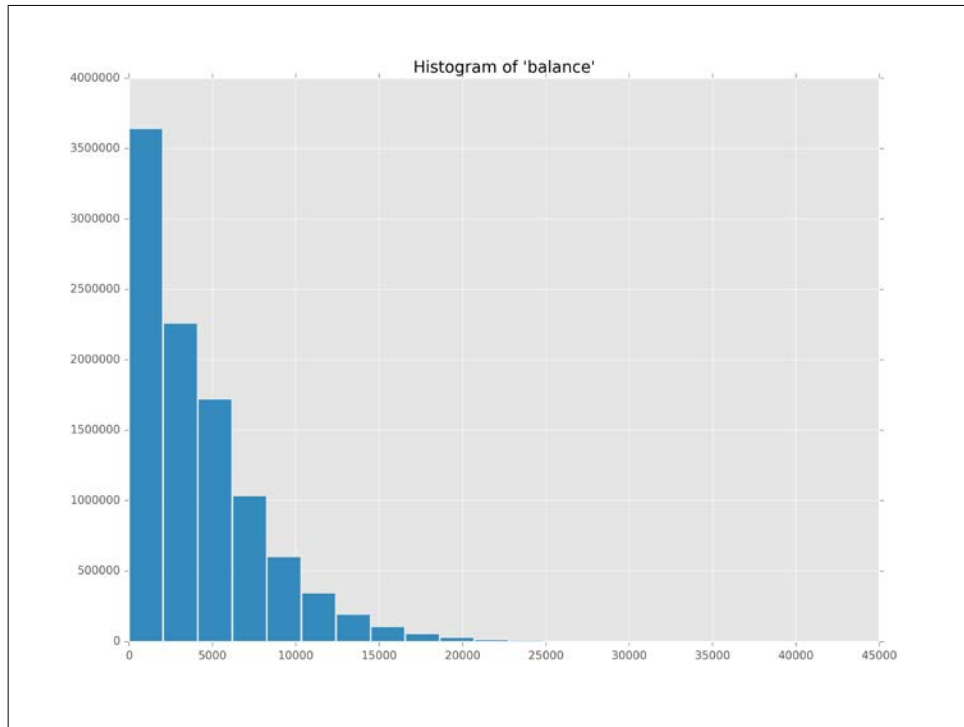
If the number of rows in your dataset is counted in billions, then the second option might not be attainable. Thus, you need to aggregate the data first:

```
hists = fraud_df.select('balance').rdd.flatMap(
    lambda row: row
).histogram(20)
```

To plot the histogram, you can simply call `matplotlib`, as shown in the following code:

```
data = {
    'bins': hists[0][:-1],
    'freq': hists[1]
}
plt.bar(data['bins'], data['freq'], width=2000)
plt.title('Histogram of \'balance\'')
```

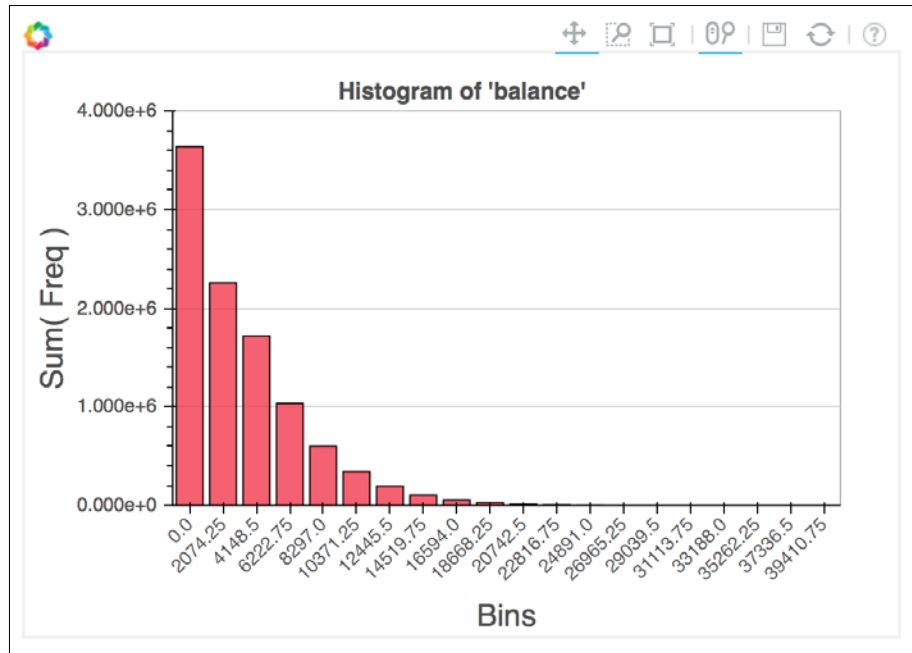
This will produce the following chart:



In a similar manner, a histogram can be created with Bokeh:

```
b_hist = chrt.Bar(  
    data,  
    values='freq', label='bins',  
    title='Histogram of \'balance\'')  
chrt.show(b_hist)
```

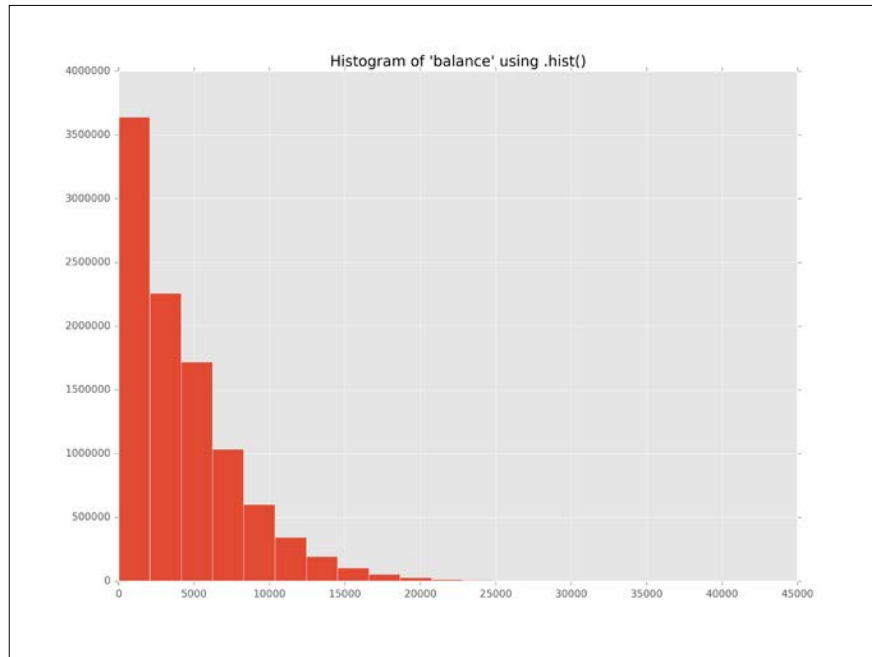
Since Bokeh uses D3.js in the background, the resulting chart is interactive:



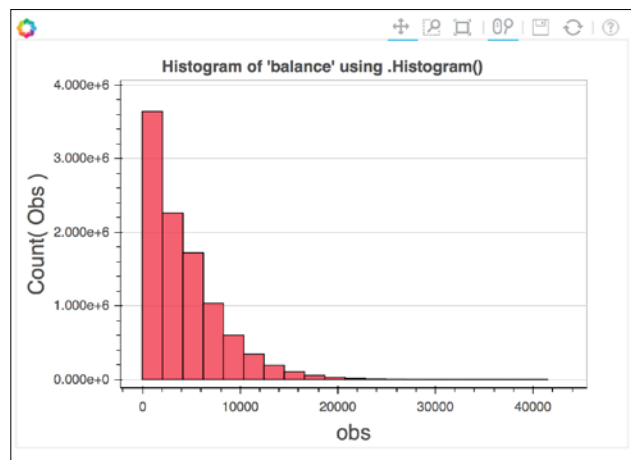
If your data is small enough to fit on the driver (although we would argue it would normally be faster to use the previous method), you can bring the data and use the `.hist(...)` (from matplotlib) or `.Histogram(...)` (from Bokeh) methods:

```
data_driver = {
    'obs': fraud_df.select('balance').rdd.flatMap(
        lambda row: row
    ).collect()
}
plt.hist(data_driver['obs'], bins=20)
plt.title('Histogram of \'balance\' using .hist()')
b_hist_driver = chrt.Histogram(
    data_driver, values='obs',
    title='Histogram of \'balance\' using .Histogram()',
    bins=20
)
chrt.show(b_hist_driver)
```

This will produce the following chart for `matplotlib`:



For Bokeh, the following chart will be generated:



Interactions between features

Scatter charts allow us to visualize interactions between up to three variables at a time (although we will be only presenting a 2D interaction in this section).



You should rarely revert to 3D visualizations unless you are dealing with some temporal data and you want to observe changes over time. Even then, we would rather discretize the time data and present a series of 2D charts, as interpreting 3D charts is somewhat more complicated and (most of the time) confusing.

Since PySpark does not offer any visualization modules on the server side, and trying to plot billions of observations at the same time would be highly impractical, in this section we will sample the dataset at 0.02% (roughly 2,000 observations).



Unless you chose a stratified sampling, you should create at least three to five samples at a predefined sampling fraction so you can check if your sample is somewhat representative of your dataset—that is, that the differences between your samples are not big.

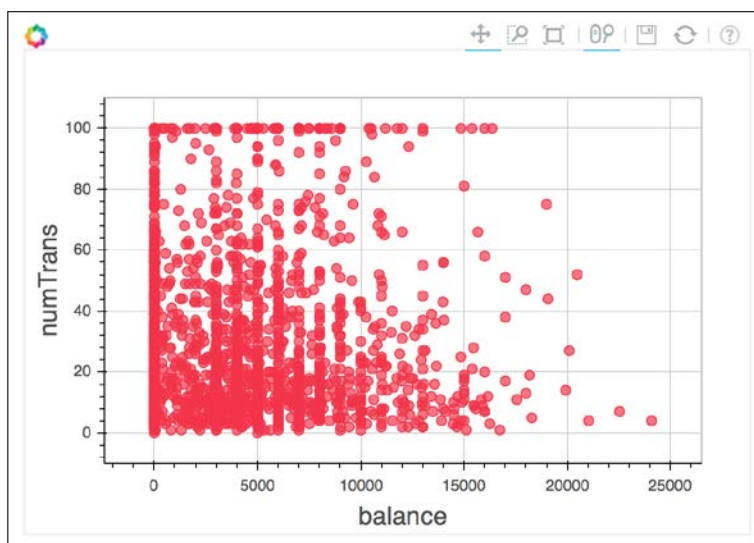
In this example, we will sample our fraud dataset at 0.02% given 'gender' as a strata:

```
data_sample = fraud_df.sampleBy(
    'gender', {1: 0.0002, 2: 0.0002}
).select(numerical)
```

To put multiple 2D charts in one go, you can use the following code:

```
data_multi = dict([
    (elem, data_sample.select(elem).rdd \
     .flatMap(lambda row: row).collect())
    for elem in numerical
])
sctr = chrt.Scatter(data_multi, x='balance', y='numTrans')
chrt.show(sctr)
```

The preceding code will produce the following chart:



As you can see, there are plenty of fraudulent transactions that had 0 balance but many transactions—that is, a fresh card and big spike of transactions. However, no specific pattern can be shown apart from some *banding* occurring at \$1,000 intervals.

Summary

In this chapter, we looked at how to clean and prepare your dataset for modeling by identifying and tackling datasets with missing values, duplicates, and outliers. We also looked at how to get a bit more familiar with your data using tools from PySpark (although this is by no means a full manual on how to analyze your datasets). Finally, we showed you how to chart your data.

We will use these (and more) techniques in the next two chapters, where we will be building machine learning models.

5

Introducing MLlib

In the previous chapter, we learned how to prepare the data for modeling. In this chapter, we will actually use some of that learning to build a classification model using the MLlib package of PySpark.

MLlib stands for Machine Learning Library. Even though MLlib is now in a maintenance mode, that is, it is not actively being developed (and will most likely be deprecated later), it is warranted that we cover at least some of the features of the library. In addition, MLlib is currently the only library that supports training models for streaming.



Starting with Spark 2.0, ML is the main machine learning library that operates on DataFrames instead of RDDs as is the case for MLlib. The documentation for MLlib can be found here: <http://spark.apache.org/docs/latest/api/python/pyspark.mllib.html>.

In this chapter, you will learn how to do the following:

- Prepare the data for modeling with MLlib
- Perform statistical testing
- Predict survival chances of infants using logistic regression
- Select the most predictable features and train a random forest model

Overview of the package

At the high level, MLlib exposes three core machine learning functionalities:

- **Data preparation:** Feature extraction, transformation, selection, hashing of categorical features, and some natural language processing methods
- **Machine learning algorithms:** Some popular and advanced regression, classification, and clustering algorithms are implemented
- **Utilities:** Statistical methods such as descriptive statistics, chi-square testing, linear algebra (sparse and dense matrices and vectors), and model evaluation methods

As you can see, the palette of available functionalities allows you to perform almost all of the fundamental data science tasks.

In this chapter, we will build two classification models: a linear regression and a random forest. We will use a portion of the US 2014 and 2015 birth data we downloaded from http://www.cdc.gov/nchs/data_access/vitalstatsonline.htm; from the total of 300 variables we selected 85 features that we will use to build our models. Also, out of the total of almost 7.99 million records, we selected a balanced sample of 45,429 records: 22,080 records where infants were reported dead and 23,349 records with infants alive.



The dataset we will use in this chapter can be downloaded from http://www.tomdrabas.com/data/LearningPySpark/births_train.csv.gz.

Loading and transforming the data

Even though MLlib is designed with RDDs and DStreams in focus, for ease of transforming the data we will read the data and convert it to a DataFrame.



The DStreams are the basic data abstraction for Spark Streaming (see <http://bit.ly/2jIDT2A>)

Just like in the previous chapter, we first specify the schema of our dataset.



Note that here (for brevity), we only present a handful of features. You should always check our GitHub account for this book for the latest version of the code: <https://github.com/drabastomek/learningPySpark>.

Here's the code:

```
import pyspark.sql.types as typ
labels = [
    ('INFANT_ALIVE_AT_REPORT', typ.StringType()),
    ('BIRTH_YEAR', typ.IntegerType()),
    ('BIRTH_MONTH', typ.IntegerType()),
    ('BIRTH_PLACE', typ.StringType()),
    ('MOTHER_AGE_YEARS', typ.IntegerType()),
    ('MOTHER_RACE_6CODE', typ.StringType()),
    ('MOTHER_EDUCATION', typ.StringType()),
    ('FATHER_COMBINED_AGE', typ.IntegerType()),
    ('FATHER_EDUCATION', typ.StringType()),
    ('MONTH_PRECARE_RECODE', typ.StringType()),
    ...
    ('INFANT_BREASTFED', typ.StringType())
]
schema = typ.StructType([
    typ.StructField(e[0], e[1], False) for e in labels
])
```

Next, we load the data. The `.read.csv(...)` method can read either uncompressed or (as in our case) GZipped comma-separated values. The `header` parameter set to `True` indicates that the first row contains the header, and we use the `schema` to specify the correct data types:

```
births = spark.read.csv('births_train.csv.gz',
                        header=True,
                        schema=schema)
```

There are plenty of features in our dataset that are strings. These are mostly categorical variables that we need to somehow convert to a numeric form.



You can glimpse over the original file schema specification here: ftp://ftp.cdc.gov/pub/Health_Statistics/NCHS/Dataset_Documentation/DVS/natality/UserGuide2015.pdf.

We will first specify our recode dictionary:

```
recode_dictionary = {
    'YNU': {
        'Y': 1,
        'N': 0,
        'U': 0
    }
}
```

Our goal in this chapter is to predict whether the 'INFANT_ALIVE_AT_REPORT' is either 1 or 0. Thus, we will drop all of the features that relate to the infant and will try to predict the infant's chances of surviving only based on the features related to its mother, father, and the place of birth:

```
selected_features = [
    'INFANT_ALIVE_AT_REPORT',
    'BIRTH_PLACE',
    'MOTHER_AGE_YEARS',
    'FATHER_COMBINED_AGE',
    'CIG_BEFORE',
    'CIG_1_TRI',
    'CIG_2_TRI',
    'CIG_3_TRI',
    'MOTHER_HEIGHT_IN',
    'MOTHER_PRE_WEIGHT',
    'MOTHER_DELIVERY_WEIGHT',
    'MOTHER_WEIGHT_GAIN',
    'DIABETES_PRE',
    'DIABETES_GEST',
    'HYP_TENS_PRE',
    'HYP_TENS_GEST',
    'PREV_BIRTH_PRETERM'
]
births_trimmed = births.select(selected_features)
```

In our dataset, there are plenty of features with Yes/No/Unknown values; we will only code Yes to 1; everything else will be set to 0.

There is also a small problem with how the number of cigarettes smoked by the mother was coded: as 0 means the mother smoked no cigarettes before or during the pregnancy, between 1-97 states the actual number of cigarette smoked, 98 indicates either 98 or more, whereas 99 identifies the unknown; we will assume the unknown is 0 and recode accordingly.

So next we will specify our recoding methods:

```
import pyspark.sql.functions as func
def recode(col, key):
    return recode_dictionary[key][col]
def correct_cig(feats):
    return func \
        .when(func.col(feats) != 99, func.col(feats)) \
        .otherwise(0)
rec_integer = func.udf(recode, typ.IntegerType())
```

The `recode` method looks up the correct key from the `recode_dictionary` (given the key) and returns the corrected value. The `correct_cig` method checks when the value of the feature `feat` is not equal to 99 and (for that situation) returns the value of the feature; if the value is equal to 99, we get 0 otherwise.

We cannot use the `recode` function directly on a `DataFrame`; it needs to be converted to a UDF that Spark will understand. The `rec_integer` is such a function: by passing our specified `recode` function and specifying the return value data type, we can use it then to encode our Yes/No/Unknown features.

So, let's get to it. First, we'll correct the features related to the number of cigarettes smoked:

```
births_transformed = births_trimmed \
    .withColumn('CIG_BEFORE', correct_cig('CIG_BEFORE')) \
    .withColumn('CIG_1_TRI', correct_cig('CIG_1_TRI')) \
    .withColumn('CIG_2_TRI', correct_cig('CIG_2_TRI')) \
    .withColumn('CIG_3_TRI', correct_cig('CIG_3_TRI'))
```

The `.withColumn(...)` method takes the name of the column as its first parameter and the transformation as the second one. In the previous cases, we do not create new columns, but reuse the same ones instead.

Now we will focus on correcting the Yes/No/Unknown features. First, we will figure out which these are with the following snippet:

```
cols = [(col.name, col.dataType) for col in births_trimmed.schema]
YNU_cols = []
for i, s in enumerate(cols):
    if s[1] == typ.StringType():
        dis = births.select(s[0]) \
            .distinct() \
            .rdd \
            .map(lambda row: row[0]) \
```

```
.collect()
if 'Y' in dis:
    YNU_cols.append(s[0])
```

First, we created a list of tuples (`cols`) that hold column names and corresponding data types. Next, we loop through all of these and calculate distinct values of all string columns; if a 'Y' is within the returned list, we append the column name to the `YNU_cols` list.

DataFrames can transform the features in bulk while selecting features. To present the idea, consider the following example:

```
births.select([
    'INFANT_NICU_ADMISSION',
    rec_integer(
        'INFANT_NICU_ADMISSION', func.lit('YNU')
    ) \
    .alias('INFANT_NICU_ADMISSION_RECODE')])
).take(5)
```

Here's what we get in return:

```
Out[8]: [Row(INFANT_NICU_ADMISSION='Y', INFANT_NICU_ADMISSION_RECODE=1),
Row(INFANT_NICU_ADMISSION='Y', INFANT_NICU_ADMISSION_RECODE=1),
Row(INFANT_NICU_ADMISSION='U', INFANT_NICU_ADMISSION_RECODE=0),
Row(INFANT_NICU_ADMISSION='N', INFANT_NICU_ADMISSION_RECODE=0),
Row(INFANT_NICU_ADMISSION='U', INFANT_NICU_ADMISSION_RECODE=0)]
```

We select the 'INFANT_NICU_ADMISSION' column and we pass the name of the feature to the `rec_integer` method. We also alias the newly transformed column as 'INFANT_NICU_ADMISSION_RECODE'. This way we will also confirm that our UDF works as intended.

So, to transform all the `YNU_cols` in one go, we will create a list of such transformations, as shown here:

```
exprs_YNU = [
    rec_integer(x, func.lit('YNU')).alias(x)
    if x in YNU_cols
    else x
    for x in births_transformed.columns
]
births_transformed = births_transformed.select(exprs_YNU)
```

Let's check if we got it correctly:

```
births_transformed.select(YNU_cols[-5:]).show(5)
```

Here's what we get:

DIABETES_PRE	DIABETES_GEST	HYP_TENS_PRE	HYP_TENS_GEST	PREV_BIRTH_PRETERM
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	1
0	0	0	0	0

only showing top 5 rows

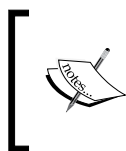
Looks like everything worked as we wanted it to work, so let's get to know our data better.

Getting to know your data

In order to build a statistical model in an informed way, an intimate knowledge of the dataset is necessary. Without knowing the data it is possible to build a successful model, but it is then a much more arduous task, or it would require more technical resources to test all the possible combinations of features. Therefore, after spending the required 80% of the time cleaning the data, we spend the next 15% getting to know it!

Descriptive statistics

I normally start with descriptive statistics. Even though the DataFrames expose the `.describe()` method, since we are working with `MLlib`, we will use the `.colStats(...)` method.



A word of warning: the `.colStats(...)` calculates the descriptive statistics based on a sample. For real world datasets this should not really matter but if your dataset has less than 100 observations you might get some strange results.

The method takes an `RDD` of data to calculate the descriptive statistics of and return a `MultivariateStatisticalSummary` object that contains the following descriptive statistics:

- `count()`: This holds a row count
- `max()`: This holds maximum value in the column
- `mean()`: This holds the value of the mean for the values in the column

- `min()`: This holds the minimum value in the column
- `normL1()`: This holds the value of the L1-Norm for the values in the column
- `normL2()`: This holds the value of the L2-Norm for the values in the column
- `numNonzeros()`: This holds the number of nonzero values in the column
- `variance()`: This holds the value of the variance for the values in the column



You can read more about the L1- and L2-norms here
<http://bit.ly/2jJJPJ0>

We recommend checking the documentation of Spark to learn more about these. The following is a snippet that calculates the descriptive statistics of the numeric features:

```
import pyspark.mllib.stat as st
import numpy as np
numeric_cols = ['MOTHER_AGE_YEARS', 'FATHER_COMBINED_AGE',
                'CIG_BEFORE', 'CIG_1_TRI', 'CIG_2_TRI', 'CIG_3_TRI',
                'MOTHER_HEIGHT_IN', 'MOTHER_PRE_WEIGHT',
                'MOTHER_DELIVERY_WEIGHT', 'MOTHER_WEIGHT_GAIN'
                ]
numeric_rdd = births_transformed\
                .select(numeric_cols)\
                .rdd \
                .map(lambda row: [e for e in row])
mllib_stats = st.Statistics.colStats(numeric_rdd)
for col, m, v in zip(numeric_cols,
                    mllib_stats.mean(),
                    mllib_stats.variance()):
    print('{0}: \t{1:.2f} \t {2:.2f}'.format(col, m, np.sqrt(v)))
```

The preceding code produces the following result:

MOTHER_AGE_YEARS:	28.30	6.08	
FATHER_COMBINED_AGE:	44.55	27.55	
CIG_BEFORE:	1.43	5.18	
CIG_1_TRI:	0.91	3.83	
CIG_2_TRI:	0.70	3.31	
CIG_3_TRI:	0.58	3.11	
MOTHER_HEIGHT_IN:	65.12	6.45	
MOTHER_PRE_WEIGHT:	214.50	210.21	
MOTHER_DELIVERY_WEIGHT:		223.63	180.01
MOTHER_WEIGHT_GAIN:	30.74	26.23	

As you can see, mothers, compared to fathers, are younger: the average age of mothers was 28 versus over 44 for fathers. A good indication (at least for some of the infants) was that many mothers quit smoking while being pregnant; it is horrifying, though, that there still were some that continued smoking.

For the categorical variables, we will calculate the frequencies of their values:

```
categorical_cols = [e for e in births_transformed.columns
                    if e not in numeric_cols]
categorical_rdd = births_transformed\
    .select(categorical_cols)\
    .rdd \
    .map(lambda row: [e for e in row])
for i, col in enumerate(categorical_cols):
    agg = categorical_rdd \
        .groupBy(lambda row: row[i]) \
        .map(lambda row: (row[0], len(row[1])))
    print(col, sorted(agg.collect(),
                      key=lambda el: el[1],
                      reverse=True))
```

Here is what the results look like:

```
INFANT_ALIVE_AT_REPORT [(1, 23349), (0, 22080)]
BIRTH_PLACE [('1', 44558), ('4', 327), ('3', 224), ('2', 136), ('7', 91), ('5', 74), ('6', 11), ('9', 8)]
DIABETES_PRE [(0, 44881), (1, 548)]
DIABETES_GEST [(0, 43451), (1, 1978)]
HYP_TENS_PRE [(0, 44348), (1, 1081)]
HYP_TENS_GEST [(0, 43302), (1, 2127)]
PREV_BIRTH_PRETERM [(0, 43088), (1, 2341)]
```

Most of the deliveries happened in hospital (BIRTH_PLACE equal to 1). Around 550 deliveries happened at home: some intentionally ('BIRTH_PLACE' equal to 3), and some not ('BIRTH_PLACE' equal to 4).

Correlations

Correlations help to identify collinear numeric features and handle them appropriately. Let's check the correlations between our features:

```
corrs = st.Statistics.corr(numeric_rdd)
for i, el in enumerate(corrs > 0.5):
    correlated = [
        (numeric_cols[j], corrs[i][j])
        for j, e in enumerate(el)
        if e == 1.0 and j != i]
    if len(correlated) > 0:
```

```
for e in correlated:
    print('{0}-to-{1}: {2:.2f}' \
          .format(numeric_cols[i], e[0], e[1]))
```

The preceding code will calculate the correlation matrix and will print only those features that have a correlation coefficient greater than 0.5: the `corrs > 0.5` part takes care of that.

Here's what we get:

```
CIG_BEFORE-to-CIG_1_TRI: 0.83
CIG_BEFORE-to-CIG_2_TRI: 0.72
CIG_BEFORE-to-CIG_3_TRI: 0.62
CIG_1_TRI-to-CIG_BEFORE: 0.83
CIG_1_TRI-to-CIG_2_TRI: 0.87
CIG_1_TRI-to-CIG_3_TRI: 0.76
CIG_2_TRI-to-CIG_BEFORE: 0.72
CIG_2_TRI-to-CIG_1_TRI: 0.87
CIG_2_TRI-to-CIG_3_TRI: 0.89
CIG_3_TRI-to-CIG_BEFORE: 0.62
CIG_3_TRI-to-CIG_1_TRI: 0.76
CIG_3_TRI-to-CIG_2_TRI: 0.89
MOTHER_PRE_WEIGHT-to-MOTHER_DELIVERY_WEIGHT: 0.54
MOTHER_PRE_WEIGHT-to-MOTHER_WEIGHT_GAIN: 0.65
MOTHER_DELIVERY_WEIGHT-to-MOTHER_PRE_WEIGHT: 0.54
MOTHER_DELIVERY_WEIGHT-to-MOTHER_WEIGHT_GAIN: 0.60
MOTHER_WEIGHT_GAIN-to-MOTHER_PRE_WEIGHT: 0.65
MOTHER_WEIGHT_GAIN-to-MOTHER_DELIVERY_WEIGHT: 0.60
```

As you can see, the 'CIG_...' features are highly correlated, so we can drop most of them. Since we want to predict the survival chances of an infant as soon as possible, we will keep only the 'CIG_1_TRI'. Also, as expected, the weight features are also highly correlated and we will only keep the 'MOTHER_PRE_WEIGHT':

```
features_to_keep = [
    'INFANT_ALIVE_AT_REPORT',
    'BIRTH_PLACE',
    'MOTHER_AGE_YEARS',
    'FATHER_COMBINED_AGE',
    'CIG_1_TRI',
    'MOTHER_HEIGHT_IN',
    'MOTHER_PRE_WEIGHT',
    'DIABETES_PRE',
    'DIABETES_GEST',
    'HYP_TENS_PRE',
    'HYP_TENS_GEST',
    'PREV_BIRTH_PRETERM'
]
births_transformed = births_transformed.select([e for e in features_
to_keep])
```

Statistical testing

We cannot calculate correlations for the categorical features. However, we can run a Chi-square test to determine if there are significant differences.

Here's how you can do it using the `.chiSqTest(...)` method of `MLlib`:

```
import pyspark.mllib.linalg as ln
for cat in categorical_cols[1:]:
    agg = births_transformed \
        .groupby('INFANT_ALIVE_AT_REPORT') \
        .pivot(cat) \
        .count()
    agg_rdd = agg \
        .rdd \
        .map(lambda row: (row[1:])) \
        .flatMap(lambda row:
            [0 if e == None else e for e in row]) \
        .collect()
    row_length = len(agg.collect()[0]) - 1
    agg = ln.Matrices.dense(row_length, 2, agg_rdd)

test = st.Statistics.chiSqTest(agg)
print(cat, round(test.pValue, 4))
```

We loop through all the categorical variables and pivot them by the 'INFANT_ALIVE_AT_REPORT' feature to get the counts. Next, we transform them into an RDD, so we can then convert them into a matrix using the `pyspark.mllib.linalg` module. The first parameter to the `.Matrices.dense(...)` method specifies the number of rows in the matrix; in our case, it is the length of distinct values of the categorical feature.

The second parameter specifies the number of columns: we have two as our 'INFANT_ALIVE_AT_REPORT' target variable has only two values.

The last parameter is a list of values to be transformed into a matrix.

Here's an example that shows this more clearly:

```
print(ln.Matrices.dense(3,2, [1,2,3,4,5,6]))
```

The preceding code produces the following matrix:

```
DenseMatrix([[ 1.,  4.],
              [ 2.,  5.],
              [ 3.,  6.]])
```

Once we have our counts in a matrix form, we can use the `.chiSqTest(...)` to calculate our test.

Here's what we get in return:

```
BIRTH_PLACE 0.0
DIABETES_PRE 0.0
DIABETES_GEST 0.0
HYP_TENS_PRE 0.0
HYP_TENS_GEST 0.0
PREV_BIRTH_PRETERM 0.0
```

Our tests reveal that all the features should be significantly different and should help us predict the chance of survival of an infant.

Creating the final dataset

Therefore, it is time to create our final dataset that we will use to build our models. We will convert our DataFrame into an RDD of LabeledPoints.

A LabeledPoint is a MLlib structure that is used to train the machine learning models. It consists of two attributes: label and features.

The label is our target variable and features can be a NumPy array, list, `pyspark.mllib.linalg.SparseVector`, `pyspark.mllib.linalg.DenseVector`, or `scipy.sparse` column matrix.

Creating an RDD of LabeledPoints

Before we build our final dataset, we first need to deal with one final obstacle: our 'BIRTH_PLACE' feature is still a string. While any of the other categorical variables can be used as is (as they are now dummy variables), we will use a hashing trick to encode the 'BIRTH_PLACE' feature:

```
import pyspark.mllib.feature as ft
import pyspark.mllib.regression as reg
hashing = ft.HashingTF(7)
births_hashed = births_transformed \
    .rdd \
    .map(lambda row: [
        list(hashing.transform(row[1]).toArray())
        if col == 'BIRTH_PLACE'
        else row[i]
        for i, col
```

```

        in enumerate(features_to_keep))] \
.map(lambda row: [[e] if type(e) == int else e
                  for e in row]) \
.map(lambda row: [item for sublist in row
                  for item in sublist]) \
.map(lambda row: reg.LabeledPoint(
    row[0],
    ln.Vectors.dense(row[1:])))
)

```

First, we create the hashing model. Our feature has seven levels, so we use as many features as that for the hashing trick. Next, we actually use the model to convert our 'BIRTH_PLACE' feature into a `SparseVector`; such a data structure is preferred if your dataset has many columns but in a row only a few of them have non-zero values. We then combine all the features together and finally create a `LabeledPoint`.

Splitting into training and testing

Before we move to the modeling stage, we need to split our dataset into two sets: one we'll use for training and the other for testing. Luckily, RDDs have a handy method to do just that: `.randomSplit(...)`. The method takes a list of proportions that are to be used to randomly split the dataset.

Here is how it is done:

```
births_train, births_test = births_hashed.randomSplit([0.6, 0.4])
```

That's it! Nothing more needs to be done.

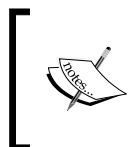
Predicting infant survival

Finally, we can move to predicting the infants' survival chances. In this section, we will build two models: a linear classifier—the logistic regression, and a non-linear one—a random forest. For the former one, we will use all the features at our disposal, whereas for the latter one, we will employ a `ChiSqSelector(...)` method to select the top four features.

Logistic regression in MLlib

Logistic regression is somewhat a benchmark to build any classification model. MLlib used to provide a logistic regression model estimated using a **stochastic gradient descent (SGD)** algorithm. This model has been deprecated in Spark 2.0 in favor of the `LogisticRegressionWithLBFGS` model.

The `LogisticRegressionWithLBFGS` model uses the **Limited-memory Broyden-Fletcher-Goldfarb-Shanno (BFGS)** optimization algorithm. It is a quasi-Newton method that approximates the BFGS algorithm.



For those of you who are mathematically adept and interested in this, we suggest perusing this blog post that is a nice walk-through of the optimization algorithms: <http://aria42.com/blog/2014/12/understanding-lbfgs>.

First, we train the model on our data:

```
from pyspark.mllib.classification \
    import LogisticRegressionWithLBFGS
LR_Model = LogisticRegressionWithLBFGS \
    .train(births_train, iterations=10)
```

Training the model is very simple: we just need to call the `.train(...)` method. The required parameters are the RDD with `LabeledPoints`; we also specified the number of iterations so it does not take too long to run.

Having trained the model using the `births_train` dataset, let's use the model to predict the classes for our testing set:

```
LR_results = (
    births_test.map(lambda row: row.label) \
    .zip(LR_Model \
    .predict(births_test \
    .map(lambda row: row.features)))
    ).map(lambda row: (row[0], row[1] * 1.0))
```

The preceding snippet creates an RDD where each element is a tuple, with the first element being the actual label and the second one, the model's prediction.

MLlib provides an evaluation metric for classification and regression. Let's check how well or how bad our model performed:

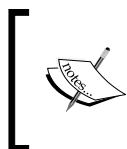
```
import pyspark.mllib.evaluation as ev
LR_evaluation = ev.BinaryClassificationMetrics(LR_results)
print('Area under PR: {0:.2f}' \
    .format(LR_evaluation.areaUnderPR))
print('Area under ROC: {0:.2f}' \
    .format(LR_evaluation.areaUnderROC))
LR_evaluation.unpersist()
```

Here's what we got:

Area under PR: 0.85
Area under ROC: 0.63

The model performed reasonably well! The 85% area under the Precision-Recall curve indicates a good fit. In this case, we might be getting slightly more predicted deaths (true and false positives). In this case, this is actually a good thing as it would allow doctors to put the expectant mother and the infant under special care.

The area under **Receiver-Operating Characteristic (ROC)** can be understood as a probability of the model ranking higher than a randomly chosen positive instance compared to a randomly chosen negative one. A 63% value can be thought of as acceptable.



For more on these metrics, we point interested readers to <http://stats.stackexchange.com/questions/7207/roc-vs-precision-and-recall-curves> and <http://gim.unmc.edu/dxtests/roc3.htm>.

Selecting only the most predictable features

Any model that uses less features to predict a class accurately should always be preferred to a more complex one. MLib allows us to select the most predictable features using a Chi-Square selector.

Here's how you do it:

```
selector = ft.ChISqSelector(4).fit(births_train)
topFeatures_train = (
    births_train.map(lambda row: row.label) \
    .zip(selector \
    .transform(births_train \
    .map(lambda row: row.features)))
    ).map(lambda row: reg.LabeledPoint(row[0], row[1]))
topFeatures_test = (
    births_test.map(lambda row: row.label) \
    .zip(selector \
    .transform(births_test \
    .map(lambda row: row.features)))
    ).map(lambda row: reg.LabeledPoint(row[0], row[1]))
```


We asked the selector to return the four most predictive features from the dataset and train the selector using the `births_train` dataset. We then used the model to extract only those features from our training and testing datasets.

The `.ChiSqSelector(...)` method can only be used for numerical features; categorical variables need to be either hashed or dummy coded before the selector can be used.

Random forest in MLlib

We are now ready to build the random forest model.

The following code shows you how to do it:

```
from pyspark.mllib.tree import RandomForest
RF_model = RandomForest \
    .trainClassifier(data=topFeatures_train,
                    numClasses=2,
                    categoricalFeaturesInfo={},
                    numTrees=6,
                    featureSubsetStrategy='all',
                    seed=666)
```

The first parameter to the `.trainClassifier(...)` method specifies the training dataset. The `numClasses` one indicates how many classes our target variable has. As the third parameter, you can pass a dictionary where the key is the index of a categorical feature in our RDD and the value for the key indicates the number of levels that the categorical feature has. The `numTrees` specifies the number of trees to be in the forest. The next parameter tells the model to use all the features in our dataset instead of keeping only the most descriptive ones, while the last one specifies the seed for the stochastic part of the model.

Let's see how well our model did:

```
RF_results = (
    topFeatures_test.map(lambda row: row.label) \
    .zip(RF_model \
        .predict(topFeatures_test \
            .map(lambda row: row.features)))
)
RF_evaluation = ev.BinaryClassificationMetrics(RF_results)
print('Area under PR: {0:.2f}' \
    .format(RF_evaluation.areaUnderPR))
```

```
print('Area under ROC: {0:.2f}' \
      .format(RF_evaluation.areaUnderROC))
model_evaluation.unpersist()
```

Here are the results:

Area under PR: 0.86
Area under ROC: 0.63

As you can see, the Random Forest model with fewer features performed even better than the logistic regression model. Let's see how the logistic regression would perform with a reduced number of features:

```
LR_Model_2 = LogisticRegressionWithLBFGS \
    .train(topFeatures_train, iterations=10)
LR_results_2 = (
    topFeatures_test.map(lambda row: row.label) \
    .zip(LR_Model_2 \
    .predict(topFeatures_test \
    .map(lambda row: row.features)))
    ).map(lambda row: (row[0], row[1] * 1.0))
LR_evaluation_2 = ev.BinaryClassificationMetrics(LR_results_2)
print('Area under PR: {0:.2f}' \
      .format(LR_evaluation_2.areaUnderPR))
print('Area under ROC: {0:.2f}' \
      .format(LR_evaluation_2.areaUnderROC))
LR_evaluation_2.unpersist()
```

The results might surprise you:

Area under PR: 0.85
Area under ROC: 0.63

As you can see, both models can be simplified and still attain the same level of accuracy. Having said that, you should always opt for a model with fewer variables.

Summary

In this chapter, we looked at the capabilities of the `MLlib` package of PySpark. Even though the package is currently in a maintenance mode and is not actively being worked on, it is still good to know how to use it. Also, for now it is the only package available to train models while streaming data. We used `MLlib` to clean up, transform, and get familiar with the dataset of infant deaths. Using that knowledge we then successfully built two models that aimed at predicting the chance of infant survival given the information about its mother, father, and place of birth.

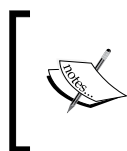
In the next chapter, we will revisit the same problem, but using the newer package that is currently the Spark recommended package for machine learning.

6

Introducing the ML Package

In the previous chapter, we worked with the MLlib package in Spark that operated strictly on RDDs. In this chapter, we move to the ML part of Spark that operates strictly on DataFrames. Also, according to the Spark documentation, the primary machine learning API for Spark is now the DataFrame-based set of models contained in the `spark.ml` package.

So, let's get to it!



In this chapter, we will reuse a portion of the dataset we played within the previous chapter. The data can be downloaded from http://www.tomdrabas.com/data/LearningPySpark/births_transformed.csv.gz.

In this chapter, you will learn how to do the following:

- Prepare transformers, estimators, and pipelines
- Predict the chances of infant survival using models available in the ML package
- Evaluate the performance of the model
- Perform parameter hyper-tuning
- Use other machine-learning models available in the package

Overview of the package

At the top level, the package exposes three main abstract classes: a `Transformer`, an `Estimator`, and a `Pipeline`. We will shortly explain each with some short examples. We will provide more concrete examples of some of the models in the last section of this chapter.

Transformer

The Transformer class, like the name suggests, *transforms* your data by (normally) appending a new column to your DataFrame.

At the high level, when deriving from the Transformer abstract class, each and every new Transformer needs to implement a `.transform(...)` method. The method, as a first and normally the only obligatory parameter, requires passing a DataFrame to be transformed. This, of course, varies *method-by-method* in the ML package: other *popular* parameters are `inputCol` and `outputCol`; these, however, frequently default to some predefined values, such as, for example, 'features' for the `inputCol` parameter.

There are many Transformers offered in the `spark.ml.feature` and we will briefly describe them here (before we use some of them later in this chapter):

- **Binarizer:** Given a threshold, the method takes a continuous variable and transforms it into a binary one.
- **Bucketizer:** Similar to the Binarizer, this method takes a list of thresholds (the `splits` parameter) and transforms a continuous variable into a multinomial one.
- **ChiSqSelector:** For the categorical target variables (think classification models), this feature allows you to select a predefined number of features (parameterized by the `numTopFeatures` parameter) that explain the variance in the target the best. The selection is done, as the name of the method suggests, using a Chi-Square test. It is one of the two-step methods: first, you need to `.fit(...)` your data (so the method can calculate the Chi-square tests). Calling the `.fit(...)` method (you pass your DataFrame as a parameter) returns a `ChiSqSelectorModel` object that you can then use to transform your DataFrame using the `.transform(...)` method.



More information on Chi-squares can be found here: http://ccnmtl.columbia.edu/projects/qmss/the_chisquare_test/about_the_chisquare_test.html.

- **CountVectorizer:** This is useful for a tokenized text (such as `[['Learning', 'PySpark', 'with', 'us'], ['us', 'us', 'us']]`). It is one of two-step methods: first, you need to `.fit(...)`, that is, learn the patterns from your dataset, before you can `.transform(...)` with the `CountVectorizerModel` returned by the `.fit(...)` method. The output from this transformer, for the tokenized text presented previously, would look similar to this: `[(4, [0, 1, 2, 3], [1.0, 1.0, 1.0, 1.0]), (4, [3], [3.0])]`.

- **DCT:** The Discrete Cosine Transform takes a vector of real values and returns a vector of the same length, but with the sum of cosine functions oscillating at different frequencies. Such transformations are useful to extract some underlying frequencies in your data or in data compression.
- **ElementwiseProduct:** A method that returns a vector with elements that are products of the vector passed to the method, and a vector passed as the `scalingVec` parameter. For example, if you had a `[10.0, 3.0, 15.0]` vector and your `scalingVec` was `[0.99, 3.30, 0.66]`, then the vector you would get would look as follows: `[9.9, 9.9, 9.9]`.
- **HashingTF:** A hashing trick transformer that takes a list of tokenized text and returns a vector (of predefined length) with counts. From PySpark's documentation:

"Since a simple modulo is used to transform the hash function to a column index, it is advisable to use a power of two as the numFeatures parameter; otherwise the features will not be mapped evenly to the columns."

- **IDF:** This method computes an **Inverse Document Frequency** for a list of documents. Note that the documents need to already be represented as a vector (for example, using either the `HashingTF` or `CountVectorizer`).
- **IndexToString:** A complement to the `StringIndexer` method. It uses the encoding from the `StringIndexerModel` object to reverse the string index to original values. As an aside, please note that this sometimes does not work and you need to specify the values from the `StringIndexer`.
- **MaxAbsScaler:** Rescales the data to be within the `[-1.0, 1.0]` range (thus, it does not shift the center of the data).
- **MinMaxScaler:** This is similar to the `MaxAbsScaler` with the difference that it scales the data to be in the `[0.0, 1.0]` range.
- **NGram:** This method takes a list of tokenized text and returns *n-grams*: pairs, triples, or *n-mores* of subsequent words. For example, if you had a `['good', 'morning', 'Robin', 'Williams']` vector you would get the following output: `['good morning', 'morning Robin', 'Robin Williams']`.
- **Normalizer:** This method scales the data to be of unit norm using the *p*-norm value (by default, it is L2).
- **OneHotEncoder:** This method encodes a categorical column to a column of binary vectors.
- **PCA:** Performs the data reduction using principal component analysis.

- **PolynomialExpansion**: Performs a polynomial expansion of a vector. For example, if you had a vector symbolically written as $[x, y, z]$, the method would produce the following expansion: $[x, x*x, y, x*y, y*y, z, x*z, y*z, z*z]$.
- **QuantileDiscretizer**: Similar to the **Bucketizer** method, but instead of passing the `splits` parameter, you pass the `numBuckets` one. The method then decides, by calculating approximate quantiles over your data, what the splits should be.
- **RegexTokenizer**: This is a string tokenizer using regular expressions.
- **RFormula**: For those of you who are avid R users, you can pass a formula such as `vec ~ alpha * 3 + beta` (assuming your `DataFrame` has the `alpha` and `beta` columns) and it will produce the `vec` column given the expression.
- **SQLTransformer**: Similar to the previous, but instead of R-like formulas, you can use SQL syntax.



The FROM statement should be selecting from `__THIS__`, indicating you are accessing the `DataFrame`. For example: `SELECT alpha * 3 + beta AS vec FROM __THIS__`.

- **StandardScaler**: Standardizes the column to have a 0 mean and standard deviation equal to 1.
- **StopWordsRemover**: Removes stop words (such as 'the' or 'a') from a tokenized text.
- **StringIndexer**: Given a list of all the words in a column, this will produce a vector of indices.
- **Tokenizer**: This is the default tokenizer that converts the string to lower case and then splits on space(s).
- **VectorAssembler**: This is a highly useful transformer that collates multiple numeric (vectors included) columns into a single column with a vector representation. For example, if you had three columns in your `DataFrame`:

```
df = spark.createDataFrame(  
    [(12, 10, 3), (1, 4, 2)],  
    ['a', 'b', 'c'])
```

The output of calling:

```
ft.VectorAssembler(inputCols=['a', 'b', 'c'],  
    outputCol='features')\  
    .transform(df) \  
    .select('features')\  
    .collect()
```

It would look as follows:

```
[Row(features=DenseVector([12.0, 10.0, 3.0])),
 Row(features=DenseVector([1.0, 4.0, 2.0]))]
```

- **VectorIndexer**: This is a method for indexing categorical columns into a vector of indices. It works in a *column-by-column* fashion, selecting distinct values from the column, sorting and returning an index of the value from the map instead of the original value.
- **VectorSlicer**: Works on a feature vector, either dense or sparse: given a list of indices, it extracts the values from the feature vector.
- **Word2Vec**: This method takes a sentence (string) as an input and transforms it into a map of {string, vector} format, a representation that is useful in natural language processing.



Note that there are many methods in the ML package that have an E letter next to it; this means the method is currently in beta (or Experimental) and it sometimes might fail or produce erroneous results. Beware.

Estimators

Estimators can be thought of as statistical models that need to be estimated to make predictions or classify your observations.

If deriving from the abstract `Estimator` class, the new model has to implement the `.fit(...)` method that fits the model given the data found in a `DataFrame` and some default or user-specified parameters.

There are a lot of estimators available in PySpark and we will now shortly describe the models available in Spark 2.0.

Classification

The ML package provides a data scientist with seven classification models to choose from. These range from the simplest ones (such as logistic regression) to more sophisticated ones. We will provide short descriptions of each of them in the following section:

- **LogisticRegression**: The benchmark model for classification. The logistic regression uses a logit function to calculate the probability of an observation belonging to a particular class. At the time of writing, the PySpark ML supports only binary classification problems.

- **DecisionTreeClassifier**: A classifier that builds a decision tree to predict a class for an observation. Specifying the `maxDepth` parameter limits the depth the tree grows, the `minInstancePerNode` determines the minimum number of observations in the tree node required to further split, the `maxBins` parameter specifies the maximum number of bins the continuous variables will be split into, and the `impurity` specifies the metric to measure and calculate the information gain from the split.
- **GBClassifier**: A **Gradient Boosted Trees** model for classification. The model belongs to the family of ensemble models: models that combine multiple weak predictive models to form a strong one. At the moment, the **GBClassifier** model supports binary labels, and continuous and categorical features.
- **RandomForestClassifier**: This model produces multiple decision trees (hence the name – forest) and uses the mode output of those decision trees to classify observations. The **RandomForestClassifier** supports both binary and multinomial labels.
- **NaiveBayes**: Based on the Bayes' theorem, this model uses conditional probability theory to classify observations. The **NaiveBayes** model in PySpark ML supports both binary and multinomial labels.
- **MultilayerPerceptronClassifier**: A classifier that mimics the nature of a human brain. Deeply rooted in the Artificial Neural Networks theory, the model is a black-box, that is, it is not easy to interpret the internal parameters of the model. The model consists, at a minimum, of three, fully connected layers (a parameter that needs to be specified when creating the model object) of artificial neurons: the input layer (that needs to be equal to the number of features in your dataset), a number of hidden layers (at least one), and an output layer with the number of neurons equal to the number of categories in your label. All the neurons in the input and hidden layers have a sigmoid activation function, whereas the activation function of the neurons in the output layer is softmax.
- **OneVsRest**: A reduction of a multiclass classification to a binary one. For example, in the case of a multinomial label, the model can train multiple binary logistic regression models. For example, if `label == 2`, the model will build a logistic regression where it will convert the `label == 2` to 1 (all remaining label values would be set to 0) and then train a binary model. All the models are then scored and the model with the highest probability wins.

Regression

There are seven models available for regression tasks in the PySpark ML package. As with classification, these range from some basic ones (such as the obligatory linear regression) to more complex ones:

- **AFTSurvivalRegression**: Fits an Accelerated Failure Time regression model. It is a parametric model that assumes that a marginal effect of one of the features accelerates or decelerates a life expectancy (or process failure). It is highly applicable for the processes with well-defined stages.
- **DecisionTreeRegressor**: Similar to the model for classification with an obvious distinction that the label is continuous instead of binary (or multinomial).
- **GBRegressor**: As with the **DecisionTreeRegressor**, the difference is the data type of the label.
- **GeneralizedLinearRegression**: A family of linear models with differing kernel functions (link functions). In contrast to the linear regression that assumes normality of error terms, the GLM allows the label to have different error term distributions: the **GeneralizedLinearRegression** model from the PySpark ML package supports gaussian, binomial, gamma, and poisson families of error distributions with a host of different link functions.
- **IsotonicRegression**: A type of regression that fits a free-form, non-decreasing line to your data. It is useful to fit the datasets with ordered and increasing observations.
- **LinearRegression**: The most simple of regression models, it assumes a linear relationship between features and a continuous label, and normality of error terms.
- **RandomForestRegressor**: Similar to either **DecisionTreeRegressor** or **GBRegressor**, the **RandomForestRegressor** fits a continuous label instead of a discrete one.

Clustering

Clustering is a family of unsupervised models that are used to find underlying patterns in your data. The PySpark ML package provides the four most popular models at the moment:

- **BisectingKMeans**: A combination of the k-means clustering method and hierarchical clustering. The algorithm begins with all observations in a single cluster and iteratively splits the data into k clusters.



Check out this website for more information on pseudo-algorithms:
<http://minethedata.blogspot.com/2012/08/bisecting-k-means.html>.

- `KMeans`: This is the famous k-mean algorithm that separates data into `k` clusters, iteratively searching for centroids that minimize the sum of square distances between each observation and the centroid of the cluster it belongs to.
- `GaussianMixture`: This method uses `k` Gaussian distributions with unknown parameters to dissect the dataset. Using the Expectation-Maximization algorithm, the parameters for the Gaussians are found by maximizing the log-likelihood function.



Beware that for datasets with many features this model might perform poorly due to the curse of dimensionality and numerical issues with Gaussian distributions.

- `LDA`: This model is used for topic modeling in natural language processing applications.

There is also one recommendation model available in PySpark ML, but we will refrain from describing it here.

Pipeline

A Pipeline in PySpark ML is a concept of an *end-to-end* transformation-estimation process (with distinct stages) that ingests some raw data (in a `DataFrame` form), performs the necessary data carpentry (transformations), and finally estimates a statistical model (estimator).



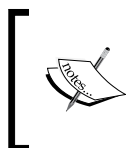
A Pipeline can be purely transformative, that is, consisting of Transformers only.

A Pipeline can be thought of as a chain of multiple discrete stages. When a `.fit(...)` method is executed on a Pipeline object, all the stages are executed in the order they were specified in the `stages` parameter; the `stages` parameter is a list of Transformer and Estimator objects. The `.fit(...)` method of the Pipeline object executes the `.transform(...)` method for the Transformers and the `.fit(...)` method for the Estimators.

Normally, the output of a preceding stage becomes the input for the following stage: when deriving from either the `Transformer` or `Estimator` abstract classes, one needs to implement the `.getOutputCol()` method that returns the value of the `outputCol` parameter specified when creating an object.

Predicting the chances of infant survival with ML

In this section, we will use the portion of the dataset from the previous chapter to present the ideas of PySpark ML.



If you have not yet downloaded the data while reading the previous chapter, it can be accessed here: http://www.tomdrabas.com/data/LearningPySpark/births_transformed.csv.gz.

In this section, we will, once again, attempt to predict the chances of the survival of an infant.

Loading the data

First, we load the data with the help of the following code:

```
import pyspark.sql.types as typ
labels = [
    ('INFANT_ALIVE_AT_REPORT', typ.IntegerType()),
    ('BIRTH_PLACE', typ.StringType()),
    ('MOTHER_AGE_YEARS', typ.IntegerType()),
    ('FATHER_COMBINED_AGE', typ.IntegerType()),
    ('CIG_BEFORE', typ.IntegerType()),
    ('CIG_1_TRI', typ.IntegerType()),
    ('CIG_2_TRI', typ.IntegerType()),
    ('CIG_3_TRI', typ.IntegerType()),
    ('MOTHER_HEIGHT_IN', typ.IntegerType()),
    ('MOTHER_PRE_WEIGHT', typ.IntegerType()),
    ('MOTHER_DELIVERY_WEIGHT', typ.IntegerType()),
    ('MOTHER_WEIGHT_GAIN', typ.IntegerType()),
    ('DIABETES_PRE', typ.IntegerType()),
    ('DIABETES_GEST', typ.IntegerType()),
    ('HYP_TENS_PRE', typ.IntegerType()),
    ('HYP_TENS_GEST', typ.IntegerType()),
    ('PREV_BIRTH_PRETERM', typ.IntegerType())
]
```

```
]
schema = typ.StructType([
    typ.StructField(e[0], e[1], False) for e in labels
])
births = spark.read.csv('births_transformed.csv.gz',
                        header=True,
                        schema=schema)
```

We specify the schema of the DataFrame; our severely limited dataset now only has 17 columns.

Creating transformers

Before we can use the dataset to estimate a model, we need to do some transformations. Since statistical models can only operate on numeric data, we will have to encode the BIRTH_PLACE variable.

Before we do any of this, since we will use a number of different feature transformations later in this chapter, let's import them all:

```
import pyspark.ml.feature as ft
```

To encode the BIRTH_PLACE column, we will use the OneHotEncoder method. However, the method cannot accept StringType columns; it can only deal with numeric types so first we will cast the column to an IntegerType:

```
births = births \
    .withColumn('BIRTH_PLACE_INT', births['BIRTH_PLACE'] \
    .cast(typ.IntegerType()))
```

Having done this, we can now create our first Transformer:

```
encoder = ft.OneHotEncoder(
    inputCol='BIRTH_PLACE_INT',
    outputCol='BIRTH_PLACE_VEC')
```

Let's now create a single column with all the features collated together. We will use the VectorAssembler method:

```
featuresCreator = ft.VectorAssembler(
    inputCols=[
        col[0]
        for col
        in labels[2:]] + \
    [encoder.getOutputCol()],
    outputCol='features'
)
```

The `inputCols` parameter passed to the `VectorAssembler` object is a list of all the columns to be combined together to form the `outputCol` — the 'features'. Note that we use the output of the encoder object (by calling the `.getOutputCol()` method), so we do not have to remember to change this parameter's value should we change the name of the output column in the encoder object at any point.

It's now time to create our first estimator.

Creating an estimator

In this example, we will (once again) use the logistic regression model. However, later in the chapter, we will showcase some more complex models from the `.classification` set of PySpark ML models, so we load the whole section:

```
import pyspark.ml.classification as cl
```

Once loaded, let's create the model by using the following code:

```
logistic = cl.LogisticRegression(  
    maxIter=10,  
    regParam=0.01,  
    labelCol='INFANT_ALIVE_AT_REPORT')
```

We would not have to specify the `labelCol` parameter if our target column had the name 'label'. Also, if the output of our `featuresCreator` was not called 'features', we would have to specify the `featuresCol` by (most conveniently) calling the `getOutputCol()` method on the `featuresCreator` object.

Creating a pipeline

All that is left now is to create a `Pipeline` and fit the model. First, let's load the `Pipeline` from the ML package:

```
from pyspark.ml import Pipeline
```

Creating a `Pipeline` is really easy. Here's how our pipeline should look like conceptually:



Converting this structure into a Pipeline is a *walk in the park*:

```
pipeline = Pipeline(stages=[
    encoder,
    featuresCreator,
    logistic
])
```

That's it! Our pipeline is now created so we can (finally!) estimate the model.

Fitting the model

Before you fit the model, we need to split our dataset into training and testing datasets. Conveniently, the DataFrame API has the `.randomSplit(...)` method:

```
births_train, births_test = births \
    .randomSplit([0.7, 0.3], seed=666)
```

The first parameter is a list of dataset proportions that should end up in, respectively, `births_train` and `births_test` subsets. The `seed` parameter provides a seed to the randomizer.



You can also split the dataset into more than two subsets as long as the elements of the list sum up to 1, and you unpack the output into as many subsets.

For example, we could split the births dataset into three subsets like this:

```
train, test, val = births.\
    randomSplit([0.7, 0.2, 0.1], seed=666)
```

The preceding code would put a random 70% of the births dataset into the `train` object, 20% would go to the `test`, and the `val` DataFrame would hold the remaining 10%.

Now it is about time to finally run our pipeline and estimate our model:

```
model = pipeline.fit(births_train)
test_model = model.transform(births_test)
```

The `.fit(...)` method of the pipeline object takes our training dataset as an input. Under the hood, the `births_train` dataset is passed first to the `encoder` object. The DataFrame that is created at the `encoder` stage then gets passed to the `featuresCreator` that creates the 'features' column. Finally, the output from this stage is passed to the `logistic` object that estimates the final model.

The `.fit(...)` method returns the `PipelineModel` object (the `model` object in the preceding snippet) that can then be used for prediction; we attain this by calling the `.transform(...)` method and passing the testing dataset created earlier. Here's what the `test_model` looks like in the following command:

```
test_model.take(1)
```

It generates the following output:

```
Out[12]: [Row(INFANT_ALIVE_AT_REPORT=0, BIRTH_PLACE='1', MOTHER_AGE_YEARS=13, FATHER_COMBINED_AGE=99, CIG_BEFORE=0, CIG_1_TRI=0, CIG_2_TRI=0, CIG_3_TRI=0, MOTHER_HEIGHT_IN=66, MOTHER_PRE_WEIGHT=133, MOTHER_DELIVERY_WEIGHT=135, MOTHER_WEIGHT_GAIN=2, DIABETES_PRE=0, DIABETES_GEST=0, HYP_TENS_PRE=0, HYP_TENS_GEST=0, PREV_BIRTH_PRETERM=0, BIRTH_PLACE_INT=1, BIRTH_PLACE_VEC=SparseVector(9, {1: 1.0}), features=SparseVector(24, {0: 13.0, 1: 99.0, 6: 66.0, 7: 133.0, 8: 135.0, 9: 2.0, 16: 1.0}), rawPrediction=DenseVector([1.0573, -1.0573]), probability=DenseVector([0.7422, 0.2578]), prediction=0.0)]
```

As you can see, we get all the columns from the Transformers and Estimators. The logistic regression model outputs several columns: the `rawPrediction` is the value of the linear combination of features and the β coefficients, the `probability` is the calculated probability for each of the classes, and finally, the `prediction` is our final class assignment.

Evaluating the performance of the model

Obviously, we would like to now test how well our model did. PySpark exposes a number of evaluation methods for classification and regression in the `.evaluation` section of the package:

```
import pyspark.ml.evaluation as ev
```

We will use the `BinaryClassificationEvaluator` to test how well our model performed:

```
evaluator = ev.BinaryClassificationEvaluator(
    rawPredictionCol='probability',
    labelCol='INFANT_ALIVE_AT_REPORT')
```

The `rawPredictionCol` can either be the `rawPrediction` column produced by the estimator or the `probability`.

Let's see how well our model performed:

```
print(evaluator.evaluate(test_model,
    {evaluator.metricName: 'areaUnderROC'}))
print(evaluator.evaluate(test_model,
    {evaluator.metricName: 'areaUnderPR'}))
```

The preceding code produces the following result:

0.7401301847095617
0.7139354342365674

The area under the ROC of 74% and area under PR of 71% shows a well-defined model, but nothing out of extraordinary; if we had other features, we could drive this up, but this is not the purpose of this chapter (nor the book, for that matter).

Saving the model

PySpark allows you to save the Pipeline definition for later use. It not only saves the pipeline structure, but also all the definitions of all the Transformers and Estimators:

```
pipelinePath = './infant_oneHotEncoder_Logistic_Pipeline'
pipeline.write().overwrite().save(pipelinePath)
```

So, you can load it up later and use it straight away to `.fit(...)` and `predict`:

```
loadedPipeline = Pipeline.load(pipelinePath)
loadedPipeline \
    .fit(births_train)\
    .transform(births_test)\
    .take(1)
```

The preceding code produces the same result (as expected):

```
Out[17]: [Row(INFANT_ALIVE_AT_REPORT=0, BIRTH_PLACE='1', MOTHER_AGE_YEARS=13,
FATHER_COMBINED_AGE=99, CIG_BEFORE=0, CIG_1_TRI=0, CIG_2_TRI=0,
CIG_3_TRI=0, MOTHER_HEIGHT_IN=66, MOTHER_PRE_WEIGHT=133, MOTHER_DE
LIVERY_WEIGHT=135, MOTHER_WEIGHT_GAIN=2, DIABETES_PRE=0, DIABETES_
GEST=0, HYP_TENS_PRE=0, HYP_TENS_GEST=0, PREV_BIRTH_PRETERM=0, BIR
TH_PLACE_INT=1, BIRTH_PLACE_VEC=SparseVector(9, {1: 1.0}), feature
s=SparseVector(24, {0: 13.0, 1: 99.0, 6: 66.0, 7: 133.0, 8: 135.0,
9: 2.0, 16: 1.0}), rawPrediction=DenseVector([1.0573, -1.0573]), p
robability=DenseVector([0.7422, 0.2578]), prediction=0.0)]
```

If you, however, want to save the estimated model, you can also do that; instead of saving the `Pipeline`, you need to save the `PipelineModel`.



Note, that not only the `PipelineModel` can be saved: virtually all the models that are returned by calling the `.fit(...)` method on an `Estimator` or `Transformer` can be saved and loaded back to be reused.

To save your model, see the following the example:

```
from pyspark.ml import PipelineModel

modelPath = './infant_oneHotEncoder_Logistic_PipelineModel'
model.write().overwrite().save(modelPath)

loadedPipelineModel = PipelineModel.load(modelPath)
test_reloadedModel = loadedPipelineModel.transform(births_test)
```

The preceding script uses the `.load(...)` method, a class method of the `PipelineModel` class, to reload the estimated model. You can compare the result of `test_reloadedModel.take(1)` with the output of `test_model.take(1)` we presented earlier.

Parameter hyper-tuning

Rarely, our first model would be the best we can do. By simply looking at our metrics and accepting the model because it passed our pre-conceived performance thresholds is hardly a scientific method for finding the best model.

A concept of parameter hyper-tuning is to find the best parameters of the model: for example, the maximum number of iterations needed to properly estimate the logistic regression model or maximum depth of a decision tree.

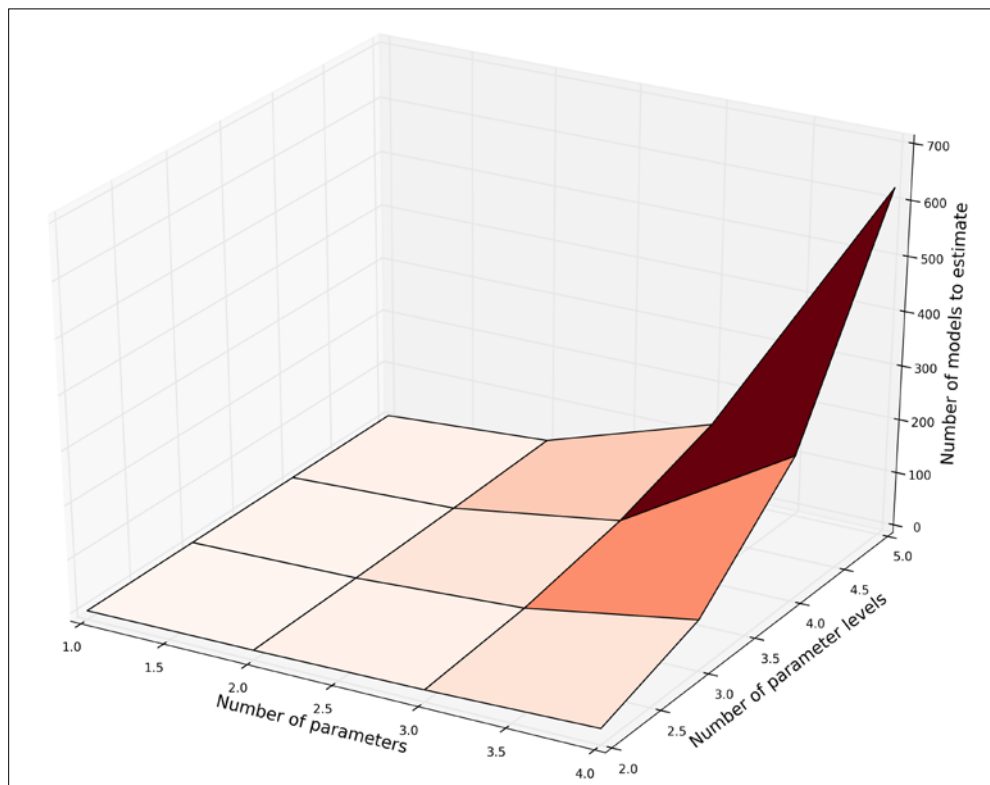
In this section, we will explore two concepts that allow us to find the best parameters for our models: grid search and train-validation splitting.

Grid search

Grid search is an exhaustive algorithm that loops through the list of defined parameter values, estimates separate models, and chooses the best one given some evaluation metric.

A note of caution should be stated here: if you define too many parameters you want to optimize over, or too many values of these parameters, it might take a lot of time to select the best model as the number of models to estimate would grow very quickly as the number of parameters and parameter values grow.

For example, if you want to fine-tune two parameters with two parameter values, you would have to fit four models. Adding one more parameter with two values would require estimating eight models, whereas adding one more additional value to our two parameters (bringing it to three values for each) would require estimating nine models. As you can see, this can quickly get out of hand if you are not careful. See the following chart to inspect this visually:



After this cautionary tale, let's get to fine-tuning our parameters space. First, we load the `.tuning` part of the package:

```
import pyspark.ml.tuning as tune
```

Next, let's specify our model and the list of parameters we want to loop through:

```
logistic = cl.LogisticRegression(
    labelCol='INFANT_ALIVE_AT_REPORT')
grid = tune.ParamGridBuilder() \
    .addGrid(logistic.maxIter,
              [2, 10, 50]) \
    .addGrid(logistic.regParam,
              [0.01, 0.05, 0.3]) \
    .build()
```

First, we specify the model we want to optimize the parameters of. Next, we decide which parameters we will be optimizing, and what values for those parameters to test. We use the `ParamGridBuilder()` object from the `.tuning` subpackage, and keep adding the parameters to the grid with the `.addGrid(...)` method: the first parameter is the parameter object of the model we want to optimize (in our case, these are `logistic.maxIter` and `logistic.regParam`), and the second parameter is a list of values we want to loop through. Calling the `.build()` method on the `.ParamGridBuilder` builds the grid.

Next, we need some way of comparing the models:

```
evaluator = ev.BinaryClassificationEvaluator(
    rawPredictionCol='probability',
    labelCol='INFANT_ALIVE_AT_REPORT')
```

So, once again, we'll use the `BinaryClassificationEvaluator`. It is time now to create the logic that will do the validation work for us:

```
cv = tune.CrossValidator(
    estimator=logistic,
    estimatorParamMaps=grid,
    evaluator=evaluator
)
```

The `CrossValidator` needs the estimator, the `estimatorParamMaps`, and the evaluator to do its job. The model loops through the grid of values, estimates the models, and compares their performance using the evaluator.

We cannot use the data straight away (as the `births_train` and `births_test` still have the `BIRTHS_PLACE` column not encoded) so we create a purely transforming Pipeline:

```
pipeline = Pipeline(stages=[encoder , featuresCreator])
data_transformer = pipeline.fit(births_train)
```

Having done this, we are ready to find the optimal combination of parameters for our model:

```
cvModel = cv.fit(data_transformer.transform(births_train))
```

The `cvModel` will return the best model estimated. We can now use it to see if it performed better than our previous model:

```
data_train = data_transformer \
    .transform(births_test)
results = cvModel.transform(data_train)
print(evaluator.evaluate(results,
    {evaluator.metricName: 'areaUnderROC'}))
print(evaluator.evaluate(results,
    {evaluator.metricName: 'areaUnderPR'}))
```

The preceding code will produce the following result:

0.7404304424804281
0.7156729757616691

As you can see, we got a slightly better result. What parameters does the best model have? The answer is a little bit convoluted, but here's how you can extract it:

```
results = [
    (
        [
            {key.name: paramValue}
            for key, paramValue
            in zip(
                params.keys(),
                params.values())
        ], metric
    )
    for params, metric
    in zip(
        cvModel.getEstimatorParamMaps(),
        cvModel.avgMetrics
    )
]
sorted(results,
    key=lambda el: el[1],
    reverse=True) [0]
```

The preceding code produces the following output:

```
Out[27]: ([{'maxIter': 50}, {'regParam': 0.01}], 2.2158632176362274)
```

Train-validation splitting

The `TrainValidationSplit` model, to select the best model, performs a random split of the input dataset (the training dataset) into two subsets: smaller training and validation subsets. The split is only performed once.

In this example, we will also use the `ChiSqSelector` to select only the top five features, thus limiting the complexity of our model:

```
selector = ft.ChiSqSelector(
    numTopFeatures=5,
    featuresCol=featuresCreator.getOutputCol(),
    outputCol='selectedFeatures',
    labelCol='INFANT_ALIVE_AT_REPORT'
)
```

The `numTopFeatures` specifies the number of features to return. We will put the selector after the `featuresCreator`, so we call the `.getOutputCol()` on the `featuresCreator`.

We covered creating the `LogisticRegression` and `Pipeline` earlier, so we will not explain how these are created again here:

```
logistic = cl.LogisticRegression(
    labelCol='INFANT_ALIVE_AT_REPORT',
    featuresCol='selectedFeatures'
)
pipeline = Pipeline(stages=[encoder, featuresCreator, selector])
data_transformer = pipeline.fit(births_train)
```

The `TrainValidationSplit` object gets created in the same fashion as the `CrossValidator` model:

```
tvsv = tune.TrainValidationSplit(
    estimator=logistic,
    estimatorParamMaps=grid,
    evaluator=evaluator
)
```

As before, we fit our data to the model, and calculate the results:

```
tvsmModel = tvs.fit(
    data_transformer \
        .transform(births_train)
)
data_train = data_transformer \
    .transform(births_test)
results = tvsmModel.transform(data_train)
print(evaluator.evaluate(results,
    {evaluator.metricName: 'areaUnderROC'}))
print(evaluator.evaluate(results,
    {evaluator.metricName: 'areaUnderPR'}))
```

The preceding code prints out the following output:

0.7334857800726642
0.7071651608758281

Well, the model with less features certainly performed worse than the full model, but the difference was not that great. Ultimately, it is a performance trade-off between a more complex model and the less sophisticated one.

Other features of PySpark ML in action

At the beginning of this chapter, we described most of the features of the PySpark ML library. In this section, we will provide examples of how to use some of the Transformers and Estimators.

Feature extraction

We have used quite a few models from this submodule of PySpark. In this section, we'll show you how to use the most useful ones (in our opinion).

NLP - related feature extractors

As described earlier, the NGram model takes a list of tokenized text and produces pairs (or n-grams) of words.

In this example, we will take an excerpt from PySpark's documentation and present how to clean up the text before passing it to the NGram model. Here's how our dataset looks like (abbreviated for brevity):



For the full view of how the following snippet looks like, please download the code from our GitHub repository: <https://github.com/drabastomek/learningPySpark>.

We copied these four paragraphs from the description of the DataFrame usage in Pipelines: <http://spark.apache.org/docs/latest/ml-pipeline.html#dataframe>.

```
text_data = spark.createDataFrame([
    ['Machine learning can be applied to a wide variety
     of data types, such as vectors, text, images, and
     structured data. This API adopts the DataFrame from
     Spark SQL in order to support a variety of data
     types.'],
    (...)
    ['Columns in a DataFrame are named. The code examples
     below use names such as "text," "features," and
     "label."'],
], ['input'])
```

Each row in our single-column DataFrame is just a bunch of text. First, we need to tokenize this text. To do so we will use the `RegexTokenizer` instead of just the `Tokenizer` as we can specify the pattern(s) we want the text to be broken at:

```
tokenizer = ft.RegexTokenizer(
    inputCol='input',
    outputCol='input_arr',
    pattern='\\s+|[,\\.\\"]')
```

The pattern here splits the text on any number of spaces, but also removes commas, full stops, backslashes, and quotation marks. A single row from the output of the tokenizer looks similar to this:

```
Out[35]: [Row(input_arr=['machine', 'learning', 'can', 'be', 'applied', 'to',
'a', 'wide', 'variety', 'of', 'data', 'types', 'such', 'as', 'vectors',
'text', 'images', 'and', 'structured', 'data', 'this', 'api', 'adopts',
'the', 'dataframe', 'from', 'spark', 'sql', 'in', 'order', 'to', 'support',
'a', 'variety', 'of', 'data', 'types'])]
```

As you can see, the `RegexTokenizer` not only splits the sentences into words, but also normalizes the text so each word is in small-caps.

However, there is still plenty of junk in our text: words such as `be`, `a`, or `to` normally provide us with nothing useful when analyzing a text. Thus, we will remove these so called stopwords using nothing else other than the `StopWordsRemover(...)`:

```
stopwords = ft.StopWordsRemover(  
    inputCol=tokenizer.getOutputCol(),  
    outputCol='input_stop')
```

The output of the method looks as follows:

```
Out[37]: [Row(input_stop=['machine', 'learning', 'applied', 'wide', 'variety',  
    'data', 'types', 'vectors', 'text', 'images', 'structured', 'data',  
    'api', 'adopts', 'dataframe', 'spark', 'sql', 'order', 'support',  
    'variety', 'data', 'types'])]
```

Now we only have the useful words. So, let's build our NGram model and the Pipeline:

```
ngram = ft.NGram(n=2,  
    inputCol=stopwords.getOutputCol(),  
    outputCol="nGrams")  
pipeline = Pipeline(stages=[tokenizer, stopwords, ngram])
```

Now that we have the pipeline, we follow in a very similar fashion as before:

```
data_ngram = pipeline \  
    .fit(text_data) \  
    .transform(text_data)  
data_ngram.select('nGrams').take(1)
```

The preceding code produces the following output:

```
Out[39]: [Row(nGrams=['machine learning', 'learning applied', 'applied wide',  
    'wide variety', 'variety data', 'data types', 'types vectors',  
    'vectors text', 'text images', 'images structured', 'structured data',  
    'data api', 'api adopts', 'adopts dataframe', 'dataframe spark',  
    'spark sql', 'sql order', 'order support', 'support variety',  
    'variety data', 'data types'])]
```

That's it. We have got our n-grams and we can now use them in further NLP processing.

Discretizing continuous variables

Ever so often, we deal with a continuous feature that is highly non-linear and really hard to fit in our model with only one coefficient.

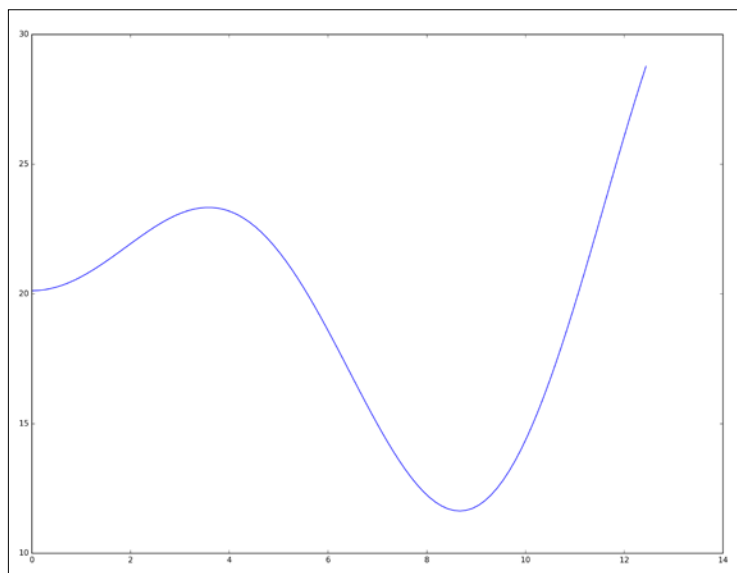
In such a situation, it might be hard to explain the relationship between such a feature and the target with just one coefficient. Sometimes, it is useful to band the values into discrete buckets.

First, let's create some fake data with the help of the following code:

```
import numpy as np
x = np.arange(0, 100)
x = x / 100.0 * np.pi * 4
y = x * np.sin(x / 1.764) + 20.1234
```

Now, we can create a DataFrame by using the following code:

```
schema = typ.StructType([
    typ.StructField('continuous_var',
                    typ.DoubleType(),
                    False
    )
])
data = spark.createDataFrame(
    [[float(e), ] for e in y],
    schema=schema)
```



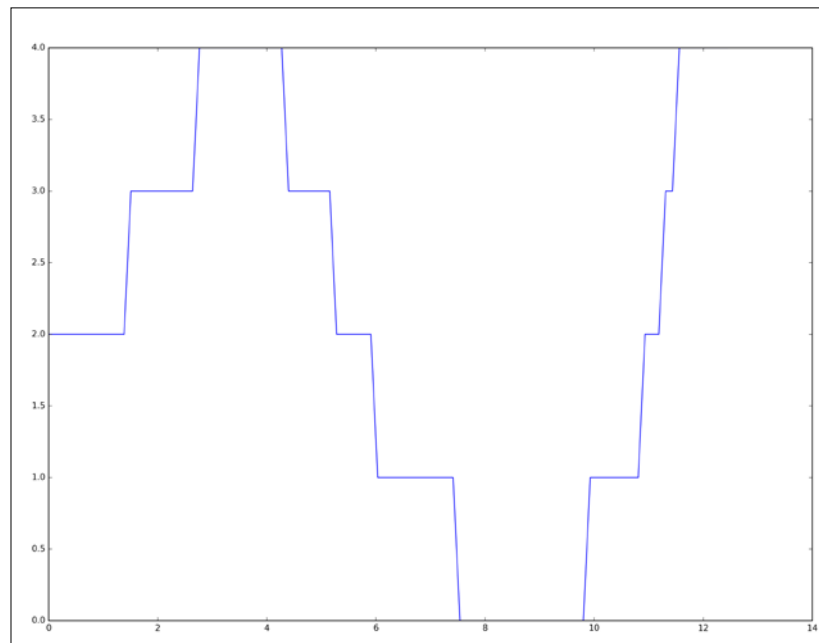
Next, we will use the `QuantileDiscretizer` model to split our continuous variable into five buckets (the `numBuckets` parameter):

```
discretizer = ft.QuantileDiscretizer(  
    numBuckets=5,  
    inputCol='continuous_var',  
    outputCol='discretized')
```

Let's see what we have got:

```
data_discretized = discretizer.fit(data).transform(data)
```

Our function now looks as follows:



We can now treat this variable as categorical and use the `OneHotEncoder` to encode it for future use.

Standardizing continuous variables

Standardizing continuous variables helps not only in better understanding the relationships between the features (as interpreting the coefficients becomes easier), but it also aids computational efficiency and protects from running into some numerical traps. Here's how you do it with PySpark ML.

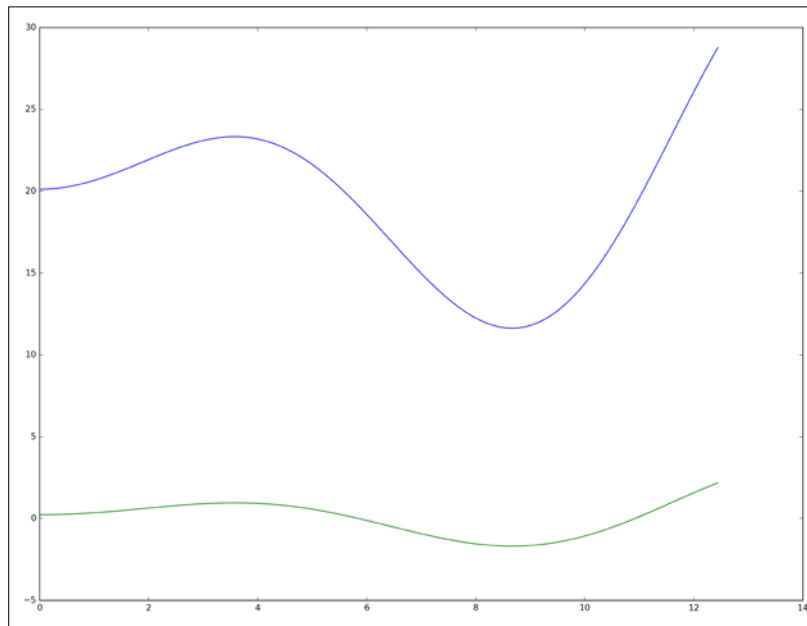
First, we need to create a vector representation of our continuous variable (as it is only a single float):

```
vectorizer = ft.VectorAssembler(  
    inputCols=['continuous_var'],  
    outputCol= 'continuous_vec')
```

Next, we build our normalizer and the pipeline. By setting the `withMean` and `withStd` to `True`, the method will remove the mean and scale the variance to be of unit length:

```
normalizer = ft.StandardScaler(  
    inputCol=vectorizer.getOutputCol(),  
    outputCol='normalized',  
    withMean=True,  
    withStd=True  
)  
pipeline = Pipeline(stages=[vectorizer, normalizer])  
data_standardized = pipeline.fit(data).transform(data)
```

Here's what the transformed data would look like:



As you can see, the data now oscillates around 0 with the unit variance (the green line).

Classification

So far we have only used the `LogisticRegression` model from PySpark ML. In this section, we will use the `RandomForestClassifier` to, once again, model the chances of survival for an infant.

Before we can do that, though, we need to cast the `label` feature to `DoubleType`:

```
import pyspark.sql.functions as func
births = births.withColumn(
    'INFANT_ALIVE_AT_REPORT',
    func.col('INFANT_ALIVE_AT_REPORT').cast(typ.DoubleType())
)
births_train, births_test = births \
    .randomSplit([0.7, 0.3], seed=666)
```

Now that we have the label converted to double, we are ready to build our model. We progress in a similar fashion as before with the distinction that we will reuse the encoder and `featureCreator` from earlier in the chapter. The `numTrees` parameter specifies how many decision trees should be in our random forest, and the `maxDepth` parameter limits the depth of the trees:

```
classifier = cl.RandomForestClassifier(
    numTrees=5,
    maxDepth=5,
    labelCol='INFANT_ALIVE_AT_REPORT')
pipeline = Pipeline(
    stages=[
        encoder,
        featuresCreator,
        classifier])
model = pipeline.fit(births_train)
test = model.transform(births_test)
```

Let's now see how the `RandomForestClassifier` model performs compared to the `LogisticRegression`:

```
evaluator = ev.BinaryClassificationEvaluator(
    labelCol='INFANT_ALIVE_AT_REPORT')
print(evaluator.evaluate(test,
    {evaluator.metricName: "areaUnderROC"}))
print(evaluator.evaluate(test,
    {evaluator.metricName: "areaUnderPR"}))
```

We get the following results:

0.7736428008521183
0.7415879154340478

Well, as you can see, the results are better than the logistic regression model by roughly 3 percentage points. Let's test how well would a model with one tree do:

```
classifier = cl.DecisionTreeClassifier(  
    maxDepth=5,  
    labelCol='INFANT_ALIVE_AT_REPORT')  
pipeline = Pipeline(stages=[  
    encoder,  
    featuresCreator,  
    classifier])  
model = pipeline.fit(births_train)  
test = model.transform(births_test)  
evaluator = ev.BinaryClassificationEvaluator(  
    labelCol='INFANT_ALIVE_AT_REPORT')  
print(evaluator.evaluate(test,  
    {evaluator.metricName: "areaUnderROC"}))  
print(evaluator.evaluate(test,  
    {evaluator.metricName: "areaUnderPR"}))
```

The preceding code gives us the following:

0.7582781726635287
0.7787580540118526

Not bad at all! It actually performed better than the random forest model in terms of the precision-recall relationship and only slightly worse in terms of the area under the ROC. We just might have found a winner!

Clustering

Clustering is another big part of machine learning: quite often, in the real world, we do not have the luxury of having the target feature, so we need to revert to an unsupervised learning paradigm, where we try to uncover patterns in the data.

Finding clusters in the births dataset

In this example, we will use the k-means model to find similarities in the births data:

```
import pyspark.ml.clustering as clus
kmeans = clus.KMeans(k = 5,
    featuresCol='features')
pipeline = Pipeline(stages=[
    assembler,
    featuresCreator,
    kmeans]
)
model = pipeline.fit(births_train)
```

Having estimated the model, let's see if we can find some differences between clusters:

```
test = model.transform(births_test)
test \
    .groupBy('prediction') \
    .agg({
        '*': 'count',
        'MOTHER_HEIGHT_IN': 'avg'
    }).collect()
```

The preceding code produces the following output:

```
Out[58]: [Row(prediction=1, avg(MOTHER_HEIGHT_IN)=66.64658634538152, count(
1)=249),
Row(prediction=3, avg(MOTHER_HEIGHT_IN)=67.69473684210526, count(
1)=475),
Row(prediction=4, avg(MOTHER_HEIGHT_IN)=65.38934651290499, count(
1)=3642),
Row(prediction=2, avg(MOTHER_HEIGHT_IN)=83.91154791154791, count(
1)=407),
Row(prediction=0, avg(MOTHER_HEIGHT_IN)=63.90958873491283, count(
1)=8948)]
```

Well, the MOTHER_HEIGHT_IN is significantly different in cluster 2. Going through the results (which we will not do here for obvious reasons) would most likely uncover more differences and allow us to understand the data better.

Topic mining

Clustering models are not limited to numeric data only. In the field of NLP, problems such as topic extraction rely on clustering to detect documents with similar topics. We will go through such an example.

First, let's create our dataset. The data is formed from randomly selected paragraphs found on the Internet: three of them deal with topics of nature and national parks, the remaining three cover technology.



The code snippet is abbreviated again, for obvious reasons. Refer to the source file on GitHub for full representation.

```
text_data = spark.createDataFrame([
    ['''To make a computer do anything, you have to write a
    computer program. To write a computer program, you have
    to tell the computer, step by step, exactly what you want
    it to do. The computer then "executes" the program,
    following each step mechanically, to accomplish the end
    goal. When you are telling the computer what to do, you
    also get to choose how it's going to do it. That's where
    computer algorithms come in. The algorithm is the basic
    technique used to get the job done. Let's follow an
    example to help get an understanding of the algorithm
    concept.'''],
    (...),
    ['''Australia has over 500 national parks. Over 28
    million hectares of land is designated as national
    parkland, accounting for almost four per cent of
    Australia's land areas. In addition, a further six per
    cent of Australia is protected and includes state
    forests, nature parks and conservation reserves. National
    parks are usually large areas of land that are protected
    because they have unspoilt landscapes and a diverse
    number of native plants and animals. This means that
    commercial activities such as farming are prohibited and
    human activity is strictly monitored.'''],
], ['documents'])
```

First, we will once again use the `RegexTokenizer` and the `StopWordsRemover` models:

```
tokenizer = ft.RegexTokenizer(
    inputCol='documents',
    outputCol='input_arr',
    pattern='\\s+|[,\\.\\"]')
stopwords = ft.StopWordsRemover(
    inputCol=tokenizer.getOutputCol(),
    outputCol='input_stop')
```


Next in our pipeline is the `CountVectorizer`: a model that counts words in a document and returns a vector of counts. The length of the vector is equal to the total number of distinct words in all the documents, which can be seen in the following snippet:

```
stringIndexer = ft.CountVectorizer(  
    inputCol=stopwords.getOutputCol(),  
    outputCol="input_indexed")  
tokenized = stopwords \  
    .transform(  
        tokenizer\  
        .transform(text_data)  
    )  
  
stringIndexer \  
    .fit(tokenized) \  
    .transform(tokenized) \  
    .select('input_indexed') \  
    .take(2)
```

The preceding code will produce the following output:

```
Out[61]: [Row(input_indexed=SparseVector(262, {2: 7.0, 6: 1.0, 8: 3.0, 10:  
3.0, 12: 3.0, 19: 1.0, 20: 1.0, 29: 1.0, 38: 1.0, 39: 2.0, 41: 2.0  
, 44: 1.0, 50: 1.0, 60: 1.0, 65: 1.0, 87: 1.0, 108: 1.0, 110: 1.0,  
112: 1.0, 114: 1.0, 116: 1.0, 139: 1.0, 149: 1.0, 150: 1.0, 162: 1  
.0, 181: 1.0, 182: 1.0, 190: 1.0, 193: 1.0, 218: 1.0, 226: 1.0, 23  
0: 1.0, 232: 1.0, 249: 1.0, 251: 1.0, 256: 1.0})),  
Row(input_indexed=SparseVector(262, {20: 1.0, 21: 1.0, 22: 2.0, 3  
2: 2.0, 33: 2.0, 36: 2.0, 48: 1.0, 49: 1.0, 55: 1.0, 63: 1.0, 72:  
1.0, 73: 1.0, 77: 1.0, 83: 1.0, 88: 1.0, 90: 1.0, 93: 1.0, 102: 1.  
0, 105: 1.0, 111: 1.0, 122: 1.0, 128: 1.0, 130: 1.0, 140: 1.0, 145  
: 1.0, 146: 1.0, 170: 1.0, 173: 1.0, 195: 1.0, 196: 1.0, 202: 1.0,  
203: 1.0, 207: 1.0, 209: 1.0, 212: 1.0, 213: 1.0, 216: 1.0, 221: 1  
.0, 224: 1.0, 225: 1.0, 228: 1.0, 231: 1.0, 237: 1.0, 241: 1.0, 24  
6: 1.0, 247: 1.0, 255: 1.0, 260: 1.0})))]
```

As you can see, there are 262 distinct words in the text, and each document is now represented by a count of each word occurrence.

It's now time to start predicting the topics. For that purpose we will use the LDA model – the **Latent Dirichlet Allocation** model:

```
clustering = clus.LDA(k=2,  
    optimizer='online',  
    featuresCol=stringIndexer.getOutputCol())
```

The `k` parameter specifies how many topics we expect to see, the `optimizer` parameter can be either `'online'` or `'em'` (the latter standing for the Expectation Maximization algorithm).

Putting these puzzles together results in, so far, the longest of our pipelines:

```
pipeline = ml.Pipeline(stages=[
    tokenizer,
    stopwords,
    stringIndexer,
    clustering]
)
```

Have we properly uncovered the topics? Well, let's see:

```
topics = pipeline \
    .fit(text_data) \
    .transform(text_data)
topics.select('topicDistribution').collect()
```

Here's what we get:

```
Out[65]: [Row(topicDistribution=DenseVector([0.0221, 0.9779])),
Row(topicDistribution=DenseVector([0.0171, 0.9829])),
Row(topicDistribution=DenseVector([0.0199, 0.9801])),
Row(topicDistribution=DenseVector([0.9923, 0.0077])),
Row(topicDistribution=DenseVector([0.9925, 0.0075])),
Row(topicDistribution=DenseVector([0.9904, 0.0096]))]
```

Looks like our method discovered all the topics properly! Do not get used to seeing such good results though: sadly, real world data is seldom that kind.

Regression

We could not finish a chapter on a machine learning library without building a regression model.

In this section, we will try to predict the `MOTHER_WEIGHT_GAIN` given some of the features described here; these are contained in the features listed here:

```
features = ['MOTHER_AGE_YEARS', 'MOTHER_HEIGHT_IN',
'MOTHER_PRE_WEIGHT', 'DIABETES_PRE',
'DIABETES_GEST', 'HYP_TENS_PRE',
'HYP_TENS_GEST', 'PREV_BIRTH_PRETERM',
'CIG_BEFORE', 'CIG_1_TRI', 'CIG_2_TRI',
'CIG_3_TRI'
]
```

First, since all the features are numeric, we will collate them together and use the `ChiSqSelector` to select only the top six most important features:

```
featuresCreator = ft.VectorAssembler(  
    inputCols=[col for col in features[1:]],  
    outputCol='features'  
)  
selector = ft.ChiSqSelector(  
    numTopFeatures=6,  
    outputCol="selectedFeatures",  
    labelCol='MOTHER_WEIGHT_GAIN'  
)
```

In order to predict the weight gain, we will use the gradient boosted trees regressor:

```
import pyspark.ml.regression as reg  
regressor = reg.GBTRegressor(  
    maxIter=15,  
    maxDepth=3,  
    labelCol='MOTHER_WEIGHT_GAIN')
```

Finally, again, we put it all together into a Pipeline:

```
pipeline = Pipeline(stages=[  
    featuresCreator,  
    selector,  
    regressor])  
weightGain = pipeline.fit(births_train)
```

Having created the `weightGain` model, let's see if it performs well on our testing data:

```
evaluator = ev.RegressionEvaluator(  
    predictionCol="prediction",  
    labelCol='MOTHER_WEIGHT_GAIN')  
print(evaluator.evaluate(  
    weightGain.transform(births_test),  
    {evaluator.metricName: 'r2'}))
```

We get the following output:

0.48862170400240335

Sadly, the model is no better than a flip of a coin. It looks that without additional independent features that are better correlated with the `MOTHER_WEIGHT_GAIN` label, we will not be able to explain its variance sufficiently.

Summary

In this chapter, we went into details of how to use PySpark ML: the official main machine learning library for PySpark. We explained what the `Transformer` and `Estimator` are, and showed their role in another concept introduced in the ML library: the `Pipeline`. Subsequently, we also presented how to use some of the methods to fine-tune the hyper parameters of models. Finally, we gave some examples of how to use some of the feature extractors and models from the library.

In the next chapter, we will delve into graph theory and `GraphFrames` that help in tackling machine learning problems better represented as graphs.

7

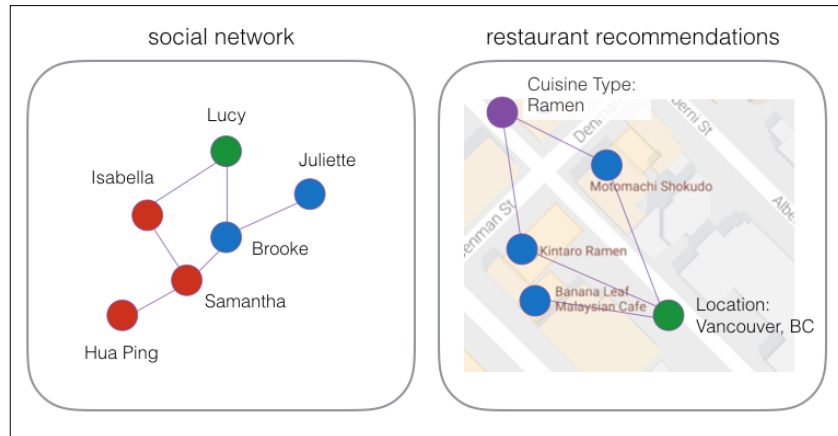
GraphFrames

Graphs are an interesting way to solve data problems because graph structures are a more intuitive approach to many classes of data problems.

In this chapter, you will learn about:

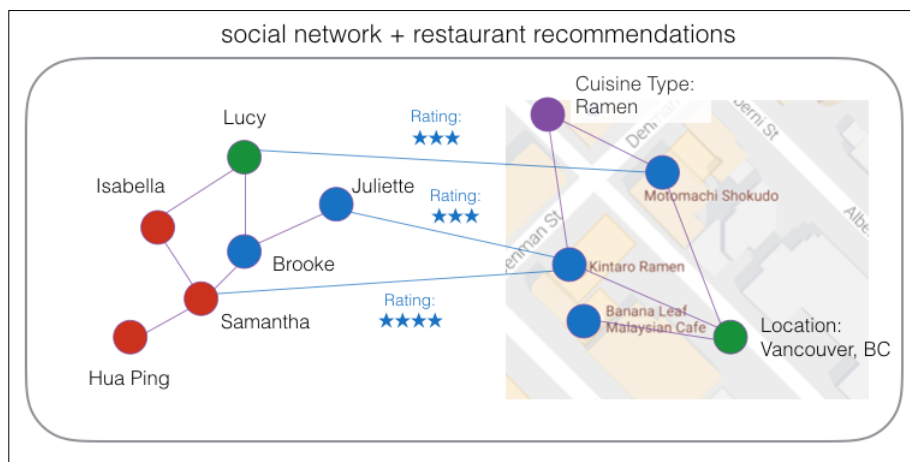
- Why use graphs?
- Understanding the classic graph problem: the flights dataset
- Understanding the graph vertices and edges
- Simple queries
- Using motif finding
- Using breadth first search
- Using PageRank
- Visualizing flights using D3

Whether traversing social networks or restaurant recommendations, it is easier to understand these data problems within the context of graph structures: vertices, edges, and properties:

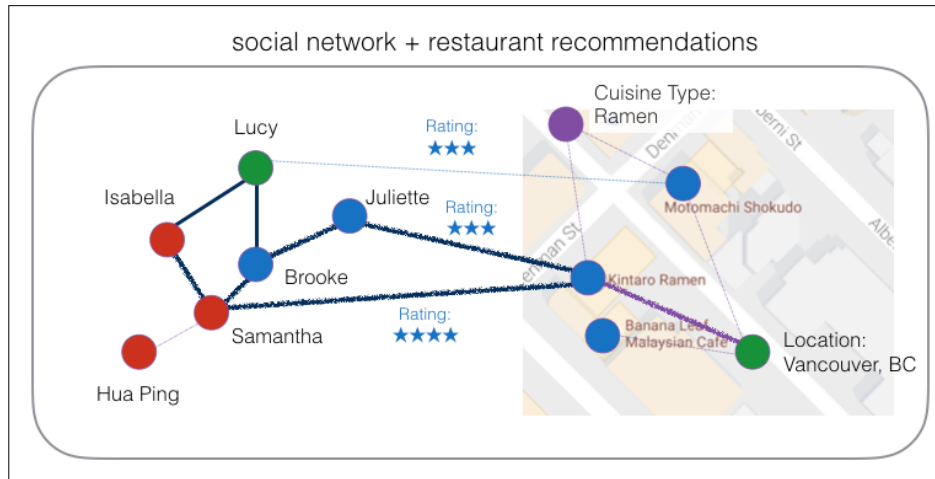


For example, within the context of **social networks**, the *vertices* are the people while the *edges* are the connections between them. Within the context of **restaurant recommendations**, the vertices (for example) involve the location, cuisine type, and restaurants while the edges are the connections between them (for example, these three restaurants are in **Vancouver, BC**, but only two of them serve ramen).

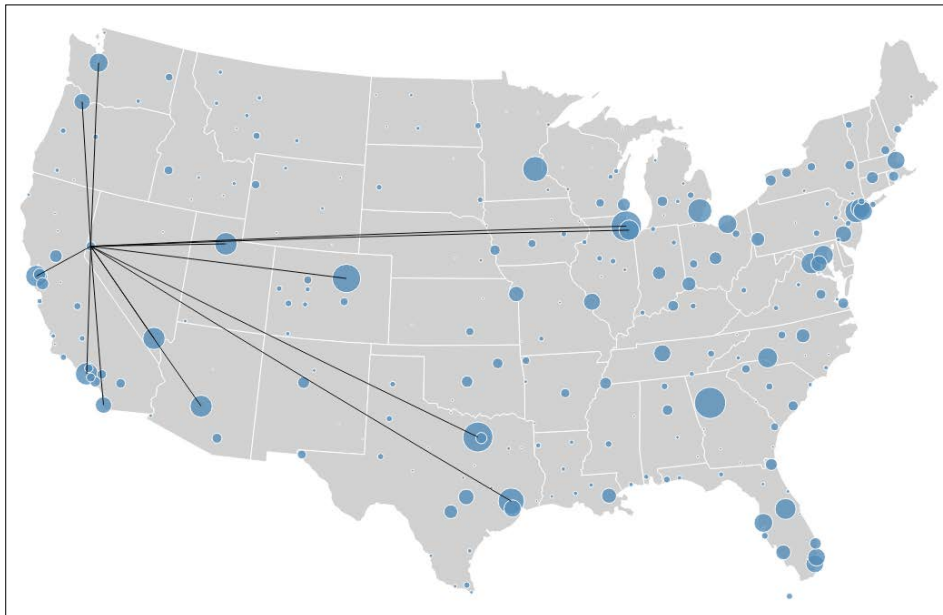
While the two graphs are seemingly disconnected, you can in fact create a social network + restaurant recommendation graph based on the reviews of friends within a social circle, as noted in the following figure:



For example, if **Isabella** wants to find a great ramen restaurant in Vancouver, traversing her friends' reviews, she will most likely choose **Kintaro Ramen**, as both **Samantha** and **Juliette** have rated the restaurant favorably:



Another classic graph problem is the analysis of flight data: airports are represented by *vertices* and flights between those airports are represented by *edges*. Also, there are numerous *properties* associated with these flights, including, but not limited to, departure delays, plane type, and carrier:



In this chapter, we will use GraphFrames to quickly and easily analyze flight performance data organized in graph structures. Because we're using graph structures, we can easily ask many questions that are not as intuitive as tabular structures, such as finding structural motifs, airport ranking using PageRank, and shortest paths between cities. GraphFrames leverages the distribution and expression capabilities of the DataFrame API to both simplify your queries and leverage the performance optimizations of the Apache Spark SQL engine.

In addition, with GraphFrames, graph analysis is available in Python, Scala, and Java. Just as important, you can leverage your existing Apache Spark skills to solve graph problems (in addition to machine learning, streaming, and SQL) instead of making a paradigm shift to learn a new framework.

Introducing GraphFrames

GraphFrames utilizes the power of Apache Spark DataFrames to support general graph processing. Specifically, the vertices and edges are represented by DataFrames allowing us to store arbitrary data with each vertex and edge. While GraphFrames is similar to Spark's GraphX library, there are some key differences, including:

- GraphFrames leverage the performance optimizations and simplicity of the DataFrame API.
- By using the DataFrame API, GraphFrames now have Python, Java, and Scala APIs. GraphX is only accessible through Scala; now all its algorithms are available in Python and Java.
- Note, at the time of writing, there was a bug preventing GraphFrames from working with Python3.x, hence we will be using Python2.x.

At the time of writing, GraphFrames is on version 0.3 and available as a Spark package (<http://spark-packages.org>) at <https://spark-packages.org/package/graphframes/graphframes>.



For more information about GraphFrames, please refer to *Introducing GraphFrames* at <https://databricks.com/blog/2016/03/03/introducing-graphframes.html>.

Installing GraphFrames

If you are running your job from a Spark CLI (for example, `spark-shell`, `pyspark`, `spark-sql`, `spark-submit`), you can use the `--packages` command, which will extract, compile, and execute the necessary code for you to use the GraphFrames package.

For example, to use the latest GraphFrames package (version 0.3) with Spark 2.0 and Scala 2.11 with spark-shell, the command is:

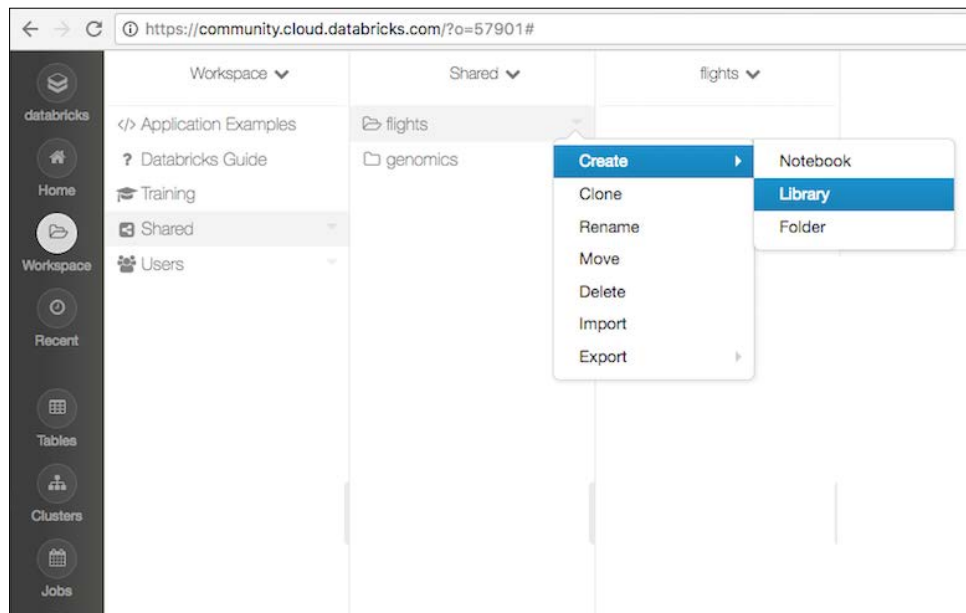
```
> $SPARK_HOME/bin/spark-shell --packages graphframes:graphframes:0.3.0-spark2.0-s_2.11
```

If you are using a notebook service, you may need to install the package first. For example, the following section shows the steps to install the GraphFrames library within the free Databricks Community Edition (<http://databricks.com/try-databricks>).

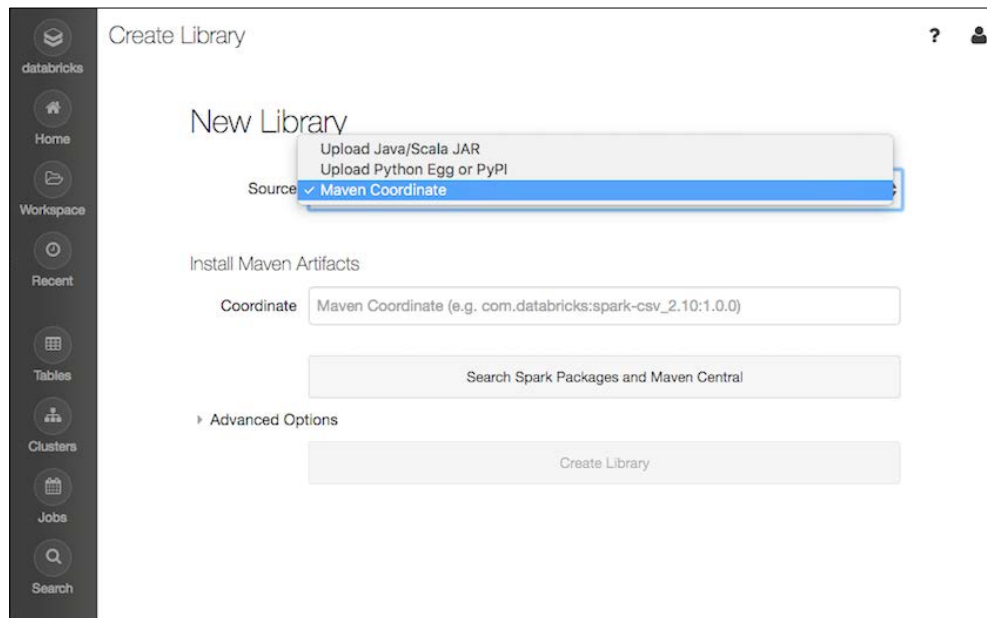
Creating a library

Within Databricks, you can create a library that is comprised of a Scala/Java JAR, Python Egg, or Maven Coordinate (including the Spark package).

To start, go to your **Workspace** within **databricks**, right-click the folder you want to create the library in (in this case, **flights**), click **Create**, and then click **Library**:



In the **Create Library** dialog, choose within the **Source** dropdown, **Maven Coordinate** as noted in the following diagram:



Maven is a tool that is used to build and manage Java-based projects such as the GraphFrames project. Maven coordinates uniquely identify those projects (or dependencies or plug-ins) so you can quickly find the project within a Maven repository; for example, <https://mvnrepository.com/artifact/graphframes/graphframes>.

From here, you can click the **Search Spark Packages and Maven Central** button and search for the GraphFrames package. Ensure that you match the GraphFrames version of Spark (for example, Spark 2.0) and Scala (for example, Scala 2.11) with your Spark cluster.

You can also enter the Maven coordinate for the GraphFrames Spark package if you already know it. For Spark 2.0 and Scala 2.11, enter the following coordinate:

```
graphframes:graphframes:0.3.0-spark2.0-s_2.11
```

Once entered, click on **Create Library**, as noted in the following screenshot:

Source: Maven Coordinate

Install Maven Artifacts

Coordinate: graphframes:graphframes:0.2.0-spark2.0-s_2.11

Search Spark Packages and Maven Central

Advanced Options

Create Library

Note that this is a one-time installation task for the GraphFrames Spark package (as part of a library). Once it is installed, you can by default automatically auto-attach the package to any Databricks cluster that you create:

graphframes-0.2.0-spark2.0-s_2.11

graphframes-0.2.0-spark2.0-s_2.11 | Delete

Artifacts

- graphframes-0.2.0-spark2.0-s_2.11.jar
- scala-logging-api_2.11-2.1.2.jar
- scala-logging-slf4j_2.11-2.1.2.jar
- slf4j-api-1.7.7.jar

Clusters

☒ Attach automatically to all clusters.

Attach	Name	Status
--------	------	--------

Preparing your flights dataset

For this flights sample scenario, we will make use of two sets of data:

- *Airline On-Time Performance and Causes of Flight Delays*: [<http://bit.ly/2ccJPPM>] This dataset contains scheduled and actual departure and arrival times, and delay causes as reported by US air carriers. The data is collected by the Office of Airline Information, **Bureau of Transportation Statistics (BTS)**.
- *Open Flights: Airports and airline data*: [<http://openflights.org/data.html>] This dataset contains the list of US airport data including the IATA code, airport name, and airport location.

We will create two DataFrames – airports and departureDelays—which will make up our **vertices** and **edges** of our GraphFrame, respectively. We will be creating this flights sample application using Python.

As we are using a Databricks notebook for our example, we can make use of the `/databricks-datasets/location`, which contains numerous sample datasets. You can also download the data from:

- `departureDelays.csv`: <http://bit.ly/2ejPr8k>
- `airportCodes`: <http://bit.ly/2ePAdKT>

In this example, we are creating two variables denoting the file paths for our Airports and Departure Delays data, respectively. Then we will load these datasets and create the respective Spark DataFrames; note for both of these files, we can easily infer the schema:

```
# Set File Paths
tripdelaysFilePath = "/databricks-datasets/flights/departuredelays.csv"
airportsnaFilePath = "/databricks-datasets/flights/airport-codes-na.txt"

# Obtain airports dataset
# Note, this dataset is tab-delimited with a header
airportsna = spark.read.csv(airportsnaFilePath, header='true',
inferSchema='true', sep='\t')
airportsna.createOrReplaceTempView("airports_na")

# Obtain departure Delays data
# Note, this dataset is comma-delimited with a header
departureDelays = spark.read.csv(tripdelaysFilePath, header='true')
departureDelays.createOrReplaceTempView("departureDelays")
departureDelays.cache()
```

Once we loaded the `departureDelays` DataFrame, we also cache it so we can include some additional filtering of the data in a performant manner:

```
# Available IATA codes from the departedelays sample dataset
tripIATA = spark.sql("select distinct iata from (select distinct
origin as iata from departureDelays union all select distinct
destination as iata from departureDelays) a")
tripIATA.createOrReplaceTempView("tripIATA")
```

The preceding query allows us to build a distinct list with origin city IATA codes (for example, Seattle = 'SEA', San Francisco = 'SFO', New York JFK = 'JFK', and so on). Next, we only include airports that had a trip occur within the `departureDelays` DataFrame:

```
# Only include airports with atleast one trip from the
# `departureDelays` dataset
airports = spark.sql("select f.IATA, f.City, f.State, f.Country from
airports_na f join tripIATA t on t.IATA = f.IATA")
airports.createOrReplaceTempView("airports")
airports.cache()
```

By building the distinct list of origin airport codes, we can build the airports DataFrame to contain only the airport codes that exist in the `departureDelays` dataset. The following code snippet generates a new DataFrame (`departureDelays_geo`) that is comprised of key attributes including date of flight, delays, distance, and airport information (origin, destination):

```
# Build `departureDelays_geo` DataFrame
# Obtain key attributes such as Date of flight, delays, distance,
# and airport information (Origin, Destination)
departureDelays_geo = spark.sql("select cast(f.date as int) as
tripid, cast(concat(concat(concat(concat('2014-',
concat(concat(substr(cast(f.date as string), 1, 2), '-')),
substr(cast(f.date as string), 3, 2)), ''), substr(cast(f.date as
string), 5, 2)), ':'), substr(cast(f.date as string), 7, 2)), ':00')
as timestamp) as `localdate`, cast(f.delay as int), cast(f.distance
as int), f.origin as src, f.destination as dst, o.city as city_src,
d.city as city_dst, o.state as state_src, d.state as state_dst from
departuredelays f join airports o on o.iata = f.origin join airports d
on d.iata = f.destination")

# Create Temporary View and cache
departureDelays_geo.createOrReplaceTempView("departureDelays_geo")
departureDelays_geo.cache()
```

To take a quick peek into this data, you can run the `show` method as shown here:

```
# Review the top 10 rows of the `departureDelays_geo` DataFrame
departureDelays_geo.show(10)
```

▶ (2) Spark Jobs

tripid	localdate	delay	distance	src	dst	city_src	city_dst	state_src	state_dst
1011111	2014-01-01 11:11:...	-5	221	MSP	INL	Minneapolis	International Falls	MN	MN
1021111	2014-01-02 11:11:...	7	221	MSP	INL	Minneapolis	International Falls	MN	MN
1031111	2014-01-03 11:11:...	0	221	MSP	INL	Minneapolis	International Falls	MN	MN
1041925	2014-01-04 19:25:...	0	221	MSP	INL	Minneapolis	International Falls	MN	MN
1061115	2014-01-06 11:15:...	33	221	MSP	INL	Minneapolis	International Falls	MN	MN
1071115	2014-01-07 11:15:...	23	221	MSP	INL	Minneapolis	International Falls	MN	MN
1081115	2014-01-08 11:15:...	-9	221	MSP	INL	Minneapolis	International Falls	MN	MN
1091115	2014-01-09 11:15:...	11	221	MSP	INL	Minneapolis	International Falls	MN	MN
1101115	2014-01-10 11:15:...	-3	221	MSP	INL	Minneapolis	International Falls	MN	MN
112015	2014-01-11 20:15:...	-7	221	MSP	INL	Minneapolis	International Falls	MN	MN

only showing top 10 rows

Building the graph

Now that we've imported our data, let's build our graph. To do this, we're going to build the structure for our vertices and edges. At the time of writing, GraphFrames requires a specific naming convention for vertices and edges:

- The column representing the *vertices* needs to have the name `id`. In our case, the vertices of our flight data are the airports. Therefore, we will need to rename the IATA airport code to `id` in our `airports` DataFrame.
- The columns representing the *edges* need to have a source (`src`) and destination (`dst`). For our flight data, the edges are the flights, therefore the `src` and `dst` are the origin and destination columns from the `departureDelays_geo` DataFrame.

To simplify the edges for our graph, we will create the `tripEdges` DataFrame with a subset of the columns available within the `departureDelays_Geo` DataFrame. As well, we created a `tripVertices` DataFrame that simply renames the `IATA` column to `id` to match the GraphFrame naming convention:

```
# Note, ensure you have already installed
# the GraphFrames spark-package
from pyspark.sql.functions import *
from graphframes import *
```

```
# Create Vertices (airports) and Edges (flights)
tripVertices = airports.withColumnRenamed("IATA", "id").distinct()
tripEdges = departureDelays_geo.select("tripid", "delay", "src",
"dst", "city_dst", "state_dst")

# Cache Vertices and Edges
tripEdges.cache()
tripVertices.cache()
```

Within Databricks, you can query the data using the `display` command. For example, to view the `tripEdges` DataFrame, the command is as follows:

```
display(tripEdges)
```

The output is as follows:

▶ (2) Spark Jobs

tripid	delay	src	dst	city_dst	state_dst
1011111	-5	MSP	INL	International Falls	MN
1021111	7	MSP	INL	International Falls	MN
1031111	0	MSP	INL	International Falls	MN
1041925	0	MSP	INL	International Falls	MN
1061115	33	MSP	INL	International Falls	MN
1071115	23	MSP	INL	International Falls	MN
1081115	-9	MSP	INL	International Falls	MN
1091115	11	MSP	INL	International Falls	MN
1101115	-3	MSP	INL	International Falls	MN

Showing the first 1000 rows.

Now that we have the two DataFrames, we can create a GraphFrame using the `GraphFrame` command:

```
tripGraph = GraphFrame(tripVertices, tripEdges)
```

Executing simple queries

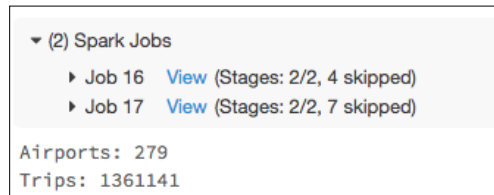
Let's start with a set of simple graph queries to understand flight performance and departure delays.

Determining the number of airports and trips

For example, to determine the number of airports and trips, you can run the following commands:

```
print "Airports: %d" % tripGraph.vertices.count()
print "Trips: %d" % tripGraph.edges.count()
```

As you can see from the results, there are 279 airports with 1.36 million trips:



Determining the longest delay in this dataset

To determine the longest delayed flight in the dataset, you can run the following query with the result of 1,642 minutes (that's more than 27 hours!):

```
tripGraph.edges.groupBy().max("delay")
```

```
# Output
```

```
+-----+
|max(delay)|
+-----+
|      1642|
+-----+
```

Determining the number of delayed versus on-time/early flights

To determine the number of delayed versus on-time (or early) flights, you can run the following queries:

```
print "On-time / Early Flights: %d" % tripGraph.edges.filter("delay <= 0").count()
print "Delayed Flights: %d" % tripGraph.edges.filter("delay > 0").count()
```

with the results nothing that almost 43% of the flights were delayed!

▼ (2) Spark Jobs	
▶ Job 18	View (Stages: 2/2, 7 skipped)
▶ Job 19	View (Stages: 2/2, 7 skipped)
On-time / Early Flights: 780469	
Delayed Flights: 580672	

What flights departing Seattle are most likely to have significant delays?

Digging further in this data, let's find out the top five destinations for flights departing from Seattle that are most likely to have significant delays. This can be achieved through the following query:

```
tripGraph.edges\
  .filter("src = 'SEA' and delay > 0")\
  .groupBy("src", "dst")\
  .avg("delay")\
  .sort(desc("avg(delay)"))\
  .show(5)
```

As you can see in the following results: Philadelphia (PHL), Colorado Springs (COS), Fresno (FAT), Long Beach (LGB), and Washington D.C (IAD) are the top five cities with flights delayed originating from Seattle:

```

▶ (1) Spark Jobs
+-----+
|src|dst|          avg(delay)|
+-----+
|SEA|PHL|55.666666666666664|
|SEA|COS| 43.53846153846154|
|SEA|FAT| 43.03846153846154|
|SEA|LGB| 39.39705882352941|
|SEA|IAD|37.733333333333334|
+-----+
only showing top 5 rows

```

What states tend to have significant delays departing from Seattle?

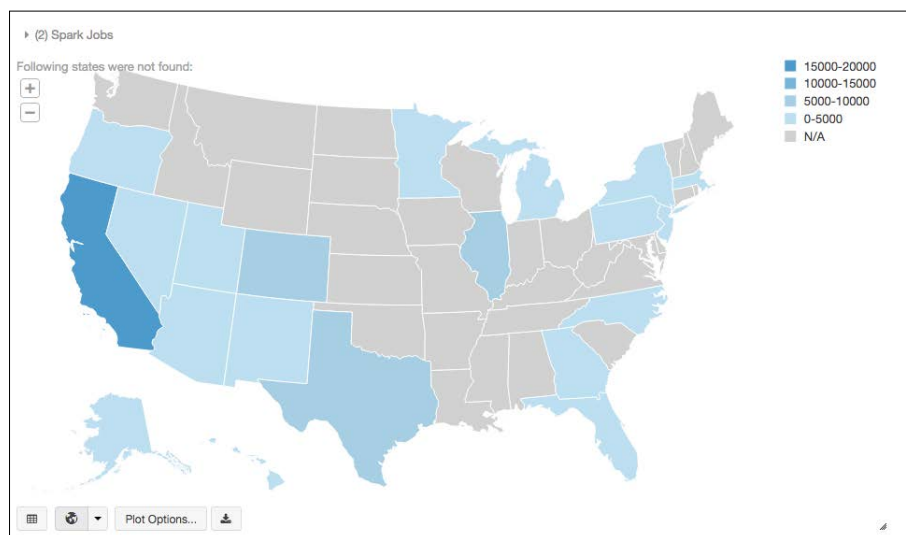
Let's find which states have the longest cumulative delays (with individual delays > 100 minutes) originating from Seattle. This time we will use the `display` command to review the data:

```
# States with the longest cumulative delays (with individual
# delays > 100 minutes) (origin: Seattle)
display(tripGraph.edges.filter("src = 'SEA' and delay > 100"))
```

► (2) Spark Jobs

tripid	delay	src	dst	city_dst	state_dst
3201938	108	SEA	BUR	Burbank	CA
3201655	107	SEA	SNA	Orange County	CA
1011950	123	SEA	OAK	Oakland	CA
1021950	194	SEA	OAK	Oakland	CA
1021615	317	SEA	OAK	Oakland	CA
1021755	385	SEA	OAK	Oakland	CA
1031950	283	SEA	OAK	Oakland	CA
1031615	364	SEA	OAK	Oakland	CA
1031325	130	SEA	OAK	Oakland	CA
1061755	107	SEA	OAK	Oakland	CA

Using the Databricks `display` command, we can also quickly change from this table view to a map view of the data. As can be seen in the following figure, the state with the most cumulative delays originating from Seattle (in this dataset) is California:

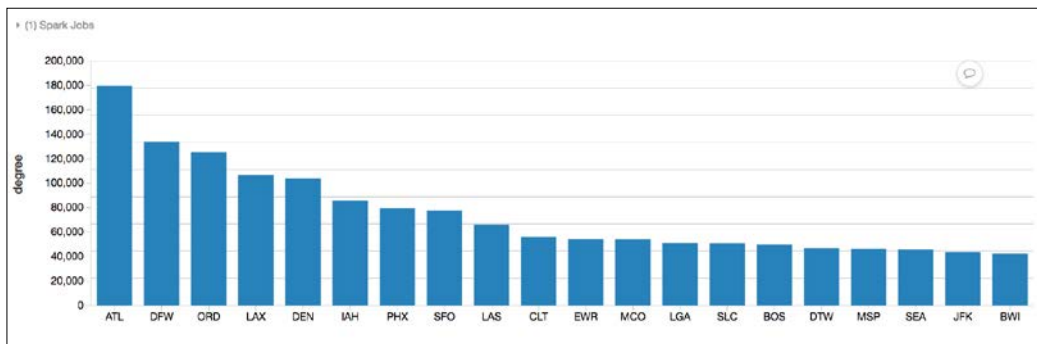


Understanding vertex degrees

Within the context of graph theory, the degrees around a vertex are the number of edges around the vertex. In our *flights* example, the degrees are then the total number of edges (that is, flights) to the vertex (that is, airports). Therefore, if we were to obtain the top 20 vertex degrees (in descending order) from our graph, then we would be asking for the top 20 busiest airports (most flights in and out) from our graph. This can be quickly determined using the following query:

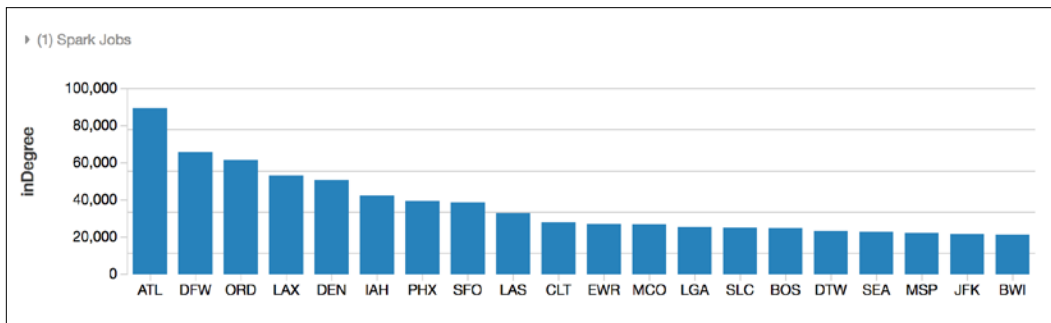
```
display(tripGraph.degrees.sort(desc("degree")).limit(20))
```

Because we're using the `display` command, we can quickly view a bar graph of this data:



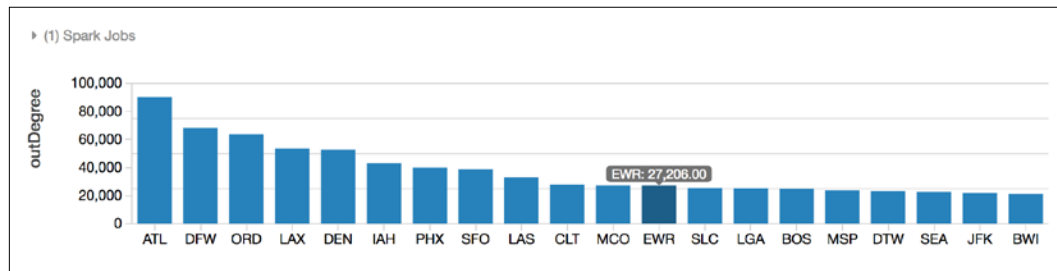
Diving into more details, here are the top 20 `inDegrees` (that is, incoming flights):

```
display(tripGraph.inDegrees.sort(desc("inDegree")).limit(20))
```



While here are the top 20 outDegrees (that is, outgoing flights):

```
display(tripGraph.outDegrees.sort(desc("outDegree")).limit(20))
```



Interestingly, while the top 10 airports (Atlanta/ATL to Charlotte/CLT) are ranked the same for incoming and outgoing flights, the ranks of the next 10 airports change (for example, Seattle/SEA is 17th for incoming flights, but 18th for outgoing).

Determining the top transfer airports

An extension of understanding vertex degrees for airports is to determine the top transfer airports. Many airports are used as transfer points instead of being the final destination. An easy way to calculate this is by calculating the ratio of `inDegrees` (the number of flights to the airport) and / `outDegrees` (the number of flights leaving the airport). Values close to 1 may indicate many transfers, whereas values <1 indicate many outgoing flights and values >1 indicate many incoming flights.

Note that this is a simple calculation that does not consider timing or scheduling of flights, just the overall aggregate number within the dataset:

```
# Calculate the inDeg (flights into the airport) and
# outDeg (flights leaving the airport)
inDeg = tripGraph.inDegrees
outDeg = tripGraph.outDegrees

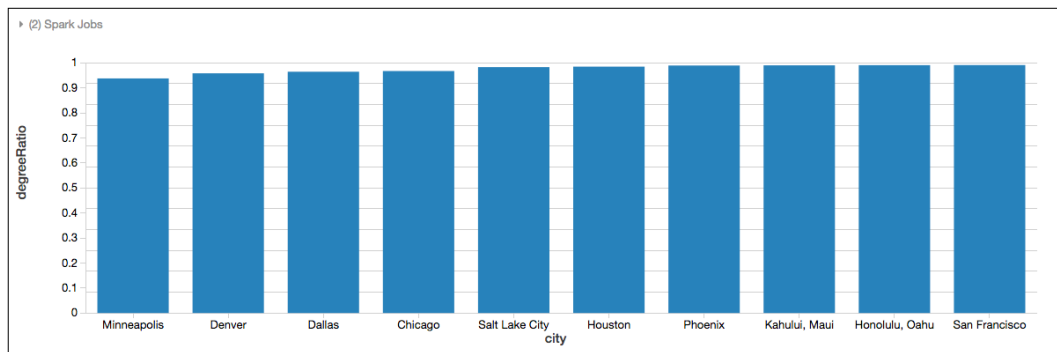
# Calculate the degreeRatio (inDeg/outDeg)
degreeRatio = inDeg.join(outDeg, inDeg.id == outDeg.id) \
    .drop(outDeg.id) \
    .selectExpr("id", "double(inDegree)/double(outDegree) as
degreeRatio") \
    .cache()

# Join back to the 'airports' DataFrame
# (instead of registering temp table as above)
```

```
transferAirports = degreeRatio.join(airports, degreeRatio.id ==
airports.IATA) \
  .selectExpr("id", "city", "degreeRatio") \
  .filter("degreeRatio between 0.9 and 1.1")

# List out the top 10 transfer city airports
display(transferAirports.orderBy("degreeRatio").limit(10))
```

The output of this query is a bar chart of the top 10 transfer city airports (that is, hub airports):



This makes sense since these airports are major hubs for national airlines (for example, Delta uses **Minneapolis** and **Salt Lake City** as its hub, Frontier uses **Denver**, American uses **Dallas** and **Phoenix**, United uses **Houston**, **Chicago**, and **San Francisco**, and Hawaiian Airlines uses **Kahului** and **Honolulu** as its hubs).

Understanding motifs

To easily understand the complex relationship of city airports and the flights between each other, we can use `motifs` to find patterns of airports (for example, vertices) connected by flights (that is, edges). The result is a `DataFrame` in which the column names are given by the motif keys. Note that motif finding is one of the new graph algorithms supported as part of `GraphFrames`.

For example, let's determine the delays that are due to **San Francisco International Airport (SFO)**:

```
# Generate motifs
motifs = tripGraphPrime.find("(a)-[ab]->(b); (b)-[bc]->(c)") \
  .filter("(b.id = 'SFO') and (ab.delay > 500 or bc.delay > 500) and
bc.tripid > ab.tripid and bc.tripid < ab.tripid + 10000")
```

```
# Display motifs
display(motifs)
```

Breaking down the preceding query, the `(x)` represents the vertex (that is, airport) while the `[xy]` represents the edge (that is, flights between airports). Therefore, to determine the delays that are due to SFO, use the following:

- The vertex `(b)` represents the airport in the middle (that is, SFO)
- The vertex `(a)` represents the origin airport (within the dataset)
- The vertex `(c)` represents the destination airport (within the dataset)
- The edge `[ab]` represents the flight between `(a)` (that is, origin) and `(b)` (that is, SFO)
- The edge `[bc]` represents the flight between `(b)` (that is, SFO) and `(c)` (that is, destination)

Within the `filter` statement, we put in some rudimentary constraints (note that this is an over simplistic representation of flight paths):

- `b.id = 'SFO'` denotes that the middle vertex `(b)` is limited to just SFO airport
- `(ab.delay > 500 or bc.delay > 500)` denotes that we are limited to flights that have delays greater than 500 minutes
- `(bc.tripid > ab.tripid and bc.tripid < ab.tripid + 10000)` denotes that the `(ab)` flight must be before the `(bc)` trip and within the same day. The `tripid` was derived from the date time, thus explaining why it could be simplified this way

The output of this query is noted in the following figure:

► (8) Spark Jobs

a	ab	b	bc	c
[MSY,New Orleans,...]	[1011751,-4,MSY,SFO]	[SFO,San Francisc...]	[1021507,536,SFO,...]	[JFK,New York,NY,...]
[MSY,New Orleans,...]	[1201725,2,MSY,SFO]	[SFO,San Francisc...]	[1211508,593,SFO,...]	[JFK,New York,NY,...]
[MSY,New Orleans,...]	[2091725,87,MSY,SFO]	[SFO,San Francisc...]	[2092110,740,SFO,...]	[MIA,Miami,FL,USA]
[MSY,New Orleans,...]	[2091725,87,MSY,SFO]	[SFO,San Francisc...]	[2092230,636,SFO,...]	[JFK,New York,NY,...]
[MSY,New Orleans,...]	[2121725,15,MSY,SFO]	[SFO,San Francisc...]	[2131420,504,SFO,...]	[SAN,San Diego,CA...]
[BUR,Burbank,CA,USA]	[1011828,88,BUR,SFO]	[SFO,San Francisc...]	[1021507,536,SFO,...]	[JFK,New York,NY,...]

The following is a simplified abridged subset from this query where the columns are the respective motif keys:

a	ab	b	bc	c
Houston (IAH)	IAH -> SFO (-4) [1011126]	San Francisco (SFO)	SFO -> JFK (536) [1021507]	New York (JFK)
Tuscon (TUS)	TUS -> SFO (-5) [1011126]	San Francisco (SFO)	SFO -> JFK (536) [1021507]	New York (JFK)

Referring to the TUS > SFO > JFK flight, you will notice that while the flight from Tuscon to San Francisco departed 5 minutes early, the flight from San Francisco to New York JFK was delayed by 536 minutes.

By using motif finding, you can easily search for structural patterns in your graph; by using GraphFrames, you are using the power and speed of DataFrames to distribute and perform your query.

Determining airport ranking using PageRank

Because GraphFrames is built on top of GraphX, there are several algorithms that we can immediately leverage. PageRank was popularized by the Google Search Engine and created by Larry Page. To quote Wikipedia:

"PageRank works by counting the number and quality of links to a page to determine a rough estimate of how important the website is. The underlying assumption is that more important websites are likely to receive more links from other websites."

While the preceding example refers to web pages, this concept readily applies to any graph structure whether it is created from web pages, bike stations, or airports. Yet the interface via GraphFrames is as simple as calling a method. `GraphFrames.PageRank` will return the PageRank results as a new column appended to the *vertices* DataFrame to simplify our downstream analysis.

As there are many flights and connections through the various airports included in this dataset, we can use the PageRank algorithm to have Spark traverse the graph iteratively to compute a rough estimate of how important each airport is:

```
# Determining Airport ranking of importance using 'pageRank'
ranks = tripGraph.pageRank(resetProbability=0.15, maxIter=5)

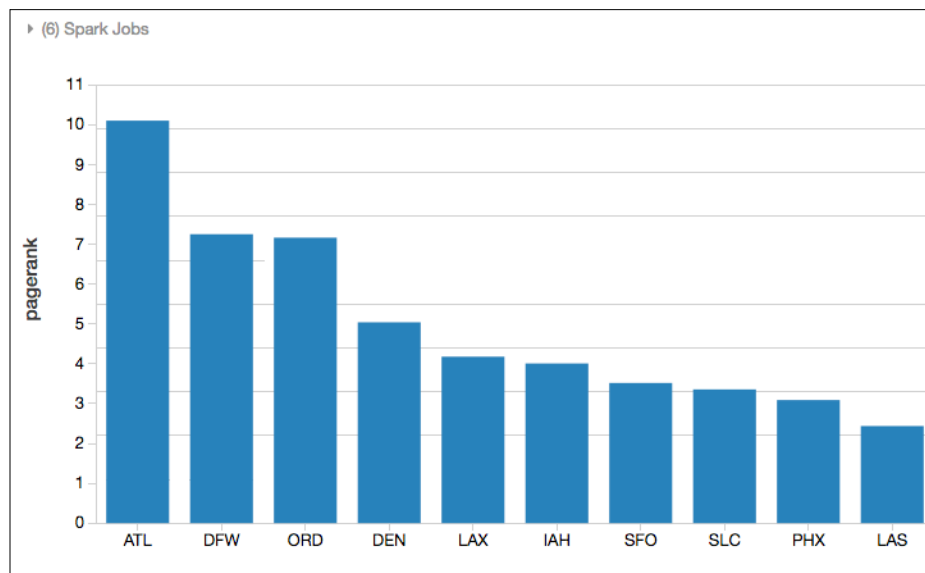
# Display the pageRank output
display(ranks.vertices.orderBy(ranks.vertices.pagerank.desc()).
limit(20))
```

Note that `resetProbability = 0.15` represents the probability of resetting to a random vertex (this is the default value) while `maxIter = 5` is a set number of iterations.



For more information on PageRank parameters, please refer to Wikipedia > Page Rank at <https://en.wikipedia.org/wiki/PageRank>.

The results of the PageRank are noted in the following bar graph:



In terms of airport ranking, the PageRank algorithm has determined that ATL (Hartsfield-Jackson Atlanta International Airport) is the most important airport in the United States. This observation makes sense as ATL is not only the busiest airport in the United States (<http://bit.ly/2eTGHs4>), but it is also the busiest airport in the world (2000-2015) (<http://bit.ly/2eTGDsy>).

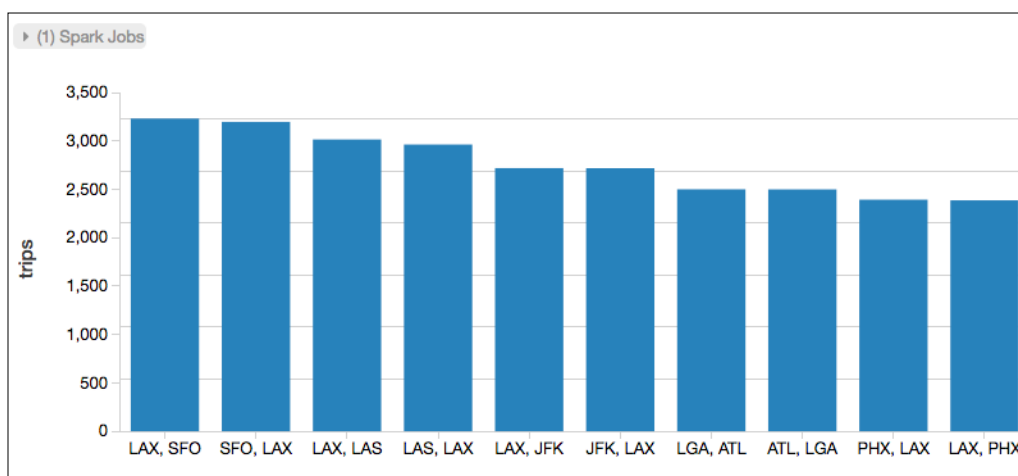
Determining the most popular non-stop flights

Expanding upon our `tripGraph` `GraphFrame`, the following query will allow us to find the most popular non-stop flights in the US (for this dataset):

```
# Determine the most popular non-stop flights
import pyspark.sql.functions as func
topTrips = tripGraph \
    .edges \
    .groupBy("src", "dst") \
    .agg(func.count("delay").alias("trips"))

# Show the top 20 most popular flights (single city hops)
display(topTrips.orderBy(topTrips.trips.desc()).limit(20))
```

Note, while we are using the `delay` column, we're just actually doing a count of the number of trips. Here's the output:



As can be observed from this query, the two most frequent non-stop flights are between LAX (Los Angeles) and SFO (San Francisco). The fact that these flights are so frequent indicates their importance in the airline market. As noted in the New York Times article from April 4, 2016, *Alaska Air Sees Virgin America as Key to West Coast* (<http://nyti.ms/2ealuzR>), acquiring slots at these two airports was one of the reasons why Alaska Airlines purchased Virgin Airlines. Graphs are not just fun but also contain potentially powerful business insight!

Using Breadth-First Search

The **Breadth-first search (BFS)** is a new algorithm as part of GraphFrames that finds the shortest path from one set of vertices to another. In this section, we will use BFS to traverse our `tripGraph` to quickly find the desired vertices (that is, airports) and edges (that is, flights). Let's try to find the shortest number of connections between cities based on the dataset. Note that these examples do not consider time or distance, just hops between cities. For example, to find the number of direct flights between Seattle and San Francisco, you can run the following query:

```
# Obtain list of direct flights between SEA and SFO
filteredPaths = tripGraph.bfs(
  fromExpr = "id = 'SEA'",
  toExpr = "id = 'SFO'",
  maxPathLength = 1)

# display list of direct flights
display(filteredPaths)
```

`fromExpr` and `toExpr` are the expressions indicating the origin and destination airports (that is, SEA and SFO, respectively). The `maxPathLength = 1` indicates that we only want one edge between the two vertices, that is, a non-stop flight between Seattle and San Francisco. As noted in the following results, there are many direct flights between Seattle and San Francisco:

» (14) Spark Jobs		
from	e0	to
» ("id":"SEA","City":"Seattle","State":"WA","Country":"USA")	» ("tripid":1010710,"delay":31,"src":"SEA","dst":"SFO","city_dst":"San Francisco","state_dst":"CA")	» ("id":"SFO","City":"San Francisco","State":"CA","Country":"USA")
» ("id":"SEA","City":"Seattle","State":"WA","Country":"USA")	» ("tripid":1012125,"delay":-4,"src":"SEA","dst":"SFO","city_dst":"San Francisco","state_dst":"CA")	» ("id":"SFO","City":"San Francisco","State":"CA","Country":"USA")
» ("id":"SEA","City":"Seattle","State":"WA","Country":"USA")	» ("tripid":1011640,"delay":-5,"src":"SEA","dst":"SFO","city_dst":"San Francisco","state_dst":"CA")	» ("id":"SFO","City":"San Francisco","State":"CA","Country":"USA")
» ("id":"SEA","City":"Seattle","State":"WA","Country":"USA")	» ("tripid":1010610,"delay":-4,"src":"SEA","dst":"SFO","city_dst":"San Francisco","state_dst":"CA")	» ("id":"SFO","City":"San Francisco","State":"CA","Country":"USA")
» ("id":"SEA","City":"Seattle","State":"WA","Country":"USA")	» ("tripid":1011230,"delay":-2,"src":"SEA","dst":"SFO","city_dst":"San Francisco","state_dst":"CA")	» ("id":"SFO","City":"San Francisco","State":"CA","Country":"USA")

But how about if we want to determine the number of direct flights between San Francisco and Buffalo? Running the following query will note that there are no results, that is, no direct flights between the two cities:

```
# Obtain list of direct flights between SFO and BUF
filteredPaths = tripGraph.bfs(
  fromExpr = "id = 'SFO'",
  toExpr = "id = 'BUF'",
  maxPathLength = 1)

# display list of direct flights
display(filteredPaths)
```

Once we modify the preceding query to `maxPathLength = 2`, that is, one layover, then you will see a lot more flight options:

```
# display list of one-stop flights between SFO and BUF
filteredPaths = tripGraph.bfs(
  fromExpr = "id = 'SFO'",
  toExpr = "id = 'BUF'",
  maxPathLength = 2)

# display list of flights
display(filteredPaths)
```

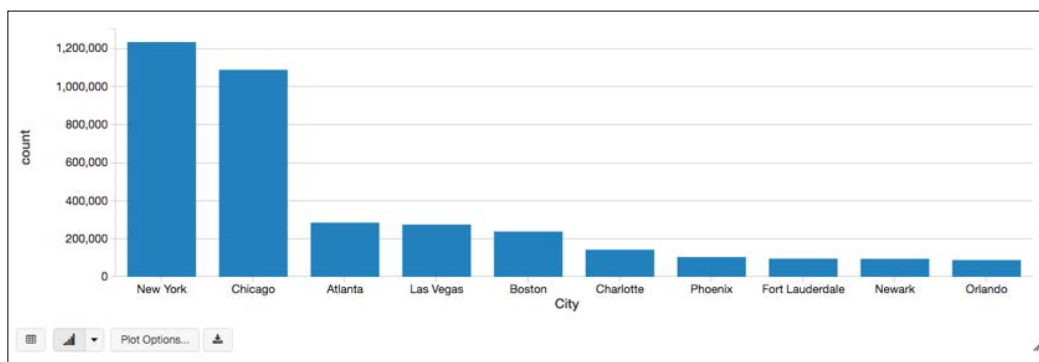
The following table provides an abridged version of the output from this query:

From	Layover	To
SFO	MSP (Minneapolis)	BUF
SFO	EWR (Newark)	BUF
SFO	JFK (New York)	BUF
SFO	ORD (Chicago)	BUF
SFO	ATL (Atlanta)	BUF
SFO	LAS (Las Vegas)	BUF
SFO	BOS (Boston)	BUF

But now that I have my list of airports, how can I determine which layover airports are more popular between SFO and BUF? To determine this, you can now run the following query:

```
# Display most popular layover cities by descending count
display(filteredPaths.groupBy("v1.id", "v1.City").count().
  orderBy(desc("count")).limit(10))
```

The output is shown in the following bar chart:



Visualizing flights using D3

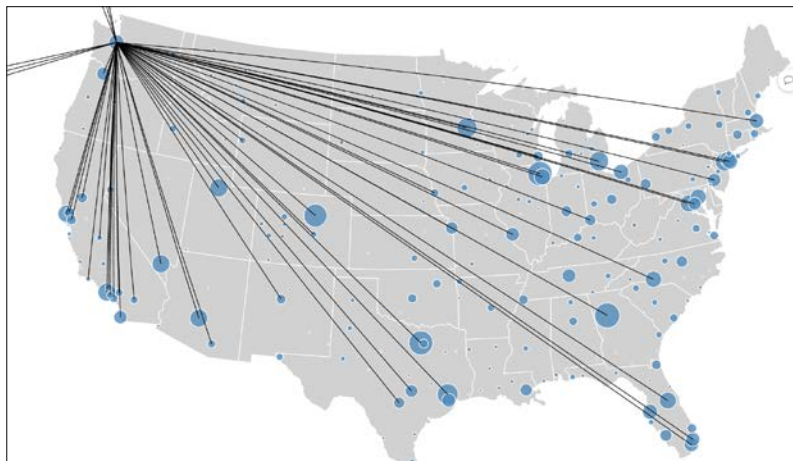
To get a powerful and fun visualization of the flight paths and connections in this dataset, we can leverage the Airports D3 visualization (<https://mbostock.github.io/d3/talk/20111116/airports.html>) within our Databricks notebook. By connecting our GraphFrames, DataFrames, and D3 visualizations, we can visualize the scope of all the flight connections as noted for all on-time or early departing flights within this dataset.

The blue circles represent the vertices (that is, airports) where the size of the circle represents the number of edges (that is, flights) in and out of those airports. The black lines are the edges themselves (that is, flights) and their respective connections to the other vertices (that is, airports). Note for any edges that go offscreen, they are representing vertices (that is, airports) in the states of Hawaii and Alaska.

For this to work, we first create a scala package called `d3a` that is embedded in our notebook (you can download it from here: <http://bit.ly/2kPkXkc>). Because we're using Databricks notebooks, we can make Scala calls within our PySpark notebook:

```
%scala
// On-time and Early Arrivals
import d3a._
graphs.force(
  height = 800,
  width = 1200,
  clicks = sql("""select src, dst as dest, count(1) as count from
departureDelays_geo where delay <= 0 group by src, dst""").as[Edge])
```

The results of the preceding query for on-time and early arrivals flights are visualized in the following screenshot:



You can hover over the airports (blue circle, vertex) in the airports D3 visualization where the lines are the edges (flights). The preceding visualization is a snapshot when hovering over Seattle (SEA) airport; while the following visualization is a snapshot when hovering over Los Angeles (LAX) airport:



Summary

As you can see in this chapter, you can easily perform a lot of powerful data analysis by executing queries against graph structures. With GraphFrames, you can leverage the power, simplicity, and performance of the DataFrame API against your graph problems.

For more information on GraphFrames, please refer to the following resources:

- *Introducing GraphFrames* (<http://bit.ly/2dBPhKn>)
- *On-Time Flight Performance with GraphFrames for Apache Spark* (<http://bit.ly/2c804ZD>)
- *On-Time Flight Performance with GraphFrames for Apache Spark (Spark 2.0) Notebook* (<http://bit.ly/2kPkXkc>)
- *GraphFrames Overview* (<http://graphframes.github.io/>)
- *Pygraphframes documentation* (<http://graphframes.github.io/api/python/graphframes.html>)
- *GraphX Programming Guide* (<http://spark.apache.org/docs/latest/graphx-programming-guide.html>)

In the next chapter, we will expand our PySpark horizon into the area of Deep Learning with the focus on TensorFlow and TensorFrames.

8

TensorFrames

This chapter will provide a high-level primer on the burgeoning field of Deep Learning and the reasons why it is important. It will provide the fundamentals surrounding feature learning and neural networks required for deep learning. As well, this chapter will provide a quick start for TensorFrames for Apache Spark.

In this chapter, you will learn about:

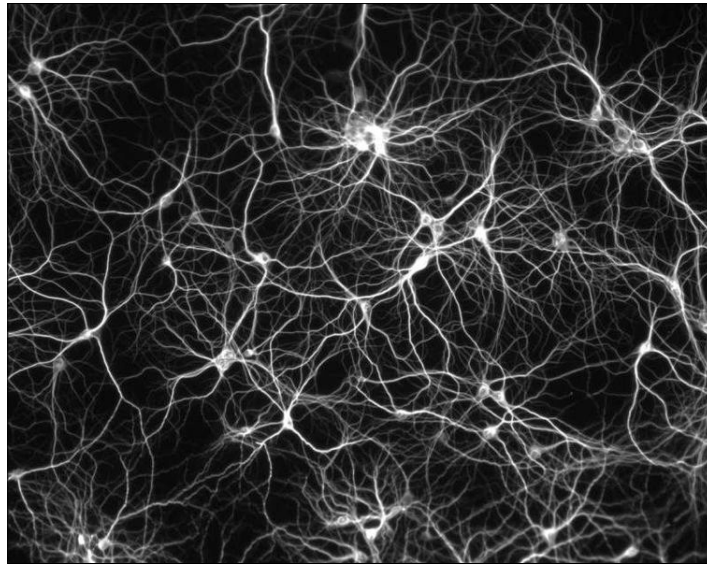
- What is Deep Learning?
- A primer on feature learning
- What is feature engineering?
- What is TensorFlow?
- Introducing TensorFrames
- TensorFrames – quick start

As you can see in the preceding breakdown, we will be initially discussing deep learning – more specifically we will start with neural networks.

What is Deep Learning?

Deep Learning is part of a family of machine learning techniques based on *learning* representations of data. Deep Learning is loosely based on our brain's own neural networks, the purpose of this structure is to provide a large number of highly interconnected elements (in biological systems, this would be the neurons in our brains); there are approximately 100 billion neurons in our brain, each connected to approximately 10,000 other neurons, resulting in a mind-boggling 10^{15} synaptic connections. These elements work together to solve problems through learning processes – examples include pattern recognition and data classification.

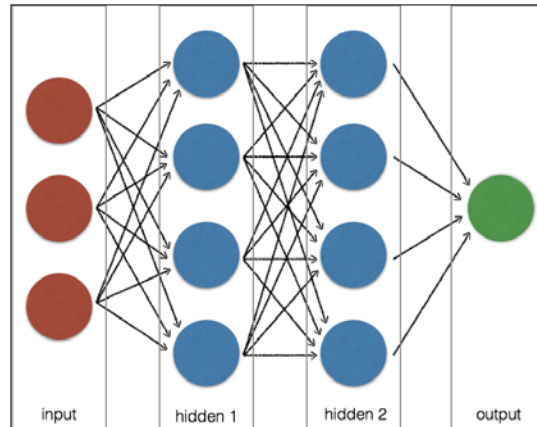
Learning within this architecture involves modifications of the connections between the interconnected elements similar to how our own brains make adjustments to the synaptic connections between neurons:



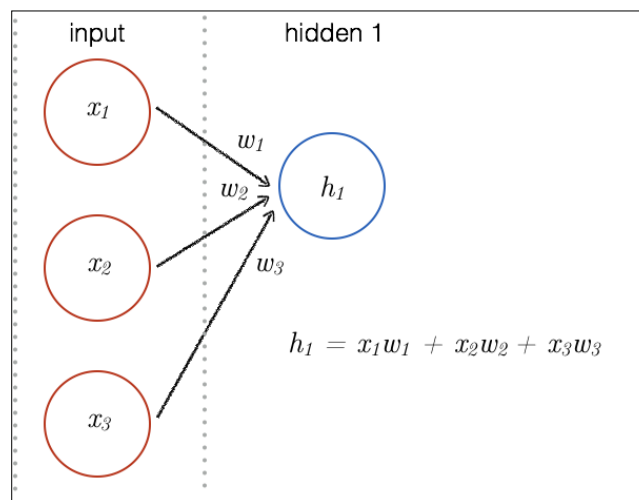
Source: *Wikimedia Commons: File: Réseau de neurones.jpg*; https://commons.wikimedia.org/wiki/File:Réseau_de_neurones.jpg.

The traditional algorithmic approach involves programming known steps or quantities, that is, you already know the steps to solve a specific problem, now repeat the solution and make it run faster. Neural networks are an interesting paradigm because neural networks learn by example and are not actually programmed to perform a specific task per se. This makes the training process in neural networks (and Deep Learning) very important in that you must provide good examples for the neural network to learn from otherwise it will "learn" the wrong thing (that is, provide unpredictable results).

The most common approach to building an artificial neural network involves the creation of three layers: input, hidden, and outer; as noted in the following diagram:



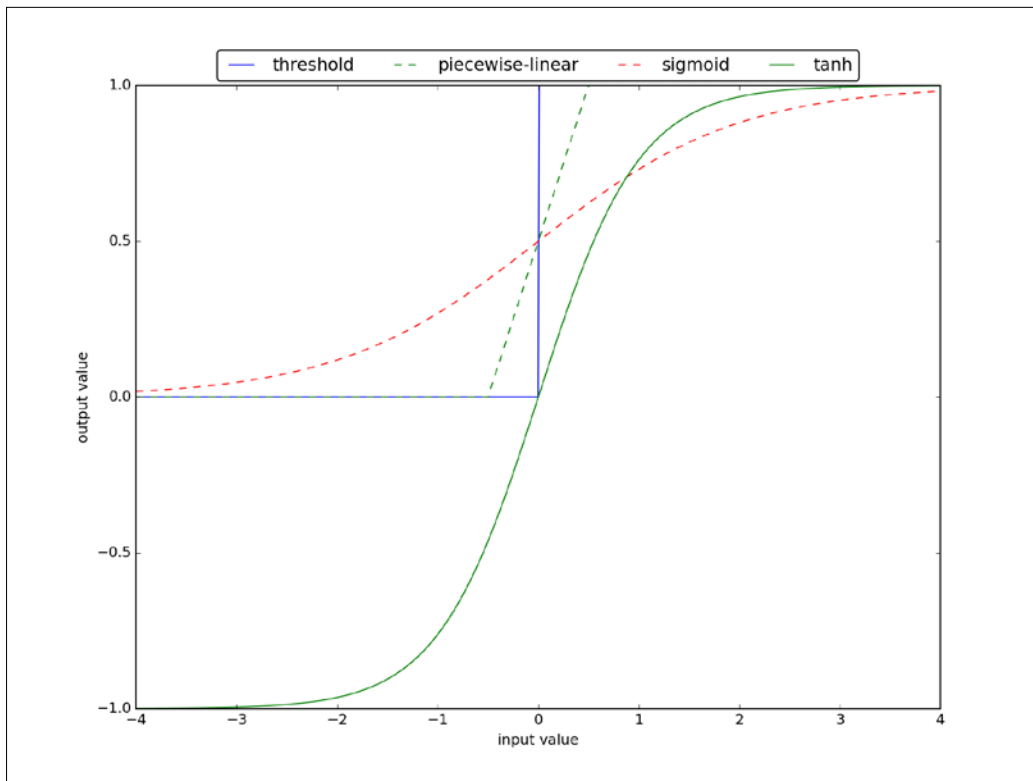
Each layer is comprised of one or more nodes with connections (that is, flow of data) between each of these nodes, as noted in the preceding diagram. Input nodes are passive in that they receive data, but do not modify the information. The nodes in the hidden and output layers will actively modify the data. For example, the connections from the three nodes in the input layer to one of the nodes in the first hidden layer is noted in the following diagram:



Referring to a signal processing neural network example, each input (denoted as x_i) has a weight applied to it (w_i), which produces a new value. In this case, one of the hidden nodes (h_1) is the result of three modified input nodes:

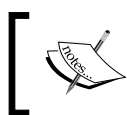
$$h_1 = x_1 w_1 + x_2 w_2 + x_3 w_3$$

There is also a bias applied to the sum in a form of a constant that also gets adjusted during the training process. The sum (the h_1 in our example) passes through so-called activation function that determines the output of the neuron. Some examples of such activation functions are presented in the following image:



This process is repeated for each node in the hidden layers as well as the output layer. The output node is the accumulation of all the weights applied to the input values for every active layer node. The learning process is the result of many iterations running in parallel, applying and reapplying these weights (in this scenario).

Neural networks appear in all the different sizes and shapes. The most popular are single- and multi-layer feedforward networks that resemble the one presented earlier; such structures (even with two layers and one neuron!) neuron in the output layer are capable of solving simple regression problems (such as linear and logistic) to highly complex regression and classification tasks (with many hidden layers and a number of neurons). Another type commonly used are self-organizing maps, sometimes referred to as Kohonen networks, due to Teuvo Kohonen, a Finnish researcher who first proposed such structures. The structures are trained *without-a-teacher*, that is, they do not require a target (an unsupervised learning paradigm). Such structures are used most commonly to solve clustering problems where the aim is to find an underlying pattern in the data.



For more information about neural network types, we suggest checking this document: <http://www.ieee.cz/knihovna/Zhang/Zhang100-ch03.pdf>

Note that there are many other interesting deep learning libraries in addition to TensorFlow; including, but not limited, to Theano, Torch, Caffe, Microsoft Cognitive Toolkit (CNTK), mxnet, and DL4J.

The need for neural networks and Deep Learning

There are many potential applications with neural networks (and Deep Learning). Some of the more popular ones include facial recognition, handwritten digit identification, game playing, speech recognition, language translation, and object classification. The key aspect here is that it involves learning and pattern recognition.

While neural networks have been around for a long time (at least within the context of the history of computer science), they have become more popular now because of the overarching themes: advances and availability of distributed computing and advances in research:

- **Advances and availability of distributed computing and hardware:**
Distributed computing frameworks such as Apache Spark allows you to complete more training iterations faster by being able to run more models in parallel to determine the optimal parameters for your machine learning models. With the prevalence of GPUs – graphic processing units that were originally designed for displaying graphics – these processors are adept at performing the resource intensive mathematical computations required for machine learning. Together with cloud computing, it becomes easier to harness the power of distributed computing and GPUs due to the lower up-front costs, minimal time to deployment, and easier to deploy elastic scale.

- **Advances in deep learning research:** These hardware advances have helped return neural networks to the forefront of data sciences with projects such as TensorFlow as well as other popular ones such as Theano, Caffe, Torch, Microsoft Cognitive Toolkit (CNTK), mxnet, and DL4J.

To dive deeper into these topics, two good references include:

- *Lessons Learned from Deploying Deep Learning at Scale* (<http://blog.algorithmia.com/deploying-deep-learning-cloud-services/>): This blog post by the folks at Algorithmia discuss their learnings on deploying deep learning solutions at scale.
- *Neural Networks by Christos Stergio and Dimitrios Siganos* (<http://bit.ly/2hNSWar>): A great primer on neural networks.

As noted previously, Deep Learning is part of a family of machine learning methods based on learning representations of data. In the case of learning representations, this can also be defined as *feature learning*. What makes Deep Learning so exciting is that it has the potential to replace or minimize the need for *manual* feature engineering. Deep Learning will allow the machine to not just learn a specific task, but also learn the *features* needed for that task. More succinctly, automating feature engineering or teaching machines *to learn how to learn* (a great reference on feature learning is Stanford's Unsupervised Feature Learning and Deep Learning tutorial: <http://deeplearning.stanford.edu/tutorial/>).

Breaking these concepts down to the fundamentals, let's start with a *feature*. As observed in Christopher Bishop's *Pattern Recognition and machine learning* (Berlin: Springer. ISBN 0-387-31073-8. 2006) and as noted in the previous chapters on MLlib and ML, a feature is a measurable property of the phenomenon being observed.

If you are more familiar in the domain of statistics, a *feature* would be in reference to the independent variables ($x_1, x_2, \dots, x_n$) within a stochastic linear regression model:

$$y_i = a + bx_2 + \dots + bx_n + e_i$$

In this specific example, y is the dependent variable and x_i are the independent variables.

Within the context of machine learning scenarios, examples of features include:

- **Restaurant recommendations:** Features include the reviews, ratings, and other content and user profile attributes related to the restaurant. A good example of this model is the *Yelp Food Recommendation System*: <http://cs229.stanford.edu/proj2013/SawantPai-YelpFoodRecommendationSystem.pdf>).

- **Handwritten Digit recognition:** Features include block wise histograms (count of pixels along 2D directions), holes, stroke detection, and so on. Examples include:
 - *Handwritten Digit Classification:* <http://ttic.uchicago.edu/~smaji/projects/digits/>
 - *Recognizing Handwritten Digits and Characters:* http://cs231n.stanford.edu/reports/vishnu_final.pdf
- **Image Processing:** Features include the points, edges, and objects within the image; some good examples include:
 - Seminar: Feature extraction by André Aichert, <http://home.in.tum.de/~aichert/featurepres.pdf>
 - University of Washington Computer Science & Engineering CSE455: Computer Vision Lecture 6, <https://courses.cs.washington.edu/courses/cse455/09wi/Lects/lect6.pdf>

Feature engineering is about determining which of these features (for example, in statistics, the independent variables) are important in defining the model that you are creating. Typically, it involves the process of using domain knowledge to create the features to allow the ML models to work.

Coming up with features is difficult, time-consuming, requires expert knowledge. "Applied machine learning" is basically feature engineering.

– Andrew Ng, *Machine Learning and AI via Brain simulations* (http://helper.ipam.ucla.edu/publications/gss2012/gss2012_10595.pdf)

What is feature engineering?

Typically, performing feature engineering involves concepts such as feature selection (selecting a subset of the original feature set) or feature extraction (building a new set of features from the original feature set):

- In *feature selection*, based on domain knowledge, you can filter the variables that you think define the model (for example, predicting football scores based on number of turnovers). Often data analysis techniques such as regression and classification can also be used to help you determine this.

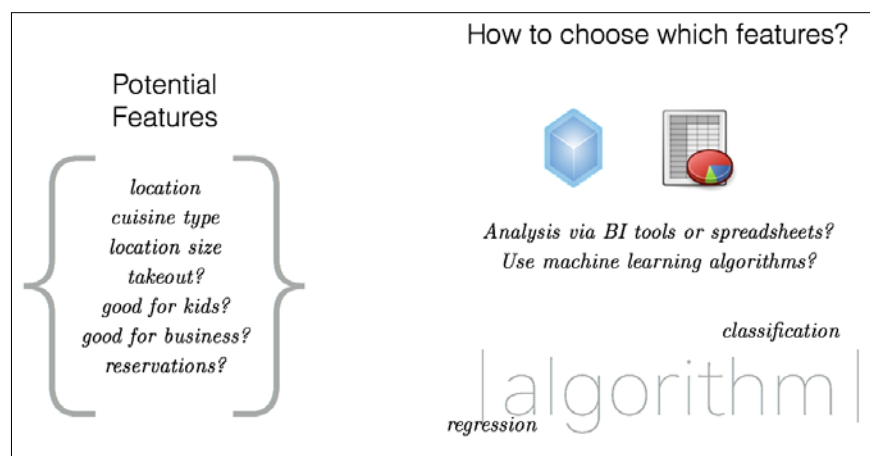
- In *feature extraction*, the idea is to transform the data from a high dimensional space (that is, many different independent variables) to a smaller space of fewer dimensions. Continuing the football analogy, this would be the quarterback rating, which is based on several selected features (e.g. completions, touchdowns, interceptions, average gain per pass attempt, etc.). A common approach for feature extraction within the linear data transformation space is **principal component analysis (PCA)**: <http://spark.apache.org/docs/latest/mllib-dimensionality-reduction.html#principal-component-analysis-pca>. Other common mechanisms include:
 - *Nonlinear dimensionality reduction*: https://en.wikipedia.org/wiki/Nonlinear_dimensionality_reduction
 - *Multilinear subspace learning*: https://en.wikipedia.org/wiki/Multilinear_subspace_learning



A good reference on the topic of feature selection versus feature extraction is *What is dimensionality reduction? What is the difference between feature selection and extraction?* <http://datascience.stackexchange.com/questions/130/what-is-dimensionality-reduction-what-is-the-difference-between-feature-selecti/132#132>

Bridging the data and algorithm

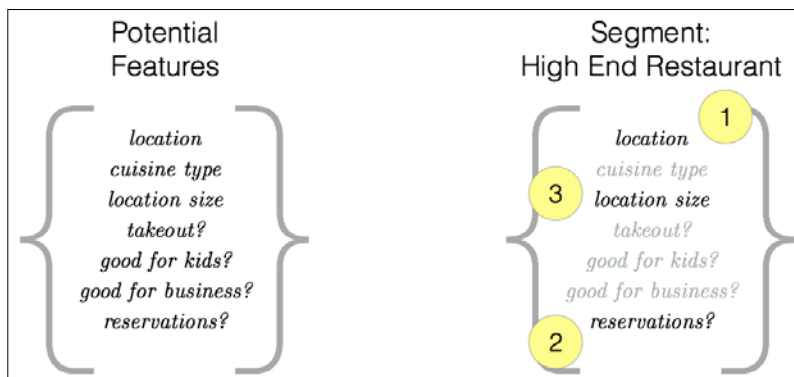
Let's bridge the feature and feature engineering definitions within the context of feature selection using the example of restaurant recommendations:



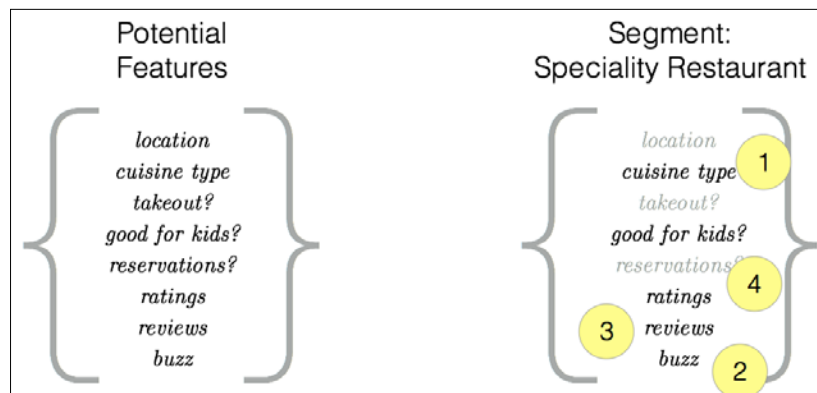
While this is a simplified model, the analogy describes the basic premise of applied machine learning. It would be up to a data scientist to analyze the data to determine the key features of this restaurant recommendation model.

In our restaurant recommendation case, while it may be easy to presume that geolocation and cuisine type are major factors, it will require some digging into the data to understand how the user (that is, restaurant-goer) has chosen their preference for a restaurant. Different restaurants often have different characteristics or weights for the mode.

For example, the key features for high-end restaurant catering businesses are often related to location (that is, proximity to their customer's location), the ability to make reservations for large parties, and the diversity of the wine list:



Meanwhile, for specialty restaurants, often few of those previous factors are involved; instead, the focus is on the reviews, ratings, social media buzz, and, possibly whether the restaurant is good for kids:

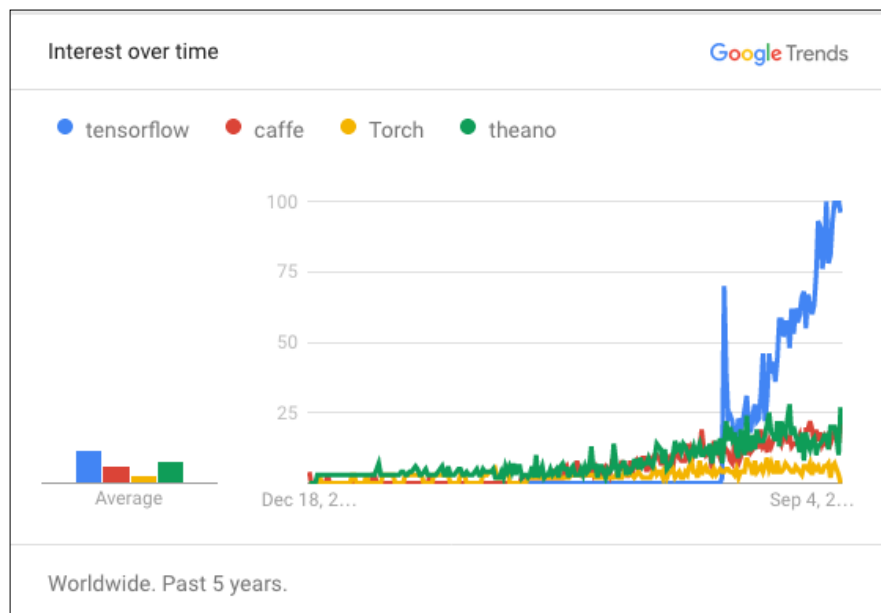


The ability to segment these different restaurants (and their target audience) is a key facet of applied machine learning. It can be an arduous process where you try different models and algorithms with different variables and weights and then retry after iteratively training and testing many different combinations. But note how this time consuming iterative approach itself is its own process that can potentially be automated? This is the key context of building algorithms of helping machines *learn to learn*: Deep Learning has the potential to automating the learning process when building our models.

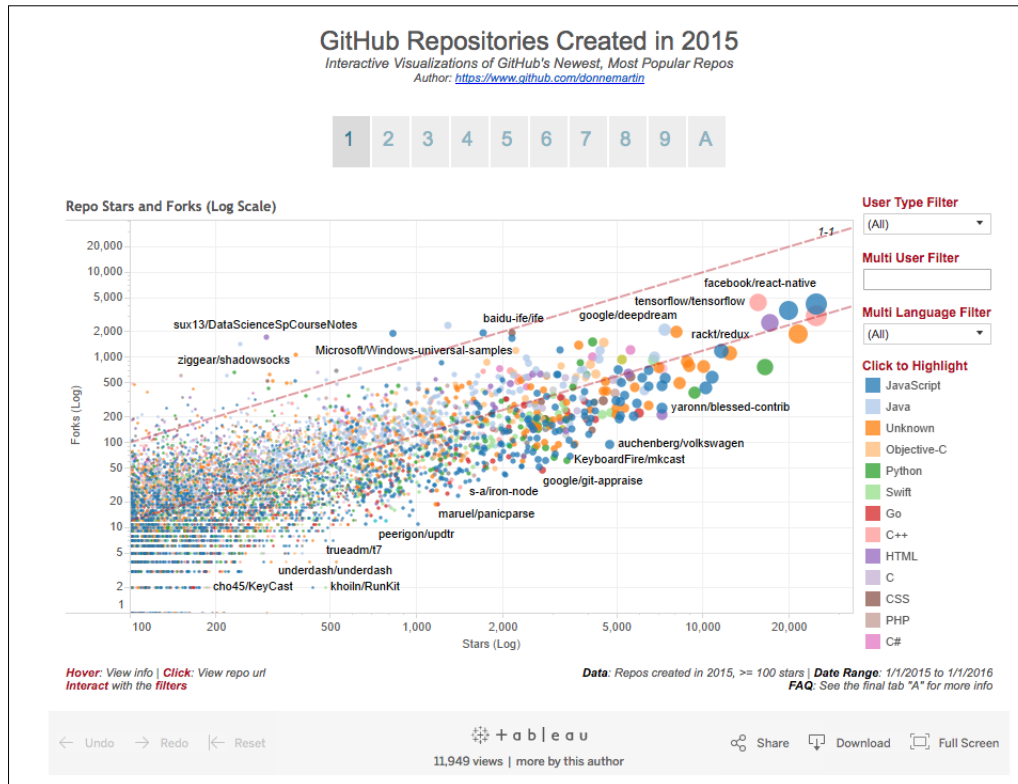
What is TensorFlow?

TensorFlow is a Google open source software library for numerical computation using data flow graphs; that is, an open source machine learning library focusing on Deep Learning. Based loosely on neural networks, TensorFlow is the culmination of the work of Google's Brain Team researchers and engineers to apply Deep Learning to Google products and build production models for various Google teams including (but not limited to) search, photos, and speech.

Built on C++ with a Python interface, it has quickly become one of the most popular Deep Learning projects in a short amount of time. The following screenshot denotes a Google Trends comparison between four popular deep learning libraries; note the spike around November 8th - 14th, 2015 (when TensorFlow was announced) and the rapid rise over the last year (this snapshot was taken late December 2016):

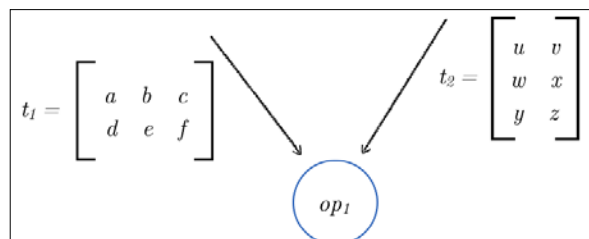


Another way to measure the popularity of TensorFlow is to note that TensorFlow is the most popular machine learning framework on GitHub per <http://www.theverge.com/2016/4/13/11420144/google-machine-learning-tensorflow-upgrade>. Note that TensorFlow was only released in November 2015 and in two months it had already become the most popular forked ML GitHub repository. In the following diagram, you can review the GitHub Repositories Created in 2015 (Interactive Visualization) per <http://donnemartin.com/viz/pages/2015>:



As noted previously, TensorFlow performs numerical computation using data flow graphs. When thinking about graph (as per the previous chapter on GraphFrames), the node (or vertices) of this graph represent mathematical operations while the graph edges represent the multidimensional arrays (that is, tensors) that communicate between the different nodes (that is, mathematical operations).

Referring to the following diagram, t_1 is a **2x3** matrix while t_2 is a **3x2** matrix; these are the tensors (or edges of the tensor graph). The node is the mathematical operations represented as op_1 :



In this example, op_1 is a matrix multiplication operation represented by the following diagram, though this could be any of the many mathematics operations available in TensorFlow:

$$op_1 = \begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix} \times \begin{bmatrix} u & v \\ w & x \\ y & z \end{bmatrix}$$

Together, to perform your numerical computations within the graph, there is a flow of multidimensional arrays (that is, tensors) between the mathematical operations (nodes) - that is, the flow of tensors, or *TensorFlow*.

To better understand how TensorFlow works, let's start by installing TensorFlow within your Python environment (initially sans Spark). For the full instructions, please refer to TensorFlow | Download and Setup: https://www.tensorflow.org/versions/r0.12/get_started/os_setup.html.

For this chapter, let's focus on the Python `pip` package management system installation on Linux or Mac OS.

Installing Pip

Ensure that you have installed `pip`; if have not, please use the following commands to install the Python package installation manager for Ubuntu/Linux:

```
# Ubuntu/Linux 64-bit
$ sudo apt-get install python-pip python-dev
```

For Mac OS, you would use the following commands:

```
# macOS
$ sudo easy_install pip
$ sudo easy_install --upgrade six
```

Note, for Ubuntu/Linux, you may also want to upgrade `pip` as the `pip` within the Ubuntu repository is old and may not be compatible with newer packages. To do this, you can run the command:

```
# Ubuntu/Linux pip upgrade
$ pip install --upgrade pip
```

Installing TensorFlow

To install TensorFlow (with `pip` already installed), you only need to execute the following command:

```
$ pip install tensorflow
```

If you have a computer that has GPU support, you can *instead* use the following command:

```
$ pip install tensorflow-gpu
```

Note that if the preceding command does not work, there are specific instructions to install TensorFlow with GPU support based on your Python version (that is, 2.7, 3.4, or 3.5) and GPU support.

For example, if I wanted to install TensorFlow on Python 2.7 with GPU enabled on Mac OS, execute the following commands:

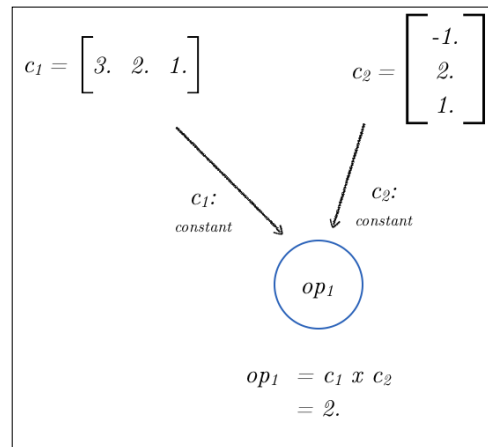
```
# macOS, GPU enabled, Python 2.7:
$ export TF_BINARY_URL=https://storage.googleapis.com/tensorflow/mac/gpu/tensorflow_gpu-0.12.0rc1-py2-none-any.whl
# Python 2
$ sudo pip install --upgrade $TF_BINARY_URL
```



Please refer to https://www.tensorflow.org/versions/r0.12/get_started/os_setup.html for the latest installation instructions.

Matrix multiplication using constants

To better describe tensors and how TensorFlow works, let's start with a matrix multiplication calculation involving two constants. As noted in the following diagram, we have c_1 (**3x1** matrix) and c_2 (**1x3** matrix), where the operation (op_1) is a matrix multiplication:



We will now define c_1 (**1x3** matrix) and c_2 (**3x1** matrix) using the following code:

```
# Import TensorFlow
import tensorflow as tf
# Setup the matrix
# c1: 1x3 matrix
# c2: 3x1 matrix
c1 = tf.constant([[3., 2., 1.]])
c2 = tf.constant([[ -1.], [2.], [1.]])
```

Now that we have our constants, let's run our matrix multiplication using the following code. Within the context of a TensorFlow graph, recall that the nodes in the graph are called operations (or ops). The following matrix multiplication is the ops, while the two matrices (c_1 , c_2) are the tensors (typed multi-dimensional array). An op takes zero or more tensors as its input, performs the operation such as a mathematical calculation, with the output being zero or more tensors in the form of numpy ndarray objects (<http://www.numpy.org/>) or tensorflow::Tensor interfaces in C, C++:

```
# m3: matrix multiplication (m1 x m3)
mp = tf.matmul(c1, c2)
```

Now that this TensorFlow graph has been established, execution of this operation (for example, in this case, the matrix multiplication) is done within the context of a session; the session places the graph ops into the CPU or GPU (that is, devices) to be executed:

```
# Launch the default graph
s = tf.Session()

# run: Execute the ops in graph
r = s.run(mp)
print(r)
```

With the output being:

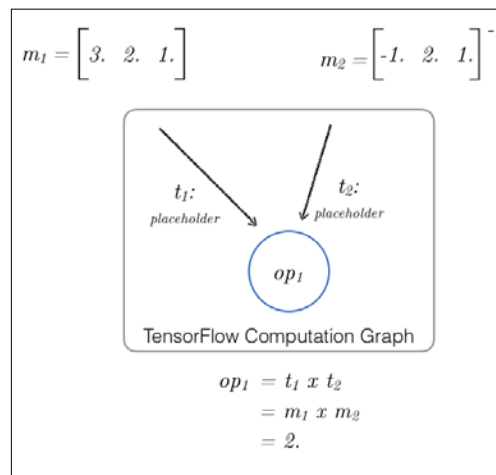
```
# [[ 2.]]
```

Once you have completed your operations, you can close the session:

```
# Close the Session when completed
s.close()
```

Matrix multiplication using placeholders

Now we will perform the same task as before, except this time, we will use tensors instead of constants. As noted in the following diagram, we will start off with two matrices (m_1 : 3×1 , m_2 : 1×3) using the same values as in the previous section:



Within TensorFlow, we will use `placeholder` to define our two tensors as per the following code snippet:

```
# Setup placeholder for your model
#   t1: placeholder tensor
#   t2: placeholder tensor
t1 = tf.placeholder(tf.float32)
t2 = tf.placeholder(tf.float32)

# t3: matrix multiplication (m1 x m3)
tp = tf.matmul(t1, t2)
```

The advantage of this approach is that, with placeholders you can use the same operations (that is, in this case, the matrix multiplication) with tensors of different sizes and shape (provided they meet the criteria of the operation). Like the operations in the previous section, let's define two matrices and execute the graph (with a simplified session execution).

Running the model

The following code snippet is similar to the code snippet in the previous section, except that it now uses placeholders instead of constants:

```
# Define input matrices
m1 = [[3., 2., 1.]]
m2 = [[-1.], [2.], [1.]]

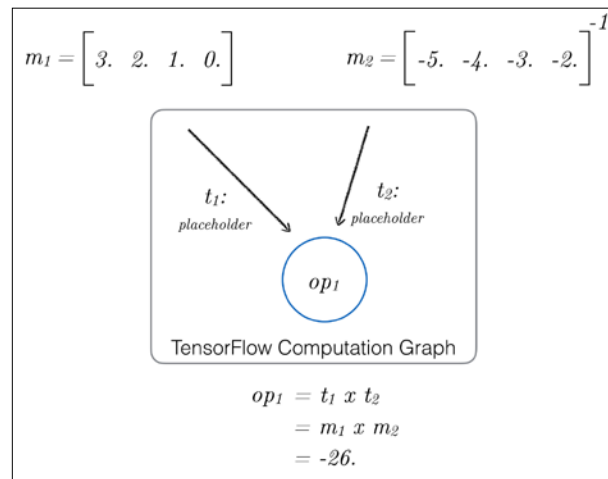
# Execute the graph within a session
with tf.Session() as s:
    print(s.run([tp], feed_dict={t1:m1, t2:m2}))
```

With the output being both the value, as well as the data type:

```
[array([[ 2.]]), dtype=float32)]
```

Running another model

Now that we have a graph (albeit a simple one) using placeholders, we can use different tensors to perform the same operation using different input matrices. As noted in the following figure, we have `m1` (4x1) and `m2` (1x4):



Because we're using placeholders, we can easily reuse the same graph within a new session using new input:

```
# setup input matrices
m1 = [[3., 2., 1., 0.]]
m2 = [[-5.], [-4.], [-3.], [-2.]]
# Execute the graph within a session
with tf.Session() as s:
    print(s.run([tp], feed_dict={t1:m1, t2:m2}))
```

With the output being:

```
[array([[ -26.]], dtype=float32)]
```

Discussion

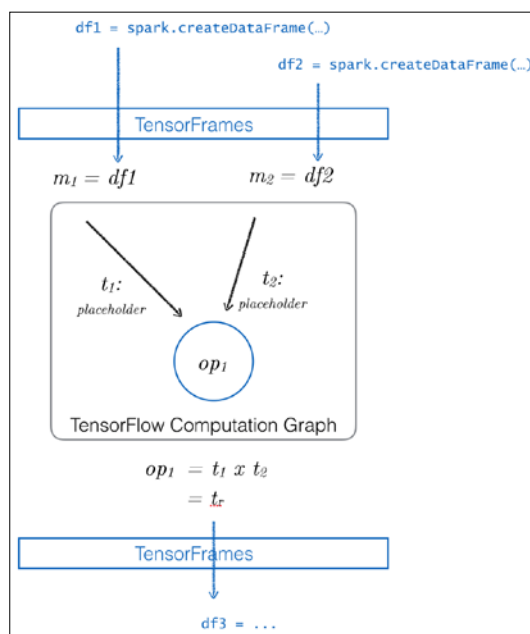
As noted previously, TensorFlow provides users with the ability to perform deep learning using Python libraries by representing computations as graphs where the tensors represent the data (edges of the graph) and operations represent what is to be executed (for example, mathematical computations) (vertices of the graph).

For more information, please refer to:

- TensorFlow | Get Started | Basic Usage https://www.tensorflow.org/get_started/basic_usage
- Shannon McCormick's Neural Network and Google TensorFlow <http://www.slideshare.net/ShannonMcCormick4/neural-networks-and-google-tensor-flow>

Introducing TensorFrames

At the time of writing, TensorFrames is an experimental binding for Apache Spark; it was introduced in early 2016, shortly after the release of TensorFlow. With TensorFrames, one can manipulate Spark DataFrames with TensorFlow programs. Referring to the tensor diagrams in the previous section, we have updated the figure to show how Spark DataFrames work with TensorFlow, as shown in the following diagram:



As noted in the preceding diagram, TensorFrames provides a bridge between Spark DataFrames and TensorFlow. This allows you to take your DataFrames and apply them as input into your TensorFlow computation graph. TensorFrames also allows you to take the TensorFlow computation graph output and push it back into DataFrames so you can continue your downstream Spark processing.

In terms of common usage scenarios for TensorFrames, these typically include the following:

Utilize TensorFlow with your data

The integration of TensorFlow and Apache Spark with TensorFrames allows data scientists to expand their analytics, streaming, graph, and machine learning capabilities to include Deep Learning via TensorFlow. This allows you to both train and deploy models at scale.

Parallel training to determine optimal hyperparameters

When building deep learning models, there are several configuration parameters (that is, hyperparameters) that impact on how the model is trained. Common in Deep Learning/artificial neural networks are hyperparameters that define the learning rate (if the rate is high it will learn quickly, but it may not take into account highly variable input - that is, it will not learn well if the rate and variability in the data is too high) and the number of neurons in each layer of your neural network (too many neurons results in noisy estimates, while too few neurons will result in the network not learning well).

As observed in *Deep Learning with Apache Spark and TensorFlow* (<https://databricks.com/blog/2016/01/25/deep-learning-with-apache-spark-and-tensorflow.html>), using Spark with TensorFlow to help find the best set of hyperparameters for neural network training resulted in an order of magnitude reduction in training time and a 34% lower error rate for the handwritten digit recognition dataset.

For more information on Deep Learning and hyperparameters, please refer to:

- *Optimizing Deep Learning Hyper-Parameters Through an Evolutionary Algorithm*
<http://ornloda.github.io/MLHPC2015/presentations/4-Steven.pdf>
- *CS231n Convolutional Network Networks for Visual Recognition*
<http://cs231n.github.io/>
- *Deep Learning with Apache Spark and TensorFlow* <https://databricks.com/blog/2016/01/25/deep-learning-with-apache-spark-and-tensorflow.html>

At the time of writing, TensorFrames is officially supported as of Apache Spark 1.6 (Scala 2.10), though most contributions are currently focused on Spark 2.0 (Scala 2.11). The easiest way to use TensorFrames is to access it via Spark Packages (<https://spark-packages.org>).

TensorFrames – quick start

After all this preamble, let's jump start our use of TensorFrames with this quick start tutorial. You can download and use the full notebook within Databricks Community Edition at <http://bit.ly/2hwGyuC>.

You can also run this from the PySpark shell (or other Spark environments), like any other Spark package:

```
# The version we're using in this notebook
$SPARK_HOME/bin/pyspark --packages tjhunter:tensorframes:0.2.2-s_2.10

# Or use the latest version
$SPARK_HOME/bin/pyspark --packages
databricks:tensorframes:0.2.3-s_2.10
```

Note, you will only use one of the above commands (that is, not both). For more information, please refer to the `databricks/tensorframes` GitHub repository (<https://github.com/databricks/tensorframes>).

Configuration and setup

Please follow the configuration and setup steps in the following order:

Launching a Spark cluster

Launch a Spark cluster using Spark 1.6 (Hadoop 1) and Scala 2.10. This has been tested with Spark 1.6, Spark 1.6.2, and Spark 1.6.3 (Hadoop 1) on Databricks Community Edition (<http://databricks.com/try-databricks>).

Creating a TensorFrames library

Create a library to attach TensorFrames 0.2.2 to your cluster: `tensorframes-0.2.2-s_2.10`. Refer to *Chapter 7, GraphFrames* to recall how to create a library.

Installing TensorFlow on your cluster

In a notebook, run one of the following commands to install TensorFlow. This has been tested with TensorFlow 0.9 CPU edition:

- TensorFlow 0.9, Ubuntu/Linux 64-bit, CPU only, Python 2.7:

```
/databricks/python/bin/pip install https://storage.googleapis.com/
tensorflow/linux/cpu/tensorflow-0.9.0rc0-cp27-none-linux_x86_64.
whl
```
- TensorFlow 0.9, Ubuntu/Linux 64-bit, GPU enabled, Python 2.7:

```
/databricks/python/bin/pip install https://storage.googleapis.com/
tensorflow/linux/gpu/tensorflow-0.9.0rc0-cp27-none-linux_x86_64.
whl
```

The following is the `pip` install command that will install TensorFlow on to the Apache Spark driver:

```
%sh
/databricks/python/bin/pip install https://storage.googleapis.com/
tensorflow/linux/cpu/tensorflow-0.9.0rc0-cp27-none-linux_x86_64.whl
```

A successful installation should have something similar to the following output:

```
Collecting tensorflow==0.9.0rc0 from https://storage.googleapis.
com/tensorflow/linux/cpu/tensorflow-0.9.0rc0-cp27-none-linux_
x86_64.whl Downloading https://storage.googleapis.com/tensorflow/
linux/cpu/tensorflow-0.9.0rc0-cp27-none-linux_x86_64.whl (27.6MB)
Requirement already satisfied (use --upgrade to upgrade):
numpy>=1.8.2 in /databricks/python/lib/python2.7/site-packages (from
tensorflow==0.9.0rc0) Requirement already satisfied (use --upgrade
to upgrade): six>=1.10.0 in /usr/lib/python2.7/dist-packages
(from tensorflow==0.9.0rc0) Collecting protobuf==3.0.0b2 (from
tensorflow==0.9.0rc0) Downloading protobuf-3.0.0b2-py2.py3-none-any.
whl (326kB) Requirement already satisfied (use --upgrade to upgrade):
wheel in /databricks/python/lib/python2.7/site-packages (from
tensorflow==0.9.0rc0) Requirement already satisfied (use --upgrade to
upgrade): setuptools in /databricks/python/lib/python2.7/site-packages
(from protobuf==3.0.0b2->tensorflow==0.9.0rc0) Installing collected
packages: protobuf, tensorflow Successfully installed protobuf-3.0.0b2
tensorflow-0.9.0rc0
```

Upon successful installation of TensorFlow, detach and reattach the notebook where you just ran this command. Your cluster is now configured; you can run pure TensorFlow programs on the driver, or TensorFlow examples on the whole cluster.

Using TensorFlow to add a constant to an existing column

This is a simple TensorFlow program where the `op` is to perform a simple addition. Note that the original source code can be found in the `databricks/tensorframes` GitHub repository. This is in reference to the TensorFlow `Readme.md` | *How to Run in Python* section (<https://github.com/databricks/tensorframes#how-to-run-in-python>).

The first thing we will do is import TensorFlow, TensorFlow, and `pyspark.sql` row to create a DataFrame based on an RDD of floats:

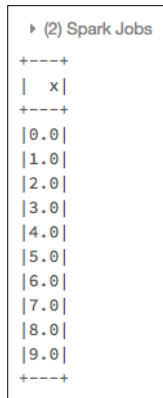
```
# Import TensorFlow, TensorFlow, and Row
import tensorflow as tf
import tensorflow as tfs
from pyspark.sql import Row
```

```
# Create RDD of floats and convert into DataFrame `df`
rdd = [Row(x=float(x)) for x in range(10)]
df = sqlContext.createDataFrame(rdd)
```

To view the `df` DataFrame generated by the RDD of floats, we can use the `show` command:

```
df.show()
```

This produces the following result:



```
▶ (2) Spark Jobs
+----+
|  x  |
+----+
|0.0|
|1.0|
|2.0|
|3.0|
|4.0|
|5.0|
|6.0|
|7.0|
|8.0|
|9.0|
+----+
```

Executing the Tensor graph

As noted previously, this tensor graph consists of adding 3 to the tensor created by the `df` DataFrame generated by the RDD of floats. We will now execute the following code snippet:

```
# Run TensorFlow program executes:
#   The 'op' performs the addition (i.e. 'x' + '3')
#   Place the data back into a DataFrame
with tf.Graph().as_default() as g:

#   The placeholder that corresponds to column 'x'.
#   The shape of the placeholder is automatically
#   inferred from the DataFrame.
x = tfs.block(df, "x")

# The output that adds 3 to x
z = tf.add(x, 3, name='z')

# The resulting `df2` DataFrame
df2 = tfs.map_blocks(z, df)
```

```
# Note that 'z' is the tensor output from the
# 'tf.add' operation
print z

## Output
Tensor("z:0", shape=(?,), dtype=float64)
```

Here are some specific call outs for the preceding code snippet:

- `x` utilizes `tf$block`, where the `block` builds a block placeholder based on the content of a column in a DataFrame
- `z` is the output tensor from the TensorFlow `add` method (`tf.add`)
- `df2` is the new DataFrame, which adds an extra column to the `df` DataFrame with the `z` tensor block by block

While `z` is the tensor (as noted in the preceding output), for us to work with the results of the TensorFlow program, we will utilize the `df2` dataframe. The output from `df2.show()` is as follows:

▸ (2) Spark Jobs

z	x
3.0	0.0
4.0	1.0
5.0	2.0
6.0	3.0
7.0	4.0
8.0	5.0
9.0	6.0
10.0	7.0
11.0	8.0
12.0	9.0

Blockwise reducing operations example

In this next section, we will show how to work with blockwise reducing operations. Specifically, we will compute the sum and min of field vectors, working with blocks of rows for more efficient processing.

Building a DataFrame of vectors

First, we will create a one-column DataFrame of vectors:

```
# Build a DataFrame of vectors
data = [Row(y=[float(y), float(-y)]) for y in range(10)]
df = sqlContext.createDataFrame(data)
df.show()
```

The output is as follows:

```
▶ (2) Spark Jobs
+-----+
|          y|
+-----+
|[0.0, 0.0]|
|[1.0, -1.0]|
|[2.0, -2.0]|
|[3.0, -3.0]|
|[4.0, -4.0]|
|[5.0, -5.0]|
|[6.0, -6.0]|
|[7.0, -7.0]|
|[8.0, -8.0]|
|[9.0, -9.0]|
+-----+
```

Analysing the DataFrame

We need to analyze the DataFrame to determine what its shape is (that is, dimensions of the vectors). For example, in the following snippet, we use the `tfs.print_schema` command for the `df` DataFrame:

```
# Print the information gathered by TensorFlow to check the content of
the DataFrame
tfs.print_schema(df)

## Output
root
|-- y: array (nullable = true) double[?,?]
```

Notice the `double[?,?]`, meaning that TensorFlow does not know the dimensions of the vectors:

```
# Because the dataframe contains vectors, we need to analyze it
# first to find the dimensions of the vectors.
df2 = tfs.analyze(df)
```

```
# The information gathered by TF can be printed
# to check the content:
tfs.print_schema(df2)

## Output
root
|-- y: array (nullable = true) double[?,2]
```

Upon analysis of the `df2` DataFrame, TensorFlow has inferred that `y` contains vectors of size 2. For small tensors (scalars and vectors), TensorFrames usually infers the shapes of the tensors without requiring a preliminary analysis. If it cannot do so, an error message will indicate that you need to run the DataFrame through `tfs.analyze()` first.

Computing elementwise sum and min of all vectors

Now, let's analyze the `df` DataFrame to compute the sum and the elementwise min of all the vectors using `tf.reduce_sum` and `tf.reduce_min`:

- `tf.reduce_sum`: Computes the sum of elements across dimensions of a tensor, for example, if `x = [[3, 2, 1], [-1, 2, 1]]` then `tf.reduce_sum(x) ==> 8`. More information can be found at: https://www.tensorflow.org/api_docs/python/math_ops/reduction#reduce_sum.
- `tf.reduce_min`: Computes the minimum of elements across dimensions of a tensor, for example, if `x = [[3, 2, 1], [-1, 2, 1]]` then `tf.reduce_min(x) ==> -1`. More information can be found at: https://www.tensorflow.org/api_docs/python/math_ops/reduction#reduce_min.

The following code snippet allows us to perform efficient elementwise reductions using TensorFlow, where the source data is within a DataFrame:

```
# Note: First, let's make a copy of the 'y' column.
# This is an inexpensive operation in Spark 2.0+
df3 = df2.select(df2.y, df2.y.alias("z"))

# Execute the Tensor Graph
with tf.Graph().as_default() as g:

    # The placeholders.
    # Note the special name that end with '_input':
    y_input = tfs.block(df3, 'y', tf_name="y_input")
    z_input = tfs.block(df3, 'z', tf_name="z_input")
```



```
# Perform elementwise sum and minimum
y = tf.reduce_sum(y_input, [0], name='y')
z = tf.reduce_min(z_input, [0], name='z')

# The resulting dataframe
(data_sum, data_min) = tfs.reduce_blocks([y, z], df3)

# The final results are numpy arrays:
print "Elementwise sum: %s and minimum: %s " % (data_sum, data_min)

## Output
Elementwise sum: [ 45. -45.] and minimum: [ 0. -9.]
```

With a few lines of TensorFlow code with TensorFrames, we can take the data stored within the `df` DataFrame and execute a Tensor Graph to perform element wise sum and min, merge the data back into a DataFrame, and (in our case) print out the final values.

Summary

In this chapter, we have reviewed the fundamentals of neural networks and Deep Learning, including the components of feature engineering. With all this new excitement in Deep Learning, we introduced TensorFlow and how it can work closely together with Apache Spark through TensorFrames.

TensorFrames is a powerful deep learning tool that allows data scientists and engineers to work with TensorFlow with data stored in Spark DataFrames. This allows you to expand the capabilities of Apache Spark to a powerful deep learning toolset that is based on the learning process of neural networks. To help continue your Deep Learning journey, the following are some great TensorFlow and TensorFrames resources:

- TensorFlow: <https://www.tensorflow.org/>
- TensorFlow | Get Started: https://www.tensorflow.org/get_started/
- TensorFlow | Guides: <https://www.tensorflow.org/tutorials/>
- Deep Learning on Databricks: <https://databricks.com/blog/2016/12/21/deep-learning-on-databricks.html>
- TensorFrames (GitHub): <https://github.com/databricks/tensorframes>
- TensorFrames User Guide: <https://github.com/databricks/tensorframes/wiki/TensorFrames-user-guide>

9

Polyglot Persistence with Blaze

Our world is complex and no single approach exists that solves all problems. Likewise, in the data world one cannot solve all problems with one piece of technology.

Nowadays, any big technology company uses (in one form or another) a MapReduce paradigm to sift through terabytes (or even petabytes) of data collected daily. On the other hand, it is much easier to store, retrieve, extend, and update information about products in a document-type database (such as MongoDB) than it is in a relational database. Yet, persisting transaction records in a relational database aids later data summarizing and reporting.

Even these simple examples show that solving a vast array of business problems requires adapting to different technologies. This means that you, as a database manager, data scientist, or data engineer, would have to learn all of these separately if you were to solve your problems with the tools that are designed to solve them easily. This, however, does not make your company agile and is prone to errors and lots of tweaking and hacking needing to be done to your system.

Blaze abstracts most of the technologies and exposes a simple and elegant data structure and API.

In this chapter, you will learn:

- How to install Blaze
- What polyglot persistence is about
- How to abstract data stored in files, pandas DataFrames, or NumPy arrays

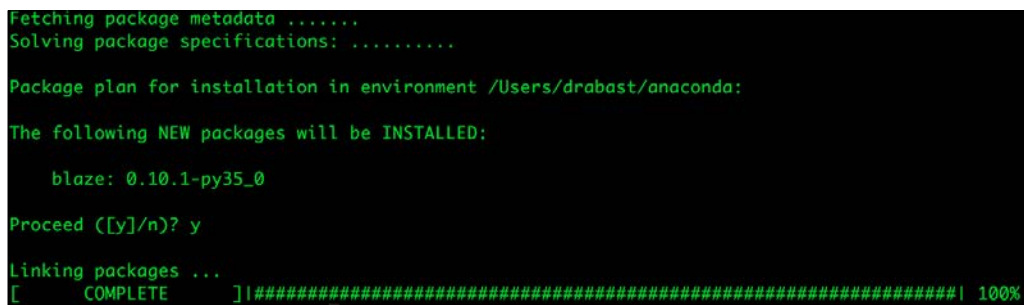
- How to work with archives (GZip)
- How to connect to SQL (PostgreSQL and SQLite) and No-SQL (MongoDB) databases with Blaze
- How to query, join, sort, and transform the data, and perform simple summary statistics

Installing Blaze

If you run Anaconda it is easy to install Blaze. Just issue the following command in your CLI (see the Bonus *Chapter 1, Installing Spark* if you do not know what a CLI is):

```
conda install blaze
```

Once the command is issued, you will see a screen similar to the following screenshot:



```
Fetching package metadata .....
Solving package specifications: .....

Package plan for installation in environment /Users/drabast/anaconda:

The following NEW packages will be INSTALLED:

    blaze: 0.10.1-py35_0

Proceed ([y]/n)? y

Linking packages ...
[ COMPLETE ]|#####| 100%
```

We will later use Blaze to connect to the PostgreSQL and MongoDB databases, so we need to install some additional packages that Blaze will use in the background.

We will install SQL Alchemy and PyMongo, both of which are part of Anaconda:

```
conda install sqlalchemy
```

```
conda install pymongo
```

All that is now left to do is to import Blaze itself in our notebook:

```
import blaze as bl
```

Polyglot persistence

Neal Ford introduced the, somewhat similar, term polyglot programming in 2006. He used it to illustrate the fact that there is no such thing as a one-size-fits-all solution and advocated using multiple programming languages that were more suitable for certain problems.

In the parallel world of data, any business that wants to remain competitive needs to adapt a range of technologies that allows it to solve the problems in a minimal time, thus minimizing the costs.

Storing transactional data in Hadoop files is possible, but makes little sense. On the other hand, processing petabytes of Internet logs using a **Relational Database Management System (RDBMS)** would also be ill-advised. These tools were designed to tackle specific types of tasks; even though they can be co-opted to solve other problems, the cost of adapting the tools to do so would be enormous. It is a virtual equivalent of trying to fit a square peg in a round hole.

For example, consider a company that sells musical instruments and accessories online (and in a network of shops). At a high-level, there are a number of problems that a company needs to solve to be successful:

1. Attract customers to its stores (both virtual and physical).
2. Present them with relevant products (you would not try to sell a drum kit to a pianist, would you?!).
3. Once they decide to buy, process the payment and organize shipping.

To solve these problems a company might choose from a number of available technologies that were designed to solve these problems:

1. Store all the products in a document-based database such as MongoDB, Cassandra, DynamoDB, or DocumentDB. There are multiple advantages of document databases: flexible schema, sharding (breaking bigger databases into a set of smaller, more manageable ones), high availability, and replication, among others.
2. Model the recommendations using a graph-based database (such as Neo4j, Tinkerpop/Gremlin, or GraphFrames for Spark): such databases reflect the factual and abstract relationships between customers and their preferences. Mining such a graph is invaluable and can produce a more tailored offering for a customer.

3. For searching, a company might use a search-tailored solution such as Apache Solr or ElasticSearch. Such a solution provides fast, indexed text searching capabilities.
4. Once a product is sold, the transaction normally has a well-structured schema (such as product name, price, and so on.) To store such data (and later process and report on it) relational databases are best suited.

With polyglot persistence, a company always chooses the right tool for the right job instead of trying to coerce a single technology into solving all of its problems.

Blaze, as we will see, abstracts these technologies and introduces a simple API to work with, so you do not have to learn the APIs of each and every technology you want to use. It is, in essence, a great working example of polyglot persistence.



To see how others do it, check out <http://www.slideshare.net/Couchbase/couchbase-at-ebay-2014>

or

<http://www.slideshare.net/bijoor1/case-study-polyglot-persistence-in-pharmaceutical-industry>.

Abstracting data

Blaze can abstract many different data structures and expose a single, easy-to-use API. This helps to get a consistent behavior and reduce the need to learn multiple interfaces to handle data. If you know pandas, there is not really that much to learn, as the differences in the syntax are subtle. We will go through some examples to illustrate this.

Working with NumPy arrays

Getting data from a NumPy array into the DataShape object of Blaze is extremely easy. First, let's create a simple NumPy array: we first load NumPy and then create a matrix with two rows and three columns:

```
import numpy as np
simpleArray = np.array([
    [1,2,3],
    [4,5,6]
])
```

Now that we have an array, we can abstract it with Blaze's DataShape structure:

```
simpleData_np = bl.Data(simpleArray)
```

That's it! Simple enough.

In order to peek inside the structure you can use the `.peek()` method:

```
simpleData_np.peak()
```

You should see an output similar to what is shown in the following screenshot:

```
Out[4]: array([[1, 2, 3],
               [4, 5, 6]])
```

You can also use (familiar to those of you versed in pandas' syntax) the `.head(...)` method.



The difference between `.peek()` and `.head(...)` is that `.head(...)` allows the specification of the number of rows as its only parameter, whereas `.peek()` does not allow that and will always print the top 10 records.

If you want to retrieve the first column of your DataShape, you can use indexing:

```
simpleData_np[0]
```

You should see a table, as shown here:

```
Out[6]:
```

	None
0	1
1	2
2	3

On the other hand, if you were interested in retrieving a row, all you would have to do (like in NumPy) is transpose your DataShape:

```
simpleData_np.T[0]
```

What you will then get is presented in the following figure:

Out[7]:		None
0	1	
1	4	

Notice that the name of the column is `None`. `DataShapes`, just like `pandas`' `DataFrames`, support named columns. Thus, let's specify the names of our fields:

```
simpleData_np = bl.Data(simpleArray, fields=['a', 'b', 'c'])
```

Now you can retrieve the data simply by calling the column by its name:

```
simpleData_np['b']
```

In return, you will get the following output:

Out[9]:		b
0	2	
1	5	

As you can see, defining the fields transposes the NumPy array and, now, each element of the array forms a *row*, unlike when we first created the `simpleData_np`.

Working with pandas' DataFrame

Since `pandas`' `DataFrame` internally uses NumPy data structures, translating a `DataFrame` to `DataShape` is effortless.

First, let's create a simple `DataFrame`. We start by importing `pandas`:

```
import pandas as pd
```

Next, we create a `DataFrame`:

```
simpleDf = pd.DataFrame([
    [1,2,3],
    [4,5,6]
], columns=['a', 'b', 'c'])
```

We then transform it into a `DataShape`:

```
simpleData_df = bl.Data(simpleDf)
```

You can retrieve data in the same manner as with the DataShape created from the NumPy array. Use the following command:

```
simpleData_df['a']
```

Then, it will produce the following output:

```
Out[13]:
```

	a
0	1
1	4

Working with files

A DataShape object can be created directly from a .csv file. In this example, we will use a dataset that consists of 404,536 traffic violations that happened in the Montgomery county of Maryland.



We downloaded the data from <https://catalog.data.gov/dataset/traffic-violations-56dda> on 8/23/16; the dataset is updated daily, so the number of traffic violations might differ if you retrieve the dataset at a later date.

We store the dataset in the `../Data` folder locally. However, we modified the dataset slightly so we could store it in the MongoDB: in its original form, with date columns, reading data back from MongoDB caused errors. We filed a bug with Blaze to fix this issue <https://github.com/blaze/blaze/issues/1580>:

```
import odo
traffic = bl.Data('../Data/TrafficViolations.csv')
```

If you do not know the names of the columns in any dataset, you can get these from the DataShape. To get a list of all the fields, you can use the following command:

```
print(traffic.fields)
```

```
['Stop_month', 'Stop_day', 'Stop_year', 'Stop_hr', 'Stop_min', 'Stop_sec', 'Agency', 'SubAgency', 'Description', 'Location', 'Latitude', 'Longitude', 'Accident', 'Belts', 'Personal_Injury', 'Property_Damage', 'Fatal', 'Commercial_License', 'HAZMAT', 'Commercial_Vehicle', 'Alcohol', 'Work_Zone', 'State', 'VehicleType', 'Year', 'Make', 'Model', 'Color', 'Violation_Type', 'Charge', 'Article', 'Contributed_To_Accident', 'Race', 'Gender', 'Driver_City', 'Driver_State', 'DL_State', 'Arrest_Type', 'Geolocation']
```




Those of you familiar with pandas would easily recognize the similarity between the `.fields` and `.columns` attributes, as these work in essentially the same way - they both return the list of columns (in the case of pandas DataFrame), or the list of fields, as columns are called in the case of Blaze DataShape.

Blaze can also read directly from a GZipped archive, saving space:

```
traffic_gz = bl.Data('../Data/TrafficViolations.csv.gz')
```

To validate that we get exactly the same data, let's retrieve the first two records from each structure. You can either call the following:

```
traffic.head(2)
```

Or you can choose to call:

```
traffic_gz.head(2)
```

It produces the same results (columns abbreviated here):

Out[17]:

	Stop_month	Stop_day	Stop_year	Stop_hr	Stop_min	Stop_sec	Agency
0	9	30	2014	23	51	0	MCP
1	3	31	2015	23	59	0	MCP

It is easy to notice, however, that it takes significantly more time to retrieve the data from the archived file because Blaze needs to decompress the data.

You can also read from multiple files at one time and create one big dataset. To illustrate this, we have split the original dataset into four GZipped datasets by year of violation (these are stored in the `../Data/Years` folder).

Blaze uses `odo` to handle saving DataShapes to a variety of formats. To save `traffic` data for traffic violations by year you can call `odo` like this:

```
import odo
for year in traffic.Stop_year.distinct().sort():
    odo.odo(traffic[traffic.Stop_year == year],
            '../Data/Years/TrafficViolations_{0}.csv.gz'\
            .format(year))
```

The preceding instruction saves the data into a GZip archive, but you can save it to any of the formats mentioned earlier. The first argument to the `.odo(...)` method specifies the input object (in our case, the DataShape with traffic violations that occurred in 2013), the second argument is the output object - the path to the file we want to save the data to. As we are about to learn - storing data is not limited to files only.

To read from multiple files you can use the asterisk character `*`:

```
traffic_multiple = bl.Data(
    '../Data/Years/TrafficViolations_*.csv.gz')
traffic_multiple.head(2)
```

The preceding snippet, once again, will produce a familiar table:

Out[19]:		Stop_month	Stop_day	Stop_year	Stop_hr	Stop_min	Stop_sec	Agency
0	3	29	2013	17	34	0	MCP	
1	8	12	2013	8	41	0	MCP	

Blaze reading capabilities are not limited to `.csv` or GZip files only: you can read data from JSON or Excel files (both, `.xls` and `.xlsx`), HDFS, or `bcolz` formatted files.



To learn more about the `bcolz` format, check its documentation at <https://github.com/Blosc/bcolz>.

Working with databases

Blaze can also easily read from SQL databases such as PostgreSQL or SQLite. While SQLite would normally be a local database, the PostgreSQL can be run either locally or on a server.

Blaze, as mentioned earlier, uses `odo` in the background to handle the communication to and from the databases.



`odo` is one of the requirements for Blaze and it gets installed along with the package. Check it out here <https://github.com/blaze/odo>.

In order to execute the code in this section, you will need two things: a running local instance of a PostgreSQL database, and a locally running MongoDB database.



In order to install PostgreSQL, download the package from <http://www.postgresql.org/download/> and follow the installation instructions for your operating system found there.

To install MongoDB, go to <https://www.mongodb.org/downloads> and download the package; the installation instructions can be found here <http://docs.mongodb.org/manual/installation/>.

Before you proceed, we assume that you have a PostgreSQL database up and running at `http://localhost:5432/`, and MongoDB database running at `http://localhost:27017`.

We have already loaded the traffic data to both of the databases and stored them in the `traffic` table (PostgreSQL) or the `traffic` collection (MongoDB).



If you do not know how to upload your data, I have explained this in my other book <https://www.packtpub.com/big-data-and-business-intelligence/practical-data-analysis-cookbook>.

Interacting with relational databases

Let's read the data from the PostgreSQL database now. The **Uniform Resource Identifier (URI)** for accessing a PostgreSQL database has the following syntax `postgresql://<user_name>:<password>@<server>:<port>/<database>::<table>`.

To read the data from PostgreSQL, you just wrap the URI around `.Data(...)` - Blaze will take care of the rest:

```
traffic_psql = bl.Data(
    'postgresql://{0}:{1}@localhost:5432/drabast::traffic'\
    .format('<your_username>', '<your_password>')
)
```

We use Python's `.format(...)` method to fill in the string with the appropriate data.



Substitute your credentials to access your PostgreSQL database in the previous example. If you want to read more about the `.format(...)` method, you can check out the Python 3.5 documentation <https://docs.python.org/3/library/string.html#format-string-syntax>.

It is quite easy to output the data to either the PostgreSQL or SQLite databases. In the following example, we will output traffic violations that involved cars manufactured in 2016 to both PostgreSQL and SQLite databases. As previously noted, we will use `odo` to manage the transfers:

```
traffic_2016 = traffic_psql[traffic_psql['Year'] == 2016]
# Drop commands
# odo.drop('sqlite:///traffic_local.sqlite::traffic2016')
# odo.drop('postgresql://{0}:{1}@localhost:5432/drabast::traffic'\
# .format('<your_username>', '<your_password>'))
# Save to SQLite
odo.odo(traffic_2016,
    'sqlite:///traffic_local.sqlite::traffic2016')
# Save to PostgreSQL
odo.odo(traffic_2016,
    'postgresql://{0}:{1}@localhost:5432/drabast::traffic'\
    .format('<your_username>', '<your_password>'))
```

In a similar fashion to `pandas`, to filter the data, we effectively select the `Year` column (the `traffic_psql['Year']` part of the first line) and create a Boolean flag by checking whether each and every record in that column equals 2016. By indexing the `traffic_psql` object with such a truth vector, we extract only the records where the corresponding value equals `True`.

The two commented out lines should be uncommented if you already have the `traffic2016` tables in your databases; otherwise `odo` will append the data to the end of the table.

The URI for SQLite is slightly different than the one for PostgreSQL; it has the following syntax `sqlite://</relative/path/to/db.sqlite>::<table_name>`.

Reading data from the SQLite database should be trivial for you by now:

```
traffic_sqlt = bl.Data(  
    'sqlite:///traffic_local.sqlite::traffic2016'  
)
```

Interacting with the MongoDB database

MongoDB has gained lots of popularity over the years. It is a simple, fast, and flexible document-based database. The database is a go-to storage solution for all full-stack developers, using the `MEAN.js` stack: M here stands for Mongo (see <http://meanjs.org>).

Since Blaze is meant to work in a very familiar way no matter what your data source, reading from MongoDB is very similar to reading from PostgreSQL or SQLite databases:

```
traffic_mongo = bl.Data(  
    'mongodb://localhost:27017/packt::traffic'  
)
```

Data operations

We have already presented some of the most common methods you will use with DataShapes (for example, `.peek()`), and ways to filter the data based on the column value. Blaze has implemented many methods that make working with any data extremely easy.

In this section, we will review a host of other commonly used ways of working with data and methods associated with them. For those of you coming from pandas and/or SQL, we will provide a respective syntax where equivalents exist.

Accessing columns

There are two ways of accessing columns: you can get a single column at a time by accessing them as if they were a DataShape attribute:

```
traffic.Year.head(2)
```

The preceding script produces the following output:

Out[28]:

	Year
0	2014.0
1	2003.0

You can also use indexing that allows the selection of more than one column at a time:

```
(traffic[['Location', 'Year', 'Accident', 'Fatal', 'Alcohol']]
.head(2))
```

This generates the following output:

Out[29]:

	Location	Year	Accident	Fatal	Alcohol
0	PARK RD AT HUNGERFORD DR	2014.0	No	No	No
1	CONNECTICUT AT METROPOLITAN AVE	2003.0	No	No	No

The preceding syntax would be the same for pandas DataFrames. For those of you unfamiliar with Python and pandas API, please note three things here:

1. To specify multiple columns, you need to enclose them in another list: note the double brackets `[ [ and ] ]`.
2. If the chain of all methods does not fit on one line (or you want to break the chain for better readability) you have two choices: either enclose the whole chain of methods in brackets `(...)` where the `...` is the chain of all methods, or, before breaking into the new line, put the backslash character `\` at the end of every line in the chain. We prefer the latter and will use that in our examples from now on.
3. Note that the equivalent SQL code would be:

```
SELECT *
FROM traffic
LIMIT 2
```

Symbolic transformations

The beauty of Blaze comes from the fact that it can operate *symbolically*. What this means is that you can specify transformations, filters, or other operations on your data and store them as object(s). You can then *feed* such object with almost any form of data conforming to the original schema, and Blaze will return the transformed data.

For example, let's select all the traffic violations that occurred in 2013, and return only the 'Arrest_Type', 'Color', and 'Charge` columns. First, if we could not reflect the schema from an already existing object, we would have to specify the schema manually. To do this, we will use the `.symbol(...)` method to achieve that; the first argument to the method specifies a symbolic name of the transformation (we prefer keeping it the same as the name of the object, but it can be anything), and the second argument is a long string that specifies the schema in a `<column_name>: <column_type>` fashion, separated by commas:

```
schema_example = bl.symbol('schema_exempl',
                             '{id: int, name: string}')
```

Now, you could use the `schema_example` object and specify some transformations. However, since we already have an existing traffic dataset, we can *reuse* the schema by using `traffic.dshape` and specifying our transformations:

```
traffic_s = bl.symbol('traffic', traffic.dshape)
traffic_2013 = traffic_s[traffic_s['Stop_year'] == 2013] [
    ['Stop_year', 'Arrest_Type', 'Color', 'Charge']
]
```

To present how this works, let's read the original dataset into pandas' DataFrame:

```
traffic_pd = pd.read_csv('../Data/TrafficViolations.csv')
```

Once read, we pass the dataset directly to the `traffic_2013` object and perform the computation using the `.compute(...)` method of Blaze; the first argument to the method specifies the transformation object (ours is `traffic_2013`) and the second parameter is the data that the transformations are to be performed on:

```
bl.compute(traffic_2013, traffic_pd).head(2)
```

Here is the output of the preceding snippet:

Out[33]:					
	Stop_year	Arrest_Type	Color	Charge	
73	2013	A - Marked Patrol	SILVER	13-409(b)	
215	2013	B - Unmarked Patrol	BLACK	21-309(b)	

You can also pass a list of lists or a list of NumPy arrays. Here, we use the `.values` attribute of the DataFrame to access the underlying list of NumPy arrays that form the DataFrame:

```
bl.compute(traffic_2013, traffic_pd.values)[0:2]
```

This code will produce precisely what we would expect:

```
[2013 'A - Marked Patrol' 'SILVER' '13-409(b)']
[2013 'B - Unmarked Patrol' 'BLACK' '21-309(b)']
```

Operations on columns

Blaze allows for easy mathematical operations to be done on numeric columns. All the traffic violations cited in the dataset occurred between 2013 and 2016. You can check that by getting all the distinct values for the `Stop_year` column using the `.distinct()` method. The `.sort()` method sorts the results in an ascending order:

```
traffic['Stop_year'].distinct().sort()
```

The preceding code produces the following output table:

Out[35]:

	Stop_year
2	2013
0	2014
1	2015
3	2016

An equivalent syntax for pandas would be as follows:

```
traffic['Stop_year'].unique().sort()
```

For SQL, use the following code:

```
SELECT DISTINCT Stop_year
FROM traffic
```

You can also make some mathematical transformations/arithmetic to the columns. Since all the traffic violations occurred after year 2000, we can subtract 2000 from the `Stop_year` column without losing any accuracy:

```
traffic['Stop_year'].head(2) - 2000
```

Here is what you should get in return:

Out[36]:

	Stop_year
0	14
1	15

The same could be attained from pandas `DataFrame` with an identical syntax (assuming `traffic` was of pandas `DataFrame` type). For SQL, the equivalent would be:

```
SELECT Stop_year - 2000 AS Stop_year
FROM traffic
```

However, if you want to do some more complex mathematical operations (for example, `log` or `pow`) then you first need to use the one provided by Blaze (that, in the background, will translate your command to a suitable method from NumPy, math, or pandas).

For example, if you wanted to log-transform the `Stop_year` you need to use this code:

```
bl.log(traffic['Stop_year']).head(2)
```

This will produce the following output:

Out[37]:							
	<table><tr><th></th><th>Stop_year</th></tr><tr><td>0</td><td>7.607878</td></tr><tr><td>1</td><td>7.608374</td></tr></table>		Stop_year	0	7.607878	1	7.608374
	Stop_year						
0	7.607878						
1	7.608374						

Reducing data

Some reduction methods are also available, such as `.mean()` (that calculates the average), `.std` (that calculates standard deviation), or `.max()` (that returns the maximum from the list). Executing the following code:

```
traffic['Stop_year'].max()
```

It will return the following output:

Out[38]: 2016

If you had a pandas `DataFrame` you can use the same syntax, whereas for SQL the same could be done with the following code:

```
SELECT MAX(Stop_year) AS Stop_year_max
FROM traffic
```

It is also quite easy to add more columns to your dataset. Say, you wanted to calculate the age of the car (in years) at the time when the violation occurred. First, you would take the `Stop_year` and subtract the `Year of manufacture`.

In the following code snippet, the first argument to the `.transform(...)` method is the DataShape, the transformation is to be performed on, and the other(s) would be a list of transformations.

```
traffic = bl.transform(traffic,
                      Age_of_car = traffic.Stop_year - traffic.Year)
traffic.head(2)
```



In the source code of the `.transform(...)` method such lists would be expressed as `*args` as you could specify more than one column to be created in one go. The `*args` argument to any method would take any number of subsequent arguments and treat it as if it was a list.

The above code produces the following table:

Out[9]:			
	Stop_year	Year	Age_of_car
0	2014	2014.0	0.0
1	2015	2003.0	12.0

An equivalent operation in pandas could be attained through the following code:

```
traffic['Age_of_car'] = traffic.apply(
    lambda row: row.Stop_year - row.Year,
    axis = 1
)
```

For SQL you can use the following code:

```
SELECT *
    , Stop_year - Year AS Age_of_car
FROM traffic
```

If you wanted to calculate the average age of the car involved in a fatal traffic violation and count the number of occurrences, you can perform a `group by` operation using the `.by(...)` operation:

```
bl.by(traffic['Fatal'],
     Fatal_AvgAge=traffic.Age_of_car.mean(),
     Fatal_Count =traffic.Age_of_car.count()
)
```

The first argument to `.by(...)` specifies the column of the DataShape to perform the aggregation by, followed by a series of aggregations we want to get. In this example, we select the `Age_of_car` column and calculate an average and count the number of rows per each value of the `'Fatal'` column.

The preceding script produces the following aggregation:

Out[40]:			
	Fatal	Fatal_AvgAge	Fatal_Count
0	No	9.580998	404418
1	Yes	8.798246	116

For pandas, an equivalent would be as follows:

```
traffic\  
  .groupby('Fatal')['Age_of_car']\  
  .agg({  
      'Fatal_AvgAge': np.mean,  
      'Fatal_Count':  np.count_nonzero  
  })
```

For SQL, it would be as follows:

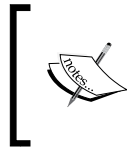
```
SELECT Fatal  
      , AVG(Age_of_car)    AS Fatal_AvgAge  
      , COUNT(Age_of_car) AS Fatal_Count  
FROM traffic  
GROUP BY Fatal
```

Joins

Joining two DataShapes is straightforward as well. To present how this is done, although the same result could be attained differently, we first select all the traffic violations by violation type (the `violation` object) and the traffic violations involving belts (the `belts` object):

```
violation = traffic[  
    ['Stop_month', 'Stop_day', 'Stop_year',  
     'Stop_hr', 'Stop_min', 'Stop_sec', 'Violation_Type']]  
belts = traffic[  
    ['Stop_month', 'Stop_day', 'Stop_year',  
     'Stop_hr', 'Stop_min', 'Stop_sec', 'Belts']]
```

Now, we join the two objects on the six date and time columns.



The same effect could have been attained if we just simply selected the two columns: `Violation_type` and `Belts` in one go. However, this example is to show the mechanics of the `.join(...)` method, so bear with us.

The first argument to the `.join(...)` method is the first DataShape we want to join with, the second argument is the second DataShape, while the third argument can be either a single column or a list of columns to perform the join on:

```
violation_belts = bl.join(violation, belts,
                          ['Stop_month', 'Stop_day', 'Stop_year',
                           'Stop_hr', 'Stop_min', 'Stop_sec'])
```

Once we have the full dataset in place, let's check how many traffic violations involved belts and what sort of punishment was issued to the driver:

```
bl.by(violation_belts[['Violation_Type', 'Belts']],
      Violation_count=violation_belts.Belts.count()
).sort('Violation_count', ascending=False)
```

Here's the output of the preceding script:

Out[43]:

	Violation_Type	Belts	Violation_count
0	Citation	No	989728
5	Warning	No	439490
2	ESERO	No	56447
1	Citation	Yes	35596
6	Warning	Yes	12245
3	ESERO	Yes	1327
4	SERO	No	3

The same could be achieved in pandas with the following code:

```
violation.merge(belts,
                on=['Stop_month', 'Stop_day', 'Stop_year',
                    'Stop_hr', 'Stop_min', 'Stop_sec']) \
.groupby(['Violation_type', 'Belts']) \
.agg({
    'Violation_count': np.count_nonzero
}) \
.sort('Violation_count', ascending=False)
```

With SQL, you would use the following snippet:

```
SELECT innerQuery.*
FROM (
    SELECT a.Violation_type
        , b.Belts
        , COUNT() AS Violation_count
    FROM violation AS a
    INNER JOIN belts AS b
        ON      a.Stop_month = b.Stop_month
           AND a.Stop_day   = b.Stop_day
           AND a.Stop_year  = b.Stop_year
           AND a.Stop_hr    = b.Stop_hr
           AND a.Stop_min   = b.Stop_min
           AND a.Stop_sec   = b.Stop_sec
    GROUP BY Violation_type
        , Belts
) AS innerQuery
ORDER BY Violation_count DESC
```

Summary

The concepts presented in this chapter are just the beginning of the road to using Blaze. There are many other ways it can be used and data sources it can connect with. Treat this as a starting point to build your understanding of polyglot persistence.

Note, however, that these days most of the concepts explained in this chapter can be attained natively within Spark, as you can use SQLAlchemy directly within Spark making it easy to work with a variety of data sources. The advantage of doing so, despite the initial investment of learning the API of SQLAlchemy, is that the data returned will be stored in a Spark DataFrame and you will have access to everything that PySpark has to offer. This, by no means, implies that you never should never use Blaze: the choice, as always, is yours.

In the next chapter, you will learn about streaming and how to do it with Spark. Streaming has become an increasingly important topic these days, as, daily (true as of 2016), the world produces roughly 2.5 exabytes of data (source: <http://www.northeastern.edu/levelblog/2016/05/13/how-much-data-produced-every-day/>) that need to be ingested, processed and made sense of.

10

Structured Streaming

This chapter will provide a jump-start on the concepts behind Spark Streaming and how this has evolved into Structured Streaming. An important aspect of Structured Streaming is that it utilizes Spark DataFrames. This shift in paradigm will make it easier for Python developers to start working with Spark Streaming.

In this chapter, you will learn:

- What is Spark Streaming?
- Why do we need Spark Streaming?
- What is the Spark Streaming application data flow?
- Simple streaming application using DStream
- A quick primer on Spark Streaming global aggregations
- Introducing Structured Streaming

Note, for the initial sections of this chapter, the example code used will be in Scala, as this was how most Spark Streaming code was written. When we start focusing on Structured Streaming, we will work with Python examples.

What is Spark Streaming?

At its core, Spark Streaming is a scalable, fault-tolerant streaming system that takes the RDD batch paradigm (that is, processing data in batches) and speeds it up. While it is a slight over-simplification, basically Spark Streaming operates in mini-batches or batch intervals (from 500ms to larger interval windows).

As noted in the following diagram, Spark Streaming receives an input data stream and internally breaks that data stream into multiple smaller batches (the size of which is based on the *batch interval*). The Spark engine processes those batches of input data to a result set of batches of processed data.



Source: Apache Spark Streaming Programming Guide at: <http://spark.apache.org/docs/latest/streaming-programming-guide.html>

The key abstraction for Spark Streaming is Discretized Stream (DStream), which represents the previously mentioned small batches that make up the stream of data. DStreams are built on RDDs, allowing Spark developers to work within the same context of RDDs and batches, only now applying it to their streaming problems. Also, an important aspect is that, because you are using Apache Spark, Spark Streaming integrates with MLlib, SQL, DataFrames, and GraphX.

The following figure denotes the basic components of Spark Streaming:



Source: Apache Spark Streaming Programming Guide at: <http://spark.apache.org/docs/latest/streaming-programming-guide.html>

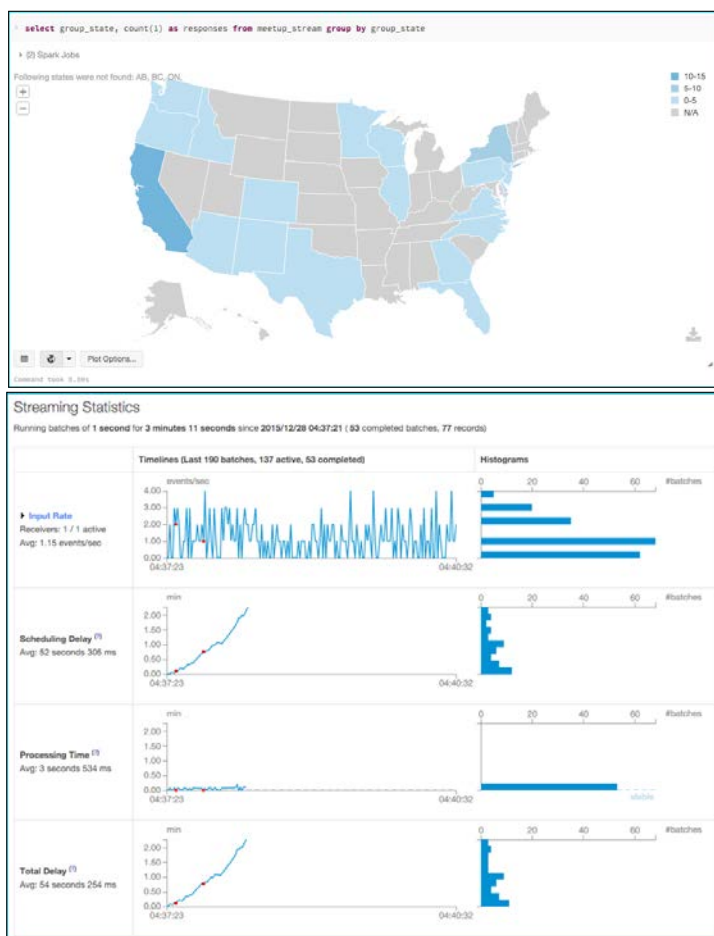
Spark Streaming is a high-level API that provides fault-tolerant *exactly-once* semantics for stateful operations. Spark Streaming has built in *receivers* that can take on many sources, with the most common being Apache Kafka, Flume, HDFS/S3, Kinesis, and Twitter. For example, the most commonly used integration between Kafka and Spark Streaming is well documented in the Spark Streaming + Kafka Integration Guide found at: <https://spark.apache.org/docs/latest/streaming-kafka-integration.html>.

Also, you can create your own *custom receiver*, such as the Meetup Receiver (<https://github.com/actions/meetup-stream/blob/master/src/main/scala/receiver/MeetupReceiver.scala>), which allows you to read the Meetup Streaming API (https://www.meetup.com/meetup_api/docs/stream/2/rsvps/) using Spark Streaming.

Watch the Meetup Receiver in Action

If you are interested in seeing the Spark Streaming Meetup Receiver in action, you can refer to the Databricks notebooks at: <https://github.com/dennyglee/databricks/tree/master/notebooks/Users/denny%40databricks.com/content/Streaming%20Meetup%20RSVPs> which utilize the previously mentioned Meetup Receiver.

The following is a screenshot of the notebook in action left window, while viewing the Spark UI (Streaming Tab) on the right.



You will be able to use Spark Streaming to receive Meetup RSVPs from around the country (or world) and get a near real-time summary of Meetup RSVPs by state (or country). Note, these notebooks are currently written in Scala.

Why do we need Spark Streaming?

As noted by Tathagata Das – committer and member of the project management committee (PMC) to the Apache Spark project and lead developer of Spark Streaming – in the Datanami article *Spark Streaming: What is It and Who's Using it* (<https://www.datanami.com/2015/11/30/spark-streaming-what-is-it-and-whos-using-it/>), there is a *business need* for streaming. With the prevalence of online transactions and social media, as well as sensors and devices, companies are generating and processing more data at a faster rate.

The ability to develop actionable insight at scale and in real time provides those businesses with a competitive advantage. Whether you are detecting fraudulent transactions, providing real-time detection of sensor anomalies, or reacting to the next viral tweet, streaming analytics is becoming increasingly important in data scientists' and data engineer's toolbox.

The reason Spark Streaming is itself being rapidly adopted is because Apache Spark unifies all of these disparate data processing paradigms (Machine Learning via ML and MLlib, Spark SQL, and Streaming) within the same framework. So, you can go from training machine learning models (ML or MLlib), to scoring data with these models (Streaming) and perform analysis using your favourite BI tool (SQL) – all within the same framework. Companies including Uber, Netflix, and Pinterest often showcase their Spark Streaming use cases:

- *How Uber Uses Spark and Hadoop to Optimize Customer Experience*: <https://www.datanami.com/2015/10/05/how-uber-uses-spark-and-hadoop-to-optimize-customer-experience/>
- *Spark and Spark Streaming at Netflix*: <https://spark-summit.org/2015/events/spark-and-spark-streaming-at-netflix/>
- *Can Spark Streaming survive Chaos Monkey?* <http://techblog.netflix.com/2015/03/can-spark-streaming-survive-chaos-monkey.html>
- *Real-time analytics at Pinterest*: <https://engineering.pinterest.com/blog/real-time-analytics-pinterest>

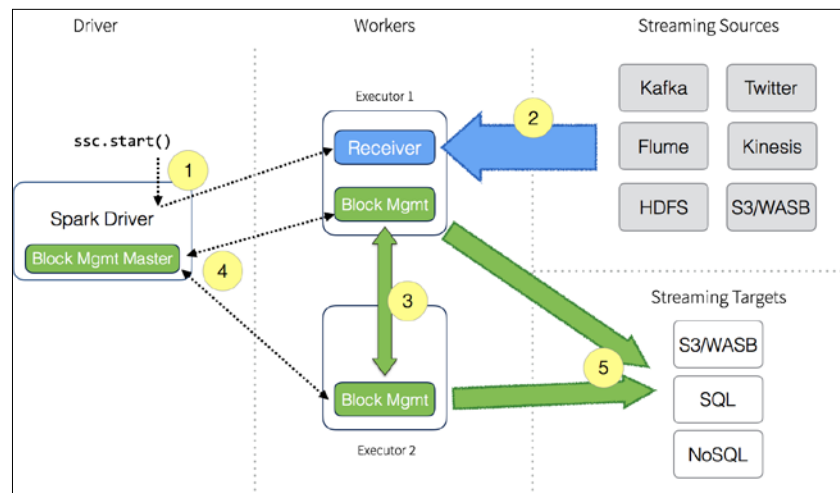
Currently, there are four broad use cases surrounding Spark Streaming:

- **Streaming ETL**: Data is continuously being cleansed and aggregated prior to being pushed downstream. This is commonly done to reduce the amount of data to be stored in the final data store.
- **Triggers**: Real-time detection of behavioral or anomaly events trigger immediate and downstream actions. For example, a device that is within the proximity of a detector or beacon will trigger an alert.

- **Data enrichment:** Real-time data joined to other datasets allowing for richer analysis. For example, including real-time weather information with flight information to build better travel alerts.
- **Complex sessions and continuous learning:** Multiple sets of events associated with real-time streams are continuously analyzed and/or updating machine learning models. For example, the stream of user activity associated with an online game that allows us to better segment the user.

What is the Spark Streaming application data flow?

The following figure provides the data flow between the Spark driver, workers, streaming sources and targets:



It all starts with the Spark Streaming Context, represented by `ssc.start()` in the preceding figure:

1. When the Spark Streaming Context starts, the driver will execute a long-running task on the executors (that is, the Spark workers).
2. The **Receiver** on the executors (**Executor 1** in this diagram) receives a data stream from the Streaming Sources. With the incoming data stream, the receiver divides the stream into blocks and keeps these blocks in memory.
3. These blocks are also replicated to another executor to avoid data loss.

4. The block ID information is transmitted to the **Block Management Master** on the driver.
5. For every batch interval configured within Spark Streaming Context (commonly this is every 1 second), the driver will launch Spark tasks to process the blocks. Those blocks are then persisted to any number of target data stores, including cloud storage (for example, S3, WASB, and so on), relational data stores (for example, MySQL, PostgreSQL, and so on), and NoSQL stores.

Suffice it to say, there are a lot of moving parts for a streaming application that need to be continually optimized and configured. Most of the documentation for Spark Streaming is more complete in Scala, so, as you are working with the Python APIs, you may sometimes need to reference the Scala version of the documentation instead. If this happens to you, please file a bug and/or fill out a PR if you have a proposed fix (<https://issues.apache.org/jira/browse/spark/>).

For a deeper dive on this topic, please refer to:

1. *Spark 1.6 Streaming Programming Guide*: <https://spark.apache.org/docs/1.6.0/streaming-programming-guide.html>
2. *Tathagata Das' Deep Dive with Spark Streaming (Spark Meetup 2013-06-17)*: <http://www.slideshare.net/spark-project/deep-divewithsparkstreaming-tathagatadassparkmeetup20130617>

Simple streaming application using DStreams

Let's create a simple word count example using Spark Streaming in Python. For this example, we will be working with DStream – the Discretized Stream of small batches that make up the stream of data. The example used for this section of the book can be found in its entirety at: https://github.com/drabastomek/learningPySpark/blob/master/Chapter10/streaming_word_count.py.

This word count example will use the Linux / Unix `nc` command – it is a simple utility that reads and writes data across network connections. We will use two different bash terminals, one using the `nc` command to send words to our computer's local port (9999) and one terminal that will run Spark Streaming to receive those words and count them. The initial set of commands for our script are noted here:

1. `# Create a local SparkContext and Streaming Contexts`
2. `from pyspark import SparkContext`
3. `from pyspark.streaming import StreamingContext`
- 4.

```

5. # Create sc with two working threads
6. sc = SparkContext("local[2]", "NetworkWordCount")
7.
8. # Create local StreamingContext with batch interval of 1 second
9. ssc = StreamingContext(sc, 1)
10.
11. # Create DStream that connects to localhost:9999
12. lines = ssc.socketTextStream("localhost", 9999)

```

Here are some important call outs for the preceding commands:

1. The `StreamingContext` on line 9 is the entry point into Spark Streaming
2. The 1 of `... (sc, 1)` on line 9 is the *batch interval*; in this case, we are running micro-batches every second.
3. The `lines` on line 12 is the `DStream` representing the data stream extracted via the `ssc.socketTextStream`.
4. As noted in the description, the `ssc.socketTextStream` is the Spark Streaming method to review a text stream for a particular socket; in this case, your local computer on socket 9999.

The next few lines of code (as described in the comments), split the `lines DStream` into words and then, using `RDDs`, count each word in each batch of data and print this information out to the console (line number 9):

```

1. # Split lines into words
2. words = lines.flatMap(lambda line: line.split(" "))
3.
4. # Count each word in each batch
5. pairs = words.map(lambda word: (word, 1))
6. wordCounts = pairs.reduceByKey(lambda x, y: x + y)
7.
8. # Print the first ten elements of each RDD in this DStream
9. wordCounts.pprint()

```

The final set of lines of the code start Spark Streaming (`ssc.start()`) and then await a termination command to stop running (for example, `<Ctrl><C>`). If no termination command is sent, then the Spark Streaming program will continue running.

```

# Start the computation
ssc.start()

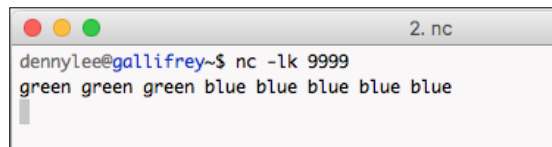
# Wait for the computation to terminate
ssc.awaitTermination()

```

Now that you have your script, as noted earlier, open two terminal windows – one for your `nc` command, and one for your Spark Streaming Program. To start the `nc` command, from one of your terminals, type:

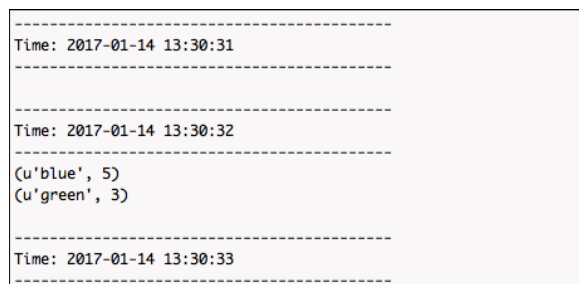
```
nc -lk 9999
```

Everything you type from this point onwards in that terminal will be transmitted to port 9999, as noted in the following screenshot:

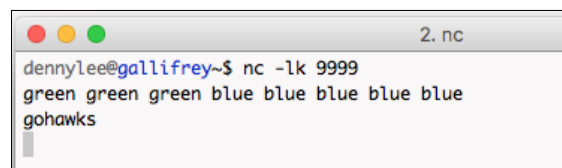


In this example (as noted previously), I typed the words **green** three times and **blue** five times. From the other terminal screen, let's run the Python streaming script you just created. In this example, I named the script `streaming_word_count.py`.
`./bin/spark-submit streaming_word_count.py localhost 9999.`

The command will run the `streaming_word_count.py` script, reading your local computer (that is, `localhost`) port 9999 to receive any words sent to that socket. As you have already sent information to the port on the first screen, shortly after starting up the script, your Spark Streaming program will read the words sent to port 9999 and perform a word count as noted in the following screenshot:



The `streaming_word_count.py` script will continue to read and print any new information to the console. Going back to our first terminal (with the `nc` command), we now can type our next set of words, as noted in the following screenshot:



Reviewing the streaming script in the second terminal, you will notice that this script continues to run every second (that is, the configured *batch interval*), and you will notice the calculated word count for *gohawks* a few seconds later:

```
-----  
Time: 2017-01-14 13:30:31  
-----  
  
-----  
Time: 2017-01-14 13:30:32  
-----  
(u'blue', 5)  
(u'green', 3)  
-----  
  
-----  
Time: 2017-01-14 13:30:33  
-----  
  
-----  
Time: 2017-01-14 13:30:34  
-----  
  
-----  
Time: 2017-01-14 13:30:35  
-----  
(u'gohawks', 1)  
-----  
  
-----  
Time: 2017-01-14 13:30:36  
-----  
  
-----  
Time: 2017-01-14 13:30:37  
-----
```

With this relatively simple script, now you can see Spark Streaming in action with Python. But if you continue typing words into the `nc` terminal, you will notice that this information is not aggregated. For example, if we continue to write *green* in the `nc` terminal (as noted here):

```
dennylee@gallifrey~$ nc -lk 9999  
green green blue blue blue blue blue  
gohawks  
green green  
█
```

The Spark Streaming terminal will report the current snapshot of data; that is, the two additional green values (as noted here):

```
-----
Time: 2017-01-16 17:19:38
-----

-----
Time: 2017-01-16 17:19:39
-----
(u'blue', 5)
(u'green', 2)
-----

Time: 2017-01-16 17:19:40
-----
(u'gohawks', 1)
-----

Time: 2017-01-16 17:19:41
-----

-----
Time: 2017-01-16 17:19:42
-----

-----
Time: 2017-01-16 17:19:43
-----
(u'green', 2)
-----
```

What did not happen was the concept of global aggregations, where we would keep *state* for this information. What this means is that, instead of reporting 2 new greens, we could get Spark Streaming to give us the overall counts of green, for example, 7 greens, 5 blues, and 1 gohawks. We will talk about global aggregations in the form of `UpdateStateByKey` / `mapWithState` in the next section.



For other good PySpark Streaming examples, check out:

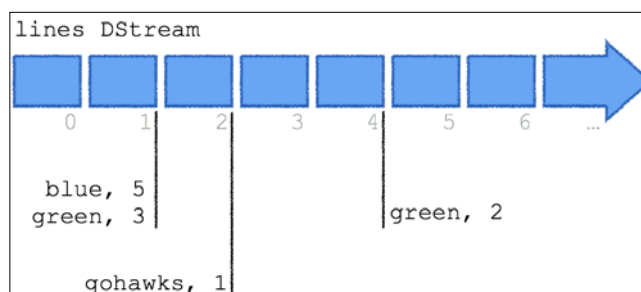
Network Wordcount (in Apache Spark GitHub repo): https://github.com/apache/spark/blob/master/examples/src/main/python/streaming/network_wordcount.py

Python Streaming Examples: <https://github.com/apache/spark/tree/master/examples/src/main/python/streaming>

S3 FileStream Wordcount (Databricks notebook): https://docs.cloud.databricks.com/docs/latest/databricks_guide/index.html#07%20Spark%20Streaming/06%20FileStream%20Word%20Count%20-%20Python.html

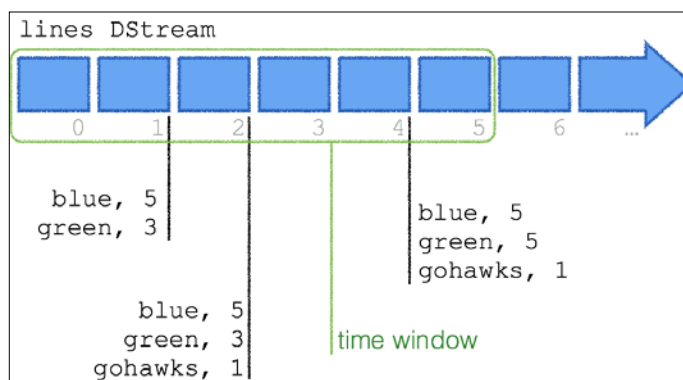
A quick primer on global aggregations

As noted in the previous section, so far, our script has performed a point in time streaming word count. The following diagram denotes the **lines DStream** and its micro-batches as per how our script had executed in the previous section:



At the 1 second mark, our Python Spark Streaming script returned the value of $\{(blue, 5), (green, 3)\}$, at the 2 second mark it returned $\{(gohawks, 1)\}$, and at the 4 second mark, it returned $\{(green, 2)\}$. But what if you had wanted the aggregate word count over a specific time window?

The following figure represents us calculating a stateful aggregation:

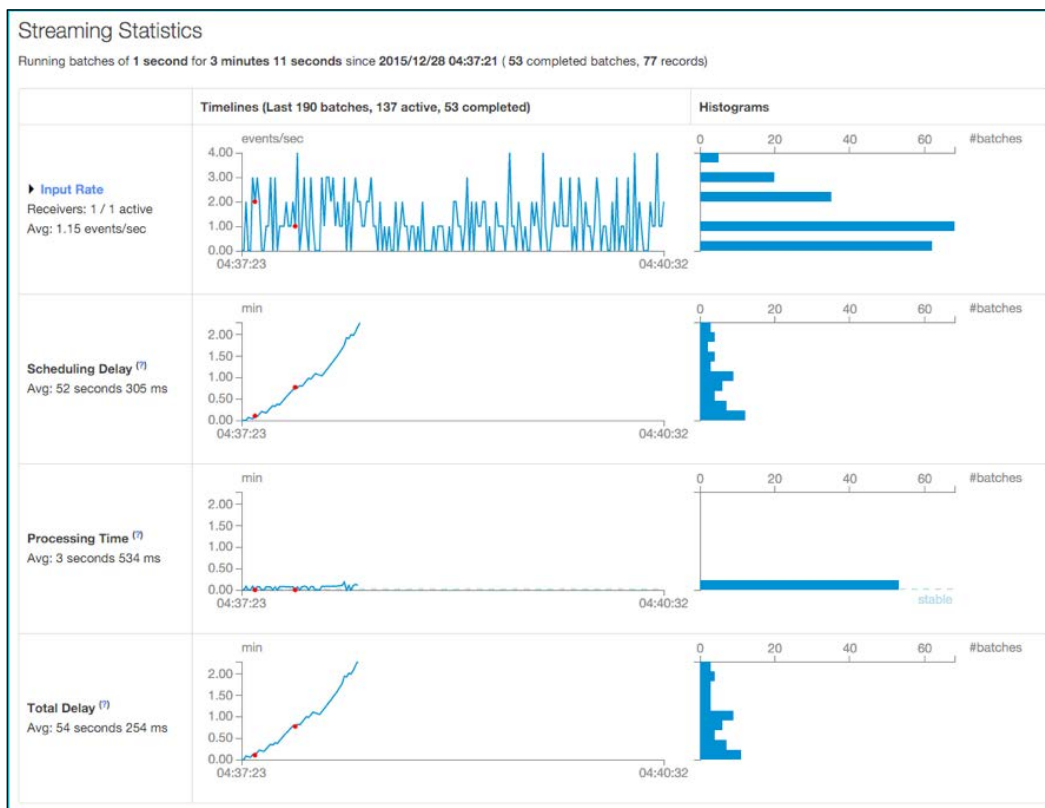


In this case, we have a time window between 0-5 seconds. Note, that in our script we have not got the specified time window: each second, we calculate the cumulative sum of the words. Therefore, at the 2 second mark, the output is not just the green and blue from the 1 second mark, but it also includes the gohawks from the 2 second mark: $\{(blue, 5), (green, 3), (gohawks, 1)\}$. At the 4 second mark, the additional 2 greens provide us a total of $\{(blue, 5), (green, 5), (gohawks, 1)\}$.

Structured Streaming

For those of you who regularly work with relational databases, this seems to be just a `GROUP BY, SUM()` statement. Yet, in the case of streaming analytics, the duration to persist the data long enough to run a `GROUP BY, SUM()` statement is longer than the *batch interval* (for example, 1 second). This means that we would constantly be running behind and trying to catch up with the data stream.

For example, if you were to run the *1. Streaming and DataFrames.scala* Databricks notebook at <https://github.com/dennyglee/databricks/blob/master/notebooks/Users/denny%40databricks.com/content/Streaming%20Meetup%20RSVPs/1.%20Streaming%20and%20DataFrames.scala>, and you were to view the Streaming jobs in the Spark UI, you would get something like the following figure:



Notice in the graph that the **Scheduling Delay** and **Total Delay** numbers are rapidly increasing (for example, average Total Delay is **54 seconds 254 ms** and the actual Total Delay is > 2min) and way outside the *batch interval* threshold of 1 second. The reason we see this delay is because, inside the streaming code for that notebook, we had also run the following code:

```
// Populate `meetup_stream` table
sqlContext.sql("insert into meetup_stream select * from meetup_stream_
json")
```

That is, inserting any new chunks of data (that is, 1 second RDD micro-batches), converting them into a DataFrame (meetup_stream_json table), and inserting the data into a persistent table (meetup_stream table). Persisting the data in this fashion led to slow streaming performance with the ever-increasing scheduling delays. To solve this problem via *streaming analytics*, this is where creating global aggregations via UpdateStateByKey (Spark 1.5 and before) or mapWithState (Spark 1.6 onwards) come in.



For more information on Spark Streaming visualizations, please take the time to review *New Visualizations for Understanding Apache Spark Streaming Applications*: <https://databricks.com/blog/2015/07/08/new-visualizations-for-understanding-apache-spark-streaming-applications.html>.

Knowing this, let's re-write the original `streaming_word_count.py` so that we now have a *stateful* version called `stateful_streaming_word_count.py`; you can get the full version of this script at https://github.com/drabastomek/learningPySpark/blob/master/Chapter10/stateful_streaming_word_count.py.

The initial set of commands for our script are noted here:

```
1. # Create a local SparkContext and Streaming Contexts
2. from pyspark import SparkContext
3. from pyspark.streaming import StreamingContext
4.
5. # Create sc with two working threads
6. sc = SparkContext("local[2]", "StatefulNetworkWordCount")
7.
8. # Create local StreamingContext with batch interval of 1 sec
9. ssc = StreamingContext(sc, 1)
10.
11. # Create checkpoint for local StreamingContext
12. ssc.checkpoint("checkpoint")
```

```
13.
14. # Define updateFunc: sum of the (key, value) pairs
15. def updateFunc(new_values, last_sum):
16.     return sum(new_values) + (last_sum or 0)
17.
18. # Create DStream that connects to localhost:9999
19. lines = ssc.socketTextStream("localhost", 9999)
```

If you recall `streaming_word_count.py`, the primary differences start at line 11:

- The `ssc.checkpoint("checkpoint")` on line 12 configures a Spark Streaming *checkpoint*. To ensure that Spark Streaming is fault tolerant due to its continual operation, it needs to checkpoint enough information to fault-tolerant storage, so it can recover from failures. Note, we will not dive deep into this concept (though more information is available in the following *Tip* section), as many of these configurations will be abstracted away with Structured Streaming.
- The `updateFunc` on line 15 tells the program to update the application's *state* (later in the code) via `UpdateStateByKey`. In this case, it is returning a sum of the previous value (`last_sum`) and the sum of the new values (`sum(new_values) + (last_sum or 0)`).
- At line 19, we have the same `ssc.socketTextStream` as the previous script.



For more information on Spark Streaming *checkpoint*, some good references are:

Spark Streaming Programming Guide > Checkpoint: <https://spark.apache.org/docs/1.6.0/streaming-programming-guide.html#checkpointing>

Exploring Stateful Streaming with Apache Spark: <http://asyncified.io/2016/07/31/exploring-stateful-streaming-with-apache-spark/>

The final section of the code is as follows:

```
1. # Calculate running counts
2. running_counts = lines.flatMap(lambda line: line.split(" "))\
3.     .map(lambda word: (word, 1))\
4.     .updateStateByKey(updateFunc)
5.
6. # Print the first ten elements of each RDD generated in this
7. # stateful DStream to the console
8. running_counts.pprint()
```

```

9.
10. # Start the computation
11. ssc.start()
12.
13. # Wait for the computation to terminate
14. ssc.awaitTermination()

```

While lines 10-14 are identical to the previous script, the difference is that we now have a `running_counts` variable that splits to get the words and runs a map function to count each word in each batch (in the previous script this was the `words` and `pairs` variables).

The primary difference is the use of the `updateStateByKey` method, which will execute the previously noted `updateFunc` that performs the sum. `updateStateByKey` is Spark Streaming's method to perform calculations against your stream of data and update the state for each key in a performant manner. It is important to note that you would typically use `updateStateByKey` for Spark 1.5 and earlier; the performance of these *stateful* global aggregations is proportional to the *size of the state*. From Spark 1.6 onwards, you should use `mapWithState`, as the performance is proportional to the *size of the batch*.



Note, there is more code typically involved with `mapWithState` (in comparison to `updateStateByKey`), hence the examples were written using `updateStateByKey`.

For more information about stateful Spark Streaming, including the use of `mapWithState`, please refer to:

Stateful Network Wordcount Python example: https://github.com/apache/spark/blob/master/examples/src/main/python/streaming/stateful_network_wordcount.py

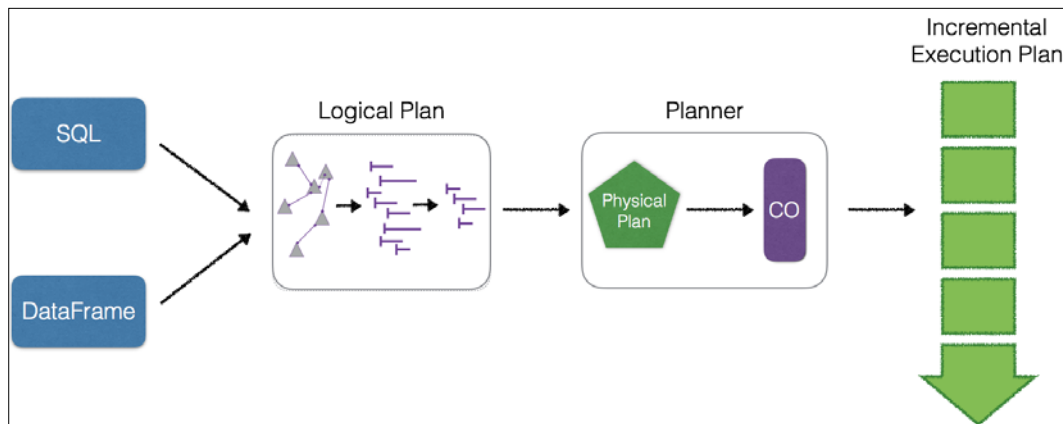
Global Aggregation using `mapWithState` (Scala): https://docs.cloud.databricks.com/docs/latest/databricks_guide/index.html#07%20Spark%20Streaming/12%20Global%20Aggregations%20-%20mapWithState.html

Word count using `mapWithState` (Scala): <https://docs.cloud.databricks.com/docs/spark/1.6/examples/Streaming%20mapWithState.html>

Faster Stateful Stream Processing in Apache Spark Streaming: <https://databricks.com/blog/2016/02/01/faster-stateful-stream-processing-in-apache-spark-streaming.html>

Introducing Structured Streaming

With Spark 2.0, the Apache Spark community is working on simplifying streaming by introducing the concept of *structured streaming* which bridges the concepts of streaming with Datasets/DataFrames (as noted in the following diagram):



As noted in earlier chapters on DataFrames, the execution of SQL and/or DataFrame queries within the Spark SQL Engine (and Catalyst Optimizer) revolves around building a logical plan, building numerous physical plans, the engine choosing the correct physical plan based on its cost optimizer, and then generating the code (i.e. *code gen*) that will deliver the results in a performant manner. What *Structured Streaming* introduces is the concept of an **Incremental Execution Plan**. When working with blocks of data, structured streaming repeatedly applies the execution plan for every new set of blocks it receives. By running in this manner, the engine can take advantage of the optimizations included within Spark DataFrames/Datasets and apply them to an incoming data stream. It will also be easier to integrate other DataFrame optimized components of Spark, including ML Pipelines, GraphFrames, TensorFrames, and many others.

Using structured streaming will also simplify your code. For example, the following is a pseudo-code example *batch aggregation* that reads a data stream from S3 and saves it to a MySQL database:

```
logs = spark.read.json('s3://logs')

logs.groupBy(logs.UserId).agg(sum(logs.Duration))
.write.jdbc('jdbc:mysql://...')
```

The following is a pseudo-code example for a *continuous aggregation*:

```
logs = spark.readStream.json('s3://logs').load()

sq = logs.groupBy(logs.UserId).agg(sum(logs.Duration))
.writeStream.format('json').start()
```

The reason for creating the `sq` variable is that it allows you to check the status of your structured streaming job and terminate it, as per the following:

```
# Will return true if the `sq` stream is active
sq.isActive

# Will terminate the `sq` stream
sq.stop()
```

Let's take the stateful streaming word count script that had used `updateStateByKey` and make it a structured streaming word count script; you can get the complete `structured_streaming_word_count.py` script at: https://github.com/drabastomek/learningPySpark/blob/master/Chapter10/structured_streaming_word_count.py.

As opposed to the previous scripts, we are now working with the more familiar `DataFrames` code as noted here:

```
# Import the necessary classes and create a local SparkSession
from pyspark.sql import SparkSession
from pyspark.sql.functions import explode
from pyspark.sql.functions import split

spark = SparkSession \
    .builder \
    .appName("StructuredNetworkWordCount") \
    .getOrCreate()
```

The first lines of the script import the necessary classes and establish the current `SparkSession`. But, as opposed to the previous streaming scripts, as in the next lines of the script noted here, you do not need to establish a `Streaming Context` as this is already included within the `SparkSession`:

```
1. # Create DataFrame representing the stream of input lines
2. # from connection to localhost:9999
3. lines = spark\
4.     .readStream\
5.     .format('socket')\
6.     .option('host', 'localhost')\
```

```
7.  .option('port', 9999)\
8.  .load()
9.
10. # Split the lines into words
11. words = lines.select(
12.     explode(
13.         split(lines.value, ' ')
14.     ).alias('word')
15. )
16.
17. # Generate running word count
18. wordCounts = words.groupBy('word').count()
```

Instead, the streaming portion of the code is initiated by calling `readStream` in line 4.

- Lines 3-8 initiate the *reading* of the data stream from port 9999, just like the previous two scripts
- Instead of running RDD `flatMap`, `map`, and `reduceByKey` functions to split the lines read into words and count each word in each batch, we can use the PySpark SQL functions `explode` and `split` as noted in lines 10-15
- Instead of running `updateStateByKey` or creating an `updateFunc` as per the stateful streaming word count script, we can generate the running word count with a familiar DataFrame `groupBy` statement and `count()`, as noted in lines 17-18

To output this data to the console, we will use `writeStream`, as noted here:

```
1. # Start running the query that prints the
2. # running counts to the console
3. query = wordCounts\
4.     .writeStream\
5.     .outputMode('complete')\
6.     .format('console')\
7.     .start()
8.
9. # Await Spark Streaming termination
10. query.awaitTermination()
```

Instead of using `pprint()`, we're explicitly calling out `writeStream` to write the stream, and defining the format and output mode. While it is a little longer to write, these methods and properties are syntactically similar with other DataFrame calls and you would only need to change the `outputMode` and `format` properties to save it to a Database, file system, console, and so on. Finally, as noted in line 10, we will run `awaitTermination` to await to cancel this streaming job.

Let's go back and run our `nc` job in the first terminal:

```
$ nc -lk 9999
green green green blue blue blue blue blue
gohawks
green green
```

Check the following output. As you can see, you get the advantages of stateful streaming but using the more familiar DataFrame API:

```
-----+-----+
Batch: 0
-----+-----+
| word|count|
-----+-----+
|green|  3|
|blue|  5|
-----+-----+

-----+-----+
Batch: 1
-----+-----+
| word|count|
-----+-----+
|green|  3|
|blue|  5|
|gohawks| 1|
-----+-----+

-----+-----+
Batch: 2
-----+-----+
| word|count|
-----+-----+
|green|  5|
|blue|  5|
|gohawks| 1|
-----+-----+
```


Summary

It is important to note that Structured Streaming is currently (at the time of writing) not production-ready. It is, however, a paradigm shift in Spark that will hopefully make it easier for data scientists and data engineers to build **continuous applications**. While not explicitly called out in the previous sections, when working with streaming applications, there are many potential problems that you will need to design for, such as late events, partial outputs, state recovery on failure, distributed reads and writes, and so on. With structured streaming, many of these issues will be abstracted away to make it easier for you to build *continuous applications*.

We encourage you to try Spark Structured Streaming so you will be able to easily build streaming applications as structured streaming matures. As Reynold Xin noted in his Spark Summit 2016 East presentation *The Future of Real-Time in Spark* (<http://www.slideshare.net/rxin/the-future-of-realtime-in-spark>):

"The simplest way to perform streaming analytics is not having to reason about streaming."

For more information, here are some additional Structured Streaming resources:

- *PySpark 2.1 Documentation: pyspark.sql.module*: <http://spark.apache.org/docs/2.1.0/api/python/pyspark.sql.html>
- *Introducing Apache Spark 2.1*: <https://databricks.com/blog/2016/12/29/introducing-apache-spark-2-1.html>
- *Structuring Apache Spark 2.0: SQL, DataFrames, Datasets and Streaming - by Michael Armbrust*: <http://www.slideshare.net/databricks/structuring-spark-dataframes-datasets-and-streaming-62871797>
- *Structured Streaming Programming Guide*: <http://spark.apache.org/docs/latest/streaming-programming-guide.html>
- *Structured Streaming (aka Streaming DataFrames) [SPARK-8360]*: <https://issues.apache.org/jira/browse/SPARK-8360>
- *Structured Streaming Programming Abstraction, Semantics, and APIs Apache JIRA*: <https://issues.apache.org/jira/secure/attachment/12793410/StructuredStreamingProgrammingAbstractionSemanticsandAPIs-ApacheJIRA.pdf>

In the next chapter we will show you how to modularize and package up your PySpark application and submit it for execution programmatically.

11

Packaging Spark Applications

So far we have been working with a very convenient way of developing code in Spark - the Jupyter notebooks. Such an approach is great when you want to develop a proof of concept and document what you do along the way.

However, Jupyter notebooks will not work if you need to schedule a job, so it runs every hour. Also, it is fairly hard to package your application as it is not easy to split your script into logical chunks with well-defined APIs - everything sits in a single notebook.

In this chapter, we will learn how to write your scripts in a reusable form of modules and submit jobs to Spark programmatically.

Before you begin, however, you might want to check out the *Bonus Chapter 2, Free Spark Cloud Offering* where we provide instructions on how to subscribe and use either Databricks' Community Edition or Microsoft's HDInsight Spark offerings; the instructions on how to do so can be found here: <https://www.packtpub.com/sites/default/files/downloads/FreeSparkCloudOffering.pdf>.

In this chapter you will learn:

- What the `spark-submit` command is
- How to package and deploy your app programmatically
- How to modularize your Python code and submit it along with PySpark script

The `spark-submit` command

The entry point for submitting jobs to Spark (be it locally or on a cluster) is the `spark-submit` script. The script, however, allows you not only to submit the jobs (although that is its main purpose), but also kill jobs or check their status.



Under the hood, the `spark-submit` command passes the call to the `spark-class` script that, in turn, starts a launcher Java application. For those interested, you can check the GitHub repository for Spark: <https://github.com/apache/spark/blob/master/bin/spark-submit>.

The `spark-submit` command provides a unified API for deploying apps on a variety of Spark supported cluster managers (such as Mesos or Yarn), thus relieving you from configuring your application for each of them separately.

On the general level, the syntax looks as follows:

```
spark-submit [options] <python file> [app arguments]
```

We will go through the list of all the options soon. The `app arguments` are the parameters you want to pass to your application.



You can either parse the parameters from the command line yourself using `sys.argv` (after `import sys`) or you can utilize the `argparse` module for Python.

Command line parameters

You can pass a host of different parameters for Spark engine when using `spark-submit`.



In what follows we will cover only the parameters specific for Python (as `spark-submit` can also be used to submit applications written in Scala or Java and packaged as `.jar` files).

We will now go through the parameters one-by-one so you have a good overview of what you can do from the command line:

- `--master`: Parameter used to set the URL of the master (head) node. Allowed syntax is:
 - `local`: Used for executing your code on your local machine. If you pass `local`, Spark will then run in a single thread (without leveraging any parallelism). On a multi-core machine you can specify either, the exact number of cores for Spark to use by stating `local [n]` where `n` is the number of cores to use, or run Spark spinning as many threads as there are cores on the machine using `local [*]`.

- `spark://host:port`: It is a URL and a port for the Spark standalone cluster (that does not run any job scheduler such as Mesos or Yarn).
 - `mesos://host:port`: It is a URL and a port for the Spark cluster deployed over Mesos.
 - `yarn`: Used to submit jobs from a head node that runs Yarn as the workload balancer.
- `--deploy-mode`: Parameter that allows you to decide whether to launch the Spark driver process locally (using `client`) or on one of the worker machines inside the cluster (using the `cluster` option). The default for this parameter is `client`. Here's an excerpt from Spark's documentation that explains the differences with more specificity (source: <http://bit.ly/2hTtDVE>):

A common deployment strategy is to submit your application from [a screen session on] a gateway machine that is physically co-located with your worker machines (e.g. Master node in a standalone EC2 cluster). In this setup, client mode is appropriate. In client mode, the driver is launched directly within the spark-submit process which acts as a client to the cluster. The input and output of the application is attached to the console. Thus, this mode is especially suitable for applications that involve the REPL (e.g. Spark shell).

Alternatively, if your application is submitted from a machine far from the worker machines (e.g. locally on your laptop), it is common to use cluster mode to minimize network latency between the drivers and the executors. Currently, standalone mode does not support cluster mode for Python applications.
 - `--name`: Name of your application. Note that if you specified the name of your app programmatically when creating `SparkSession` (we will get to that in the next section) then the parameter from the command line will be overridden. We will explain the precedence of parameters shortly when discussing the `--conf` parameter.
 - `--py-files`: Comma-delimited list of `.py`, `.egg` or `.zip` files to include for Python apps. These files will be delivered to each executor for use. Later in this chapter we will show you how to package your code into a module.
 - `--files`: Command gives a comma-delimited list of files that will also be delivered to each executor to use.

- `--conf`: Parameter to change a configuration of your app dynamically from the command line. The syntax is `<Spark property>=<value for the property>`. For example, you can pass `--conf spark.local.dir=/home/SparkTemp/` or `--conf spark.app.name=learningPySpark`; the latter would be an equivalent of submitting the `--name` property as explained previously.



Spark uses the configuration parameters from three places: the parameters from the `SparkConf` you specify when creating `SparkContext` within your app take the highest precedence, then any parameter that you pass to the `spark-submit` script from the command line, and lastly, any parameter that is specified in the `conf/spark-defaults.conf` file.

- `--properties-file`: File with a configuration. It should have the same set of properties as the `conf/spark-defaults.conf` file as it will be read instead of it.
- `--driver-memory`: Parameter that specifies how much memory to allocate for the application on the driver. Allowed values have a syntax similar to the 1,000M, 2G. The default is 1,024M.
- `--executor-memory`: Parameter that specifies how much memory to allocate for the application on each of the executors. The default is 1G.
- `--help`: Shows the help message and exits.
- `--verbose`: Prints additional debug information when running your app.
- `--version`: Prints the version of Spark.

In a Spark standalone with `cluster` deploy mode only, or on a cluster deployed over Yarn, you can use the `--driver-cores` that allows specifying the number of cores for the driver (default is 1). In a Spark standalone or Mesos with `cluster` deploy mode only you also have the opportunity to use either of these:

- `--supervise`: Parameter that, if specified, will restart the driver if it is lost or fails. This also can be set in Yarn by setting the `--deploy-mode` to `cluster`
- `--kill`: Will finish the process given its `submission_id`
- `--status`: If this command is specified, it will request the status of the specified app

In a Spark standalone and Mesos only (with the `client` deploy mode) you can also specify the `--total-executor-cores`, a parameter that will request the number of cores specified for all executors (not each). On the other hand, in a Spark standalone and YARN, only the `--executor-cores` parameter specifies the number of cores per executor (defaults to 1 in YARN mode, or to all available cores on the worker in standalone mode).

In addition, when submitting to a YARN cluster you can specify:

- `--queue`: This parameter specifies a queue on YARN to submit the job to (default is `default`)
- `--num-executors`: Parameter that specifies how many executor machines to request for the job. If dynamic allocation is enabled, the initial number of executors will be at least the number specified.

Now that we have discussed all the parameters it is time to put it into practice.

Deploying the app programmatically

Unlike the Jupyter notebooks, when you use the `spark-submit` command, you need to prepare the `SparkSession` yourself and configure it so your application runs properly.

In this section, we will learn how to create and configure the `SparkSession` as well as how to use modules external to Spark.



If you have not created your free account with either Databricks or Microsoft (or any other provider of Spark) do not worry - we will be still using your local machine as this is easier to get us started. However, if you decide to take your application to the cloud it will literally only require changing the `--master` parameter when you submit the job.

Configuring your SparkSession

The main difference between using Jupyter and submitting jobs programmatically is the fact that you have to create your Spark context (and Hive, if you plan to use HiveQL), whereas when running Spark with Jupyter the contexts are automatically started for you.

In this section, we will develop a simple app that will use public data from Uber with trips made in the NYC area in June 2016; we downloaded the dataset from https://s3.amazonaws.com/nyc-tlc/trip+data/yellow_tripdata_2016-06.csv (beware as it is an almost 3GB file). The original dataset contains 11 million trips, but for our example we retrieved only 3.3 million and selected only a subset of all available columns.



The transformed dataset can be downloaded from http://www.tomdrabas.com/data/LearningPySpark/uber_data_nyc_2016-06_3m_partitioned.csv.zip. Download the file and unzip it to the Chapter13 folder from GitHub. The file might look strange as it is actually a directory containing four files inside that, when read by Spark, will form one dataset.

So, let's get to it!

Creating SparkSession

Things with Spark 2.0 have become slightly simpler than with previous versions when it comes to creating SparkContext. In fact, instead of creating a SparkContext explicitly, Spark currently uses SparkSession to expose higher-level functionality. Here's how you do it:

```
from pyspark.sql import SparkSession

spark = SparkSession \
    .builder \
    .appName('CalculatingGeoDistances') \
    .getOrCreate()

print('Session created')
```

The preceding code is all that you need!



If you want to use RDD API you still can. However, you do not need to create a SparkContext anymore as SparkSession starts one under the hood. To get the access you can simply call (borrowing from the preceding example): `sc = spark.sparkContext`.

In this example, we first create the `SparkSession` object and call its `.builder` internal class. The `.appName(...)` method allows us to give our application a name, and the `.getOrCreate()` method either creates or retrieves an already created `SparkSession`. It is a good convention to give your application a meaningful name as it helps to (1) find your application on a cluster and (2) creates less confusion for everyone.



Under the hood, the Spark session creates a `SparkContext` object. When you call `.stop()` on `SparkSession` it actually terminates the `SparkContext` within.

Modularizing code

Building your code in such a way so it can be reused later is always a good thing. The same can be done with Spark - you can modularize your methods and then reuse them at a later point. It also aids readability of your code and its maintainability.

In this example, we will build a module that would do some calculations on our dataset: It will compute the *as-the-crow-flies* distance (in miles) between the pickup and drop-off locations (using the Haversine formula), and also will convert the calculated distance from miles into kilometers.



More on the Haversine formula can be found here: <http://www.movable-type.co.uk/scripts/latlong.html>.

So, first, we will build a module.

Structure of the module

We put the code for our extraneous methods inside the `additionalCode` folder.



Check out the GitHub repository for this book if you have not done so already <https://github.com/drabastomek/learningPySpark/tree/master/Chapter11>.

The tree for the folder looks as follows:

```
additionalCode/
├── setup.py
└── utilities
    ├── __init__.py
    ├── base.py
    ├── converters
    │   ├── __init__.py
    │   └── distance.py
    └── geoCalc.py
2 directories, 6 files
```

As you can see, it has a structure of a somewhat normal Python package: At the top we have the `setup.py` file so we can package up our module, and then inside we have our code.

The `setup.py` file in our case looks as follows:

```
from setuptools import setup

setup(
    name='PySparkUtilities',
    version='0.1dev',
    packages=['utilities', 'utilities/converters'],
    license='''
        Creative Commons
        Attribution-Noncommercial-Share Alike license''',
    long_description='''
        An example of how to package code for PySpark'''
)
```

We will not delve into details here on the structure (which on its own is fairly self-explanatory): You can read more about how to define `setup.py` files for other projects here https://pythonhosted.org/an_example_pypi_project/setuptools.html.

The `__init__.py` file in the utilities folder has the following code:

```
from .geoCalc import geoCalc
__all__ = ['geoCalc', 'converters']
```

It effectively exposes the `geoCalc.py` and `converters` (more on these shortly).

Calculating the distance between two points

The first method we mentioned uses the Haversine formula to calculate the direct distance between any two points on a map (Cartesian coordinates). The code that does this lives in the `geoCalc.py` file of the module.

The `calculateDistance(...)` is a static method of the `geoCalc` class. It takes two geo-points, expressed as either a tuple or a list with two elements (latitude and longitude, in that order), and uses the Haversine formula to calculate the distance. The Earth's radius necessary to calculate the distance is expressed in miles so the distance calculated will also be in miles.

Converting distance units

We build the utilities package so it can be more universal. As a part of the package we expose methods to convert between various units of measurement.



At this time we limit it to the distance only, but the functionality can be further extended to other domains such as area, volume, or temperature.

For ease of use, any class implemented as a `converter` should expose the same interface. That is why it is advised that such a class derives from our `BaseConverter` class (see `base.py`):

```
from abc import ABCMeta, abstractmethod

class BaseConverter(metaclass=ABCMeta):
    @staticmethod
    @abstractmethod
    def convert(f, t):
        raise NotImplementedError
```

It is a purely abstract class that cannot be instantiated: Its sole purpose is to force the derived classes to implement the `convert(...)` method. See the `distance.py` file for details of the implementation. The code should be self-explanatory for someone proficient in Python so we will not be going through it step-by-step here.

Building an egg

Now that we have all our code in place we can package it. The documentation for PySpark states that you can pass `.py` files (using the `--py-files` switch) to the `spark-submit` script separated by commas. However, it is much more convenient to package our module into a `.zip` or an `.egg`. This is when the `setup.py` file comes handy - all you have to do is to call this inside the `additionalCode` folder:

```
python setup.py bdist_egg
```

If all goes well you should see three additional folders: `PySparkUtilities.egg-info`, `build`, and `dist` - we are interested in the file that sits in the `dist` folder: The `PySparkUtilities-0.1.dev0-py3.5.egg`.



After running the preceding command, you might find that the name of your `.egg` file is slightly different as you might have a different Python version. You can still use it in your Spark jobs, but you will have to adapt the `spark-submit` command to reflect the name of your `.egg` file.

User defined functions in Spark

In order to do operations on `DataFrames` in PySpark you have two options: Use built-in functions to work with data (most of the time it will be sufficient to achieve what you need and it is recommended as the code is more performant) or create your own user-defined functions.

To define a UDF you have to wrap the Python function within the `.udf(...)` method and define its return value type. This is how we do it in our script (check the `calculatingGeoDistance.py` file):

```
import utilities.geoCalc as geo
from utilities.converters import metricImperial

getDistance = func.udf(
    lambda lat1, long1, lat2, long2:
        geo.calculateDistance(
            (lat1, long1),
            (lat2, long2)
        )
)

convertMiles = func.udf(lambda m:
    metricImperial.convert(str(m) + ' mile', 'km'))
```

We can then use such functions to calculate the distance and convert it to miles:

```
uber = uber.withColumn(
    'miles',
    getDistance(
        func.col('pickup_latitude'),
        func.col('pickup_longitude'),
        func.col('dropoff_latitude'),
        func.col('dropoff_longitude')
    )
)

uber = uber.withColumn(
    'kilometers',
    convertMiles(func.col('miles'))
)
```

Using the `.withColumn(...)` method we create additional columns with the values of interest to us.



A word of caution needs to be stated here. If you use the PySpark built-in functions, even though you call them Python objects, underneath that call is translated and executed as Scala code. If, however, you write your own methods in Python, it is not translated into Scala and, hence, has to be executed on the driver. This causes a significant performance hit. Check out this answer from Stack Overflow for more details: <http://stackoverflow.com/questions/32464122/spark-performance-for-scala-vs-python>.

Let's now put all the puzzles together and finally submit our job.

Submitting a job

In your CLI type the following (we assume you keep the structure of the folders unchanged from how it is structured on GitHub):

```
./launch_spark_submit.sh \
--master local[4] \
--py-files additionalCode/dist/PySparkUtilities-0.1.dev0-py3.5.egg \
calculatingGeoDistance.py
```

We owe you some explanation for the `launch_spark_submit.sh` shell script. In Bonus Chapter 1, *Installing Spark*, we configured our Spark instance to run Jupyter (by setting the `PYSPARK_DRIVER_PYTHON` system variable to `jupyter`). If you were to simply use `spark-submit` on a machine configured in such a way, you would most likely get some variation of the following error:

```
jupyter: 'calculatingGeoDistance.py' is not a Jupyter command
```

Thus, before running the `spark-submit` command we first have to unset the variable and then run the code. This would quickly become extremely tiring so we automated it with the `launch_spark_submit.sh` script:

```
#!/bin/bash

unset PYSPARK_DRIVER_PYTHON
spark-submit $*
export PYSPARK_DRIVER_PYTHON=jupyter
```

As you can see, this is nothing more than a wrapper around the `spark-submit` command.

If all goes well, you will see the following *stream of consciousness* appearing in your CLI:

```
17/01/08 20:51:55 INFO SparkContext: Running Spark version 2.1.0
17/01/08 20:51:55 WARN NativeCodeLoader: Unable to load native-heapdump library for your platform... using builtin-java classes where applicable
17/01/08 20:51:56 INFO SecurityManager: Changing view acls to: drabast
17/01/08 20:51:56 INFO SecurityManager: Changing modify acls to: drabast
17/01/08 20:51:56 INFO SecurityManager: Changing view acls groups to:
17/01/08 20:51:56 INFO SecurityManager: Changing modify acls groups to:
17/01/08 20:51:56 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabled; users with view permissions: Set(drabast); groups with view permissions: Set(); users with modify permissions: Set(drabast); groups with modify permissions: Set()
17/01/08 20:51:56 INFO Utils: Successfully started service 'sparkDriver' on port 52919.
17/01/08 20:51:56 INFO SparkEnv: Registering MapOutputTracker
17/01/08 20:51:56 INFO SparkEnv: Registering BlockManagerMaster
17/01/08 20:51:56 INFO BlockManagerMasterEndpoint: Using org.apache.spark.storage.DefaultTopologyMapper for getting topology information
17/01/08 20:51:56 INFO BlockManagerMasterEndpoint: BlockManagerMasterEndpoint up
17/01/08 20:51:56 INFO DiskBlockManager: Created local directory at /private/var/folders/_g/wy0_l8n54mz_ktg1pgj_bhj80000gg/T/blockmgr-ffe4fbf9-73d1-4e56-8568-79a68ee52123
17/01/08 20:51:56 INFO MemoryStore: MemoryStore started with capacity 366.3 MB
17/01/08 20:51:56 INFO SparkEnv: Registering OutputCommitCoordinator
17/01/08 20:51:56 INFO Utils: Successfully started service 'SparkUI' on port 4040.
17/01/08 20:51:56 INFO SparkUI: Bound SparkUI to 0.0.0.0, and started at http://192.168.0.109:4040
17/01/08 20:51:56 INFO SparkContext: Added file file:/Users/drabast/Documents/Publishing/TST/Ch13/calculatingGeoDistance.py at file:/Users/drabast/Documents/Publishing/TST/Ch13/calculatingGeoDistance.py with timestamp 1483937516630
17/01/08 20:51:56 INFO Utils: Copying /Users/drabast/Documents/Publishing/TST/Ch13/calculatingGeoDistance.py to /private/var/folders/_g/wy0_l8n54mz_ktg1pgj_bhj80000gg/T/spark-79d9b524-250a-4c92-b292-a80cccd661de/userFiles-a095112f-2ac5-45b9-abf8-09c7db1a6d9d/calculatingGeoDistance.py
17/01/08 20:51:56 INFO SparkContext: Added file file:/Users/drabast/Documents/Publishing/TST/Ch13/additionalCode/dist/PySparkUtilities-0.1.dev0-py3.5.egg at file:/Users/drabast/Documents/Publishing/TST/Ch13/additionalCode/dist/PySparkUtilities-0.1.dev0-py3.5.egg with timestamp 1483937516655
17/01/08 20:51:56 INFO Utils: Copying /Users/drabast/Documents/Publishing/TST/Ch13/additionalCode/dist/PySparkUtilities-0.1.dev0-py3.5.egg to /private/var/folders/_g/wy0_l8n54mz_ktg1pgj_bhj80000gg/T/spark-79d9b524-250a-4c92-b292-a80cccd661de/userFiles-a095112f-2ac5-45b9-abf8-09c7db1a6d9d/PySparkUtilities-0.1.dev0-py3.5.egg
17/01/08 20:51:56 INFO Executor: Starting executor ID driver on host localhost
17/01/08 20:51:56 INFO Utils: Successfully started service 'org.apache.spark.network.netty.NettyBlockTransferService' on port 52920.
17/01/08 20:51:56 INFO NettyBlockTransferService: Server created on 192.168.0.109:52920
17/01/08 20:51:56 INFO BlockManager: Using org.apache.spark.storage.RandomBlockReplicationPolicy for block replication policy
17/01/08 20:51:56 INFO BlockManagerMaster: Registering BlockManager BlockManagerId(driver, 192.168.0.109, 52920, None)
17/01/08 20:51:56 INFO BlockManagerMasterEndpoint: Registering block manager 192.168.0.109:52920 with 366.3 MB RAM, BlockManagerId(driver, 192.168.0.109, 52920, None)
17/01/08 20:51:56 INFO BlockManagerMaster: Registered BlockManager BlockManagerId(driver, 192.168.0.109, 52920, None)
17/01/08 20:51:56 INFO BlockManager: Initialized BlockManager: BlockManagerId(driver, 192.168.0.109, 52920, None)
17/01/08 20:51:56 INFO SharedState: Warehouse path is 'file:/Users/drabast/Documents/Publishing/TST/Ch13/spark-warehouse/'.
Session created
17/01/08 20:51:57 INFO MemoryStore: Block broadcast_0 stored as values in memory (estimated size 127.1 KB, free 366.2 MB)
17/01/08 20:51:57 INFO MemoryStore: Block broadcast_0_piece0 stored as bytes in memory (estimated size 14.3 KB, free 366.2 MB)
17/01/08 20:51:57 INFO BlockManagerInfo: Added broadcast_0_piece0 in memory on 192.168.0.109:52920 (size: 14.3 KB, free: 366.3 MB)
17/01/08 20:51:57 INFO SparkContext: Created broadcast_0 from csv at NativeMethodAccessorImpl.java:0
17/01/08 20:51:57 INFO FileInputFormat: Total input paths to process : 4
```

There's a host of useful things that you can get from reading the output:

- Current version of Spark: 2.1.0
- Spark UI (what will be useful to track the progress of your job) is started successfully on `http://localhost:4040`
- Our `.egg` file was added successfully to the execution
- The `uber_data_nyc_2016-06_3m_partitioned.csv` was read successfully
- Each start and stop of jobs and tasks are listed

Once the job finishes, you will see something similar to the following:

```
17/01/08 20:52:09 INFO TaskSchedulerImpl: Removed taskSet 4.0, whose tasks have all completed, from pool
17/01/08 20:52:09 INFO DAGScheduler: ResultStage 4 (ShowString at NativeMethodAccessorImpl.java:0) finished in 0.726 s
17/01/08 20:52:09 INFO DAGScheduler: Job 3 finished: showString at NativeMethodAccessorImpl.java:0, took 5.491970 s
17/01/08 20:52:09 INFO CodeGenerator: Code generated in 22.037969 ms

+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|VendorID|tpep_pickup_datetime|pickup_longitude|pickup_latitude|dropoff_longitude|dropoff_latitude|total_amount|    miles|    kilometers|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|1|2016-06-09 21:06:...|-73.99520111083984|40.7394005090332|-73.99320220947266|40.76264190673828|14.16|1.6031238964762755|2.579977824050715|
|1|2016-06-09 21:06:...|-74.00510406494141|40.73751831054688|-73.99778747538894|40.756744384765625|11.16|1.3825857369344510|2.2250560602210383|
|1|2016-06-09 21:06:...|-73.99408721923827|40.75905227661133|-73.98001008328121|40.78688322143555|13.31|2.007767579000166|3.23118706803284|
|1|2016-06-09 21:06:...|-73.99886322021484|40.73970613208088|-74.00420379638672|40.71982955932617|12.31|1.4014676217552341|2.254435082660556|
|1|2016-06-09 21:09:...|-73.97091674804686|40.75846862792969|-73.95187377929686|40.77882276611328|11.76|1.6788976160321079|2.7019238049755767|
|1|2016-06-09 21:13:...|-73.97908020019531|40.7470703125|-73.98967742919923|40.7038459777832|24.81|3.037812203195951|4.888884842340184|
|1|2016-06-09 21:13:...|-73.99645233154298|40.75335693359375|-73.9559555053711|40.77219772338867|21.35|2.4873338365295776|4.002975785815857|
|1|2016-06-09 21:05:...|-73.98802947998048|40.74996948242188|-74.00271606445312|40.76051330566406|11.15|1.0591140833475323|1.7044788953508512|
|1|2016-06-09 21:06:...|-74.00428771972656|40.73029708862305|-73.98162078857422|40.75013732910156|15.36|1.8131873277192654|2.9180421467410333|
|1|2016-06-09 21:06:...|-73.9819107055664|40.7568244934082|-73.94727325439453|40.7801170349121|14.31|2.4240392225577776|3.9011129785880243|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 10 rows

17/01/08 20:52:09 INFO SparkUI: Stopped Spark web UI at http://192.168.0.109:4040
17/01/08 20:52:09 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
17/01/08 20:52:09 INFO MemoryStore: MemoryStore cleared
17/01/08 20:52:09 INFO BlockManager: BlockManager stopped
17/01/08 20:52:09 INFO BlockManagerMaster: BlockManagerMaster stopped
17/01/08 20:52:09 INFO OutputCommitCoordinator$OutputCommitCoordinatorEndpoint: OutputCommitCoordinator stopped!
17/01/08 20:52:09 INFO SparkContext: Successfully stopped SparkContext
17/01/08 20:52:10 INFO ShutdownHookManager: Shutdown hook called
17/01/08 20:52:10 INFO ShutdownHookManager: Deleting directory /private/var/folders/_g/wy0_l8n54mz_ktg1pgj_bhj80000gq/T/spark-79d9b524-250a-4c92-b292-a80cccd661de
17/01/08 20:52:10 INFO ShutdownHookManager: Deleting directory /private/var/folders/_g/wy0_l8n54mz_ktg1pgj_bhj80000gq/T/spark-79d9b524-250a-4c92-b292-a80cccd661de/pyspark-90f5eb26-3b03-4951-8e19-2d733d8237c0
endeavour:CH13 drcobast$
```

From the preceding screenshot, we can read that the distances are reported correctly. You can also see that the Spark UI process has now been stopped and all the clean up jobs have been performed.

Monitoring execution

When you use the `spark-submit` command, Spark launches a local server that allows you to track the execution of the job. Here's what the window looks like:

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
2	csv at NativeMethodAccessorImpl.java:0	2017/01/08 21:05:33	0.4 s	0/4				

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
1	csv at NativeMethodAccessorImpl.java:0	2017/01/08 21:05:33	13 ms	1/1	23.7 MB			
0	csv at NativeMethodAccessorImpl.java:0	2017/01/08 21:05:33	0.2 s	1/1	23.7 MB			

At the top you can switch between the **Jobs** or **Stages** view; the **Jobs** view allows you to track the distinct jobs that are executed to complete the whole script, while the **Stages** view allows you to track all the stages that are executed.

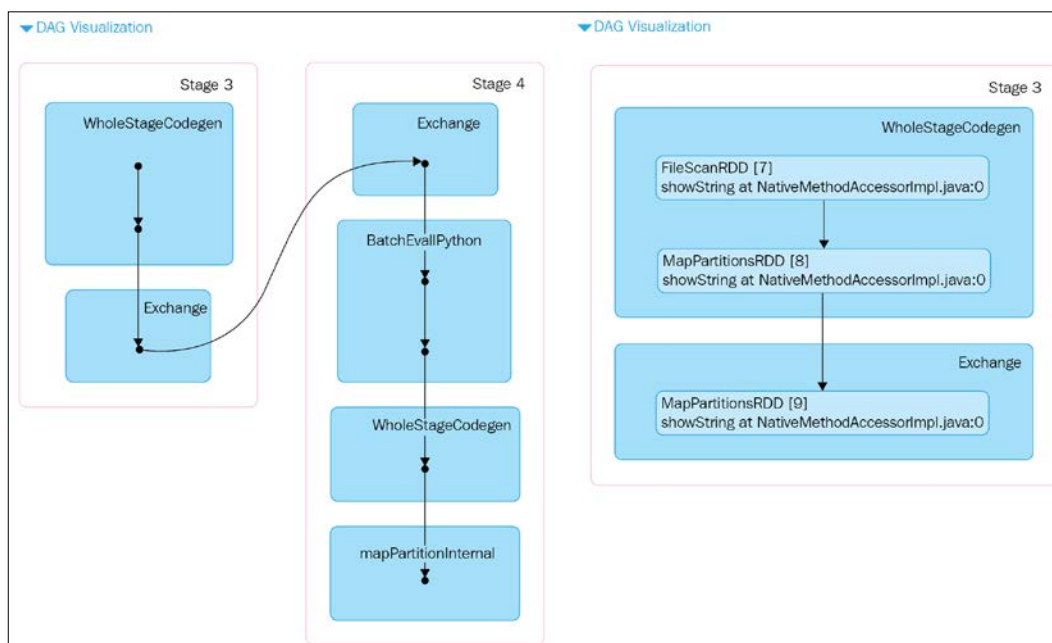
You can also peak inside each stage execution profile and track each task execution by clicking on the link of the stage. In the following screenshot, you can see the execution profile for Stage 3 with four tasks running:

Executor ID	Address	Task Time	Total Tasks	Failed Tasks	Killed Tasks	Succeeded Tasks
0	driver / localhost	2017/01/08 21:07:19	2 s			
1	driver / localhost	2017/01/08 21:07:19	2 s			
2	driver / localhost	2017/01/08 21:07:19	2 s			
3	driver / localhost	2017/01/08 21:07:19	2 s			



In a cluster setup instead of **driver/localhost** you would see the driver number and host's IP address.

Inside a job or a stage, you can click on the DAG Visualization to see how your job or stage gets executed (the following chart on the left shows the **Job** view, while the one on the right shows the **Stage** view):



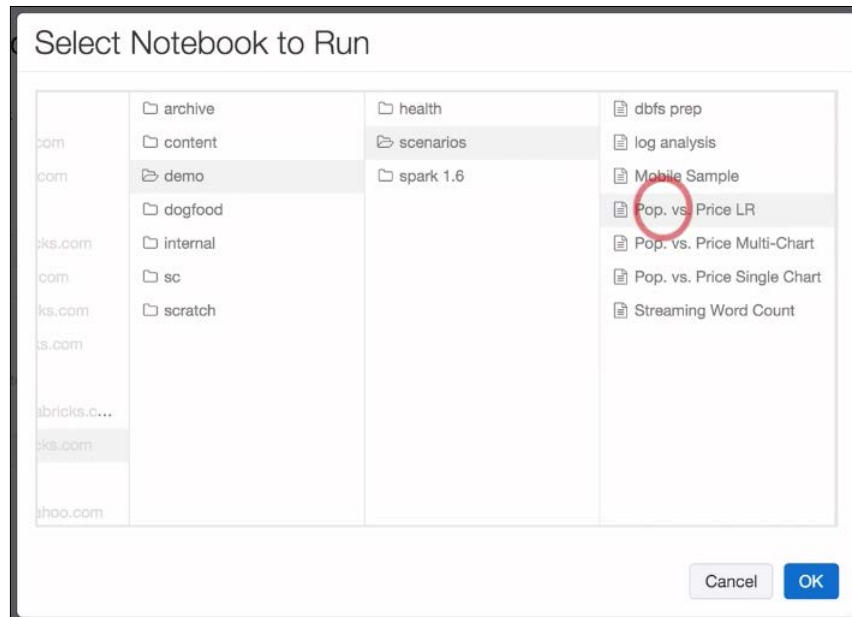
Databricks Jobs

If you are using the Databricks product, an easy way to go from development from your Databricks notebooks to production is to use the Databricks Jobs feature. It will allow you to:

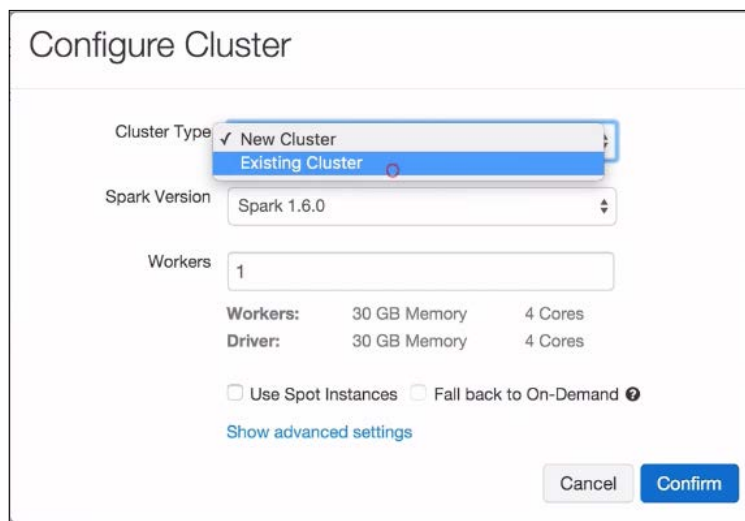
- Schedule your Databricks notebook to run on an existing or new cluster
- Schedule at your desired frequency (from minutes to months)
- Schedule time out and retries for your job
- Be alerted when the job starts, completes, and/or errors out
- View historical job runs as well as review the history of the individual notebook job runs

This capability greatly simplifies the scheduling and production workflow of your job submissions. Note that you will need to upgrade your Databricks subscription (from Community edition) to use this feature.

To use this feature, go to the Databricks **Jobs** menu and click on **Create Job**. From here, fill out the job name and then choose the notebook that you want to turn into a job, as shown in the following screenshot:



Once you have chosen your notebook, you can also choose whether to use an existing cluster that is running or have the job scheduler launch a **New Cluster** specifically for this job, as shown in the following screenshot:



Configure Cluster

Cluster Type: ✓ New Cluster Existing Cluster

Spark Version: Spark 1.6.0

Workers: 1

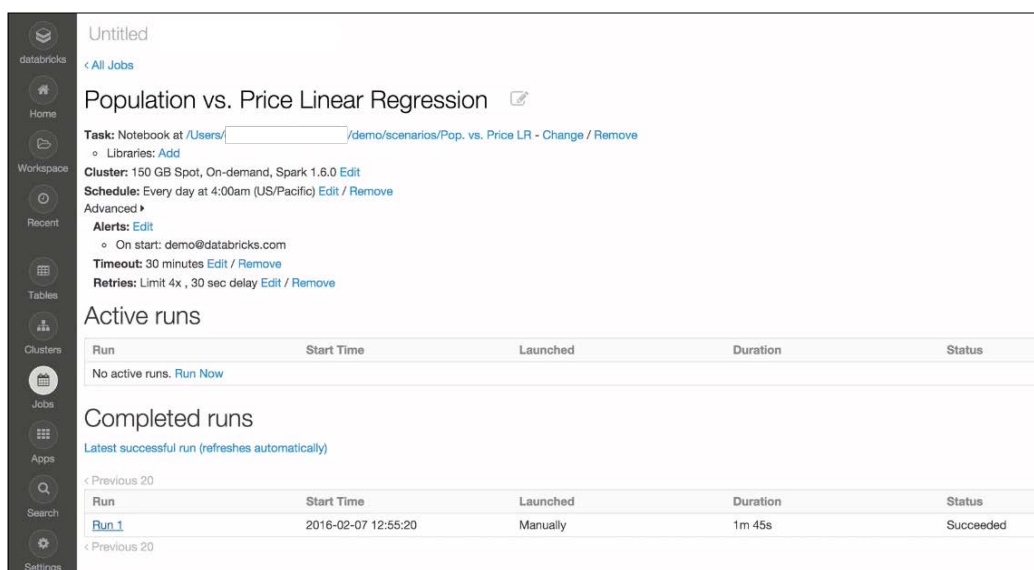
Workers: 30 GB Memory 4 Cores
Driver: 30 GB Memory 4 Cores

☐ Use Spot Instances ☐ Fall back to On-Demand ?

[Show advanced settings](#)

Cancel Confirm

Once you have chosen your notebook and cluster; you can set the schedule, alerts, timeout, and retries. Once you have completed setting up your job, it should look something similar to the **Population vs. Price Linear Regression Job**, as noted in the following screenshot:



Untitled

< All Jobs

Population vs. Price Linear Regression

Task: Notebook at /Users/[redacted]/demo/scenarios/Pop. vs. Price LR - Change / Remove

Libraries: Add

Cluster: 150 GB Spot, On-demand, Spark 1.6.0 Edit

Schedule: Every day at 4:00am (US/Pacific) Edit / Remove

Advanced

Alerts: Edit

- On start: demo@databricks.com

Timeout: 30 minutes Edit / Remove

Retries: Limit 4x, 30 sec delay Edit / Remove

Active runs

Run	Start Time	Launched	Duration	Status
No active runs. Run Now				

Completed runs

[Latest successful run \(refreshes automatically\)](#)

< Previous 20

Run	Start Time	Launched	Duration	Status
Run 1	2016-02-07 12:55:20	Manually	1m 45s	Succeeded

< Previous 20

You can test the job by clicking on the **Run Now** link under **Active runs** to test your job.

As noted in the **Meetup Streaming RSVPs** Job, you can view the history of your completed runs; as shown in the screenshot, for this notebook there are **50** completed job runs:

Meetup Streaming RSVPs

Task: Notebook at /Users/[redacted]/content/Streaming Meetup RSVPs/4c. Reports and Dashboards (mapWithState, Country) - Change / Remove

Cluster: demo-2 (120 GB, Running, Spark 1.6.x) Edit

Schedule: Every day at 1:00am (US/Pacific) Edit / Remove

Advanced ▶

Active runs

Run	Start Time	Launched	Duration	Status
No active runs. Run Now				

Completed runs

Latest successful run (refreshes automatically)

< Previous 20 Next 20 >

Run	Start Time	Launched	Duration	Status
Run 50	2016-02-21 20:04:00	By scheduler	8s	Succeeded
Run 49	2016-02-21 20:03:00	By scheduler	7s	Succeeded
Run 48	2016-02-21 20:02:00	By scheduler	7s	Succeeded
Run 47	2016-02-21 20:01:00	By scheduler	7s	Succeeded
Run 46	2016-02-21 20:00:00	By scheduler	13s	Succeeded
Run 45	2016-02-21 19:59:00	By scheduler	7s	Succeeded
Run 44	2016-02-21 19:58:00	By scheduler	7s	Succeeded
Run 43	2016-02-21 19:57:00	By scheduler	7s	Succeeded

By clicking on the job run (in this case, **Run 50**), you can see the results of that job run. Not only can you view the start time, duration, and status, but also the results for that specific job:

Run 50 of Meetup Streaming RSVPs

< All Jobs / Meetup Streaming RSVPs View Code Results Only Dashboard: Meetup Streaming Dashboard

Started: 2016-02-21 20:04:00

Duration: 8s

Status: Succeeded

Task: Notebook at /Users/[redacted]/content/Streaming Meetup RSVPs/4c. Reports and Dashboards (mapWithState, Country)

Cluster: (Unavailable) - View Spark UI

Output

Reports and Dashboards using mapWithState (Country)

This is the third notebook of the **Streaming Meetup RSVPs** set of notebooks. The purpose of this notebook is to create a set of reports and dashboards using Spark SQL from the DataFrames generated by the Streaming Meetup RSVPs Streaming applications.

Meetup Sources

- Based off of the [Meetup RSVP Ticker](#)
- Reference: [Meetup Streaming API > RSVPs](#) %md ## Meetup Streaming Example

```
> select * from batch_meetup_stream_country order by count desc limit 20;
```

group_country	count
US	5472
RU	634
CA	516



REST Job Server

A popular way to run jobs is to also use REST APIs. If you are using Databricks, you can run your jobs using the Databricks REST APIs. If you prefer to manage your own job server, a popular open source REST Job Server is `spark-jobserver` - a RESTful interface for submitting and managing Apache Spark jobs, jars, and job contexts. The project recently (at the time of writing) was updated so it can handle PySpark jobs.

For more information, please refer to <https://github.com/spark-jobserver/spark-jobserver>.

Summary

In this chapter, we walked you through the steps on how to submit applications written in Python to Spark from the command line. The selection of the `spark-submit` parameters has been discussed. We also showed you how you can package your Python code and submit it alongside your PySpark script. Furthermore, we showed you how you can track the execution of your job.

In addition, we also provided a quick overview of how to run Databricks notebooks using the Databricks Jobs feature. This feature simplifies the transition from development to production, allowing you to take your notebook and execute it as an end-to-end workflow.

This brings us to the end of this book. We hope you enjoyed the journey, and that the material contained herein will help you start working with Spark using Python. Good luck!

Index

Symbols

- `.collect()` method 29
- `.count()` method 31
- `.distinct()` transformation 26
- `.filter(...)` transformation 25
- `.flatMap(...)` transformation 26
- `.foreach(...)` method 32
- `.format(...)`
 - URL 193
- `.leftOuterJoin(...)` transformation 27, 28
- `.map(...)` transformation 24, 25
- `.reduce(...)` method 29, 30
- `.repartition(...)` transformation 28
- `.sample(...)` transformation 27
- `.saveAsTextFile(...)` method 31, 32
- `.take(...)` method 29

A

actions

- `.collect()` method 29
- `.count()` method 31
- `.foreach(...)` method 32
- `.reduce(...)` method 29, 30
- `.saveAsTextFile(...)` method 31, 32
- `.take(...)` method 29
 - about 18, 29
 - reference link 5

airportCodes

- URL 138

airport ranking

- determining, with PageRank 149, 150

Airports D3 visualization

- URL 154

Apache Spark

- about 2
- reference link 3
- URL, for issues 208

Apache Spark 2.0 Architecture

- about 8
- continuous applications 14
- Datasets, unifying with DataFrames 9, 10
- Project Tungsten 2 11, 12
- references 8
- SparkSession 10
- Structured Streaming 13

Apache Spark APIs

- about 3
- Catalyst Optimizer 6
- Datasets 6
- Resilient Distributed Dataset (RDD) 4, 5

Apache Spark Jobs

- about 3
- DataFrames 6
- execution process 3
- Project Tungsten 7

application, deploying

- about 227
- code, modularizing 229
- execution, monitoring 236
- job, submitting 233, 235
- Sparksession, configuring 227
- Sparksession, creating 228

associative 30

B

bcolz format

- URL 191

birth data

- correlations, calculating 87, 88
- descriptive statistics, calculating 85-87
- final dataset, creating 90
- knowledge, obtaining 85
- loading 80-85
- RDD, creating of LabeledPoints 90
- splitting, into training and testing 91
- statistical test, executing 89, 90
- transforming 80-85
- URL 80, 97, 105

Blaze

- installing 184

block-wise reducing operations

- about 179
- DataFrame, analyzing 180
- DataFrame, building of vectors 180
- elementwise min, computing of
 - vectors 181, 182
- elementwise sum, computing of
 - vectors 181, 182

Breadth-first search (BFS)

- about 152
- using 152, 153

Bureau of Transportation Statistics (BTS) 138

C

Catalyst Optimizer

- about 6, 35, 36
- reference link 36
- references 7

Chi-square

- URL 98

classification

- about 101, 122, 123
- DecisionTreeClassifier 102
- GBClassifier 102
- LogisticRegression 101
- MultilayerPerceptronClassifier 102
- NaiveBayes 102
- OneVsRest 102
- RandomForestClassifier 102

clustering

- about 103, 123
- BisectingKMeans 103

- clusters, searching in births dataset 124

- GaussianMixture 104

- KMeans 104

- LDA 104

- topic mining 124-127

code, application deploying

- .egg, building 232

- distance, calculating between two
 - points 231

- distance units, converting 231

- module structure 229, 230

- user defined functions, in Spark 232, 233

Code Generation 12

command line, parameters

- conf 226

- deploy mode 225

- driver-memory 226

- executor-memory 226

- files 225

- help 226

- kill 226

- master 224

- name 225

- properties-file 226

- py-files 225

- status 226

- supervise 226

- verbose 226

- version 226

commutative 30

constants

- used, for matrix multiplication 170

continuous applications 14

continuous variables

- discretizing 119, 120

- standardizing 120, 121

correlations

- calculating 70, 71

Cost-based Optimizer framework

- references 36

D

DAG scheduler

- reference link 4

data abstraction

- about 186

- databases, working with 192
- files, working with 189-191
- NumPy arrays, working with 186-188
- pandas' DataFrame, working 188, 189
- databases**
 - MongoDB databases, interacting with 194
 - relational databases, interacting with 193
 - working, with 192
- Databricks Community Edition**
 - reference link 49
 - URL 135, 223
- Databricks Jobs 237-240**
- databricks/tensorframes GitHub repository**
 - URL 176
- DataFrame API**
 - filter statements, executing 46
 - number of rows 46
 - querying, with 46
- DataFrames**
 - about 6
 - creating 38, 39
 - custom JSON data, creating 38
 - DataFrame API query, using 42
 - PySpark, speeding up with 36, 37
 - reference link 37, 41, 117
 - simple queries, executing 42
 - SQL query, writing 42, 43
 - temporary table, creating 39-41
- DataFrames, relating with Tungsten**
 - URL 38
- data lineage**
 - URL 5
- data operations**
 - about 194
 - columns, accessing 194, 195
 - data, reducing 198, 199
 - joins 200, 201
 - operations, performing on
 - columns 197, 198
 - symbolic transformations 195, 196
- dataset**
 - about 6, 66
 - correlations, calculating 70, 71
 - descriptive statistics, calculating 67-70
 - unifying, with DataFrames 9, 10
 - URL 67
- Deep Learning**
 - about 157-161
 - data and algorithm, bridging 164-166
 - feature engineering 163
 - need for 161, 162
 - reference link 162, 175
- departureDelays.csv**
 - URL 138
- descriptive statistics**
 - calculating 67-70
- distributed computing**
 - advances 161
 - availability 161
- Distributed File System (HDFS) 38**
- DStreams**
 - about 80
 - URL 80
 - used, for Spark Streaming 208-212
- duplicates**
 - checking for 56-59
- E**
- edges 138**
- estimators**
 - about 101
 - classification 101, 102
 - clustering 103, 104
 - regression 103
- F**
- Faster Stateful Stream Processing**
 - URL 217
- feature engineering 163**
- feature extraction**
 - about 116, 164
 - continuous variables, discretizing 119, 120
 - continuous variables,
 - standardizing 120, 121
 - NLP related feature extractors 116-118
 - reference link 164
- feature learning 162**
- features**
 - about 162
 - Handwritten Digit recognition 163
 - Image Processing 163

- references 162
- restaurant recommendations 162
- feature selection** 163
- flights dataset**
 - preparing 138, 139
- functions**
 - URL 59

G

- global aggregations**
 - about 213-217
 - URL 217
- Gradient Boosted Trees** 102
- graph**
 - building 140, 141
- GraphFrames**
 - about 134
 - installing 134, 135
 - library, creating 135-137
 - references 134
- graph queries**
 - executing 141
 - flight delays, determining 143
 - longest delay, determining in dataset 142
 - number of airports, determining 142
 - number of delayed flights, versus on-time flights, determining 142, 143
 - number of trips, determining 142
 - states, determining for significant delays from SEA 144
- grid search** 111-114

H

- Hartsfield-Jackson Atlanta International Airport (ATL)**
 - references 150
- Haversine formula**
 - URL 229
- histograms** 72-75
- hyperparameters**
 - reference link 175

I

- Incremental Execution Plan** 218
- infant survival**
 - most predictable features, selecting 93
 - predicting 91
 - predicting, with logistic regression
 - in MLlib 91, 92
 - predicting, with random forest
 - in MLlib 94, 95
- infant survival prediction, with ML**
 - data, loading 105, 106
 - estimator, creating 107
 - model, fitting 108, 109
 - model, saving 110, 111
 - performance, evaluating 109
 - performing 105
 - pipeline, creating 107
 - transformers, creating 106
- International Air Transport Association(IATA) code** 51
- Inverse Document Frequency (IDF)** 99

J

- joins** 200-202

L

- L1-Norm**
 - reference link 86
- L2-Norm**
 - reference link 86
- Lambda expressions**
 - about 21, 22
 - reference link 21, 25
- Latent Dirichlet Allocation** 126
- learning PySpark**
 - URL 49
- Limited-memory Broyden-Fletcher-Goldfarb-Shanno (BFGS)** 92
 - URL 92
- lines DStream** 213
- local mode, versus cluster mode**
 - URL 24
- logistic regression**
 - used, for predicting infant survival 92

M

matrix multiplication

- with constants 170
- with placeholders 171-173

Maven repository

- URL 136

Meetup Streaming API

- URL 204

missing observations

- checking 60-64

MLlib

- about 79
- data preparation 80
- infant survival, predicting with logistic regression 91, 92
- infant survival, predicting with random forest 94, 95
- machine learning algorithms 80
- overview 80
- URL 79
- utilities 80

ML package

- classification 122, 123
- clustering 123
- estimators 101
- feature extraction 116
- features 116
- overview 97
- Pipeline 104
- regression 127, 128
- Transformer class 98-100

MongoDB

- URL 192

MongoDB database

- interacting, with 194

Mortality dataset

- URL 19

motifs 147-149

N

neural networks

- need for 161, 162
- reference link 161

NLP related feature extractors 116-118

NumPy arrays

- working with 186-188

O

odo

- URL 192

on-time flight performance

- flights, visualizing with D3 154, 155
- popular non-stop flights, determining 151
- references 138, 151, 154

on-time flight performance, use cases

- about 49
- airports, joining 50, 51
- data, visualizing 52, 53
- flight performance, joining 50, 51
- source datasets, preparing 50

outliers

- checking 64-66

P

PageRank

- airport ranking, determining 149, 150
- reference link 150

pandas' DataFrame

- working, with 188, 189

parameter hyper-tuning

- about 111
- grid search 111-114
- train-validation splitting 115, 116

pip

- installing 168

Pipeline 104

placeholders

- used, for matrix multiplication 171-173

Polyglot persistence

- about 185, 186
- references 186

Population vs. Price Linear

Regression Job 239

PostgreSQL

- URL 192

principal component analysis (PCA)

- about 164
- URL 164

project management committee (PMC) 206

Project Tungsten

- about 7, 36
- improvements 11
- references 8, 12, 36

Project Tungsten 2

- about 11
- improvements 12

pseudo-algorithm

- URL 104

PySpark

- speeding up, with DataFrames 36, 37

PySpark performance, improving

- URL 35

pyspark.sql.DataFrame

- URL 53

pyspark.sql.functions

- URL 53

pyspark.sql.types

- URL 67

Python

- communicating, to RDD 34, 35

Python Dataset

- URL 54

R

random forest

- used, for predicting infant survival 94, 95

Receiver-Operating Characteristic (ROC)

- about 93
- URL 93

record schema

- URL 19

regression

- about 103, 127, 128
- AFTSurvivalRegression 103
- DecisionTreeRegressor 103
- GBRegressor 103
- GeneralizedLinearRegression 103
- IsotonicRegression 103
- LinearRegression 103
- RandomForestRegressor 103

Regular Expressions

- reference link 22

Relational Database Management System (RDBMS) 35, 185

relational databases

- interacting, with 193

Resilient Distributed

Datasets (RDDs) 4, 5

- communicating, to Python 34, 35

- creating 18, 19

- files, reading from 20, 21

- global scope, versus local scope 23, 24

- internal functions 17, 18

- interoperating, with 43

- Lambda expressions 21, 22

- schema 20

- schema, inferring with reflection 43, 44

- schema, specifying
 - programmatically 44, 45

Row object 43

S

S3 FileStream Wordcount

(Databricks notebook)

- URL 212

setup.py files

- URL 230

social networks 132

Spark Dataset API 53, 54

Spark Packages

- URL 175

Spark performance

- URL, for Scala vs Python 233

spark rdd

- reference link, for removing elements 22

SparkSession

- about 10
- configuring 227
- creating 228

SparkSession, using in Apache Spark

- URL 33

Spark Streaming

- about 203, 205
- application data flow 207, 208
- DStreams, using 208-212
- need for 206
- reference link 204
- references 206
- URL 204, 216
- use cases 206

Spark Streaming, use cases

- complex sessions 207
- continuous learning 207
- data enrichment 207

- Streaming ETL 206
- triggers 206
- spark-submit command**
 - about 223, 224
 - command line parameters 224
 - URL 224
- SQL**
 - filter statement, executing with where
 - clause 48, 49
 - number of rows 47
 - querying, with 47
 - references 49
- Stateful Network Wordcount Python**
 - URL 217
- Stateful Streaming**
 - URL 216
- statistical model**
 - reference link 69
- stochastic gradient descent (SGD) 91**
- Structured Streaming**
 - about 13, 218-220
 - reference link 13
 - URL 219
- Structuring Spark**
 - URL 36

T

- TensorFlow**
 - about 166-168
 - constant, adding 177
 - installing 169
 - matrix multiplication, with constants 170
 - matrix multiplication, with placeholders 171-173
 - pip, installing 168
 - references 173
 - tensor graph, executing 178, 179
 - URL 168, 169
- TensorFrames**
 - about 174
 - block-wise reducing operations 179
 - configuration 176
 - constant, adding with TensorFlow 177
 - library, creating 176
 - optimal hyperparameters, determining via
 - parallel training 175

- reference link 175-177
- setup 176
- Spark cluster, launching 176
- TensorFlow, installing on cluster 176
- TensorFlow, utilizing with data 174
- using 175
- tf.reduce_min**
 - about 181
 - URL 181
- tf.reduce_sum**
 - about 181
 - URL 181
- topic mining 124-127**
- top transfer airports**
 - determining 146, 147
- Traffic Violations**
 - URL 189
- train-validation splitting 115, 116**
- transformations**
 - about 24
 - .distinct() 26
 - .filter(...) 25
 - .flatMap(...) 26
 - .leftOuterJoin(...) 27, 28
 - .map(...) 24
 - reference link 5
 - .repartition(...) 28
 - .sample(...) 27
 - URL, for methods 24
- Transformer class**
 - about 98
 - Binarizer 98
 - ChiSqSelector 98
 - CountVectorizer 98
 - DCT 99
 - ElementwiseProduct 99
 - HashingTF 99
 - IDF 99
 - IndexToString 99
 - MaxAbsScaler 99
 - MinMaxScaler 99
 - NGram 99
 - Normalizer 99
 - OneHotEncoder 99
 - PCA 99
 - PolynomialExpansion 100
 - QuantileDiscretizer 100

- RegexTokenizer 100
- RFormula 100
- SQLTransformer 100
- StandardScaler 100
- StopWordsRemover 100
- StringIndexer 100
- Tokenizer 100
- VectorAssembler 100
- VectorIndexer 101
- VectorSlicer 101
- Word2Vec 101

U

Uniform Resource Identifier (URI) 192

V

vertex degrees
states, about 145, 146

vertices 138

visualization

- about 71
- histograms 72-75
- used, for interacting between features 76, 77

VS14MORT.txt file

- URL 19

W

where clause

- filter statement, executing 48, 49

Word count

- URL 217

Y

YARN cluster

- num-executors parameter 227
- queue parameter 227